The Systems Engineer's Path

A Five-Volume Book — from senior engineer to systems architect: distributed systems, database internals, and platform design from first principles

A training book · with worked examples, build-alongs, exercises, and full solutions

Preface

This is the second volume. The first, *The Data Engineer's Path*, took a computer-science graduate and trained them into a **senior data engineer** — someone who can hold a company's data platform and reason about the systems they operate. This volume takes that engineer the next step: to a **data architect and staff-level systems engineer** who understands those systems *from the inside*, can *build* them, not just use them, and can *design platforms* and defend the trade-offs at scale. Where Volume 1 taught "the shuffle is expensive," A Five-Volume Book shows you the exchange-operator machinery underneath it, has you build a query engine that shuffles, and teaches you to architect the platform it runs on.

It assumes Volume 1 (or an equivalent senior data engineer) and cross-references it generously, so it is usable on its own. It is denser and goes deeper — but it is written for someone senior, so it respects your time: no re-teaching, heavy signposting, and every abstraction motivated by the problem that forced it into existence.

Four threads run through the whole volume.

- **Think from first principles.** Derive a system's behavior from its underlying model — the failure model, the storage model, the consistency model — not from a tool's documentation. You will see a *first-principles* callout that traces each mechanism to the constraint that produced it.
- **Reason about cost, complexity, and scale.** Now with formal tools — amplification and the RUM conjecture, queueing theory and Little's law, the Universal Scalability Law — and back-of-the-envelope estimation at architect scale. A *cost & scale* callout runs throughout.
- **Design under trade-offs.** Every architecture is a deliberate set of trade-offs against constraints and non-functional requirements; naming and defending them *is* the job. A *design trade-off* callout makes each one explicit, and recurring "design review" exercises have you critique and defend real architectures.
- **Communicate the design.** Architecture decision records, diagrams, and the ability to make a skeptical room trust a design — the third thread of Volume 1, elevated. (The leadership and technical-strategy half of the architect role is the subject of Volume 5; this volume stays technical.)

Correctness is non-negotiable, as in Volume 1. Every citation names a real paper, specification, book with free-legal access, or documented system, with the correct author and venue; numbers are dated because hardware and pricing age; fast-moving claims carry their date; and where the field is genuinely unsettled the text says so. The sources are the canon of systems: the distributed-systems and database papers, *Designing Data-Intensive Applications*, Petrov's *Database Internals*, the CMU and MIT graduate courses, and the official specs.

How to use this book. Read it front to back and do the work. Each chapter motivates its topic, develops it with worked examples and traces, and closes with a *key ideas* box, *exercises* with full solutions, and pointers into the primary literature. Several parts end with a **build-along** — a mini Raft module, an LSM store, a query engine, a Kubernetes operator — because at this level, reading is not enough: you understand a system when you have built a small one. The build-alongs are in Python for approachability, with pointers to how production engines differ in Rust and C++.

Contents

VOL 1 · SYSTEMS PROGRAMMING & PERFORMANCE · PART I

The Machine & the Cost Model

What a program actually runs on: CPU, memory hierarchy, caches, and mechanical sympathy — the real cost model that the RAM abstraction hides.

Chapter 1 — The Real Machine vs. the RAM Model

Before you start: nothing — this is the beginning of Volume 1. · **Pairs with:** Chapter 2 (the CPU up close), Chapter 3 (the memory hierarchy and caches). Together these three install the cost model the rest of the volume optimizes against.

BEFORE YOU READ ON — PREDICT FIRST

No prior chapter to recall, so instead make three *guesses* and write them down; the point is to have a number in your head to be surprised by. (1) A modern CPU core can do roughly how many simple operations — additions, comparisons — per second: about a million, a billion, or a trillion? (2) Reading a value that happens to sit in the CPU's fastest cache versus reading one that must come from main memory (DRAM): same speed, about 10× apart, or about 100× apart? (3) When you write `x = a + b` in C, is "fetch `a` from memory" the same cost as "add"? Answers resolve over the course of this chapter.

Every algorithms course you have ever taken quietly lied to you, and the lie was a good one. It said: assume the machine can read any piece of memory, do one arithmetic operation, and write the result, each in one fixed unit of time, the same unit regardless of *which* piece of memory or *which* operation. On that assumption you learned to count operations, and counting operations told you whether an algorithm scaled. That assumption is the **RAM model**, and it is the foundation of asymptotic analysis. It is also false about the machine on your desk in ways that span six orders of magnitude — and the whole of performance engineering, the whole reason this volume exists, lives in the gap between the model and the metal.

This chapter is about that gap. We will state precisely what the RAM model assumes, see exactly where a real processor and a real memory system diverge from it, and meet the single most important divergence — the **memory wall**, the enormous and growing distance between how fast a CPU can compute and how long it takes to fetch a byte from DRAM. The goal is not to make you distrust big-O; big-O is right about what it claims. The goal is to make you fluent in the second cost model that big-O deliberately hides — the one denominated in *where the data is*, not *how many operations you do* — because at the level this book operates, that second model is where the performance is won and lost.

1.1 What the RAM model assumes

The **Random Access Machine** (RAM) model is the idealized computer that theoretical analysis runs on. It has a processor that executes instructions one at a time, in program order, and a memory of cells that the processor can read or write by address. Its defining assumption is **uniform cost**: every basic operation — an arithmetic op, a comparison, a load from memory, a store to memory — takes the same constant amount of time, usually idealized as "one unit," and crucially that unit does not depend on which address you touch. Reading cell 5 costs the same as reading cell 5,000,000,000. There is no notion of "nearby" or "far away" memory; access is, as the name says, uniformly *random-access*.

This is a spectacularly useful abstraction, and you should not give it up. Because every operation costs one unit, the running time of an algorithm is just the *number of operations* it performs, which you can count by reading the code. That is what lets you say a linear scan is $O(n)$, a comparison sort is $O(n \log n)$, and a nested-loop join is $O(n^2)$, and to know — before running anything — that the last one will eventually outrun any machine you can buy. The RAM model throws away every detail of real hardware precisely so that *growth rate* survives, and growth rate is the thing that no amount of faster hardware can fix. Nothing in this chapter contradicts that. An $O(n^2)$ algorithm is a disaster on any real machine too.

What the model buys with that simplification is a blind spot, and the blind spot is the subject of this volume. By declaring every operation equal, the RAM model asserts three things that are simply untrue of the computer in front of you:

- **All memory accesses cost the same.** They do not. On real hardware the cost of a load depends enormously on *where the data currently sits* — in a register, in a small fast cache, in a larger slower cache, in DRAM, on an SSD, or across a network — and those costs differ by factors of ten to factors of ten million.
- **One instruction takes one unit of time.** It does not. A real CPU overlaps instructions, executes them out of order, guesses the outcome of branches, and can retire several instructions in a single clock tick or stall for hundreds of ticks on a single one — the subject of Chapter 2.
- **Operations are independent and equal.** They are not. A multiply may cost more than an add; a division much more; and an instruction that depends on the result of the previous one cannot start until that result is ready, so *dependencies* between operations, not just their count, govern speed.

Each of these is a place where the abstraction leaks. This chapter takes the first one — the uniform-cost-memory assumption — because it is the largest leak by far, and because understanding it reorganizes how you think about all fast code. Chapter 2 takes the second (the CPU), and Chapter 3 takes the mechanism, caches, that turns the first assumption from "false" into "false but predictable, and therefore exploitable."

> **QUICK CHECK**
>
> State the RAM model's uniform-cost assumption in one sentence, and name the real-hardware fact that most directly contradicts it. (Answer below the next section — try it from memory first.)

1.2 The memory wall

Here is the historical fact that makes the uniform-cost assumption not just wrong but *catastrophically* wrong on a modern machine. For several decades, the speed at which a CPU can execute instructions improved much faster, year over year, than the speed at which main memory (DRAM) can deliver a requested byte. Processor throughput raced ahead; DRAM *latency* — the time from "I want the byte at this address" to "here is the byte" — improved only slowly. The two curves diverged, and the widening gap between them was named the **memory wall** by Wulf and McKee in 1995, whose short paper

"Hitting the Memory Wall" pointed out the then-uncomfortable consequence: as the gap grows, the average time to touch memory comes to dominate program performance, no matter how fast the processor's arithmetic is (Wulf & McKee, *ACM SIGARCH Computer Architecture News*, 1995).

The consequence for us is stark and it is worth stating as baldly as possible. On a modern processor, the time to fetch a single value from DRAM is on the order of a *hundred nanoseconds* — and in that same hundred nanoseconds, a single core can execute on the order of hundreds of instructions. So a load that misses every cache and goes to main memory is not "one unit" of work in any meaningful sense; measured against the arithmetic the core could otherwise be doing, it is hundreds of units. A program that spends its time chasing pointers through DRAM can leave the processor's arithmetic units idle more than 90% of the time, waiting. The CPU is not slow. It is *waiting for memory*. That single sentence explains an enormous fraction of the real-world performance problems this volume teaches you to see.

The hardware's answer to the memory wall is not faster DRAM — DRAM latency is hard physics and improves slowly. The answer is the **memory hierarchy**: small, fast, expensive memories (caches) placed close to the processor, holding copies of the data the program is most likely to touch next, backed by progressively larger, slower, cheaper memories behind them. The hierarchy does not repeal the memory wall; it *hides* it, but only for programs whose access patterns let the caches do their job. Learning to write those programs — to earn the cache's help rather than fight it — is a large part of what "systems performance" means, and it is the whole subject of Chapter 3. For now the load-bearing idea is only this: *the cost of an operation is dominated by where its data lives, and the gap between "close" and "far" is the central fact of modern performance.*

(Answer to the quick check: the RAM model assumes every basic operation, including any memory access, costs one fixed unit of time independent of the address. The fact that most directly contradicts it is the memory hierarchy — accessing data in a nearby cache is one to two orders of magnitude faster than reaching main memory, and further tiers are slower still.)

1.3 Latency, as orders of magnitude only

To reason about the real cost model you need anchors: a rough sense of how far apart the tiers of the hierarchy are. But here we must be careful, because this is exactly the kind of place where a confident, precise-looking number is worse than useless. **The absolute latencies below are approximate, they are machine-dependent, and they drift as hardware changes.** What is stable — what you should actually memorize — is the *shape*: the ordering of the tiers and the rough number of orders of magnitude between them. Treat every specific figure as "as of the mid-2020s, on a typical server-class x86-64 core, give or take a large factor."

With that caveat stated as loudly as it can be, here is the ordering. It is the single most important picture in this volume.

Where the data is	**Rough access latency (order of magnitude)**	**Relative to L1**
CPU register	< 1 ns (available essentially at once, within the pipeline)	—
L1 cache	~1 ns (a few clock cycles)	1×
L2 cache	a few ns (~10 cycles)	~10×
L3 cache	~10 ns (tens of cycles)	~10–50×
Main memory (DRAM)	~100 ns (hundreds of cycles)	~100×
Local SSD (NVMe)	tens of µs	~10^4–10^5×
Same-datacenter network round trip	hundreds of µs to ~1 ms	~10^5–10^6×

The register-through-DRAM rows follow the memory-hierarchy access times given for a specific Intel Core i7 in Bryant and O'Hallaron's *Computer Systems: A Programmer's Perspective* (CS:APP, Chapter 6), which are themselves presented there as approximate and processor-specific; the SSD and network rows are conventional community orders of magnitude of the kind popularized by the widely circulated "latency numbers every programmer should know" list (attributed to Jeff Dean, and later made time-adjustable by Colin Scott). **Use them as ratios, never as guarantees.** If your program's performance depends on whether L2 is 8 cycles or 14 on your particular chip, you are measuring, not reading a book — and measuring is exactly what Chapter 3 and the performance-engineering chapters teach.

Read the table for its shape and three facts fall out. First, the jumps are *multiplicative*, not additive: each tier is roughly 10× to 100× slower than the one above it, so the distance from register to network round trip is not "a lot" but a factor of around a million. Second, there is a cliff between on-chip (register, L1/L2/L3 — nanoseconds) and off-chip (DRAM and below — DRAM at ~100 ns, then a huge further step to SSD at microseconds), and the memory wall is the height of that cliff. Third — and this is the one that reorganizes how you write code — the difference between a cache hit and a cache miss to DRAM is the same ~100× that the difference between a good and a bad algorithm often is. You can lose two orders of magnitude of performance without changing your algorithm's big-O at all, purely by where your data lands.

THE SAME IDEA THREE WAYS: RAM MODEL VS. REAL MACHINE

1. In words. The RAM model charges one unit for every memory access. The real machine charges wildly different amounts depending on which tier of the hierarchy holds the data — from under a nanosecond in a register to ~100 ns in DRAM and far more below that.

2. As a picture. A pyramid: registers at the tiny fast apex, then L1, L2, L3, then DRAM, then SSD, then network at the wide slow base. Size grows and speed falls as you descend.

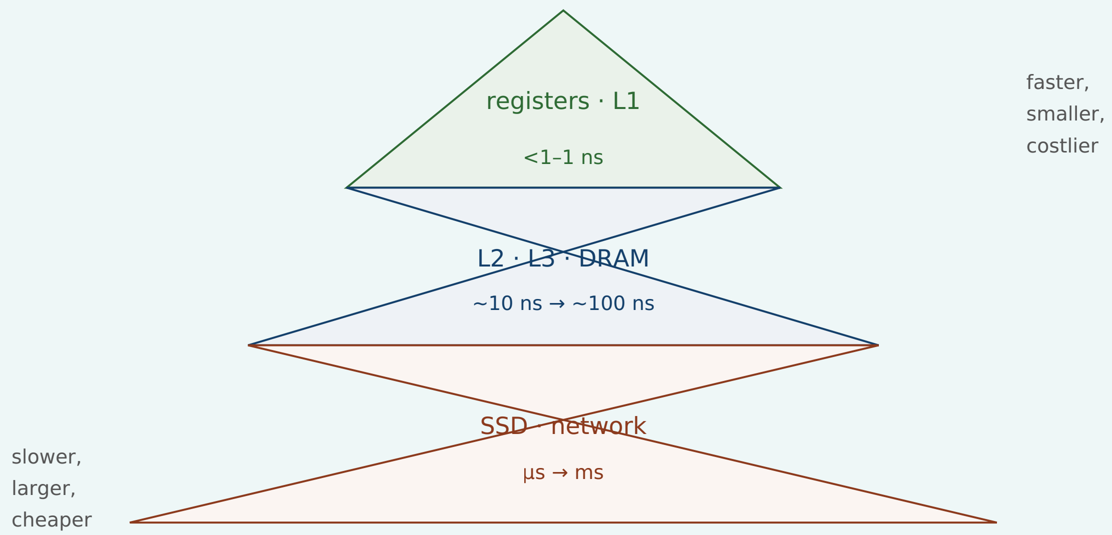

Figure 1.1: The memory hierarchy. Latency rises multiplicatively as you descend; the RAM model pretends the whole pyramid is a single flat tier.

3. As a trace. Consider summing an array of 8 million longs (64 MB), one value at a time. The RAM model says: $8{\times}10^6$ loads + $8{\times}10^6$ adds = $1.6{\times}10^7$ "units," done. The real machine says: the array does not fit in cache, so the cost is dominated by streaming 64 MB up from DRAM; the adds are essentially free, hidden under the wait for memory. Same operation count, completely different account of where the time goes — and only the second account predicts what you will actually measure.

1.4 Why "one operation = one unit of time" is a useful lie

It would be easy to draw the wrong conclusion from all this — to decide the RAM model is broken and should be discarded. That is exactly backwards, and getting the relationship right is the whole point of this chapter. The RAM model is a *useful lie*, in the precise sense that it is false in its literal claims but true and indispensable for the job it is meant to do. Its job is to predict how cost *scales*, and for that job the constant-factor differences between memory tiers are noise: whether a memory access costs 1 ns or 100 ns, an $O(n^2)$ algorithm still loses to an $O(n \log n)$ one once n is large enough. The uniform-cost lie is what makes asymptotic analysis *possible*, and asymptotic analysis is non-negotiable. You must know it and you must trust it, for what it is for.

The mistake is not using the RAM model. The mistake is using it for a question it was never built to answer: *how fast will this actually run?* The RAM model has nothing to say about constant factors, and on modern hardware the constant factors span a hundredfold, all of it hidden inside the word "one" in "one unit of time." Two algorithms with identical O(n) complexity can differ by 50× in wall-clock time because one walks memory in an order the caches love and the other scatters its accesses across DRAM — and the RAM model, by construction, cannot see the difference. So the discipline is to hold two cost models at once and know which question each one answers:

Question	Right model	Denominated in
Will this scale? Which algorithm as $n \to \infty$?	RAM / big-O	operation count
How fast will it actually run at this size on this machine?	the real cost model	where the data lives; bytes moved across tiers

This is the through-line of the entire volume. Every technique that is coming — cache-friendly data layout, avoiding pointer chasing, keeping the CPU's pipeline fed, minimizing system calls, batching I/O — is a way of respecting the second model *after* you have already satisfied the first. You pick a good algorithm with big-O; then you make it fast with mechanical sympathy. Neither replaces the other. An engineer who knows only big-O writes asymptotically optimal code that runs ten times slower than it should; an engineer who knows only micro-optimization polishes an O(n^2) disaster. You need both, in that order.

TRAP: TRUSTING THE RAM MODEL FOR PERFORMANCE

The classic error this chapter exists to prevent: reasoning that because two implementations have the same big-O, they will perform about the same — and therefore choosing between them on style or convenience rather than measuring. "They're both O(n), so it doesn't matter" is true about *scaling* and can be false about *speed* by one to two orders of magnitude, because big-O deliberately discards the very constant factor — where the data lives — that dominates real time. A linked list and an array can both be O(n) to traverse; the array is often many times faster because it streams contiguous memory the cache can prefetch, while the list chases pointers into random DRAM addresses (we will measure a version of this in Chapter 3). The fix is not to distrust big-O — it is right about scaling — but to *stop asking big-O a question it cannot answer*. When the question is "how fast," the answer comes from the memory hierarchy and, ultimately, from measurement, never from the operation count alone.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A colleague says, "Both loops are O(n), so they'll perform the same." Cover the paragraph below and articulate, in two or three sentences, why that reasoning is incomplete — using the words *cache*, *constant factor*, and *memory hierarchy*.

Model answer. "They have the same *asymptotic* cost, which tells us they scale the same way — but big-O throws away the constant factor, and on real hardware that constant is dominated by where the data lives in the memory hierarchy. If one loop streams contiguous memory and the other chases pointers into random DRAM addresses, they can differ by 10× to 100× in wall-clock time despite identical big-O, because a cache hit and a DRAM miss are about two orders of magnitude apart. So same complexity, potentially very different speed — we should measure before assuming they tie." This answer signals seniority precisely because it separates the two cost models and names which one governs the question being asked.

1.5 Mechanical sympathy: the thesis of this volume

There is a phrase for the discipline this chapter is pointing at, borrowed from motor racing and popularized in software by Martin Thompson: **mechanical sympathy**. A driver with mechanical sympathy understands enough about how the car works to get the most out of it without breaking it; a programmer with mechanical sympathy understands enough about how the machine works to write code that runs *with* the grain of the hardware rather than against it. It does not mean writing assembly, and it does not mean premature micro-optimization. It means knowing that the machine has a memory hierarchy, a pipelined out-of-order CPU, a cost for crossing into the operating system, and a real network underneath every abstraction — and letting that knowledge shape the code you write by default.

That is the promise of this volume, and this chapter is its foundation. We have established the one idea everything else builds on: *the RAM model's uniform cost is a useful lie, and real performance comes from the cost model it hides — the one where the price of an operation is set by where its data lives.* From here the volume descends into the machine to make that concrete. Chapter 2 opens up the CPU itself — pipelining, out-of-order execution, branch prediction — to explain the second broken assumption, that one instruction takes one unit of time. Chapter 3 opens up the memory hierarchy — cache lines, locality, the three kinds of miss — to turn "memory access cost varies" from a warning into a set of concrete, exploitable rules. By the end of Part 1 you will look at a loop and see not an operation count but a pattern of movement through the hierarchy, and that shift in vision is the beginning of writing fast code.

CAN YOU… WITHOUT LOOKING?

- State the RAM model's **uniform-cost assumption** and name the three real-hardware facts it gets wrong (memory is not uniform-cost; instructions do not take one unit each; operations are not independent and equal).
- Explain the **memory wall**: CPU throughput outgrew DRAM latency for decades, so average memory-access time comes to dominate performance (Wulf & McKee, 1995).
- Recite the memory hierarchy **in order** — register < L1 < L2 < L3 < DRAM < SSD < network — and say *why the numbers are orders of magnitude only* (machine- dependent, drift with hardware).
- Say why "one operation = one unit of time" is a **useful lie**: right about scaling, silent about the ~100× constant factor that decides real speed.
- Hold both cost models at once and state **which question each answers** (big-O: scaling; the real cost model: actual speed).
- Define **mechanical sympathy** and say why it comes *after* choosing a good algorithm, not instead of it.

RETRIEVAL — CLOSED BOOK

Cover everything above. Answer from memory; then check yourself against the section noted. Rate your confidence (low/med/high) before you look — an overconfident miss is your most valuable signal.

1. **R1.** What single assumption defines the RAM model, and what does adopting it buy you? (§1.1)
2. **R2.** What is the memory wall, who named it, and what is its consequence for program performance? (§1.2)
3. **R3.** Put these in order of increasing latency and give the rough order-of- magnitude gaps: DRAM, L1 cache, register, network round trip, L3 cache, SSD. Why must you present them as orders of magnitude? (§1.3)
4. **R4.** Two implementations have identical O(n) complexity. Can one be 50× slower? Explain in terms of what big-O discards. (§1.4)
5. **R5.** Which cost model answers "will it scale?" and which answers "how fast will it run?" — and in what units is each denominated? (§1.4)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP)**, Chapter 6 ("The Memory Hierarchy"). The canonical treatment; its Core i7 access- time figures are the source of this chapter's register-through-DRAM orders of magnitude, and it is careful to present them as processor-specific. The free CMU 15-213 course materials cover the same ground.
- **Wulf & McKee, "Hitting the Memory Wall: Implications of the Obvious"** (*ACM SIGARCH Computer Architecture News*, 1995). The two-page paper that named the memory wall; short, readable, and prescient.
- **Drepper, "What Every Programmer Should Know About Memory"** (2007). A long, free, seminal write-up on caches and DRAM. Dated in its specific numbers — read it for the mechanisms, not the figures — but unmatched in depth on why the hierarchy exists.
- **"Latency Numbers Every Programmer Should Know."** The community list attributed to Jeff Dean, and Colin Scott's interactive, year-adjustable version. Use strictly as orders of magnitude; the absolute numbers are dated hardware and are meant to be read as ratios.

Exercises

1.1 WARM-UP State the RAM model's uniform-cost assumption in one sentence. Then list the three things it asserts about real hardware that this chapter says are false.

1.2 WARM-UP Put these seven locations in order from fastest to slowest access, and give the rough order-of- magnitude latency for each: L2 cache, DRAM, register, network round trip (same datacenter), L1 cache, local NVMe SSD, L3 cache.

1.3 WARM-UP In two sentences, explain the memory wall and why it makes the RAM model's uniform-cost assumption worse over time rather than better.

1.4 CORE A DRAM access costs roughly 100 ns and a single core executes on the order of a few billion simple operations per second. (a) Roughly how many simple operations could the core have executed in the time of one DRAM access? (b) In one or two sentences, explain what this ratio implies for a program that spends most of its time following pointers into randomly located memory. State clearly which of your numbers are order-of-magnitude approximations.

1.5 CORE Your colleague replaced an O(n log n) sort with an O(n) bucketing pass and is puzzled that the "faster" version runs slower on their real dataset. Give two distinct explanations grounded in this chapter, and say what single action would settle which one (if either) is right.

1.6 FADED *Worked, with the last step for you.* You must argue in a design review that a proposed change — switching a hot lookup table from a hash map spread across the heap to a sorted array searched by binary search — could be faster *despite* binary search's O(log n) being asymptotically worse than the hash map's O(1). Steps 1–3 are done; complete step 4.
Step 1 (given): Both are cheap in big-O for realistic n, so scaling is not the deciding factor here — this is a constant-factor question.
Step 2 (given): The constant factor is dominated by the memory hierarchy: how many *slow* memory accesses each structure makes per lookup.
Step 3 (given): A hash map lookup typically jumps to one effectively random heap location (likely a cache miss); a binary search over a small contiguous sorted array touches a handful of locations, but they are close together and the array may sit largely in cache.
Step 4 (you): Complete the argument — state the conclusion about which is likely to make fewer costly trips to slow memory, and name the one thing you would do before trusting the argument. Explain *why that final step and not simply "pick the lower big-O."*

1.7 MIXED For each situation, decide whether the RAM/big-O model or the real cost model is the right tool, and say why in one line: (a) choosing between a nested-loop join and a hash join for tables that will grow to billions of rows; (b) explaining why two O(n) array-processing loops on a fixed 100 MB array differ by 8× in wall-clock time; (c) deciding whether an algorithm will still be viable when the input is 1000× bigger; (d) explaining why a linked-list traversal is slower than an array traversal of the same length.

1.8 MIXED A teammate presents two claims: (i) "This algorithm is O(n), so it's optimal." (ii) "This one's O(n) too but ten times faster in practice." Are these claims in contradiction? Decide which cost model each claim belongs to, explain whether both can be true at once, and state what information you would still need to choose between the two implementations for production.

1.9 CORE Define *mechanical sympathy* in your own words, and explain the ordering rule this chapter gives: why you apply it *after* choosing a good algorithm, not instead of choosing one. Give a one-sentence example of each failure mode (knowing only big-O; knowing only micro-optimization).

1.10 STRETCH The latency table in §1.3 is labelled "orders of magnitude only, machine-dependent." Give three distinct reasons the specific numbers cannot be trusted as exact for *your* machine, and explain why the *ratios* between tiers are nonetheless far more stable than the absolute values. What would you do to get numbers you could actually trust for a performance decision?

1.11 LAB On your machine, discover the real numbers instead of trusting the book. Run `getconf LEVEL1_DCACHE_LINESIZE` and `lscpu` (or `sysctl hw` on macOS) and write down your L1/L2/L3 cache sizes and the cache-line size. Then, in any language, write a loop that sums a large array that comfortably exceeds your L3 size and one that fits inside L1, timing each per element. Report the two per-element times and their ratio. State clearly which numbers are yours (the cache sizes, the measured ratio) and which came from the book (nothing — that's the point of the exercise).

Chapter 2 — The CPU Up Close

Before you start: Chapter 1 (the real machine vs. the RAM model — the two cost models). · **Pairs with:** Chapter 3 (the memory hierarchy — the other reason "one instruction = one unit" fails). This chapter attacks the RAM model's *second* false assumption.

BEFORE YOU READ ON

From Chapter 1, recall and answer from memory: (1) The RAM model makes three false assertions about real hardware; §1.1 named them. One was "all memory accesses cost the same" (the memory-hierarchy leak). What was the *second* one — the one this chapter is about? (2) Why is "one operation = one unit of time" called a *useful lie* rather than simply a mistake? (3) *Predict:* a modern CPU executes instructions one after another, finishing each before starting the next — true or false? Answers resolve in §2.1–2.2.

In Chapter 1 we said the RAM model tells three lies, dealt with the first (memory is not uniform-cost), and promised to return for the second: that *one instruction takes one unit of time*. This is the chapter. A modern CPU does not execute your instructions one at a time, in order, each in a tidy tick of the clock. It executes many at once, frequently out of their written order, and it *guesses* — commits to work before it knows whether that work is needed. Understanding this is what separates code that keeps the processor busy from code that leaves it stalling, and the difference, for the same big-O, can again be a large constant factor.

We will stay conceptual and correct. This chapter describes *mechanisms* — the pipeline, superscalar and out-of-order execution, branch prediction, instruction-level parallelism — and gives their costs only as orders of magnitude with explicit caveats, because the exact pipeline depth and misprediction penalty are microarchitecture-specific numbers that differ from chip to chip and that this book will not invent for you. The mechanisms, though, are general and stable, and knowing them changes how you read a hot loop. The through-line: **straight-line, predictable, independent work is fast because it keeps the machine full; branchy, dependent, unpredictable work is slow because it forces the machine to wait or to throw work away.**

2.1 Fetch–decode–execute, and why it isn't one step

At the coarsest level a CPU repeats a cycle: **fetch** the next instruction from memory, **decode** it into control signals, **execute** it (compute, or read/write memory), and make its results available. This is the von Neumann fetch–decode–execute loop, and if a processor really did it one instruction at a time — finish all stages for instruction 1, then start instruction 2 — then "one instruction = one unit of time" would be roughly true and this chapter would be short. Early processors worked essentially that way.

The trouble is that the stages use *different parts* of the chip. While the execute unit is busy with instruction 1, the fetch unit is idle; while instruction 1 is being decoded, the fetch unit is idle again. Doing one instruction start-to-finish before beginning the next wastes most of the hardware most of the time. The entire architecture of a fast CPU is a series of answers to the question *how do we stop leaving hardware idle?* — and each answer breaks the RAM model's tidy accounting a little more. We take them in order: pipelining (overlap the stages), superscalar (duplicate the units), out-of-order (reorder to avoid waiting), and speculation (guess so you never have to wait for a branch). Bryant and O'Hallaron develop this progression in detail in *Computer Systems: A Programmer's Perspective* (CS:APP), Chapters 4 and 5; we take the conceptual tour.

2.2 Pipelining: overlap the stages

The first idea is an assembly line. Split instruction processing into stages (conceptually: fetch, decode, execute, memory, write-back — real designs have more, and the exact count is a microarchitecture-specific number we will not invent), and let each stage work on a *different* instruction at the same time. While instruction 1 is in "execute," instruction 2 is in "decode," and instruction 3 is in "fetch." Once the pipeline is full, an instruction *completes* every clock cycle even though each individual instruction still takes several cycles to pass all the way through. Throughput goes up by roughly the number of stages, though the latency of any single instruction does not.

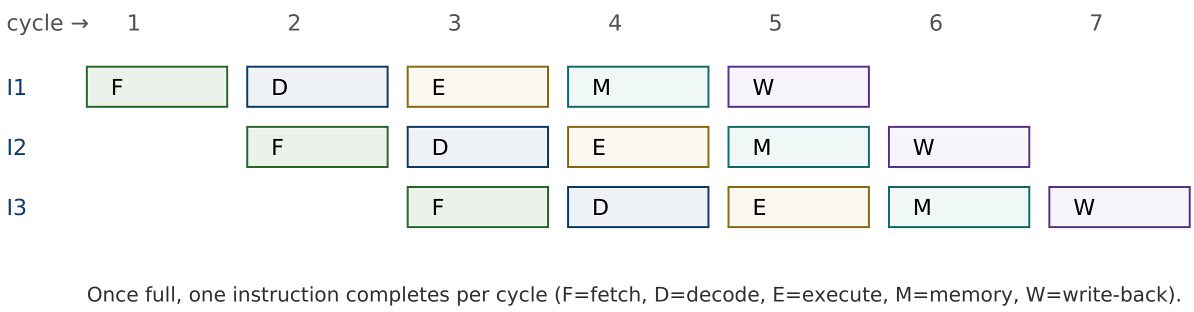

Figure 2.1: A pipeline overlaps stages so different instructions occupy different units in the same cycle. Stage names and count are illustrative; real pipelines have more stages.

Pipelining is why "one instruction = one unit of time" is doubly wrong: an instruction takes several cycles of latency, yet the machine can *complete* one (or more) per cycle. But the assembly line only runs smoothly if the next instruction is always ready and its inputs are available. Two things break that smooth flow — a dependency where an instruction needs a result that is not ready yet (a **data hazard**, which forces a stall), and a branch where the CPU does not yet know *which* instruction comes next (a **control hazard**). The rest of the chapter is about how the hardware fights these two, and how your code can help or hurt that fight.

> **QUICK CHECK**
>
> In a filled pipeline, an instruction's *latency* is several cycles but the pipeline's *throughput* is about one instruction per cycle. In one sentence, why are those two numbers different? (Answer after the next section.)

2.3 Superscalar and out-of-order: more units, smarter order

If one assembly line is good, several are better. A **superscalar** processor has multiple execution units and can fetch, decode, and issue several instructions *per cycle*, so its peak throughput is more than one instruction per cycle — provided it can find independent instructions to feed the parallel units. That proviso is everything. Two instructions can run in the same cycle only if neither needs the other's result; if instruction 2 uses instruction 1's output, they must run in sequence no matter how many units are idle.

To keep those units fed, modern CPUs execute **out of order**. Rather than stalling the whole machine when the next instruction in program order is waiting on a slow input (say, a value still coming from DRAM — recall Chapter 1: that is hundreds of cycles), the processor looks ahead in the instruction stream, finds later instructions whose inputs *are* ready, and executes those now, filling the gap. Results are then reassembled into the original program order before they become visible, so the program's observed behavior is still that of straightforward sequential execution — the reordering is invisible to correctness but decisive for speed. This is why a cache miss does not necessarily stall the CPU: if there is independent work nearby, the machine does that work while the miss is serviced, hiding the latency.

The practical lesson lands directly on how you write hot code. The processor can only overlap work it can prove is independent, and it can only look a bounded distance ahead. Code with lots of **instruction-level parallelism** (ILP) — many operations that do not depend on each other's results — gives the out-of-order engine plenty to chew on and runs near the machine's peak. Code that is a long *dependency chain* — each step needing the previous step's result — starves the parallel units no matter how many there are, because there is never more than one thing that can run next. Same instruction count, very different speed; and, as in Chapter 1, the difference is exactly the constant factor big-O throws away.

(Answer to the quick check: latency counts the cycles for one instruction to traverse all stages; throughput counts how often a new instruction completes. Because stages overlap, a new instruction can finish every cycle even though each one individually took several cycles to get through.)

THE SAME IDEA THREE WAYS: DEPENDENCY CHAINS VS. INDEPENDENT WORK

1. In words. Summing an array into a single accumulator forces each add to wait for the previous add's result — a dependency chain, low ILP. Summing into several independent partial accumulators and combining them at the end lets the superscalar, out-of-order engine run several adds per cycle — high ILP.

2. As a picture. One long chain `a→a→a→a` (each arrow "must finish before") versus four short parallel chains that only merge at the very end.

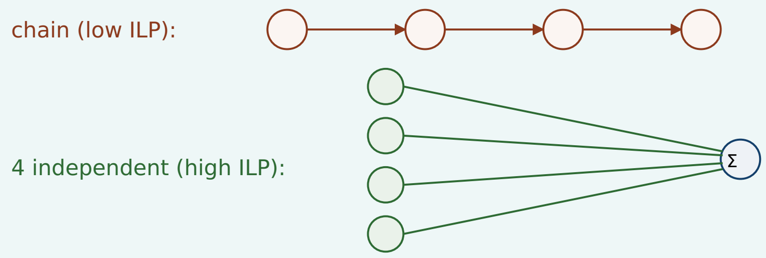

Figure 2.2: A dependency chain serializes execution; independent operations expose parallelism the CPU can exploit. Combining partial sums at the end recovers the same result.

3. As a trace. Both compute the same sum of n numbers — identical operation count, identical $O(n)$. The chained version can retire roughly one add per dependency latency; the four-accumulator version can keep several execution units busy at once, finishing in a fraction of the wall-clock time. (This is a real and measurable effect; CS:APP Chapter 5 develops it as "loop unrolling with multiple accumulators," and the performance-engineering chapters of this volume return to it.)

2.4 Branch prediction and the cost of a miss

Now the control hazard. When the CPU reaches a conditional branch — an `if`, a loop test, a `switch` — it does not yet know which way execution will go, because that depends on a condition that may not be computed yet. But the pipeline needs to fetch *something* next; it cannot sit idle for the several cycles it takes to resolve the branch. So the processor **predicts** which way the branch will go and speculatively executes down the predicted path, doing real work on the assumption that its guess is right.

Modern **branch predictors** are remarkably good — they track the history of each branch and find patterns, and on predictable branches (a loop that runs a million times and exits once; a check that is almost always false) they are right the overwhelming majority of the time, so the speculation is nearly free and the pipeline never stalls. This is why a loop with a highly predictable exit condition runs fast: the CPU correctly guesses "keep looping" every time until the one time it does not.

The cost arrives on a **misprediction**. When the branch resolves and the guess was wrong, all the speculatively executed work on the wrong path must be discarded, and the pipeline must be refilled from the correct target — the instructions already in flight are thrown away and the machine restarts fetching from the right place. The penalty is roughly proportional to how deep the pipeline is: everything that was in flight is lost. On modern deeply pipelined CPUs this penalty is on the order of *ten to a few tens of cycles* per misprediction — an order of magnitude, not a figure to quote precisely, since the exact number is specific to each microarchitecture and this book will not invent one for a particular chip. (CS:APP, Chapter 5, works a concrete example for one processor; treat any single number as machine-specific.) What matters is the shape: a mispredicted branch costs many times what a correctly predicted one does.

The consequence for your code is direct and sometimes surprising. A branch that is *unpredictable* — one that goes each way about half the time, in no pattern the predictor can learn — pays the misprediction penalty roughly half the time it executes. In a tight loop, that can dominate the loop's cost. This is the mechanism behind a famous and initially baffling observation: processing a *sorted* array can be markedly faster than processing the same data *unsorted*, when the loop body contains a data-dependent branch, because sorting makes the branch predictable (long runs of the same outcome) whereas the shuffled data makes it a coin flip the predictor cannot learn. The arithmetic is identical; only the predictability changed. When you see a performance mystery like that, the branch predictor is a prime suspect.

TRAP: THE UNPREDICTABLE BRANCH IN THE HOT LOOP

The error is to assume that because two versions of a loop do the same arithmetic, they cost the same — forgetting that a *data-dependent, unpredictable branch* inside a hot loop can add a misprediction penalty on a large fraction of iterations, dwarfing the arithmetic. It is the CPU-level cousin of the Chapter 1 trap ("same big-O, so same speed"): same operation count, very different wall-clock time, because one version keeps the pipeline full and the other keeps flushing it. The tells: a branch whose outcome depends on the data and follows no pattern, in code that runs an enormous number of times. The fixes are the standard toolkit — make the branch predictable (e.g. by sorting), or remove it entirely by turning control flow into data flow (branchless code, conditional-move or arithmetic tricks, table lookups, or SIMD masks — the performance-engineering chapters cover these). But confirm with measurement first: on *predictable* branches the predictor already makes the branch nearly free, and a branchless rewrite there just adds complexity for no gain.

2.5 What this means for the code you write

Step back and the theme of the chapter is one sentence: **the CPU runs fastest when you feed it a steady stream of predictable, independent work, and stalls when you feed it dependencies and surprises.** That single idea reorganizes a pile of otherwise-disconnected performance folklore into consequences of how the machine actually works:

- **Straight-line code with predictable branches** keeps the pipeline full and speculation cheap — fast.
- **Independent operations (high ILP)** let the superscalar, out-of-order engine run several things per cycle — fast.
- **Long dependency chains** serialize execution and starve the parallel units — slow, even at the same instruction count.
- **Unpredictable, data-dependent branches** cause mispredictions that flush the pipeline — slow, sometimes dramatically.

- **A cache miss (Chapter 1) need not stall the CPU** if there is independent work nearby for the out-of-order engine to do while it waits — which is why ILP and memory behavior are entangled, and why both matter together.

None of this changes an algorithm's big-O, and none of it should tempt you to abandon big-O — a pipeline-friendly O(n^2) is still a disaster. This is, exactly as in Chapter 1, the *second* cost model at work: after you have chosen a good algorithm, the constant factor that decides real speed is set by how well your code fits the machine — its memory hierarchy (Chapter 1 and Chapter 3) and its pipelined, speculative, superscalar core (this chapter). Chapter 3 now returns to the memory side and turns "data location matters" into the concrete, exploitable rules of caches and locality — the other half of mechanical sympathy.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. In a review, someone asks why sorting an array before a filtering loop made the loop faster, even though sorting is extra work and the loop does the same comparisons either way. Cover the paragraph below and explain it in two or three sentences, using *branch predictor*, *misprediction*, and *pipeline*.

Model answer. "The loop has a data-dependent branch. On the unsorted data that branch goes each way unpredictably, so the branch predictor is wrong about half the time, and every misprediction flushes the pipeline — a penalty on the order of tens of cycles per miss, paid on a huge fraction of iterations. Sorting makes the branch outcomes come in long predictable runs, so the predictor is almost always right and the pipeline stays full; the arithmetic is identical, but the misprediction cost nearly vanished. Net of the one-time sort, the loop got cheaper because we made it *predictable*, not because we did less arithmetic." Naming the mechanism — rather than saying "sorting is just faster" — is what marks the answer as senior.

CAN YOU… WITHOUT LOOKING?

- Explain **pipelining** and why it makes an instruction's *latency* (several cycles) differ from the pipeline's *throughput* (about one per cycle).
- Say what **superscalar** and **out-of-order** execution each add, and why both depend on finding *independent* instructions.
- Contrast a **dependency chain** (low ILP, serialized) with **independent work** (high ILP, parallel) and give the multiple-accumulator sum as the example.
- Describe **branch prediction** and the cost of a **misprediction** (flush the pipeline; penalty on the order of tens of cycles — machine-specific, an order of magnitude only).
- Explain why processing a **sorted** array can beat the same data unsorted when the loop has a data-dependent branch.
- State the chapter's one sentence: the CPU is fast on predictable, independent work and stalls on dependencies and surprises — and why this is the RAM model's *second* lie.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate your confidence before checking. An overconfident miss is the signal to reread.

1. **R1.** Why does executing one instruction fully before starting the next waste the hardware, and what is pipelining's fix? (§2.1–2.2)
2. **R2.** What must be true of two instructions for a superscalar CPU to run them in the same cycle, and what does out-of-order execution do when the next in-order instruction is stalled? (§2.3)
3. **R3.** Define instruction-level parallelism. Why is a single-accumulator sum lower ILP than a multi-accumulator sum computing the same total? (§2.3)
4. **R4.** What does a CPU do at a conditional branch before the condition is known, and what exactly is discarded on a misprediction? Give the penalty as an order of magnitude with its caveat. (§2.4)
5. **R5.** Explain the sorted-vs-unsorted array speed difference in terms of the branch predictor. Which of the two cost models from Chapter 1 does this belong to? (§2.4–2.5)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP)**, Chapter 4 ("Processor Architecture") and Chapter 5 ("Optimizing Program Performance"). Chapter 4 builds a pipelined processor from scratch; Chapter 5 covers superscalar/out- of-order machines, branch misprediction, dependency chains, and multiple accumulators, all with concrete (processor-specific) numbers. The primary source for this chapter.
- **Agner Fog, optimization manuals** (agner.org, free). Deep, empirical detail on microarchitecture, pipelines, and branch prediction across real x86 CPUs. Dated in places — verify against current parts — but the mechanism-level explanations are excellent.
- **MIT 6.172, *Performance Engineering of Software Systems*** (free OCW). The lectures on modern processors, ILP, and branch behavior connect these mechanisms to measurable speedups in real code.

Exercises

2.1 WARM-UP Name the four techniques, in the order this chapter introduces them, that a fast CPU uses to stop leaving hardware idle, and give a one-line description of what each one does.

2.2 WARM-UP Distinguish *latency* and *throughput* for a single instruction passing through a pipeline. Why can a pipeline complete about one instruction per cycle while each instruction still takes several cycles from start to finish?

2.3 WARM-UP Define a *data hazard* and a *control hazard* in one sentence each, and say which of the two branch prediction is designed to address.

2.4 CORE Two loops compute the same sum of an array of doubles. Loop A accumulates into a single variable; loop B accumulates into four independent partial sums and adds them at the end. Both do the same number of additions. Explain, using ILP and the out-of-order engine, why B can be substantially faster, and name the one condition on the additions that makes B's parallelism possible.

2.5 CORE A hot loop contains `if (x[i] > threshold) count++;`. On dataset P the branch is taken in long predictable runs; on dataset Q it is taken about half the time in no pattern. The arithmetic is identical. Explain why the loop can run several times faster on P, and estimate qualitatively where the extra time on Q goes.

2.6 COST A branch mispredicts with probability p per execution, and each misprediction costs about k cycles (an order-of-magnitude, machine-specific number — state that caveat). The loop body otherwise takes about b cycles. (a) Write the approximate average cost per iteration. (b) For what range of p does the misprediction term dominate the body? (c) Explain why making the branch predictable attacks p, and a branchless rewrite attacks k (by removing the branch), and why you would measure before choosing.

2.7 FADED *Worked, with the last step for you.* Explain why a cache miss (Chapter 1) does not always stall the CPU. Steps 1–3 given; complete step 4.
Step 1 (given): A load that misses to DRAM takes on the order of hundreds of cycles to return its value (Chapter 1).
Step 2 (given): A modern CPU executes out of order: when the next in-order instruction is waiting, it looks ahead for later instructions whose inputs are already available.
Step 3 (given): If there is independent work near the missing load, the machine can execute that work during the hundreds of cycles the miss is being serviced.
Step 4 (you): State the conclusion about whether the miss stalls the CPU, and name the property of the surrounding code that determines whether the latency gets hidden or not. Explain *why that property, and not the miss latency alone, decides the outcome.*

2.8 MIXED For each observation, decide whether the dominant mechanism is the *memory hierarchy* (Chapter 1), *branch prediction*, or an *instruction-level dependency chain* (this chapter) — and justify in one line: (a) a loop got 3× faster after the input array was sorted, with no change to the arithmetic; (b) a loop got faster after splitting one accumulator into four; (c) a traversal got much slower when the same data was stored as a linked list instead of an array; (d) a tight numeric loop with no branches and no memory pressure is already near peak throughput.

2.9 MIXED A colleague says: "My function is O(n) and yours is O(n), but yours runs 5× faster — that's impossible, big-O says they're the same." Decide which cost model resolves the paradox, list three distinct mechanisms from Chapters 1–2 that could each independently produce a 5× gap at equal big-O, and state the single action that would identify which one is responsible here.

2.10 STRETCH Speculation means the CPU does work it might have to throw away. (a) Explain why this is still a net performance win on typical code. (b) Give a case where heavy speculation on an unpredictable branch is a net *loss*. (c) Speculative execution has also had *security* consequences in real processors. Without inventing specifics, state at a high level why executing instructions before you know they should run could, in principle, leak information — and note that the precise mechanisms and mitigations are a real, separate topic you would need primary sources to describe accurately.

2.11 LAB Measure a dependency chain. In C (or any compiled language), write two loops that sum a large array of `double`: one with a single accumulator, one with four independent accumulators combined at the end. Compile with optimization *off* for the reduction (or use a language/setting that will not auto-vectorize away the difference), time both, and report the ratio. State which numbers are yours and note that the exact ratio depends on your CPU and compiler. If you cannot prevent the compiler from transforming the loop, say so and report what you observed instead — do not report a number you did not actually measure.

Chapter 3 — The Memory Hierarchy & Caches

Before you start: Chapter 1 (the memory wall and the latency hierarchy), Chapter 2 (the CPU; why a miss need not stall). · **Pairs with:** the rest of Part 1 and the performance-engineering chapters, which turn these rules into measured speedups. This chapter makes "data location matters" concrete and exploitable.

BEFORE YOU READ ON

From the previous two chapters, from memory: (1) Chapter 1 said accessing DRAM costs on the order of ~100 ns while a nearby cache hit costs ~1 ns — about how many orders of magnitude apart is that? (2) Chapter 2 said a cache miss does *not* always stall the CPU. Under what condition does the latency get hidden? (3) *Predict:* you sum every element of a large 2-D array. Does the *order* in which you visit the elements — row by row vs. column by column — change how long it takes, given that you touch exactly the same elements either way? By the end of this chapter you will have a measured answer.

Chapter 1 told you the memory wall exists and that hardware answers it with a hierarchy of caches. Chapter 2 told you the CPU can sometimes hide a miss behind other work. This chapter is where "memory access cost varies" stops being a warning and becomes a set of concrete, mechanical rules you can design against — because caches are not magic, they follow simple policies, and once you know the policies you can write code that hits the cache instead of missing it. The payoff is the largest, cheapest performance win available to most programs: not a better algorithm, just the same algorithm arranged so the data is where the CPU expects it.

We will finish with a measurement, not a claim. At the end of the chapter is a short C program that traverses the same array two ways — the cache-friendly way and the cache-hostile way — and the exact speed ratio it produced when run for this book, on a specific machine, so you can see the effect is real and roughly how big it is. The number is machine-dependent, and we will say so; but it is measured, not invented.

3.1 The cache line: memory moves in blocks, not bytes

The first and most important fact about caches overturns a picture you have probably carried since you first learned about memory: that you read one byte, or one `int`, at a time. You do not. When the CPU needs a value that is not already in cache, it does not fetch just that value — it fetches an entire aligned **cache line** (also called a cache block) containing it, and installs the whole line in the cache. On the common x86-64 processors this line is **64 bytes**; Bryant and O'Hallaron state this standard size in *Computer Systems: A Programmer's Perspective* (CS:APP, Chapter 6), and it is what `getconf LEVEL1_DCACHE_LINESIZE` reports on such machines. (It is a hardware parameter, not a law of nature — verify it on your target — but 64 bytes is the near-universal value on current x86-64.)

This single mechanism explains most of cache behavior. When you touch one byte, the seven-plus neighbors sharing its 64-byte line come along for free — they are now in cache, and touching them next costs a cheap hit instead of an expensive miss. So the cost of a memory access is not really "per access"; it is "per line brought in." A program that uses every byte of each line it fetches gets the full value of every expensive DRAM trip. A program that touches one byte per line and then moves far away wastes 63 of every 64 bytes it pays to move — it pays full price for the line and uses a fraction of it. That ratio, *useful bytes per line fetched*, is the hidden efficiency knob behind an enormous amount of performance.

> **QUICK CHECK**
>
> On a machine with 64-byte cache lines, you read one `int` (4 bytes). How many bytes does the hardware actually move into the cache, and how many neighboring `int`s are now cheap to access? (Answer after the next section.)

3.2 Locality: the property caches reward

Caches work — they successfully hide the memory wall — because real programs exhibit **locality of reference**, a strong tendency to access memory that is close in time or space to what they just accessed. There are two kinds, and CS:APP (Chapter 6) frames them as the two properties good code should have:

- **Temporal locality:** if you access a location, you are likely to access it *again soon*. A loop variable, an accumulator, a hot lookup table — accessed repeatedly, so after the first (missing) access they sit in cache and every later access is a hit.
- **Spatial locality:** if you access a location, you are likely to access *nearby* locations soon. Walking an array in order, reading struct fields together — because memory arrives a whole line at a time, spatial locality means the neighbors you fetched are exactly the ones you are about to use.

The hardware bets on both. The cache keeps recently used lines (exploiting temporal locality), and by fetching a full line it pre-loads spatial neighbors (exploiting spatial locality). Some CPUs also *prefetch* — detect a sequential access pattern and fetch lines ahead of the program's demand, so the data is already resident by the time the loop reaches it. All of this is automatic and invisible until you fight it. Your job is to write code whose access pattern *has* locality, so that the hardware's bets pay off. Code with good locality runs near the speed of cache; code that scatters its accesses runs near the speed of DRAM; and from Chapter 1 you know that is about a 100× difference, available on the same algorithm.

(Answer to the quick check: the hardware moves the full 64-byte line, so 64 bytes; that line holds 16 `int`s, so the 15 neighbors sharing the line are now cheap hits.)

3.3 Hits, misses, and the three C's

An access that finds its line already in cache is a **hit**; one that does not is a **miss**, and a miss pays to fetch the line from the next level down (another cache, or DRAM). Since misses are where the time goes, it helps to classify *why* a miss happened. The classic taxonomy, the **three C's** (presented in CS:APP, Chapter 6), sorts every miss into one of three causes:

- **Compulsory (cold) miss:** the first-ever reference to a line. It was never in cache, so the first touch must miss. Unavoidable in principle — you have to load data at least once — but you can make each compulsory miss *count* by using the whole line (spatial locality) once you have paid for it.
- **Capacity miss:** the line was in cache but got evicted because the program's working set — the data it is actively using — is simply larger than the cache. If you stream through 512 MB repeatedly and the cache holds a few MB, lines you will need again get pushed out before you return to them. The fix is to shrink the working set (e.g. process data in cache-sized *blocks*, a technique called blocking or tiling).
- **Conflict miss:** the line was evicted not because the cache was full overall, but because too many of the addresses you are using map to the *same* small set of cache slots and crowd each other out. This one depends on the cache's *associativity*, which we take next.

3.4 Associativity, conceptually

A cache cannot let any line live in any slot — searching the whole cache on every access would be too slow. Instead each memory address maps to a restricted set of slots. In a **direct-mapped** cache, each address has exactly one slot it may occupy; simple and fast, but two hot addresses that happen to map to the same slot will endlessly evict each other even if the rest of the cache is empty — that is a conflict miss. At the other extreme, a **fully associative** cache lets a line go in any slot (no conflict misses, but expensive to build). Real caches are **set-associative**: a compromise where each address maps to a small *set* of slots (say 8, an "8-way" cache) and the line may occupy any slot within its set. This kills most conflict misses while staying cheap enough to build.

You rarely need to compute set indices by hand, and this book will not turn associativity into arithmetic drills. What you need is the intuition: conflict misses are why a data structure can perform badly at one size or memory alignment and fine at another, seemingly at random. A classic symptom is a loop that slows sharply when an array's row length (its "stride") is a power of two that aligns many accesses onto the same cache sets. When performance changes with the exact size or alignment of your data rather than the amount of it, suspect conflict misses — and know the deeper treatment (and the set-index arithmetic) is in CS:APP, Chapter 6.

QUICK CHECK

Classify each miss: (a) the very first read of a freshly allocated array; (b) re-reading a 1 GB array in a loop when the cache holds only a few MB; (c) two hot variables that keep evicting each other though the cache is mostly empty. (Answer below the demonstration.)

3.5 Why arrays are fast and pointer-chasing is slow

Now the payoff that ties the chapter together, and the reason "same big-O, different speed" keeps recurring. Consider two ways to store a sequence of integers you will walk from start to finish. An **array** stores them contiguously: element $i+1$ sits right after element i in memory. A **linked list** stores each element in its own separately allocated node, connected by pointers; consecutive nodes can be anywhere in the heap, often far apart. Both take $O(n)$ to traverse — the same number of "steps" — and the RAM model says they cost the same. The real machine disagrees, sometimes by more than 10×.

Walking the array has excellent spatial locality: each 64-byte line brought in holds many consecutive elements, so most accesses are hits, and the hardware prefetcher, seeing the sequential pattern, fetches lines ahead of you so the data is waiting. Walking the linked list is **pointer-chasing**: each node's address is only known once you have read the *previous* node's pointer, so the accesses are (a) to scattered, effectively random addresses — poor spatial locality, a likely cache miss per node — and (b) a strict dependency chain, since you cannot fetch node $i+1$ until node i arrives. That dependency defeats the latency-hiding trick from Chapter 2: there is no independent work to overlap with the miss, so the CPU stalls on the full DRAM latency at every step. Poor locality and a serial dependency, together, are why pointer-chasing through memory is one of the slowest things a program can do, and why cache-friendly, array-based layouts underpin most high-performance data structures. (This is the mechanism behind the array-vs-list remark back in Chapter 1's warning box — now you know exactly why.)

3.6 A measured demonstration: row-major vs. column-major

Let us prove the effect rather than assert it, with the cleanest possible experiment: the *same* arithmetic on the *same* data, differing only in access order. Store an $N \times N$ matrix of `long` in a single contiguous block in row-major order (row 0, then row 1, …), and sum every element two ways. The **row-major** traversal walks memory in order — inner loop over columns, so consecutive accesses are adjacent in memory (stride 1), excellent spatial locality. The **column-major** traversal walks down columns — inner loop over rows, so each step jumps N elements forward in memory (stride N), touching a different cache line almost every time. Same elements, same additions, same result; only the order differs.

```
#define N 8192            /* an N x N matrix of long = 512 MiB, far larger than cache */

/* row-major: inner index j varies fastest -> contiguous, stride 1 */
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        sum_row += a[(size_t)i * N + j];

/* column-major: inner index i varies fastest -> stride of N longs per step */
for (int j = 0; j < N; j++)
    for (int i = 0; i < N; i++)
        sum_col += a[(size_t)i * N + j];
```

Both loops produced the identical sum (a correctness check that they really do the same work). Built with `gcc -O2` and run for this book on a 12th-generation Intel Core i7 (i7-12700H) laptop, on a 512 MiB matrix that dwarfs the cache, the row-major traversal took roughly **0.04 s** and the column-major traversal roughly **0.95–0.99 s** — the column-major version was about **24–26× slower** across repeated runs, for doing the very same additions in a different order. **This exact ratio is machine-dependent** — it depends on cache sizes, prefetcher, memory bandwidth, and compiler, and you should expect a different number on your hardware — but the *direction and rough magnitude* (a large multiplier, not a few percent) is the robust, reproducible result, and you can reproduce it yourself with the lab exercise. The mechanism is exactly §3.1–3.2: the row-major loop uses all 8 `long`s of every 64-byte line it fetches and lets the prefetcher run ahead; the column-major loop touches one `long` per line and throws the other 7 away, so it issues many times more line fetches and stalls on the misses it cannot hide.

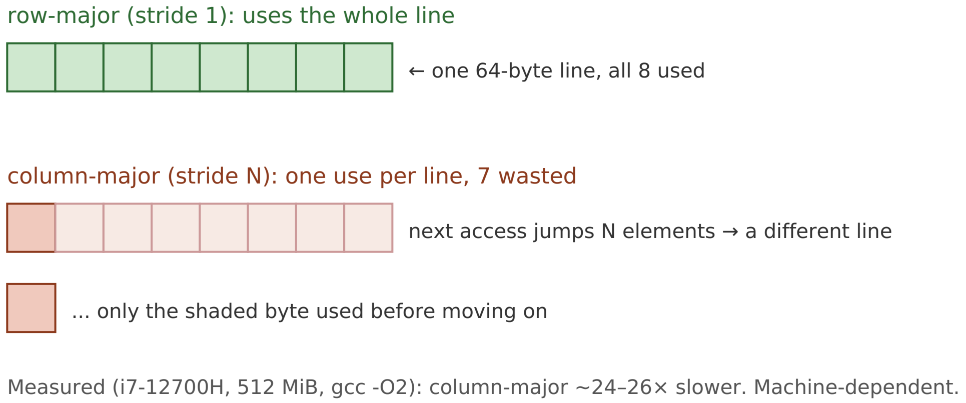

Figure 3.1: The two traversals do identical arithmetic. Row-major consumes every element of each fetched line; column-major uses one element per line and moves on, wasting most of the memory bandwidth it pays for.

(Answer to the §3.4 quick check: (a) compulsory/cold — first-ever reference; (b) capacity — the working set exceeds the cache; (c) conflict — addresses collide on the same cache set despite free space elsewhere.)

THE SAME IDEA THREE WAYS: LOCALITY PAYS

1. In words. Memory arrives one cache line at a time, so code that uses all of each line it fetches (good spatial locality) moves far less memory and hits far more often than code that touches one item per line and jumps away.

2. As a picture. Figure 3.1: a fully-consumed line vs. a line touched once and abandoned.

3. As a trace / measurement. The row-major vs. column-major experiment: identical sum, identical operation count, ~24–26× wall-clock difference on the test machine because one order has spatial locality and the other does not. Prose says "locality matters"; the number says how much.

TRAP: IGNORING CACHE LOCALITY (THE "SAME BIG-O, SO SAME SPEED" FALLACY, MEMORY EDITION)

The recurring error of this Part, now at the cache level: judging two implementations equivalent because they have the same complexity and the same operation count, while ignoring that one has good locality and the other does not. Column-major matrix traversal, linked lists walked as pointer chains, arrays-of-pointers-to-objects scattered across the heap, hash tables with poor probe locality — all can be an order of magnitude slower than a cache-friendly layout doing the identical logical work. The tell is a hot loop over a data structure larger than cache whose access pattern is *not* sequential. The fixes are layout, not algorithm: store data contiguously in the order you traverse it; keep frequently-used fields together; process large data in cache-sized blocks (blocking/tiling); prefer arrays of values to arrays of pointers. And, as always in this Part: confirm by *measuring*, on your data and your machine, rather than trusting any single ratio — including this chapter's ~25×, which is one machine's number, not a constant of nature.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A reviewer asks why you rewrote a nested loop to swap the order of the two indices when "it computes exactly the same thing." Cover the paragraph below and answer in two or three sentences, using *cache line*, *spatial locality*, and *stride*.

Model answer. "It computes the same result, but the original visited the array with a large stride, touching one element per 64-byte cache line and then jumping to a different line — so it moved roughly a line's worth of memory per element and got almost no reuse. Swapping the loop order makes the inner traversal stride-1, so each fetched line is fully consumed before we move on: far fewer misses, the prefetcher can run ahead, and we use the memory bandwidth we're already paying for. Same big-O and same arithmetic; the change is purely spatial locality, and on our data it measured about an order of magnitude faster." Naming the mechanism — line, stride, locality — rather than "I made it faster" is the senior register.

3.7 The rules, and where Part 1 goes next

Three chapters in, the cost model the RAM abstraction hides is now concrete. Chapter 1 established that where data lives dominates cost and that this is a second cost model alongside big-O. Chapter 2 opened the CPU and showed why predictable, independent work runs fast. This chapter turned "data location matters" into rules you can design against: memory moves in 64-byte lines, so use the whole line (spatial locality) and reuse hot data (temporal locality); classify your misses (compulsory, capacity, conflict) to know which fix applies (use each line, shrink the working set, avoid alignment collisions); and remember that contiguous array access is fast while pointer-chasing is slow because it has neither locality nor independent work to hide the misses. These are not micro-optimizations to sprinkle on at the end — they are how you get an order of magnitude on the same algorithm, and you can now *see* them in code.

The rest of Part 1 builds directly on this. The next chapters take the model down to how data is actually laid out in memory — bytes, alignment, the stack and the heap — so you can control locality deliberately rather than by accident, and the performance-engineering chapters return with profiling tools that *show* you the misses and mispredictions these three chapters taught you to predict. The habit to carry forward is the one this Part has drilled from three angles: when two pieces of code have the same big-O but different speed, the difference is the machine — its memory hierarchy and its pipeline — and you now know where to look.

CAN YOU… WITHOUT LOOKING?

- State the **cache line** size on common x86-64 (64 bytes, per CS:APP) and explain why memory moving in lines, not bytes, is the key fact behind cache behavior.
- Define **temporal** and **spatial** locality and say which one a full-line fetch and which one a prefetcher exploits.
- Name the **three C's** — compulsory, capacity, conflict — and give the fix each one points to.
- Explain **associativity** conceptually (direct-mapped vs. set-associative vs. fully associative) and why conflict misses make performance depend on size/alignment, not just data volume.
- Explain why **array traversal is fast and pointer-chasing is slow** in terms of locality *and* the dependency chain from Chapter 2.
- Describe the **row-major vs. column-major** demonstration, its measured order-of-magnitude result, *and* why the exact number is machine-dependent.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is a cache line, what is its common size on x86-64, and why does its existence mean cost is "per line fetched," not "per byte"? (§3.1)
2. **R2.** Define temporal and spatial locality and give a code example of each. (§3.2)
3. **R3.** Name the three C's and, for each, the design change that reduces it. (§3.3)
4. **R4.** Why can a loop's speed change sharply with the array's size or alignment even when the amount of data is the same? Which of the three C's is this? (§3.4)
5. **R5.** Give two independent reasons a linked-list traversal is slower than an array traversal of the same length, one about locality and one about dependencies. (§3.5)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP)**, Chapter 6 ("The Memory Hierarchy"). The source for this chapter's cache-line size, locality framing, three-C's taxonomy, and set-associativity arithmetic, plus the "memory mountain" experiment that visualizes read throughput across sizes and strides. The free CMU 15-213 materials cover it.
- **Drepper, "What Every Programmer Should Know About Memory"** (2007, free). The deepest freely available treatment of cache organization, associativity, prefetching, and access patterns. Its absolute numbers are dated — read for mechanism — but its analysis of why layout matters is unmatched.
- **MIT 6.172, *Performance Engineering of Software Systems*** (free OCW). Lectures on cache-efficient algorithms, blocking/tiling, and measuring cache behavior with real tools — the applied continuation of this chapter.

Exercises

3.1 WARM-UP On a machine with 64-byte cache lines, how many `double`s (8 bytes each) fit in one line? If you read one `double` from a cold line, how many neighbors become cheap hits, and which kind of locality does that reward?

3.2 WARM-UP Give a one-sentence code example of temporal locality and one of spatial locality, and say which cache mechanism (line fill, retention of recent lines, prefetching) exploits each.

3.3 WARM-UP Classify each miss as compulsory, capacity, or conflict: (a) the first read of a just-`malloc`ed buffer; (b) repeatedly scanning a 4 GB dataset with an 8 MB last-level cache; (c) a loop that slows down only when an array's row length is a particular power of two.

3.4 CORE You must sum an $N \times N$ matrix stored in row-major order. One colleague writes the inner loop over columns, another over rows. (a) Which is faster and why, in terms of cache lines and stride? (b) Both are $O(N^2)$ and do the same additions — reconcile that with a large measured speed difference. (c) Name the kind of miss the slow version suffers most.

3.5 CORE A profile shows a data structure of 5 million small objects is slow to traverse. It is currently an array of *pointers*, each pointing to a separately `malloc`ed object. Propose a layout change that would improve locality, and explain both why it reduces misses (spatial locality) and why it helps the CPU hide the remaining misses (recall the dependency argument from Chapter 2).

3.6 COST A traversal touches one 8-byte value per 64-byte cache line (stride 8 values). (a) What fraction of each fetched line does it use? (b) Compared with a stride-1 traversal of the same values, roughly how many times more memory must it move? (c) Explain how this back-of-the-envelope ratio relates to (though it will not exactly equal) the measured row-major/column-major slowdown, and name two hardware factors that make the measured number differ from this simple estimate.

3.7 FADED *Worked, with the last step for you.* Explain why the column-major traversal in §3.6 stalls the CPU more than the row-major one, tying it to Chapter 2. Steps 1–3 given; complete step 4.
Step 1 (given): Column-major access has stride N, so almost every access lands on a new cache line — a likely miss to DRAM (~100 ns, Chapter 1).
Step 2 (given): A cache miss need not stall the CPU if there is independent work nearby to overlap with it (Chapter 2 §2.3).
Step 3 (given): A summation into one accumulator, walking predictably, does have some independent memory-level parallelism — but the row-major version keeps far more useful data resident per fetched line, so it issues far fewer misses to begin with.
Step 4 (you): Complete the explanation: state which effect dominates the ~25× gap here — the sheer *number* of misses (locality) or the inability to hide each one (dependencies) — and justify your choice. Explain *why the number of misses is the primary lever in this particular experiment,* and what you would change about the code to test your claim.

3.8 MIXED For each speedup, decide whether the primary mechanism is *spatial locality / cache lines*, *a capacity miss fixed by blocking*, *branch prediction* (Chapter 2), or an *ILP / dependency chain* (Chapter 2), and justify in a line: (a) tiling a large matrix multiply into cache-sized blocks; (b) sorting the input before a data-dependent filter loop; (c) switching an array-of-structs traversal to consume each cache line fully; (d) summing into four accumulators instead of one.

3.9 MIXED Two implementations of the same O(n) aggregation differ by 15× in wall-clock time. Without running anything yet, list the candidate causes drawn from all of Chapters 1–3 (name at least three distinct mechanisms), then describe the *single* measurement you would take first to narrow it down and why that one is most informative.

3.10 STRETCH The chapter reports a specific ~24–26× row-major/column-major ratio and repeatedly insists it is machine-dependent. (a) List three properties of a machine that would change this ratio, and say for each whether it would tend to raise or lower it. (b) Explain why the *existence* and rough *magnitude* (a large multiplier) of the effect is nonetheless robust across machines, even though the exact number is not. (c) Why is reporting "about 25× on this specific CPU, and reproduce it yourself" more honest and more useful than printing a single universal multiplier?

3.11 LAB Reproduce the demonstration on your own machine. First run `getconf LEVEL1_DCACHE_LINESIZE` and `lscpu` (or the macOS `sysctl hw` equivalents) and record your cache-line size and L1/L2/L3 sizes. Then write the two-loop matrix-sum program (size it so the matrix comfortably exceeds your last-level cache), verify both loops produce the identical sum, time each, and report *your* ratio. Note how it compares to the book's ~25× and to your §3.6 stride estimate. Report only numbers you actually measured; if the compiler optimizes a loop away, change the code (e.g. consume the sum) and say what you did.

Chapter 4 — Virtual Memory & Address Translation

Before you start: Chapter 3 (caches; memory moves in lines; a miss to DRAM costs ~100 ns), Chapter 1 (the latency hierarchy). · **Pairs with:** Chapter 5 (data layout) and the operating-system Part, which builds the page tables this chapter only describes. This chapter adds one more cache to the picture — a cache for *address translations* — and shows why a huge, scattered working set can be slow for a reason the last chapter did not cover.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 3 said the CPU keeps recently used cache lines so a repeated access is a cheap hit — what is the general trick there, in one word? (2) Chapter 1 put a DRAM access at ~100 ns and an L1 hit at ~1 ns — about how many orders of magnitude apart? (3) *Predict:* the pointer `p` in your C program holds an address like `0x7ffd3a08`. When the CPU reads `*p`, do you think that number is the actual electrical address of a cell in your DRAM chips — the physical location — or something the hardware has to *translate* first? By the end of the chapter you will know which, and what the translation costs.

Every address your program has ever used is a lie — a convenient, deliberate, hardware-enforced lie. When your code holds a pointer and dereferences it, the number in that pointer is a **virtual address**: a name in a private, per-process address space that the operating system and CPU maintain, not the **physical address** of a byte in the DRAM chips. Before the load can touch memory, the hardware must *translate* virtual to physical. That translation is invisible, automatic, and — this is the point for a performance book — *not free*. This chapter explains the mechanism (why virtual memory exists, how pages and page tables implement it) and then, in the register of Part 1, isolates its cost: the translation itself is cached, in a structure called the TLB, and when that cache misses, the machine pays a price that scales with how scattered your data is. The through-line from Chapter 3 is exact — **the TLB is a cache for the page table**, and everything you learned about hits, misses, and working sets applies to it too.

4.1 Virtual vs. physical addresses: why the lie exists

On a modern system, each process runs as if it owns a large, contiguous block of memory starting near address zero, all to itself. It does not. Physical DRAM is shared among every process and the kernel, is smaller than the address space the program is allowed to name, and is handed out in scattered fragments. The **virtual address space** is the illusion that papers over this reality: the hardware's memory-management unit (MMU), configured by the OS, maps each process's virtual addresses onto whatever physical frames happen to be free, and keeps different processes' identical-looking virtual addresses pointing at different physical memory. Bryant and O'Hallaron develop this fully in *Computer Systems: A Programmer's Perspective* (CS:APP, Chapter 9, "Virtual Memory"); Arpaci-Dusseau's *Operating Systems: Three Easy Pieces* (OSTEP) devotes its whole virtualization part to it.

The illusion buys three things at once, which is why every general-purpose OS pays for it. First, **isolation**: process A literally cannot name a byte belonging to process B, because A's page tables contain no mapping to B's frames — a security and stability boundary enforced in hardware. Second, **a simple programming model**: your linker can lay out code and data at fixed virtual addresses without knowing where in physical RAM the program will actually land, and every process gets the same clean layout. Third, **flexibility the OS exploits**: a virtual page can be mapped to a physical frame, or to a page of a file on disk, or to nothing yet (allocated lazily on first touch), or shared read-only between processes (as with a common library) — all invisibly to your code. For this chapter, hold onto the first fact and the cost it implies: because the mapping is per-process and arbitrary, *the hardware must consult a translation on essentially every memory access*, and that lookup is what we now follow.

4.2 Pages and page tables

Translating every individual byte address would need a map with one entry per byte — absurd. Instead memory is divided into fixed-size **pages**: the address space is chopped into aligned blocks, and translation happens one page at a time. The near-universal base page size on x86-64 (and many other architectures) is **4 KiB**; CS:APP uses this standard size, and it is what `getconf PAGE_SIZE` reports on such systems. A virtual address then splits cleanly into two parts: the high bits are the **virtual page number** (which page), and the low bits — 12 of them for a 4 KiB page, since $2^{12} = 4096$ — are the **offset** within the page. Only the page number needs translating; the offset is the same in virtual and physical addresses, because a page maps to a physical **frame** of the same size and a byte's position within the page does not move.

The **page table** is the data structure, held in memory and owned by the OS per process, that maps virtual page numbers to physical frame numbers (plus permission and status bits: present or not, readable/writable/executable, accessed, dirty). Conceptually it is one big array indexed by virtual page number. But a flat array is impractical: a 48-bit virtual address space with 4 KiB pages has 2^{36} pages, and a flat table with an entry per page would be enormous *per process*, the overwhelming majority of it empty because most programs use a tiny, sparse fraction of their address space. The standard fix is a **multi-level (hierarchical) page table**: a tree of tables where the virtual page number is split into several fields, each indexing one level. x86-64 uses a four-level page table (a documented architectural fact, described in CS:APP). Only the branches that correspond to actually-used regions of memory need to exist, so a sparse address space costs a sparse tree instead of a giant flat array.

The cost that matters shows up right here. To translate one virtual page number through a four-level table, the hardware must read *each level in turn*: read the top-level table to find the address of the next-level table, read that to find the next, and so on — up to **four dependent memory accesses** to complete one translation on x86-64. This descent is called a **page-table walk**. And notice it is a dependency chain of the exact kind Chapter 3 warned about: each read's address comes from the previous read, so the walk cannot be parallelized, and if the table levels are not in cache, each step is its own trip toward

DRAM. Doing four dependent memory accesses *for every* load and store would erase all the performance we have been chasing — so, as always, the machine caches the result.

> **QUICK CHECK**
>
> A 4 KiB page needs how many offset bits, and why exactly that many? If a virtual address is split into a page number and an offset, which part does the page table translate, and which part is copied unchanged into the physical address? (Answer after the next section.)

4.3 The TLB: a cache for translations

The **translation lookaside buffer (TLB)** is a small, fast, on-CPU cache that stores recent virtual-page-to-physical-frame translations. It is a cache in exactly the sense of Chapter 3 — same idea, different contents. Chapter 3's data caches store recently used *cache lines* so a repeated data access is a cheap hit; the TLB stores recently used *translations* so a repeated access to the same page skips the page-table walk. On each memory access the MMU first checks the TLB with the virtual page number:

- A **TLB hit** — the translation is present — returns the physical frame in roughly a cycle, effectively free, and the access proceeds. This is the common case, because programs have locality: consecutive accesses usually fall on the same few pages (a 4 KiB page holds a lot of data), so one page's translation, cached once, serves thousands of accesses.
- A **TLB miss** — the translation is not cached — triggers the **page-table walk** of §4.2: the several dependent memory accesses to descend the table, after which the freshly found translation is installed in the TLB (evicting an older one) so nearby future accesses hit. On modern x86-64 this walk is done by hardware; on some architectures the OS handles it in software, which is costlier still.

So the cost of a TLB miss is the cost of the walk: on the order of a handful of dependent memory accesses. If those accesses hit in the data caches (the upper page-table levels are hot and small, and are cached), a miss is cheap — a few cycles. If they miss to DRAM, each is a ~100 ns trip (Chapter 1), and a walk can cost on the order of tens to hundreds of nanoseconds. The exact penalty and the exact TLB capacity are hardware parameters that vary by CPU, and this book will not quote a specific chip's numbers as if they were universal — CS:APP (Chapter 9) gives a worked example TLB and walk, and Drepper's memory paper covers TLB behavior in depth. What you must carry is the *shape*: a TLB hit is ~free, a TLB miss costs a page-table walk whose price is one to a few memory accesses and, in the bad case, is a DRAM-latency event stacked *on top of* whatever the data access itself will cost.

(Answer to the quick check: 12 offset bits, because 2^{12} = 4096 = 4 KiB, so 12 bits address every byte within a page. The page table translates the page-number bits; the offset is copied unchanged into the physical address, since a page and its frame are the same size and a byte keeps its position within the page.)

THE SAME IDEA THREE WAYS: THE TLB IS A CACHE FOR THE PAGE TABLE

1. In words. Translating an address needs a multi-step page-table walk. Rather than walk on every access, the CPU caches recent translations in the TLB; a hit skips the walk, a miss pays for it and caches the result — identical logic to a data cache, applied to translations instead of data.

2. As a picture. Figure 4.1: two parallel lookups. The data cache maps *address → line*; the TLB maps *virtual page → physical frame*. Same hit/miss structure, different key and value.

3. As a trace. Access page P: TLB miss → walk the 4-level table (up to 4 memory reads) → install P→frame in the TLB. Next 1000 accesses to page P: TLB hits, ~free. The walk's cost is amortized over every access that stays on the page — which is why sequential code barely notices the TLB and scattered code does.

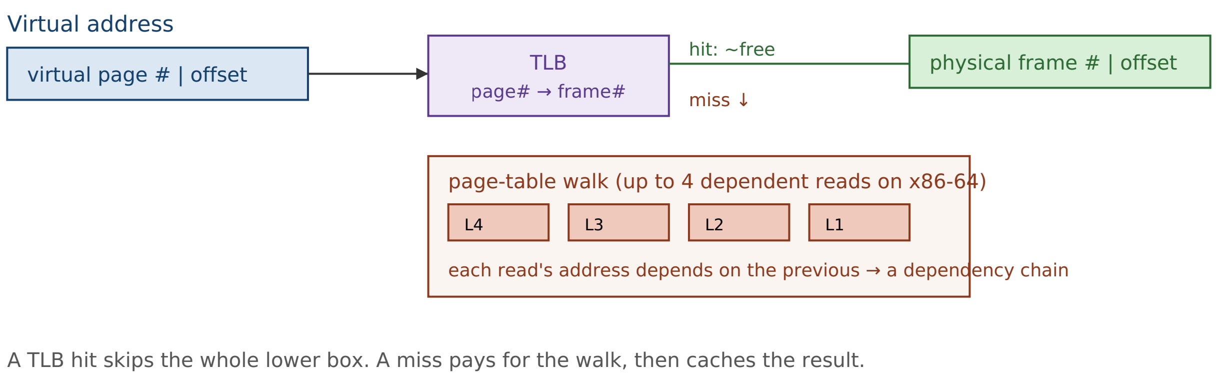

Figure 4.1: Address translation. The TLB caches virtual-page→physical-frame mappings; a hit is near-free, a miss triggers a multi-level page-table walk (a dependency chain of memory reads) whose result is then cached.

4.4 Why huge, scattered working sets thrash the TLB

Here is the performance consequence, and it is a new failure mode not covered by Chapter 3's data caches. The TLB holds only a limited number of translations — a hardware-fixed count, small relative to the number of pages a large program can touch. The total span of memory the TLB can map at once is its **reach**: roughly (number of entries) × (page size). With 4 KiB pages, even a few thousand TLB entries reach only a few megabytes of address space at a time. So a program whose hot data spans *hundreds* of megabytes, accessed in a scattered pattern that jumps from page to page, will keep missing the TLB: each jump lands on a page whose translation was long since evicted, forcing a fresh page-table walk. This is **TLB thrashing**, and it is precisely a *capacity miss* (Chapter 3's taxonomy) at the translation level — the working set of *pages* exceeds what the TLB can hold, just as a data working set can exceed the L2.

Notice the two costs now stack. A scattered access over a huge array can miss the *data* cache (pay ~100 ns to fetch the line) *and* miss the *TLB* (pay a page-table walk to even learn where the line is), and in the worst case the walk itself misses cache and goes to DRAM. This is a large part of why truly random access over a multi-hundred-megabyte structure is so much slower than the raw ~100 ns DRAM figure alone would predict — a fact the Chapter 6 capstone lab measures directly, where a random pointer-chase over 512 MiB costs well above the bare DRAM latency, with the surplus owing partly to exactly this page-walk cost. By contrast, *sequential* access barely touches the TLB: a 4 KiB page holds ~512 `long`s, so one translation serves hundreds of consecutive accesses before you cross into the next page — the same "use the whole thing you paid to bring in" principle as the cache line, one level up.

> **QUICK CHECK**
>
> Two loops touch the same 512 MiB array the same number of times: one walks it sequentially, one jumps to random pages. Which one thrashes the TLB, and why does the other one hardly use it at all? (Answer below the huge-pages section.)

4.5 Huge pages, conceptually

If TLB reach is (entries) × (page size) and you cannot easily add TLB entries (that is fixed hardware), the other lever is the page size. **Huge pages** do exactly that: instead of mapping memory in 4 KiB units, the OS can map it in much larger units — on x86-64, commonly **2 MiB**, and **1 GiB** where supported (both are real, documented x86-64 page sizes; the `pdpe1gb` CPU flag advertises 1 GiB support). One huge-page translation covers 2 MiB instead of 4 KiB, so the *same* number of TLB entries now reaches roughly 512× more memory. A working set that thrashed the TLB with 4 KiB pages may fit entirely within the TLB's reach once mapped with 2 MiB pages, turning a storm of page-table walks into TLB hits. A second, smaller benefit: a huge page's walk descends fewer levels (the translation stops at a higher level of the tree), so even the misses are a bit cheaper.

Huge pages are a real mitigation for real workloads — large in-memory databases, big hash tables, scientific arrays — where TLB misses are a measured bottleneck. They are not free lunch: they can waste memory to internal fragmentation (a 2 MiB page allocated for a few live kilobytes), can be harder for the OS to find contiguously once memory is fragmented, and help *only* when TLB misses were actually a problem. That last clause is the discipline of this whole Part: huge pages are something you reach for *after* a profiler shows page-walk cost is significant, not a switch you flip on faith. This book keeps the treatment conceptual — the mechanics of enabling them (explicit `mmap` flags vs. transparent huge pages) belong to the operating-system Part — but the model is what matters here: *bigger pages, fewer translations, more TLB reach.*

(Answer to the §4.4 quick check: the random-page jumper thrashes the TLB — each jump lands on a different page whose translation has been evicted, forcing a walk. The sequential walker hardly uses the TLB because each 4 KiB page's single translation serves the hundreds of consecutive accesses that fall on that page before it crosses to the next one.)

TRAP: IGNORING TLB COST (THE "IT'S ALL JUST DRAM LATENCY" FALLACY)

Having learned Chapter 3, it is tempting to model every scattered access as a single ~100 ns DRAM miss and stop there. That undercounts. A random access over a large working set can pay *two* independent penalties: the data-cache miss to fetch the line, *and* a TLB miss whose page-table walk is itself one to several memory accesses. When your measured cost per random access sits well above the raw DRAM latency you expected, suspect the TLB — especially if the working set spans far more memory than TLB reach (a few thousand pages × 4 KiB). The tell is a hot, random-access structure of hundreds of megabytes or more. The fixes are the ones this chapter and the last give: prefer sequential or blocked access so one translation serves many accesses (the sequential-vs-random gap is partly a TLB gap); shrink the working set; and, where a profiler confirms page-walk cost, consider huge pages. As always: confirm by measuring page-walk and TLB-miss counters, not by guessing.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "Our random lookup into the big in-memory index costs about 160 ns per access, but you told me DRAM is only ~100 ns — where does the extra time come from?" Cover the paragraph below and answer in two or three sentences, using *TLB*, *page-table walk*, and *reach*.

Model answer. "The ~100 ns is just the data fetch. Each random lookup also needs an address translation, and because the index spans far more memory than the TLB can map at once (its reach is only a few megabytes with 4 KiB pages), most lookups miss the TLB and trigger a page-table walk — several dependent memory accesses — *before* the data access even starts. That walk cost stacks on top of the DRAM latency, which is where the surplus comes from. If a profiler confirms it, mapping the index with huge pages would raise TLB reach enough to turn most of those walks back into hits." Naming the TLB and page walk — rather than "memory is just slow" — is the senior register, and it points at a concrete, measurable fix.

4.6 What to carry forward

This chapter added the last piece of the machine's cost model that Part 1 needs. The addresses your program uses are virtual, translated to physical one page at a time through a multi-level page table; the translation is cached in the TLB, so it is near-free when your accesses have page-level locality and expensive — a page-table walk of several dependent memory accesses — when they do not. A huge, scattered working set thrashes the TLB the same way a large data working set overflows the L2, and huge pages widen TLB reach as a conceptual mitigation. The unifying idea, straight from Chapter 3, is that *the TLB is one more cache*, subject to the same hit/miss/working-set logic as every other level — which is why the single habit of accessing memory sequentially or in blocks pays off twice: once in the data cache, once in the TLB.

Part 1 now has the whole machine in view: the CPU that runs your instructions (Chapter 2), the cache hierarchy that feeds it data (Chapter 3), and the translation layer that turns your addresses into real memory (this chapter). Chapter 5 spends that knowledge: it is where you deliberately *design* your data's layout — array-of- structs versus struct-of-arrays, hot/cold splitting, alignment — so that both the cache and the TLB work *for* you. Then Chapter 6 puts numbers on all of it in a capstone lab. The posture to keep: when memory is slow, ask not only "did the data miss the cache?" but also "did the *address* miss the TLB?"

CAN YOU… WITHOUT LOOKING?

- Explain the difference between a **virtual** and a **physical** address, and name the three things virtual memory buys (isolation, a simple layout, OS flexibility).
- Split a virtual address into **page number** and **offset**, say why a 4 KiB page has a 12-bit offset, and explain why page tables are **multi-level**.
- State what a **page-table walk** is, why it is a dependency chain, and roughly how many memory accesses it can take on x86-64 (up to four).
- Explain why the **TLB is a cache for the page table**, what a hit and a miss each cost (in shape, not a specific chip's numbers), and how it maps onto Chapter 3's hit/miss model.
- Define **TLB reach** and explain why a huge, scattered working set **thrashes** the TLB while sequential access barely uses it.
- Explain how **huge pages** raise TLB reach, and name a cost of using them.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is the difference between a virtual and a physical address, and why does a general-purpose OS use virtual memory at all? (§4.1)
2. **R2.** Why are page tables multi-level rather than a single flat array, and what is a page-table walk? (§4.2)
3. **R3.** In what precise sense is the TLB "a cache for the page table"? What does a hit cost and what does a miss cost, in shape? (§4.3)
4. **R4.** Define TLB reach and explain TLB thrashing using Chapter 3's miss taxonomy. Which of the three C's is it? (§4.4)
5. **R5.** How do huge pages reduce TLB misses, and when are they *not* worth it? (§4.5)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP)**, Chapter 9 ("Virtual Memory"). The source for this chapter's address-translation mechanism, multi-level page tables, TLB, and a worked example of a walk. The free CMU 15-213 materials cover it.
- **Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP)**, the virtualization part (free full text). Paging, TLBs, and page-table structures from the OS side — the complement to CS:APP's hardware view, and where the operating-system Part of this book will pick up.
- **Drepper, "What Every Programmer Should Know About Memory"** (2007, free). Includes TLB behavior, the cost of misses, and huge pages. Absolute numbers are dated — read it for mechanism.

Exercises

4.1 WARM-UP A system uses 4 KiB pages. (a) How many bits of a virtual address are the page offset? (b) If you access `a[0]` then `a[1]` (adjacent `long`s), do they need one translation or two? (c) Roughly how many consecutive `long`s can you touch before you cross a page boundary and (possibly) need a new translation?

4.2 WARM-UP In one sentence each, name the three benefits virtual memory provides, and state which one is the security/isolation boundary between processes.

4.3 WARM-UP Complete the analogy and justify it in a sentence: a data cache stores recently used ____ so a repeated data access is a cheap hit; the TLB stores recently used ____ so a repeated access to the same ____ skips the ____.

4.4 CORE Explain why a page-table walk is a *dependency chain* and why that matters for its cost. Tie it explicitly to the pointer-chasing argument from Chapter 3: what do a page-table walk and a linked-list traversal have in common?

4.5 CORE A hot in-memory hash table of 400 MiB is probed at random keys. Measurements show ~150 ns per probe, noticeably more than the ~100 ns you expected for a single DRAM miss. (a) Give two independent memory-system costs a random probe pays. (b) Explain, using *TLB reach*, why this structure is prone to TLB misses. (c) Name one change that would reduce the translation cost specifically, and one that would reduce the data-miss cost.

4.6 COST A TLB has E entries and the system uses 4 KiB pages. (a) Write the TLB reach as a formula. (b) For a working set of W bytes accessed randomly by page, under what condition (in terms of W, E, and page size) do you expect heavy TLB thrashing? (c) If you switch to 2 MiB pages without changing E, by what factor does reach change, and how does that move the threshold in (b)?

4.7 FADED *Worked, with the last step for you.* Explain why sequential access over a 512 MiB array barely stresses the TLB while random-by-page access over the same array thrashes it. Steps 1–3 given; complete step 4.
Step 1 (given): Translation happens per page (4 KiB); a page of `long`s holds ~512 elements.
Step 2 (given): The TLB reaches only (entries × 4 KiB) of address space at once — a few megabytes, far less than 512 MiB.
Step 3 (given): Sequential access stays on one page for ~512 consecutive accesses before crossing to the next, so one translation is amortized over many accesses.
Step 4 (you): Complete the argument for the random case: state how often the random walker needs a *new, uncached* translation, why the TLB's small reach means those translations are usually missing, and therefore why nearly every random access pays a page-table walk. Then state the one number you would measure to confirm the TLB — not just the data cache — is the culprit.

4.8 MIXED For each symptom, decide whether the primary cause is most likely a *data-cache capacity miss* (Ch 3), a *TLB miss / page-walk* (Ch 4), a *branch misprediction* (Ch 2), or a *dependency chain / low ILP* (Ch 2), and justify in a line: (a) a random-access loop over 800 MiB is slower than ~100 ns/access predicts, and enabling 2 MiB pages speeds it up; (b) a loop over 64 MiB re-read many times is much slower than one over 1 MiB re-read the same total number of times; (c) a sum over a linked list is far slower than over an array of the same length; (d) a tight loop with a data-dependent `if` speeds up sharply after the input is sorted.

4.9 MIXED A colleague proposes enabling huge pages everywhere "to make memory faster." Without running anything, give (a) the specific condition under which huge pages actually help, (b) two costs or downsides of huge pages, and (c) the one measurement you would take first to decide whether they are worth it for a given workload. Note which chapter's discipline this reflects.

4.10 STRETCH The chapter says the TLB "is a capacity miss at the translation level." (a) Map each of Chapter 3's three C's onto a translation-level analogue if one exists (compulsory, capacity, conflict), or say why it does not. (b) Explain why huge pages address the *capacity* analogue specifically. (c) Argue why increasing the base page size is not a free win at the system level (think of programs with small, sparse memory footprints).

4.11 LAB On your own machine, record the base page size (`getconf PAGE_SIZE`) and confirm your architecture's page-table depth and huge-page sizes from its documentation (do *not* guess — cite what you find). Then write a program that sums the same large array (comfortably larger than your last- level cache, e.g. 512 MiB) two ways: strictly sequentially, and in a random-by-page order (jump to a random page, read a few values, jump again). Time both and report *your* ns-per-access for each. If your platform exposes TLB-miss or page-walk counters (e.g. `perf stat -e dTLB-load-misses` on Linux), record them for both runs and check that the random run has far more. Report only numbers you actually measured, and note anything that surprised you.

Chapter 5 — Mechanical Sympathy & Data-Oriented Design

Before you start: Chapter 3 (cache lines, locality, the three C's), Chapter 4 (the TLB), Chapter 1 (mechanical sympathy as this volume's thesis). · **Pairs with:** Chapter 6 (the capstone lab) and the performance-engineering Part. This is where the last four chapters stop being knowledge *about* the machine and become a way of *designing your data*.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 3 — memory moves in 64-byte lines, so the cost of an access is really "per line fetched." If a loop uses only 8 of the 64 bytes in each line it brings in, roughly what fraction of the memory bandwidth it pays for is wasted? (2) Chapter 1 named this volume's thesis after a driving metaphor — what is the two-word term, and who is it attributed to? (3) *Predict:* you have 16 million "particles," each a struct with 8 fields, and you need to sum just *one* field across all of them. Does it matter for speed whether you store them as one array of whole structs, or as eight separate arrays — one per field? By the end you will have a measured answer.

Chapters 1 through 4 were reconnaissance: the CPU, the cache hierarchy, and the translation layer, each with its own cost model that the RAM abstraction hides. This chapter spends that reconnaissance. The term **mechanical sympathy** — popularized in software by Martin Thompson, and introduced back in Chapter 1 — names the posture: understand enough about how the machine works to get the most out of it. **Data-oriented design** is that posture applied to one specific and enormously consequential decision — *how you lay your data out in memory* — and its central claim, which the rest of this chapter defends, is blunt: *the fastest code respects the memory system.* You do not earn an order of magnitude here by a cleverer algorithm; you earn it by arranging the same data so the cache lines you fetch are full of bytes you actually use and the pages you touch are few. This chapter turns Chapter 3's rules into design choices — array-of-structs versus struct-of-arrays, splitting hot fields from cold, padding and alignment — and, as promised, measures one of them on real hardware.

5.1 The design question: lay data out for the access pattern

Data-oriented design inverts the habit most of us learned first. Object-oriented instinct groups data by *conceptual entity*: a `Particle` "has" a position, a velocity, a mass, so we put them in one struct and make an array of those. That models the domain cleanly — and it is often exactly wrong for performance, because *the hardware does not care about your conceptual entities; it cares about which bytes travel together in a cache line.* The design question data-oriented design asks first is not "what is the natural object?" but "**what is the hot loop's access pattern, and which bytes does it actually touch together?**" You then lay memory out to match that pattern, so every fetched line and every translated page is spent on bytes the loop needs. Everything in this chapter is a special case of that one move.

5.2 Array-of-structs vs. struct-of-arrays

The canonical layout choice. Suppose each record has several fields and you have many records. There are two natural layouts:

- **Array-of-structs (AoS):** one array, each element a full struct — all of record 0's fields, then all of record 1's, and so on. Fields of the *same record* are adjacent in memory.
- **Struct-of-arrays (SoA):** one array *per field* — every record's `x` in one contiguous array, every `y` in another, and so on. The same field across *different records* is adjacent in memory.

Which wins depends entirely on the access pattern, and neither is universally better — this is a genuine trade-off, not a rule. The deciding question is *which bytes your hot loop touches together*:

- If the hot loop **scans one field (or a few) across many records** — sum every `x`, filter by one attribute, do SIMD over one channel — **SoA wins**. In SoA those values are contiguous, so each 64-byte line is full of the values you want and the traversal is stride-1 with perfect spatial locality (§3.1–3.2). In AoS the same scan strides over the whole struct, touching a few bytes per line and wasting the rest — the column-major mistake of §3.6, in a new costume.
- If the hot loop **uses most fields of one record at a time** — process particle *i* completely, then particle *i*+1 — **AoS wins**. All of record *i*'s fields share a line or two, so one fetch brings the whole record; SoA would instead touch one element in each of many separate arrays, scattering the access across many lines and pages.

A measured demonstration: sum one field of 16 million records

Let us measure the SoA-favoring case. Define a record of 8 `double`s — position, velocity, mass, charge — which is exactly **64 bytes**, one cache line, in AoS. Make 16 million of them and sum a single field, `x`, two ways: over the array-of-structs (each step strides 64 bytes to the next record's `x`, using 8 of every 64 bytes fetched) and over a contiguous per-field array (stride-1, every byte of every line used).

```
struct Particle { double x, y, z, vx, vy, vz, mass, charge; };  /* 64 bytes = 1 line */

/* AoS: sum one field -> stride of 64 bytes, 8 of 64 bytes used per line */
for (size_t i = 0; i < N; i++) sum += aos[i].x;

/* SoA: the x field is its own contiguous array -> stride 1, whole line used */
for (size_t i = 0; i < N; i++) sum += soa_x[i];
```

Built with `gcc -O2` and run for this book on a 12th-generation Intel Core i7 (i7-12700H), over 16 million records (best of 5 timed runs), the AoS scan of one field took roughly **0.12 s** and the SoA scan roughly **0.032 s** — the SoA layout was about **3.7–4× faster** for computing the identical sum, differing only in how the data was laid out. **This exact ratio is machine-dependent** and code-dependent, and you should expect a different number on your hardware. Note it is a clear multiple, but *less* than the naive 8× you might predict from "AoS moves 8× the memory": the SoA loop is not purely bandwidth-bound — summing into a single accumulator has a dependency chain (§2.3) that caps its speed, so it cannot fully cash in the bandwidth it saves, which compresses the ratio. Remove that ceiling (multiple accumulators, vectorization) and the gap widens toward the bandwidth-bound 8×. The robust, portable result is the *direction and rough magnitude*: laying the scanned field out contiguously is a large, several-fold win, for the same arithmetic — because SoA turns a one-value-per-line scan into a whole-line scan.

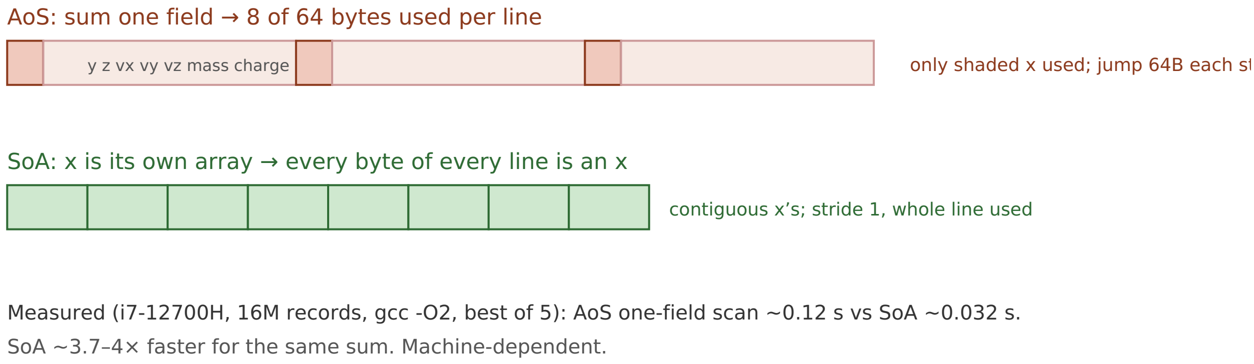

Figure 5.1: Summing one field. AoS strides over whole records, using a fraction of each fetched line; SoA stores that field contiguously, so every fetched line is full of values the loop uses.

> **QUICK CHECK**
>
> You process each record fully — read all 8 fields of particle *i*, do the physics, then move to particle *i*+1 — for every record. Which layout, AoS or SoA, is better *here*, and why does the answer flip from the sum-one-field case? (Answer after the next section.)

5.3 Hot/cold field splitting

AoS-vs-SoA is the all-or-nothing version; hot/cold splitting is the surgical one. Often a record has a few fields touched by the hot loop (the **hot** fields) and several touched rarely — logging metadata, debug info, a rarely-read description (the **cold** fields). If they all live in one struct, every cache line the hot loop fetches is padded out with cold bytes it will not use, lowering useful-bytes-per-line exactly as the AoS one-field scan did. **Hot/cold splitting** divides the record: put the hot fields in one compact struct (so more hot records fit per line and per page) and move the cold fields to a separate structure, referenced by index or pointer, touched only on the rare path. The hot loop now streams through dense hot records; the cold data is out of its way. This is the same principle as SoA — group by access frequency instead of by conceptual entity — applied at the granularity of "hot vs. cold" rather than "field by field." The tell that you want it: a struct where a hot loop reads two of twelve fields, and the other ten are dead weight in every line.

(Answer to the §5.2 quick check: ***AoS wins*** *when you use most fields of one record at a time. All of particle i's fields are adjacent, so one or two line fetches bring the whole record and every byte is used; SoA would scatter that record's fields across eight separate arrays — eight different lines and possibly pages per record. The answer flips*

because the access pattern flipped: SoA is contiguous across records for one field; AoS is contiguous across fields for one record. Match the layout to whichever direction your hot loop travels.)

5.4 Padding and alignment

Two low-level facts about how the compiler places your struct in memory, both from the C model (CS:APP, Chapter 3 covers data alignment). First, **alignment**: each type has an alignment requirement — an 8-byte `double` is typically placed at an 8-byte-aligned address — because aligned accesses are efficient and, on some architectures, misaligned ones are slow or forbidden. To honor this inside a struct, the compiler inserts **padding**: unused bytes between fields so each field lands at a legal offset, plus tail padding so an array of the struct keeps every element aligned. The consequence for layout: field *order* changes struct size. A struct declared `{ char a; double b; char c; }` can be substantially larger than its 10 bytes of real data because padding is inserted around the fields; reordering to put the large, strictly-aligned fields first and pack the small ones together can shrink it. A smaller struct means more records per cache line and per page — a direct locality win, for free, from field ordering alone.

Second, alignment interacts with the cache line. A value that straddles a 64-byte line boundary can require touching *two* lines to read one value; keeping hot structures aligned to line boundaries (and sized so they tile the line cleanly, as our 64-byte `Particle` did) avoids that. There is a tension worth naming: padding a struct *up* to a nice size can help alignment and, as we will see next, avoid a concurrency hazard — but padding also makes the struct *bigger*, which hurts the locality of a scan. Like everything in this Part, it is a trade-off resolved by knowing the access pattern and, when it matters, measuring.

THE SAME IDEA THREE WAYS: GROUP DATA BY HOW IT'S ACCESSED

1. In words. The hardware moves and translates memory in fixed blocks (lines, pages). Lay data out so the bytes a hot loop uses together are stored together — by field (SoA), by temperature (hot/cold), and by alignment — so every block you pay for is full of bytes you need.

2. As a picture. Figure 5.1: an AoS line mostly wasted on unused fields vs. an SoA line entirely full of the scanned field.

3. As a measurement. Summing one field of 16 M records: SoA ~0.032 s vs. AoS ~0.12 s on the test machine — ~3.7–4× for the identical arithmetic, purely from layout. Prose says "group by access pattern"; the number says the grouping is worth several-fold.

5.5 A preview: false sharing

One layout hazard belongs to the concurrency Part but is worth planting now, because it is the mirror image of everything above. So far, packing related data into the same cache line has been *good* — it raises useful-bytes-per-line. But when *multiple threads* write to *different* variables that happen to share one cache line, the line ping-pongs between the cores' caches: each write forces the line out of the other core's cache to maintain coherence, even though the threads touch logically independent data. This is **false**

sharing — no real data is shared, but the cache line is, and the hardware's coherence protocol pays for it. The fix runs *opposite* to the locality advice of this chapter: pad or align the per-thread variables so each lands on its *own* cache line, deliberately *spreading* them out. That single-threaded packing helps and multi-threaded sharing hurts is not a contradiction — it is the same fact (a whole line moves as a unit) cutting two ways depending on whether one core or several are writing it. The full treatment, with a measurement, waits for the concurrency Part; for now, file it: *when you add threads, revisit which variables share a line.*

TRAP: AOS WHEN YOU SCAN ONE FIELD (PACKING THE WRONG AXIS)

The recurring layout error: choosing a struct layout to model the domain cleanly, then running a hot loop that touches only one or two fields of each record — so every cache line and every page you fetch is mostly bytes the loop ignores. It is the column-major mistake (§3.6) wearing an object-oriented costume, and it silently caps a scan at a fraction of its potential bandwidth. The tells: a hot loop over a large array of multi-field structs that reads only a couple of fields; a SIMD kernel that wants one channel contiguous but finds it strided; a profiler showing high cache-miss rate on a sequential-looking loop. The fixes are the chapter's: switch the scanned field(s) to **SoA**, or **split hot from cold** so the hot loop streams dense records; and reorder fields to shrink padding. And the inverse trap for later: do *not* reflexively pack everything tight once threads are involved — that invites false sharing (§5.5). As always, confirm the layout change with a measurement on your data and machine — the ~4× here is one machine's number, not a law.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. In review, someone asks why you "denormalized a clean `Particle` struct into a bunch of parallel arrays — it's uglier and the objects made more sense." Cover the paragraph and answer in two or three sentences, using *struct-of-arrays*, *cache line*, and *access pattern*.

Model answer. "The hot loop scans a single field across all records, so with the struct layout each 64-byte line we fetched held one useful value and seven fields the loop never reads — we were paying for eight times the bandwidth we used. Splitting that field into its own contiguous array (struct-of- arrays) makes the scan stride-1, so every fetched line is full of values we sum, and it measured several times faster for the identical result. If a different loop needed whole records at once I'd keep them as structs — the layout follows the access pattern, not the domain model." Naming the access pattern and useful-bytes-per-line — rather than "arrays are faster" — is the senior register, and it makes the trade-off explicit so the reviewer can check it against the *other* loops.

5.6 The rule, and where Part 1 lands

Five chapters of machine knowledge reduce, at the design level, to one instruction: **lay your data out for the way the hot loop touches it.** Memory moves in cache lines and is translated in pages (Chapters 3–4), so the layout that keeps each fetched line and each touched page full of bytes the loop actually uses wins — array-of-structs when you use whole records, struct-of- arrays when you scan a field, hot/cold splitting when a record is mostly cold, tight field ordering to shrink padding, and (later, under threads) deliberate spreading to avoid false sharing. None of this changes an algorithm's big-O; all of it can change its speed by a large factor, as the AoS/SoA measurement showed. That is why "the fastest code respects the memory system" is not a slogan but a design discipline — the practical cash value of mechanical sympathy.

Part 1 is almost complete. You now have the machine (CPU, caches, TLB) and the design posture (data-oriented layout) to exploit it. Chapter 6 closes the Part with the skill that ties it together: **back-of-the-envelope cost estimation** — using the orders of magnitude from Chapters 1–5 to *predict* whether a workload is compute-, memory-, or I/O-bound, then measuring to confirm. It is where "respect the memory system" becomes a number you can estimate before you write the code — and the bridge into Part 2, where you take manual control of that memory in C.

CAN YOU… WITHOUT LOOKING?

- State the central question of **data-oriented design** (what does the hot loop touch together?) and connect **mechanical sympathy** (Martin Thompson) to it.
- Define **AoS** and **SoA** and say which wins for (a) scanning one field across records and (b) using whole records one at a time — and *why* each.
- Explain **hot/cold splitting** and the tell that you want it.
- Explain **padding and alignment**: why field order changes struct size, and why a smaller struct improves locality.
- Describe **false sharing** in one sentence and why its fix (spread data out) is the *opposite* of single-threaded packing.
- Recall the AoS/SoA **measured result** (~3.7–4× on the test machine), *why* it was less than the naive 8×, and why the exact number is machine-dependent.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What single question does data-oriented design ask before choosing a layout, and why is "what is the natural object?" the wrong one for performance? (§5.1)
2. **R2.** Define AoS and SoA and give the access pattern that favors each. (§5.2)
3. **R3.** What is hot/cold field splitting, and how does it raise useful-bytes-per-line? (§5.3)
4. **R4.** Why does reordering a struct's fields change its size, and why can that be a locality win? (§5.4)
5. **R5.** What is false sharing, and why is its fix the opposite of the packing advice in the rest of the chapter? (§5.5)

FURTHER READING

- **Drepper, "What Every Programmer Should Know About Memory"** (2007, free). The deepest free treatment of why layout and access pattern dominate memory performance; its sections on data structures and access patterns are the direct background for this chapter. Read for mechanism; its absolute numbers are dated.
- **MIT 6.172, *Performance Engineering of Software Systems*** (free OCW). Lectures on cache-efficient data structures, data layout, and measuring the effect — the applied continuation of this chapter, and where the performance-engineering Part picks up.
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP)**, Chapter 3 (data alignment and struct layout) and Chapter 6 (locality). The reference for how the compiler pads and aligns structs, and why alignment matters.

Exercises

5.1 WARM-UP Define array-of-structs and struct-of-arrays in one sentence each, and state which stores "the same field across different records" contiguously.

5.2 WARM-UP A record is 8 `double`s (64 bytes). You sum one field across many records in AoS. (a) How many bytes of each 64-byte line does the sum use? (b) Roughly how many times more memory does the AoS one-field scan move than an SoA scan of the same field? (c) Which kind of locality (§3.2) does SoA restore?

5.3 WARM-UP Give the tell — in one sentence — that a struct is a candidate for hot/cold field splitting.

5.4 CORE For each workload, choose AoS or SoA and justify in terms of the access pattern and cache lines: (a) a physics step that reads and writes all fields of one particle before moving to the next; (b) a query that sums one numeric column over ten million rows; (c) a SIMD kernel that multiplies every record's `x` by a scalar; (d) rendering that, per object, reads its position, color, and size together.

5.5 CORE A struct is declared `struct S { char flag; double value; int id; };`. (a) Explain why its size is larger than 1+8+4 = 13 bytes. (b) Reorder the fields to minimize padding and explain your ordering. (c) State the locality benefit of the smaller struct when you have an array of ten million of them.

5.6 COST You scan one 8-byte field of records that are S bytes each, laid out AoS, over a large array. (a) In terms of S and the 64-byte line, how many useful bytes per fetched line does the scan use, and what fraction is that? (b) As S grows, what happens to the AoS/SoA bandwidth ratio for this scan? (c) Why might the *measured* speedup be smaller than this bandwidth ratio predicts (name one reason from Chapter 2 and relate it to why this chapter's demo measured ~4×, not 8×)?

5.7 FADED *Worked, with the last step for you.* Argue which layout to choose for a mixed workload that has *two* hot loops: loop A sums one field over all records; loop B updates all fields of each record in turn. Steps 1–3 given; complete step 4.
Step 1 (given): Loop A (scan one field) favors SoA — contiguous field, whole lines used.
Step 2 (given): Loop B (whole record at a time) favors AoS — one record's fields adjacent, one fetch per record.
Step 3 (given): A single layout cannot be optimal for both; the choice depends on which loop dominates the runtime and by how much.
Step 4 (you): State how you would *decide*: what you would measure to learn which loop dominates, how hot/cold splitting or a hybrid layout might serve both partly, and why "pick the layout that helps the hotter loop, then re-measure" is the right posture rather than reasoning it out on paper. Name the discipline this reflects.

5.8 MIXED For each speedup, decide whether the primary mechanism is *SoA / whole-line use* (Ch 5), *hot/cold splitting* (Ch 5), a *TLB / huge-page effect* (Ch 4), a *capacity miss fixed by blocking* (Ch 3), or *ILP / multiple accumulators* (Ch 2), and justify in a line: (a) moving a rarely-read description field out of a hot record; (b) summing a column after switching from row-structs to column arrays; (c) tiling a matrix multiply; (d) enabling 2 MiB pages for a random-access index; (e) summing into four accumulators instead of one.

5.9 MIXED Two versions of the same aggregation over ten million records differ by 5× in wall time; both are O(n). Without running anything, list at least three distinct candidate causes drawn from Chapters 2–5 (include at least one layout cause and one from an earlier chapter), then name the single measurement you would take first and why it best narrows the field.

5.10 STRETCH This chapter both tells you to *pack* related data into shared lines (SoA, hot/cold, tight padding) and warns that packing per-thread data into a shared line causes *false sharing*. (a) Explain why these are not contradictory — reduce both to the one fact about cache lines that drives them. (b) Give a single scenario where you would deliberately pad a struct *up* to a full cache line, and say what you are trading away. (c) Why does the right choice depend on how many cores write the data?

5.11 LAB Reproduce the AoS/SoA demonstration on your own machine. Define a record of several fields (size it to a whole number of your cache line, e.g. 64 bytes), make enough records to comfortably exceed your last-level cache, and sum one field two ways: over the array-of-structs and over a contiguous per-field array. Verify both produce the identical sum, time each (best of several runs), and report *your* ratio. Then try to widen it toward the bandwidth ceiling by removing the accumulator dependency (several accumulators, or let the compiler vectorize) and report what changed. Report only numbers you actually measured; if a loop is optimized away, consume the sum and say what you did.

Chapter 6 — The Cost Model, Applied (Capstone Lab)

Before you start: all of Part 1 — Chapter 1 (the latency hierarchy), Chapter 2 (the CPU), Chapter 3 (caches), Chapter 4 (the TLB), Chapter 5 (data layout). · **Pairs with:** the performance-engineering Part, which turns these estimates into profiled, tool-verified analysis. This chapter is the capstone: you *predict* from the cost model, then *measure*, then *reconcile*.

BEFORE YOU READ ON

From memory, across the whole Part: (1) Chapter 1's orders of magnitude — an L1 hit ~1 ns, an L2 hit a few ns, a DRAM access ~100 ns. About how many times slower is DRAM than L2? (2) Chapter 3 — what does the hardware *prefetcher* do for a sequential access pattern? (3) *Predict, and write it down:* you sum a 1 MiB array (fits in L2) and a 512 MiB array (lives in DRAM), touching the same total number of elements. From the latency numbers alone, how many times slower would you guess the DRAM sum is — 2×? 10×? 100×? Hold your number; this chapter measures it, and the answer is a lesson.

Everything in Part 1 has been building one instrument: a **cost model** — a set of orders of magnitude and mechanisms that lets you estimate, before writing or profiling a line, roughly how fast a piece of code *can* go and what will limit it. This closing chapter uses that instrument for real. First we assemble the estimation toolkit and the single most useful classification in performance work — is this workload **compute-bound**, **memory-bound**, or **I/O-bound**? Then we run the capstone lab: predict the cost of two array sums from the model, measure them on real hardware, and — this is the whole point — *reconcile the surprise* when the measurement defies the naive prediction. The reconciliation is where the model earns its keep, because a cost model you cannot correct against reality is just a table of numbers to misquote.

6.1 Back-of-the-envelope estimation

Back-of-the-envelope estimation means getting the right *order of magnitude* with arithmetic you can do in your head, using the ratios from this Part — not predicting a benchmark to three digits. The toolkit is small, and every number in it is one you already have (treat all as approximate and machine-dependent; the register-through-DRAM figures anchor to CS:APP, the SSD/network ones to the community "latency numbers" list, both introduced in Chapter 1):

- **Latencies** (time to get *one* scattered item): L1 ~1 ns, L2 a few ns, L3 ~10 ns, DRAM ~100 ns, local SSD tens of µs, network round trip hundreds of µs and up — multiplicative jumps, ~10–100× per tier.
- **Bandwidths** (throughput for *streaming* many contiguous items): memory delivers on the order of *gigabytes per second* per core when accessed sequentially, because the prefetcher and full-line fetches keep the pipe full. Latency and bandwidth are

different numbers answering different questions — the distinction is the hinge of this whole chapter.

- **Compute**: a core executes on the order of a few instructions per cycle at a few GHz — so ~10^9–10^{10} simple operations per second per core (Chapters 1–2).
- **Sizes that gate tier**: which tier your data lives in is set by working-set size versus cache sizes (measure your own with `lscpu`/`getconf`). Fits in L2 → L2 speed; spills past L3 → DRAM speed; larger than RAM → SSD/I-O speed.

The method: estimate how many operations and how many bytes the work requires, divide by the relevant rate, and take the *larger* of the compute-time and memory-time estimates as your floor — because the resource in shorter supply is the bottleneck. That single comparison is the next section.

6.2 Compute-bound, memory-bound, or I/O-bound

The most useful question you can ask about a workload is *what resource is it waiting on?* Three broad answers, each pointing at a different toolkit:

- **Compute-bound:** the bottleneck is the CPU's execution units — lots of arithmetic per byte of data, data already in cache. Time scales with operations, not memory traffic. The levers are Chapter 2's: better ILP, fewer/ predictable branches, vectorization, a lower-complexity algorithm.
- **Memory-bound:** the bottleneck is moving data between the CPU and memory — little arithmetic per byte, or a working set that spills to DRAM. Time scales with bytes moved and miss counts, not with the arithmetic. The levers are Chapters 3–5's: locality, layout (AoS/SoA), blocking, and fewer, fuller cache lines and pages.
- **I/O-bound:** the bottleneck is the disk or network — the workload spends its time waiting on tens-of-microseconds-and-up operations. Time scales with I/O operations and their latency; the levers are batching, asynchrony, and reducing round trips (the low-level I/O Part).

The deciding ratio is *work per byte*: an operation that does much arithmetic on each byte it loads (a dense matrix multiply) tends compute-bound; one that does almost nothing per byte (summing an array, copying memory) tends memory-bound; one dominated by waiting on storage or a socket is I/O-bound. Crucially, *the same algorithm can change class with its input size*: a sum whose array fits in cache may be limited by the add pipeline (compute-ish), while the identical sum over an array that spills to DRAM becomes memory-bound. This is why "profile before you optimize" is not a platitude — apply the memory toolkit to a compute-bound loop and you get nothing. Estimate the class first; the estimate tells you which chapter to open.

> **QUICK CHECK**
>
> Classify each as most likely compute-, memory-, or I/O-bound: (a) summing a 500 MiB array once; (b) a dense N×N matrix multiply with N small enough to fit in cache; (c) reading 10,000 small records from an SSD one at a time. (Answer after the lab.)

6.3 The capstone lab, part 1: predict, measure, reconcile a surprise

Now the payoff. We will estimate, then measure, the cost of summing an array that fits in L2 versus one that spills to DRAM — and confront the gap between the naive prediction and the result.

The prediction (naive, from latency numbers). A 1 MiB array of `long` fits in this machine's per-core L2 (1.25 MiB); a 512 MiB array is far past the 24 MiB L3, so it lives in DRAM. If you reach for the *latency* numbers — L2 a few ns, DRAM ~100 ns — the tempting estimate is that the DRAM sum should be roughly **an order of magnitude or more slower**, maybe 10–100×. Write that prediction down; it is the one most readers make, and it is wrong — instructively.

The measurement. Summing into a `long`, touching the same total element count in both cases (the 1 MiB array swept many times, the 512 MiB array once), built with `gcc -O2` and run for this book on an i7-12700H (best of 5):

```
/* both sum the SAME number of elements; only where the data lives differs */
for (pass ...) for (i < n) sum += a[i];   /* small n: L2-resident;  big n: DRAM */
```

Working set	Time (same total elements)	ns / element	Effective GB/s
L2-resident (1 MiB, many passes)	~0.048–0.050 s	~0.71–0.75	~10.7–11.2
DRAM-resident (512 MiB, one pass)	~0.082–0.086 s	~1.22–1.28	~6.3–6.6

The DRAM sum was only about **1.7×** slower than the L2 sum — *not* the 10–100× the latency numbers suggested. (These figures are machine-dependent; the point is the *ratio*'s surprise, not the digits.)

The reconciliation — why the prediction was wrong. The naive estimate used the wrong number. The ~100 ns DRAM figure is a *latency* — the time to get *one* scattered item when you must wait for it. But a sequential sum does not wait for one item at a time: the access pattern is stride-1, so the **hardware prefetcher** (Chapter 3) sees it coming and streams lines ahead of the loop, and the CPU has many independent loads in flight. The workload is therefore **bandwidth-bound, not latency-bound** — its speed is set by how fast memory can stream contiguous data, not by the latency of a single miss. And streaming bandwidth from DRAM, while lower than from L2, is within a small factor of it; here the loop achieved ~11 GB/s out of L2 and ~6 GB/s out of DRAM (both also partly capped by the single-accumulator add dependency of §2.3, which is why neither is dramatic). Prefetching converted a would-be 100× latency gap into a ~1.7× bandwidth gap. *That* is the lesson of the capstone: **use latency for scattered access and bandwidth for streaming access** — reach for the wrong one and your estimate is off by orders of magnitude.

6.4 The capstone lab, part 2: when the memory wall really bites

So does the ~100 ns DRAM latency ever show up? Yes — precisely when the prefetcher *cannot* help, i.e. when access is scattered and dependent, so there is no stream to run ahead of and no independent work to overlap. That is the pointer-chasing pattern of Chapter 3. To expose it, chase a pointer around a random cyclic permutation (each step's address comes from the previous read) over the same two working sets — L2-resident and DRAM-resident — timing an equal number of dependent steps. Measured on the same machine (best of 3):

Working set (random pointer-chase)	ns / dependent step	DRAM/L2 ratio
L2-resident (1 MiB)	~6 ns	~24–25×
DRAM-resident (512 MiB)	~150–158 ns	

Now the latency gap appears in full: ~24–25×, and the DRAM chase costs ~150 ns per step — comfortably in the ~100 ns order of magnitude Chapter 1 gave for a DRAM access, and in fact somewhat *more*. That surplus over the bare ~100 ns is itself a Part-1 lesson: a random chase over 512 MiB also **thrashes the TLB** (Chapter 4) — each jump lands on a new page whose translation is likely evicted — so part of the ~150 ns is the page-table walk stacked on top of the data miss. Same hardware, same data sizes as §6.3; the *only* change is sequential→random access, and the cost went from a ~1.7× bandwidth gap to a ~25× latency gap. This is the whole Part in one contrast: *where your data lives* matters, but *how you walk it* decides whether the memory wall is hidden or paid in full.

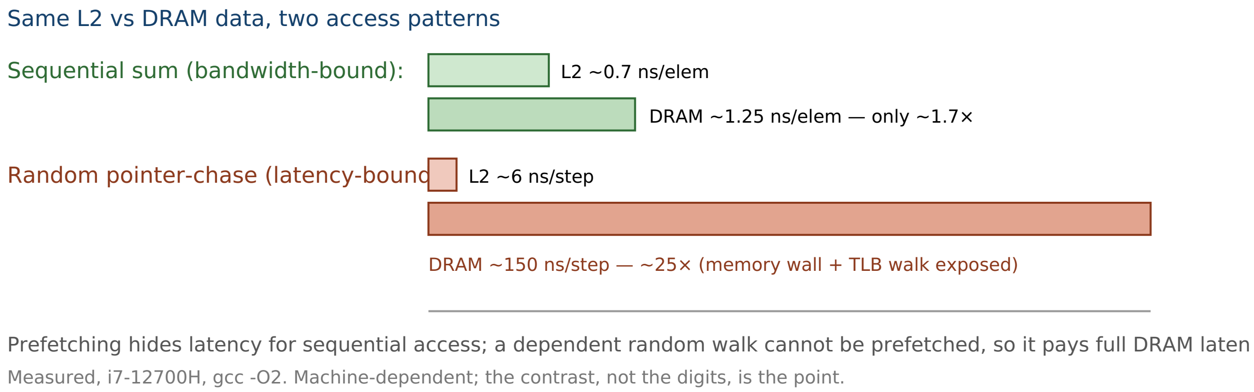

Figure 6.1: The same data sizes, two access patterns. Sequential streaming is bandwidth-bound (DRAM only ~1.7× slower than L2); a dependent random chase is latency-bound (~25×), exposing the full memory wall plus TLB cost.

THE SAME IDEA THREE WAYS: LATENCY VS. BANDWIDTH DECIDES THE ESTIMATE

1. In words. DRAM's ~100 ns is a *latency* — the wait for one scattered item. Sequential access is not a series of waits; the prefetcher streams data, so it runs at memory *bandwidth*, within a small factor of cache. Predict streaming with bandwidth, scattered access with latency.

2. As a picture. Figure 6.1: a small sequential gap (bandwidth) beside a large random gap (latency) over the identical data sizes.

3. As measurements. Sequential sum: DRAM ~1.7× L2. Random chase: DRAM ~25× L2, ~150 ns/step. The model predicted 100× from latency; the streaming case corrected it to ~1.7×; the scattered case confirmed the latency really is there when nothing hides it.

(Answer to the §6.2 quick check: (a) ***memory-bound*** *— one add per 8 bytes, streams from DRAM; (b)* ***compute-bound*** *— much arithmetic per byte, data resident in cache; (c)* ***I/O-bound*** *— dominated by per-record SSD latency, and batching/async would help.)*

TRAP: GUESSING INSTEAD OF PROFILING (AND ESTIMATING WITH THE WRONG NUMBER)

Two failures this Part has hammered, together at last. The first: *optimizing without measuring* — rewriting a loop's layout or algorithm on a hunch, when a five-minute profile would have shown the time is somewhere else entirely. The second, subtler: *estimating with the wrong model* — as in §6.3, where the ~100 ns latency number predicts a 100× gap for a workload that is actually bandwidth-bound and shows ~1.7×. A back-of-the-envelope estimate is a hypothesis about the bottleneck, not a verdict; it tells you where to *look* and which class you expect, and then you measure to confirm the class and find the real limiter. The tell that you have mis-estimated is exactly the §6.3 surprise: a measurement off from your prediction by more than a small factor — treat that gap as information (usually you used latency where bandwidth applies, or vice versa), reconcile it, and update the model. Guessing the cause and skipping the measurement is how good engineers waste days; the discipline of Part 1 is estimate, measure, reconcile, in that order.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A colleague says: "DRAM is 100× slower than cache, so our big sequential scan over the 500 MiB table must be crippled — let's cache it in a fancy structure." Cover the paragraph and answer in three or four sentences, using *latency*, *bandwidth*, *prefetcher*, and *bound*.

Model answer. "That 100× is a *latency* figure — the wait for one scattered access. Our scan is sequential, so the prefetcher streams the data and we run at memory *bandwidth*, which is within a small factor of cache; the scan is bandwidth-bound, not latency-bound, so it's probably only a couple of times slower than if it fit in cache, not 100×. Before we build anything, let's measure the scan's GB/s and confirm it's near memory bandwidth — if it is, a fancy cache buys little and the win, if any, is fewer bytes touched, not lower latency. Where latency *would* bite is if we switched to scattered, pointer-chasing access — then we'd pay the full ~100 ns-plus per step." Naming which resource bounds the workload, and measuring before building, is the senior register.

6.5 Closing Part 1: from the machine to the code that respects it

Six chapters ago the RAM model told a comforting lie: every operation costs one unit, memory is uniform, and speed follows big-O. Part 1 replaced it with the real cost model. You know the machine now — the CPU that overlaps and predicts (Chapter 2), the cache hierarchy that moves memory in lines and rewards locality (Chapter 3), the TLB that translates addresses and punishes scattered pages (Chapter 4) — and you know how to spend that knowledge: lay data out for the access pattern (Chapter 5), and estimate before you optimize, distinguishing latency from bandwidth and compute-bound from memory-bound from I/O-bound (this chapter). The habit that unifies all of it is the one the whole Part drilled: when two pieces of code have the same big-O but different speed, the difference is the machine — and you now know where to look, what to estimate, and how to confirm it by measuring.

This closes Part 1, "The Machine & the Cost Model," and opens the rest of the volume. Part 2 takes you into **C and manual memory**: pointers, the stack and heap, alignment and layout under your direct control, and undefined behavior — programming with no safety net so you understand the net. Everything you estimated here in the abstract, you will there arrange in bytes by hand. The cost model is no longer something the machine hides from you; it is a tool you carry into every line you write.

CAN YOU… WITHOUT LOOKING?

- Do a **back-of-the-envelope** estimate: turn a workload into operations and bytes, divide by the right rate, and take the larger estimate as the floor.
- Distinguish **latency** from **bandwidth** and say which applies to scattered vs. streaming access — and why using the wrong one mispredicts by orders of magnitude.
- Classify a workload as **compute-, memory-, or I/O-bound** from its work-per-byte, and name which chapter's levers each class points to.
- Explain the **§6.3 surprise**: why the sequential DRAM sum was only ~1.7× slower than the L2 sum, not 100× (prefetcher → bandwidth-bound).
- Explain the **§6.4 contrast**: why the random pointer-chase *did* show ~25× and ~150 ns/step, and what part of that surplus is TLB cost.
- State the discipline in order — **estimate, measure, reconcile** — and why a prediction-vs-measurement gap is information, not failure.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** Give the back-of-the-envelope method for deciding whether code is compute- or memory-bound, and the "work per byte" intuition behind it. (§6.1–6.2)
2. **R2.** What is the difference between latency and bandwidth, and which one governs a sequential scan? (§6.1, §6.3)
3. **R3.** Why was the sequential DRAM sum only ~1.7× slower than the L2 sum when the latency numbers suggested far more? (§6.3)
4. **R4.** Why did the random pointer-chase show ~25× and ~150 ns/step, and why is that *more* than the bare DRAM latency? (§6.4)
5. **R5.** State the estimate–measure–reconcile discipline and explain why a gap between prediction and measurement is useful rather than embarrassing. (§6.3, warning box)

FURTHER READING

- **MIT 6.172, *Performance Engineering of Software Systems*** (free OCW). The applied home of this chapter: profiling, distinguishing compute- from memory-bound work, and measuring bandwidth vs. latency with real tools — the direct continuation into the performance-engineering Part.
- **Agner Fog, optimization manuals** (free, *agner.org*). Microarchitectural detail behind the compute-bound levers — throughput vs. latency of instructions, and how many can be in flight. Dated in specifics; verify against your CPU.
- **Drepper, "What Every Programmer Should Know About Memory"** (2007, free) and **Bryant & O'Hallaron, CS:APP**, Chapters 6 and 9. The mechanisms this chapter estimates against — cache and memory bandwidth, prefetching, and translation cost. Read Drepper for mechanism, not its dated absolute numbers; the "latency numbers" list (Jeff Dean / Colin Scott) supplies the SSD/network orders of magnitude, strictly as ratios.

Exercises

6.1 WARM-UP In one sentence each, define latency and bandwidth, and say which one you would use to estimate the time to (a) fetch one random element from a huge array, and (b) stream through a huge array in order.

6.2 WARM-UP Classify each workload as compute-, memory-, or I/O-bound and give the one-line reason: (a) `memcpy` of 1 GiB; (b) computing SHA-256 over data already in L1; (c) fetching 1,000 rows one-by-one from a remote database.

6.3 WARM-UP State the estimate–measure–reconcile discipline in order, and say in one sentence why a gap between your prediction and the measurement is useful information.

6.4 CORE You must sum a 512 MiB array of `long` once. (a) Estimate the time two ways: from DRAM *latency* (~100 ns per element) and from streaming *bandwidth* (assume ~6 GB/s for this loop). (b) Which estimate is right for a sequential sum, and why? (c) By roughly what factor do the two estimates differ, and what does that factor tell you about the cost of using the wrong model?

6.5 CORE Explain the §6.3 result to a teammate: the DRAM sum was only ~1.7× slower than the L2 sum. (a) Why did the prefetcher make this bandwidth-bound rather than latency-bound? (b) Why was even the L2 sum not much faster than a naive "few-ns-per-access" guess would allow (name the §2.3 factor)? (c) What single change to the code would let it approach memory bandwidth more closely?

6.6 COST A kernel does F floating-point operations and moves B bytes. Your core can do ~10^{10} ops/s and stream ~10^{10} bytes/s. (a) Write the compute-time and memory-time estimates. (b) Give the condition on F/B (work per byte) under which the kernel is compute-bound. (c) A sum does one add per 8 bytes — is it compute- or memory-bound by this test, and does the measurement in §6.3 agree?

6.7 FADED *Worked, with the last step for you.* Reconcile why the random pointer-chase over 512 MiB cost ~150 ns/step while the sequential sum over the same array cost ~1.25 ns/element. Steps 1–3 given; complete step 4.
Step 1 (given): Both touch DRAM-resident data of the same size; the only difference is access pattern (sequential vs. dependent-random).
Step 2 (given): Sequential access is prefetchable and has many independent loads in flight, so it runs at bandwidth (§6.3).
Step 3 (given): The random chase is a dependency chain — each step's address comes from the previous read — so it cannot be prefetched or overlapped and pays full latency per step (§3.5).
Step 4 (you): Complete the reconciliation: explain why ~150 ns/step is *more* than the ~100 ns bare DRAM latency (name the Chapter 4 mechanism), and state the one measurement that would confirm that extra cost is translation, not data. Then say what this pair of results teaches about when to estimate with latency vs. bandwidth.

6.8 MIXED For each observation, name the most likely bottleneck class (compute / memory / I/O) *and* the specific mechanism from Chapters 1–6, and justify in a line: (a) a sequential scan runs at ~6 GB/s and adding threads barely helps; (b) a loop with a data-dependent branch speeds up 3× after sorting the input; (c) a random lookup into a 2 GiB index costs ~160 ns and huge pages help; (d) a dense matrix multiply that fits in cache is limited by how many multiply-adds per cycle the core issues.

6.9 MIXED A report claims a data pipeline stage is "slow because DRAM is 100× slower than cache." Given only that the stage streams sequentially through a 4 GiB file-backed array, (a) say why the 100× framing is probably the wrong model, (b) predict the rough class and the rough slowdown vs. an in-cache version, and (c) name the two measurements you would take to confirm your reframing before anyone optimizes.

6.10 STRETCH The chapter measured a sequential DRAM/L2 ratio of ~1.7× and a random one of ~25× on one machine. (a) Explain why the *random* ratio is far more portable across machines than the sequential one (think about what each is bounded by). (b) Give one machine change that would raise the sequential ratio and one that would lower the random ns-per-step. (c) Argue why reporting both ratios — with the access pattern that produced each — is more honest and more useful than a single "DRAM is N× slower" headline number.

6.11 LAB **Capstone.** On your own machine: (1) record your L2 and L3 sizes and cache-line size. (2) *Predict*, from the cost model and before running, the time to sum an L2-sized array vs. a DRAM-sized array (same total elements), stating whether you expect it bandwidth- or latency-bound and your predicted ratio — write it down first. (3) Measure both (verify identical sums; best of several runs) and report ns/element and effective GB/s. (4) Then measure the *random pointer-chase* version over the same two sizes and report ns/step. (5) *Reconcile*: compare each measurement to your prediction, explain any surprise using latency-vs-bandwidth and (for the random case) the TLB, and if you can, capture cache-miss and TLB-miss counters (e.g. `perf stat`) to support your explanation. Report only numbers you actually measured; if a loop is elided, consume the result and say what you did.

VOL 1 · SYSTEMS PROGRAMMING & PERFORMANCE · PART II

C & Manual Memory

Pointers, layout, the stack/heap, manual allocation, and undefined behavior — programming with no safety net so you understand the net.

Chapter 7 — Pointers & Memory Layout

Before you start: Chapter 4 (virtual addresses; a pointer holds a *virtual* address the hardware translates), Chapter 5 (alignment and struct padding, met there as a layout lever), Chapter 3 (cache lines and locality). · **Opens Part 2** and pairs with Chapter 8 (stack vs. heap) and Chapter 9 (manual allocation). Part 1 told you what memory *costs*; Part 2 hands you the controls. This chapter is the first: what a pointer actually is, how arithmetic on it works, and how your program's bytes are arranged in the address space.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 4 — when your C pointer holds a number like `0x7ffd3a08` and you dereference it, is that the physical address of a DRAM cell, or a *virtual* address the hardware translates first? (2) Chapter 5 said a struct like `{ char a; double b; char c; }` is bigger than its 10 bytes of real data — what is the compiler inserting, and why? (3) *Predict:* `p` is a pointer to an `int` and `q` is a pointer to a `double`. You write `p + 1` and `q + 1`. Do both advance the address by the same number of bytes? If not, what decides each step? By the end of the chapter you will be able to compute the exact byte a pointer expression lands on.

A pointer is the most demystifiable idea in systems programming, because underneath the syntax it is just a number: **a pointer is a variable that holds a memory address.** That is the whole secret. The type decoration — `int *`, `double *`, `struct Particle *` — does not change what is stored (an address); it tells the compiler what *kind* of thing lives at that address, which is exactly what it needs to know to read the right number of bytes when you dereference, and to scale arithmetic correctly when you do math on the pointer. This chapter builds the model from that one fact. We will see what a pointer holds and what dereferencing does; how pointer arithmetic silently multiplies by the pointee's size; why arrays and pointers are close cousins but *not* the same thing; how your program's memory is carved into regions (text, data, bss, heap, stack); and how `sizeof`, alignment, and struct padding decide the exact byte layout — with every size on this page *verified with the compiler* rather than asserted, because in C those sizes are implementation-defined and you must never guess them.

7.1 A pointer is an address

When you declare `int x = 42;`, the compiler arranges for `x` to live at some address in memory — say byte 0x7ffd...a0. When you then write `int *p = &x;`, the **address-of** operator `&` yields that address, and `p` stores it. So `p` holds a number that names a location; `*p` — the **dereference** — means "go to the location named by `p` and read (or write) the object there." Because `p` was declared `int *`, `*p` reads `sizeof(int)` bytes and interprets them as an `int`. Reconnecting to Chapter 4: the number in `p` is a *virtual* address in this process's address space; dereferencing it triggers the translation-and-fetch machinery we already dissected. The pointer is the programmer-visible handle on the very addresses that chapter was translating.

How big is a pointer? Big enough to hold an address on this machine. On the x86-64 machine used throughout this book, compiled with gcc, a data pointer is **8 bytes** (64 bits) — verified, not assumed:

```
printf("%zu\n", sizeof(void*));   /* prints 8 on this machine (x86-64, LP64) */
```

The C standard does *not* fix the size of a pointer — it is implementation-defined, and different pointer types need not even be the same size in general (a subtlety that rarely bites on flat-address-space machines like x86-64, where all object pointers are the same width). What the standard *does* guarantee is narrower and worth stating precisely: `void*` can hold any object pointer and round-trip it back unchanged, and `sizeof(char) == 1` by definition — a `char` is one *byte*, the unit `sizeof` counts in (cppreference, "sizeof operator"; the C standard, §6.5.3.4). Everything else about sizes we will measure.

QUICK CHECK

`int *p = &x;`. In one sentence each: what does `p` hold, and what does `*p` do? And why does the *type* `int *` matter when you write `*p`, if the stored value is "just an address"? (Answer after the next section.)

7.2 Pointer arithmetic scales by the pointee size

Here is the feature that trips up everyone once. When you add an integer to a pointer, C does *not* add that integer to the raw address. It scales: `p + n` advances the address by `n * sizeof(*p)` bytes, so that `p + n` points at the *n*-th object of that type past `p`, not the *n*-th byte. This is deliberate — it makes `p + 1` mean "the next element," which is what you almost always want — but it means the byte step depends entirely on the pointer's type. Verified on this machine:

```
int    arr[4];   /* &arr[1] - &arr[0] == 4 bytes  (sizeof(int)    == 4) */
double darr[4];  /* &darr[1]- &darr[0]== 8 bytes  (sizeof(double)== 8) */
```

Running exactly that on the book's machine, the `int` pointer stepped **4 bytes** and the `double` pointer stepped **8 bytes** — each equal to the pointee's `sizeof`, which is what "answers your prediction from the opener": no, `p + 1` and `q + 1` do *not* advance by the same number of bytes; each advances by its own element size. The mirror image is pointer *subtraction*: `q - p` for two pointers into the same array yields the number of *elements* between them (of type `ptrdiff_t`), again dividing out the element size. Indexing is just sugar for this: `arr[i]` is defined as `*(arr + i)`, so the subscript is scaled arithmetic wearing brackets.

Two precision points the C standard is strict about, both worth carrying into the UB chapter (Chapter 9). First, pointer arithmetic is only defined *within* a single array object, including the special "one past the last element" address, which you may compute and compare against but must *not* dereference (C standard §6.5.6, additive operators). Forming or dereferencing a pointer further out than one-past-the-end is undefined behavior — not "wraps around," not "reads the next variable," but genuinely undefined.

Second, the scaling is by the *static* type of the pointer, so a mismatched type computes the wrong address silently. The takeaway is mechanical and exact: *to find where `p + n` points, multiply `n` by `sizeof(*p)` and add that many bytes to `p`.*

> **THE SAME IDEA THREE WAYS: POINTER ARITHMETIC SCALES BY THE POINTEE SIZE**
>
> **1. In words.** Adding n to a pointer moves it by n elements, i.e. $n \times$ `sizeof(*p)` bytes — so the byte step is the element size, and `arr[i]` is exactly `*(arr + i)`.
>
> **2. As a picture.** Figure 7.1: two rulers over the same bytes. On the `int*` ruler each tick is 4 bytes apart; on the `double*` ruler each tick is 8 bytes apart. "+1" means "next tick," and the tick spacing is `sizeof(*p)`.
>
> **3. As a trace.** `double *q` at address 1000: `q+1` → 1000 + 1×8 = 1008; `q+3` → 1000 + 3×8 = 1024. An `int *p` at 1000: `p+3` → 1000 + 3×4 = 1012. Same "+3", different bytes, because the element size differs.

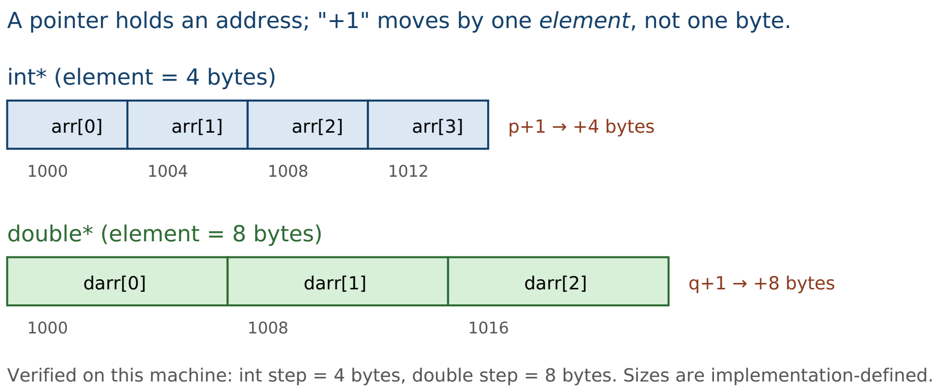

Figure 7.1: Pointer arithmetic scales by the pointee's size. "+1" advances one element — 4 bytes for `int*`, 8 for `double*` on this x86-64 machine.

(Answer to the §7.1 quick check: `p` holds the address of `x`; `*p` goes to that address and reads/writes the object there. The type `int *` matters because it tells the compiler how many bytes to read at that address and how to interpret them — four bytes as an `int` here — and it sets the scale for pointer arithmetic.)

7.3 Arrays vs. pointers, and array-to-pointer decay

Arrays and pointers feel interchangeable in C, and beginners often conclude they are the same. They are not, and the difference is a frequent source of bugs. An **array** is a block of contiguous elements; the array's name refers to that whole block, and the block *is* the storage. A **pointer** is a separate variable that holds an address, which may point at an array's first element but is its own object. The two behave alike in expressions only because of a specific rule: **array-to-pointer decay.** In most expression contexts, an array name is automatically converted ("decays") to a pointer to its first element (C standard §6.3.2.1). That is why you can write `int *p = arr;` and `arr[i]` and pass `arr` to a function expecting `int *` — the array decayed to `&arr[0]`.

The decay does *not* happen in three contexts, and those exceptions expose the difference. The sharpest is `sizeof`. Applied to an array, `sizeof` yields the *whole array*'s size in bytes; applied to a pointer, it yields the *pointer*'s size. Verified on this machine:

```
int  a10[10];
int *p = a10;
sizeof(a10);  /* == 40 : 10 ints × 4 bytes each, the whole array */
sizeof(p);    /* == 8  : just the pointer, regardless of what it points to */
```

On the book's machine those printed **40** and **8**. This is the origin of a notorious trap: passing an array to a function makes it decay to a pointer, so *inside* the function `sizeof(param)` is the pointer size (8), not the array size — the length information is lost at the call boundary, which is why C array-handling functions take an explicit length parameter. (The other two non-decay contexts: the operand of unary `&`, where `&arr` has type "pointer to array"; and a string literal used to initialize a `char` array, where it initializes the array rather than setting a pointer.) The mental model: an array is the land; a pointer is a signpost that can point at it. Usually the signpost is what travels around your program, but the two are not the same object — and `sizeof` is where the difference becomes visible.

TRAP: `SIZEOF` ON A DECAYED ARRAY (MEASURING THE SIGNPOST, NOT THE LAND)

The classic bug: `void f(int a[]) { size_t n = sizeof(a) / sizeof(a[0]); … }` intending to recover the element count. It cannot work: the parameter `a` is a *pointer* (the array decayed at the call), so `sizeof(a)` is the pointer size — 8 on this machine — and the "count" comes out as `8 / sizeof(int) = 2` regardless of the real length. The `sizeof(array)/ sizeof(array[0])` idiom is valid *only* where the true array type is in scope (same function it was declared in, not a decayed parameter). The tell: computing a length from `sizeof` on a function parameter, or on anything you received as a pointer. The fix is the one the standard library uses everywhere — pass the length explicitly alongside the pointer. Confirm your assumption by printing `sizeof` at the point of use if you are ever unsure; on this machine the decayed case prints 8, the in-scope array prints its full byte size (40 for `int[10]`).

7.4 The address-space layout: text, data, bss, heap, stack

Where do all these addresses live? A running process's virtual address space (the illusion of Chapter 4) is conventionally divided into regions, each holding a different kind of data, described in CS:APP (Chapter 9) and OSTEP's address-space chapters. From low addresses upward, the classic picture is:

- **Text** (a.k.a. code): your compiled machine instructions, plus read-only constants like string literals (often a separate `.rodata`). Typically mapped read-only and executable.
- **Data** (`.data`): global and `static` variables that have a non-zero initial value — the initial bytes are stored in the executable and loaded here.
- **BSS** (`.bss`): global and `static` variables that are zero-initialized. They take *no* space in the executable file (only a size is recorded); the OS provides zeroed pages at load. (The name is a historical assembler mnemonic.)

- **Heap**: the region for *dynamic* allocation (`malloc`), which grows upward (toward higher addresses) as you request more. This is Chapter 9's subject.
- **Stack**: automatic storage for function calls — local variables, saved return addresses, arguments — which grows *downward* (toward lower addresses) as calls nest. This is Chapter 8's subject.

You can see the ordering directly. A small program that prints the address of a function (text), an initialized global (data), a zero global (bss), a `malloc`'d block (heap), and a local (stack), built `-no-pie` so the classic fixed layout is visible, printed on this machine:

```
text  (&function)      = 0x401196
.data (&global_init)   = 0x404048
.bss  (&global_bss)    = 0x40405c
heap  (malloc'd block)  = 0x351b2a0
stack (&local)         = 0x7fffa35e9a3c
```

The ordering is exactly the textbook one: text < data < bss < heap, all low, and the stack far up near the top of the space — with the heap growing up and the stack growing down toward each other, which is why the address space leaves a large gap between them. **These specific numbers are machine-, OS-, and build-dependent, not universal.** A default position-independent-executable (PIE) build on the same machine printed entirely different, higher base addresses, because **address-space layout randomization (ASLR)** relocates the segments each run as a security measure — the *relative* structure holds, the absolute numbers do not. What to carry: your program's memory is not one undifferentiated pool; it is regions with different purposes, permissions, and growth directions, and knowing which region a pointer points into tells you a lot about its lifetime — the through-line into the next two chapters.

> **QUICK CHECK**
>
> Which region holds each of these: a `malloc`'d buffer, a function's local `int`, a `static int counter = 5;`, and the machine code of `main`? Which two regions grow *toward* each other, and in which directions? (Answer after the next section.)

7.5 `sizeof`, alignment, and struct padding — computed and verified

Now the exact byte layout. Three rules from the C object model (CS:APP Chapter 3; C standard §6.7.2.1) govern how a struct is laid out, and together they let you compute a struct's size by hand — which we will then check against the compiler.

1. **Each type has a size and an alignment.** `sizeof(T)` is how many bytes an object of type `T` occupies; its alignment is an address requirement — an object of type `T` must be placed at an address that is a multiple of `_Alignof(T)`. On x86-64 the alignment of the scalar types equals their size. Verified on this machine (gcc, x86-64 LP64):

type	char	short	int	long	long long	float	double	void*
sizeof (bytes)	1	2	4	8	8	4	8	8
_Alignof	1	2	4	8	8	4	8	8

These sizes are implementation-/ABI-defined, not guaranteed by the language. The C standard fixes only `sizeof(char) == 1` and minimum ranges (an `int` is *at least* 16 bits, a `long` at least 32); it does not promise `int` is 4 bytes. These are the values for this machine's ABI, measured — on a 16-bit or ILP64 platform they would differ, which is precisely why you print them rather than hard-code them.

1. **Members are laid out in declaration order, at increasing offsets, with the first member at offset 0.** Before each member the compiler inserts as much **padding** as needed to put that member at an offset that is a multiple of *its* alignment.
2. **The struct's own alignment is the largest alignment among its members, and its size is rounded up to a multiple of that alignment** (tail padding), so that in an array of the struct every element stays aligned.

Let us apply this to a concrete struct and predict its size before measuring:

```
struct Mix { char a; int b; char c; double d; };
```

Hand computation, using the verified alignments (char 1, int 4, double 8):

member	align	size	offset	note
`a` (char)	1	1	0	first member at 0; occupies byte 0
— padding —		3	1–3	to reach the next multiple of 4 for `b`
`b` (int)	4	4	4	4 is a multiple of 4; occupies 4–7
`c` (char)	1	1	8	occupies byte 8
— padding —		7	9–15	to reach the next multiple of 8 for `d`
`d` (double)	8	8	16	16 is a multiple of 8; occupies 16–23

Struct alignment = max(1, 4, 1, 8) = **8**; the running size is 24, already a multiple of 8, so no tail padding is needed. Predicted `sizeof(struct Mix)` = **24**, of which only 1 + 4 + 1 + 8 = 14 bytes are real data and **10 bytes are padding.** Now the check — running `offsetof` and `sizeof` on this machine printed *exactly*:

```
offsetof(a)=0  offsetof(b)=4  offsetof(c)=8  offsetof(d)=16
sizeof(struct Mix)=24   _Alignof(struct Mix)=8
```

The hand computation matches the compiler byte for byte. And it exposes the layout lever from Chapter 5: field *order* changes size. Reorder the same fields largest-alignment first —

```
struct Packed { double d; int b; char a; char c; };
```

— and the layout becomes `d` at 0, `b` at 8, `a` at 12, `c` at 13, needing only 2 bytes of tail padding to round 14 up to the next multiple of 8. Verified: `sizeof(struct Packed) = 16` on this machine. Same four fields, same data, **24 vs. 16 bytes** — a third smaller — purely from ordering, which (as Chapter 5 argued) means more records per cache line and per page for free. The point of this section is method as much as result: *you can compute a struct's size from the alignment rules, and you should verify it, because the underlying sizes are implementation-defined.*

(Answer to the §7.4 quick check: `malloc`'d buffer → heap; local `int` → stack; `static int counter = 5;` → data (non-zero initializer); machine code of `main` → text. The heap (growing up) and the stack (growing down) grow toward each other.)

7.6 Endianness: which byte comes first

One more layout fact, invisible until you look at individual bytes. When a multi-byte integer sits in memory, in what order are its bytes stored? **Little-endian** stores the least-significant byte at the lowest address; **big-endian** stores the most-significant byte first. The C standard does *not* mandate either — byte order is part of the implementation-defined representation of integer types (C standard §6.2.6) — so portable code must not assume one. x86-64 is **little-endian**, and rather than assert it, we verify it: store the 32-bit value `0x01020304` and read its bytes one at a time.

```
uint32_t x = 0x01020304u;
unsigned char *b = (unsigned char*)&x;
/* print b[0] b[1] b[2] b[3] */
```

On this machine that printed the bytes **`04 03 02 01`** — the least-significant byte (`0x04`) first, at the lowest address — confirming **little-endian**, as expected for x86-64. On a big-endian machine the same program would print `01 02 03 04`. Endianness matters the moment bytes cross a boundary the CPU does not control: reading a binary file or network packet written by a different machine, or reinterpreting memory through a different-width pointer. Within a single program on one machine you rarely notice it — arithmetic and printing hide it — but at the byte level it is real and, like the sizes above, something you *check* rather than assume. (Network protocols standardize on big-endian, "network byte order," precisely so machines of either kind can interoperate; Beej's Guide covers the `htons`/`ntohl` conversions the sockets Part will use.)

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate is debugging and says: "I have a `struct { char a; int b; char c; double d; }`, that's 14 bytes of data, but `sizeof` says 24 — is the compiler broken?" Cover the paragraph and answer in two or three sentences, using *alignment*, *padding*, and *field order*.

Model answer. "It's not broken — the compiler inserts padding so each field lands on an address that's a multiple of its alignment: the `int` needs a 4-aligned offset and the `double` an 8-aligned one, so there are 3 bytes of padding after `a` and 7 after `c`, and the whole struct rounds up to a multiple of 8 — 24 bytes, 10 of them padding. If you reorder the fields largest-alignment-first (`double, int, char, char`) it packs to 16, same data, because you stop paying to re-align. You can confirm any layout with `offsetof` and `sizeof` rather than guessing." Naming alignment and offsets — and showing you can recompute the size — is the senior register, and it turns "the compiler is weird" into a controllable layout decision.

7.7 What to carry forward

A pointer is an address in a variable; dereferencing it reads or writes the object there, using the pointer's type to know how many bytes and how to interpret them. Arithmetic on a pointer scales by the pointee's size, so `p + n` lands `n * sizeof(*p)` bytes along and `arr[i]` is just `*(arr + i)` — and stepping outside an array's bounds is undefined, not merely adventurous. Arrays and pointers are close but distinct: an array decays to a pointer to its first element in most contexts, but `sizeof` (and unary `&`) sees the real array, which is why length is lost at a function boundary. Your program's memory is regioned — text, data, bss, heap, stack — with different lifetimes and growth directions, verified here by printing addresses (and remembering ASLR moves the absolute numbers). And a struct's exact size follows from alignment and padding, which you can compute by hand and *must* verify, because the scalar sizes and byte order underneath are implementation-defined — measured here as `sizeof(struct Mix) = 24` and little-endian on this x86-64 machine.

That is the layer of control Part 1 was pointing at. Chapter 8 takes the two regions with the most interesting lifetimes — the **stack** and the **heap** — and asks where a given object should live, why stack allocation is nearly free while the heap is managed, and how a pointer to a stack local becomes a dangling bug the instant its function returns. Chapter 9 then hands you the heap's controls, `malloc` and `free`, and the precise ways manual memory management goes wrong. Everything in those chapters is pointers into these regions — which is why "a pointer is an address, and addresses live in regions with lifetimes" is the model to keep.

CAN YOU… WITHOUT LOOKING?

- State what a **pointer** holds, what **dereferencing** does, and why the pointer's **type** matters even though the stored value is "just an address."
- Compute where `p + n` points for a given pointer type (multiply *n* by `sizeof(*p)`), and say why stepping past one-past-the-end of an array is undefined.
- Explain **array-to-pointer decay**, and why `sizeof` on a function's array *parameter* gives the pointer size, not the array size.
- Name the address-space regions — **text, data, bss, heap, stack** — say what each holds, and which two grow toward each other and in which directions.
- Compute a struct's size by hand from **alignment and padding**, and explain why field *order* changes it — and why you *verify* sizes rather than assume them.
- Define **little- vs. big-endian**, state which x86-64 is, and describe the one-line experiment that checks it.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What does a pointer hold, and what two things does its *type* tell the compiler? (§7.1–7.2)
2. **R2.** How many bytes does `p + 3` advance for an `int *p` vs. a `double *p` on this machine, and what general rule gives the answer? (§7.2)
3. **R3.** What is array-to-pointer decay, and what does `sizeof` report for an array vs. for a pointer (or a decayed parameter)? (§7.3)
4. **R4.** Name the five address-space regions low-to-high and say what each holds; which two grow toward each other? (§7.4)
5. **R5.** Give the three struct-layout rules and use them to explain why `{ char; int; char; double; }` is 24 bytes on this machine. (§7.5)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP)**, Chapter 3 (machine-level data, pointers, arrays, struct alignment) and Chapter 9 (the process address space and its regions). The reference for this chapter's layout and padding rules; the free CMU 15-213 materials cover them.
- **cppreference.com** and the **C standard** — the `sizeof` operator, pointer arithmetic (§6.5.6), array-to-pointer conversion (§6.3.2.1), and object representation (§6.2.6). The authority for what the language *guarantees* vs. leaves implementation-defined; consult these before assuming any size or byte order.
- **Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP)**, the address-space chapters (free full text). The OS view of how text/data/heap/stack are arranged and why the stack and heap grow toward each other.
- **Beej's Guide to C** (free). Practical, readable coverage of pointers, arrays, and byte order, including the network-byte-order conversions used in the low-level I/O Part.

Exercises

7.1 WARM-UP In one sentence each: (a) what does a pointer variable store? (b) what does `*p` do? (c) what does `&x` produce?

7.2 WARM-UP A `double *q` holds address 2000 (`sizeof(double)==8` here). What address is `q + 4`? What address is `q + 4` if `q` is instead an `int *` (`sizeof(int)==4`)? Show the arithmetic.

7.3 WARM-UP `int a[10]; int *p = a;` on this machine. State the value of `sizeof(a)` and of `sizeof(p)`, and explain the difference in one sentence.

7.4 CORE Explain array-to-pointer decay. Then explain precisely why `size_t len(int a[]) { return sizeof(a) / sizeof(a[0]); }` does *not* return the element count, and what the returned value actually is on this machine.

7.5 CORE For `struct S { char a; short b; char c; int d; };`, compute by hand (using char 1, short 2, int 4, with alignment = size for each) the offset of every member, the total `sizeof`, and how many bytes are padding. Then state how you would verify your answer with the compiler.

7.6 CORE Reorder the fields of `struct S` from 7.5 to minimize its size, give the new `sizeof`, and explain the ordering rule you applied and why it works.

7.7 COST You have an array of N = 10 million records, and a hot loop that scans them. (a) If the record is `struct Mix` (24 bytes here), how many bytes does the array occupy? (b) If reordering shrinks it to 16 bytes, how many bytes now, and roughly how many *more* records fit in a 64-byte cache line (§3.1) and a 4 KiB page (§4.2)? (c) In one sentence, tie the size reduction to the memory-system cost model of Part 1.

7.8 FADED *Worked, with the last step for you.* Compute `sizeof(struct T { double d; char a; int b; };)` on this machine's ABI. Steps 1–3 given; complete step 4.
Step 1 (given): `d` (double, align 8) at offset 0, occupies 0–7.
Step 2 (given): `a` (char, align 1) at offset 8, occupies byte 8.
Step 3 (given): `b` (int, align 4) needs an offset that is a multiple of 4; the next such offset after byte 8 is 12, so 3 bytes of padding go at 9–11 and `b` occupies 12–15.
Step 4 (you): State the struct's alignment, whether any tail padding is needed to round the size up to a multiple of that alignment, the final `sizeof`, and the total padding bytes. Then name the one function you would call to confirm each member's offset.

7.9 MIXED For each symptom, decide which chapter's mechanism is the primary cause and justify in a line: (a) `sizeof(param)` inside a function returns 8 no matter how large the caller's array was; (b) a random-access loop over an 800 MiB table is far slower than ~100 ns/access predicts, and huge pages speed it up; (c) a struct is unexpectedly 24 bytes when its fields sum to 14; (d) reading a binary file written on a different architecture yields byte-swapped integers.

7.10 MIXED Without running code, decide for each what value (or category of value) is produced and why, drawing on Chapters 4–7: (a) the number stored in a pointer — is it a physical or virtual address? (b) `&arr[5] - &arr[2]` for an `int arr[10]`; (c) whether the absolute address printed by a PIE build is stable across runs, and what causes the answer; (d) whether `sizeof(int)` is guaranteed by the C standard to be 4.

7.11 STRETCH The chapter says pointer arithmetic is only defined within a single array object (plus one-past-the-end). (a) Explain why `&arr[10]` for `int arr[10]` is legal to *form* but not to *dereference*. (b) Give a plausible reason the standard makes going further undefined rather than defining it as "wraps" or "reads adjacent memory" (think about the optimizer and about Chapter 4's segmented address space). (c) Connect this to the out-of-bounds bug you will meet in Chapter 9.

7.12 LAB On your own machine, write and run a program that prints: (a) `sizeof` and `_Alignof` for `char, short, int, long, double, void*`; (b) the `offsetof` of every member and the `sizeof` of a struct of your choice with at least three differently-aligned fields, and verify it against your hand computation; (c) the byte order of a known 32-bit value to determine your machine's endianness. Report *your* measured numbers and label the machine/ABI; note any that differ from this chapter's x86-64 values and why they might.

Chapter 8 — The Stack & the Heap

Before you start: Chapter 7 (a pointer is an address; the address-space regions, including stack and heap), Chapter 4 (virtual memory), Chapter 5 (locality). · **Pairs with:** Chapter 9 (the heap's controls, `malloc`/`free`). Chapter 7 named the regions; this chapter takes the two with real lifetimes — the **stack** and the **heap** — and answers where an object should live, what each costs, and how a pointer that outlives its storage becomes one of C's classic bugs.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 7 — of the five address-space regions, which two grow *toward* each other, and in which directions? (2) Chapter 4 — a local variable and a `malloc`'d block both have *virtual* addresses; does that mean they have the same *lifetime*? Guess what determines when each stops being valid. (3) *Predict:* a function creates a local `int n`, and returns `&n` (its address) to the caller. The caller then reads through that pointer. Do you expect that to reliably give back the value of `n` — and if not, what has gone wrong by the time the caller looks? By the end you will be able to state precisely why this is a bug.

Every object in a C program has a **storage duration** — a lifetime — and getting lifetimes wrong is the source of a whole family of bugs that the previous chapter's "a pointer is just an address" model does not, by itself, warn you about. A pointer stays a valid handle only as long as the object it points at is alive; the moment that object's lifetime ends, the pointer becomes **dangling**, and using it is undefined behavior. This chapter is about the two storage durations you manage most directly. **Automatic** storage — the stack — is created and destroyed for you as functions run, is essentially free to allocate, and has a lifetime tied strictly to a block's execution. **Dynamic** storage — the heap — you allocate and free by hand, is managed by a real allocator with real cost, and lives exactly as long as you keep it alive. We will see how the call stack works and why pushing a frame is nearly free; why the heap is different and must be managed; what stack overflow is; and then the payoff — the lifetime rules that make "return a pointer to a local" a bug, shown correct and broken, and compiled to see what actually happens.

8.1 Automatic storage: the call stack and stack frames

When a function is called, the machine needs somewhere to put that call's local variables, its arguments, the address to return to when it finishes, and some bookkeeping. That somewhere is a **stack frame** (or activation record), pushed onto the **call stack** — the stack region from Chapter 7. The stack is a last-in-first-out structure that mirrors the call structure of the program: calling a function *pushes* a new frame on top; returning *pops* it off. Because calls nest (a function calls a function calls a function) and returns unwind in the reverse order, LIFO is exactly the right discipline, and the region grows *downward* (toward lower addresses) as frames stack up, as Chapter 7's address printout showed the stack sitting high and the CPU pushing toward the heap.

The hardware makes this cheap. A CPU register, the **stack pointer**, holds the address of the current top of the stack. Allocating a frame for a new call is, in essence, *subtracting* the frame's size from the stack pointer — moving it down to reserve that many bytes — and deallocating on return is adding it back. That is the sense in which stack allocation is "free": reserving space for all of a function's locals is a single arithmetic adjustment of one register, not a search or a bookkeeping operation. There is no per-variable cost and nothing to track; the locals are simply the bytes between the old and new stack-pointer positions. CS:APP (Chapter 3) develops the exact frame layout and the call/return instruction sequence; the shape to hold is: **a call pushes a frame by bumping a pointer, a return pops it by bumping the pointer back, and the whole frame's storage appears and vanishes together.**

Two consequences follow immediately. First, automatic variables have a lifetime tied to their block: they come into existence when execution enters the block (or function) and **cease to exist when execution leaves it** — the frame is popped, and those bytes are no longer reserved for them (C standard §6.2.4, automatic storage duration). Second, that is why locals are so cheap and why you do not free them: the pop reclaims the entire frame at once. The flip side — the danger — is that the storage is reclaimed *whether or not any pointer still refers to it*. Hold onto that; it is the whole of §8.4.

> **QUICK CHECK**
>
> Why is allocating a function's local variables often described as "free," in terms of what the CPU actually does to reserve the space? And what event reclaims all of a function's locals at once? (Answer after the next section.)

8.2 Dynamic storage: the heap, and why it must be managed

The stack's discipline — allocate on call, free on return, strict LIFO — is exactly what makes it cheap, and also what makes it insufficient. Plenty of data does not follow call structure: a buffer whose size you only learn at runtime, a data structure that must outlive the function that built it, a result you want to hand back to a caller. For these you need storage whose lifetime you control independently of the call stack. That is the **heap**, and you obtain it through the allocation functions `malloc` and friends (the subject of Chapter 9): `malloc(n)` asks for *n* bytes and returns a pointer to them; that block stays alive until *you* explicitly `free` it, across as many function calls and returns as you like.

This freedom is not free. Unlike the stack — where "allocate" is a pointer bump and "free" is a pointer bump back — the heap must track, out of a large pool, which regions are in use and which are available, in whatever arbitrary order you allocate and free them. That tracking is the job of the **allocator**, a nontrivial piece of software (typically inside the C library) that maintains data structures — free lists, size classes — to satisfy requests and reclaim freed blocks. Because allocations and frees can interleave in any order, the allocator must search for a suitable free block, may split or coalesce blocks, and can leave the pool **fragmented** (free space chopped into pieces too small to be useful). So a heap allocation is a genuine function call doing real work, orders of magnitude more expensive than a stack allocation, and the memory it returns must be explicitly returned or it is **leaked**. Chapter 9 opens the allocator up; for now the contrast is the point.

THE SAME IDEA THREE WAYS: STACK VS. HEAP

1. In words. The stack is automatic storage with LIFO lifetime tied to calls, allocated/freed by bumping the stack pointer — nearly free, but a local dies when its function returns. The heap is manually managed storage with a lifetime *you* control, allocated by a real allocator — flexible, but costly and yours to free.

2. As a picture. Figure 8.1: the stack, a tidy pile of frames growing down, each pushed/popped whole; the heap, a pool of scattered blocks (some in use, some free) that an allocator hands out and reclaims in any order.

3. As a table. Stack: allocate = pointer bump, free = automatic on return, lifetime = the block, cost ≈ free, failure = overflow. Heap: allocate = `malloc` (search/split), free = manual `free`, lifetime = until you free it, cost = real, failures = leak / use-after-free / double-free (Ch 9).

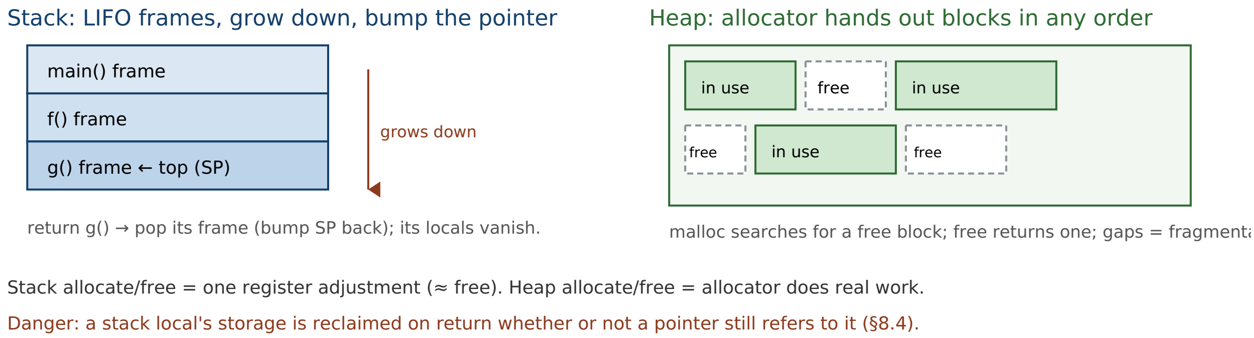

Figure 8.1: The stack pushes/pops whole frames by bumping a pointer (nearly free, LIFO lifetime); the heap is a managed pool the allocator hands out and reclaims in any order (flexible, costly, yours to free).

8.3 Stack overflow

The stack is fast but **bounded**: the OS reserves a finite region for it (commonly on the order of a few megabytes per thread by default on Linux, though the exact size is configurable and platform-dependent — check `ulimit -s` rather than assume a number). If the program keeps pushing frames without returning — most often through unbounded or runaway **recursion**, or by declaring a very large local array — the stack pointer eventually runs past the end of that reserved region. This is **stack overflow**: the access falls on a page with no valid mapping (often a deliberately unmapped *guard page*), the hardware faults, and the OS typically terminates the process (on Linux, a segmentation fault). The two everyday causes are worth naming: recursion with no base case or too deep a recursion for the input, and large automatic arrays (declaring `int buf[10000000];` as a local tries to reserve tens of megabytes on a few-megabyte stack). The fixes match the causes: bound the recursion (or convert it to iteration with an explicit heap-backed structure), and put large or runtime-sized buffers on the *heap*, where the space is far larger and the size can be chosen at runtime. Stack overflow is the stack's characteristic failure, just as leaks and use-after-free (Ch 9) are the heap's.

(Answer to the §8.1 quick check: stack allocation is "free" because reserving space for a call's locals is just subtracting the frame size from the stack-pointer register — one arithmetic adjustment, no per-variable work. Returning from the function — popping the frame by moving the stack pointer back — reclaims all of that function's locals at once.)

8.4 Lifetimes and the dangling-pointer bug: why you can't return a pointer to a local

Now the payoff, and one of the most important bugs in this book. Because a stack frame is reclaimed the instant its function returns (§8.1), **a pointer to a local variable is only valid until that function returns.** After the return, the local's storage is no longer reserved for it — the frame has been popped, the stack pointer moved back, and those bytes are free to be reused by the very next function call. A pointer still holding that address is a **dangling pointer**: it names storage whose lifetime has ended. Reading or writing through it is **undefined behavior** (C standard §6.2.4: the lifetime of an object with automatic storage duration ends when execution leaves its block; using a pointer to an object past its lifetime is UB). This is the answer to the opener's prediction — returning `&n` and dereferencing it later does *not* reliably give back `n`, because `n` no longer exists.

Here is the bug and its correct counterpart, side by side:

```
/* BUG: returns the address of a local; the frame is gone on return. */
int *make_counter_bug(void) {
    int n = 0;
    return &n;          /* dangling: n dies when this function returns */
}

/* CORRECT: the caller owns the storage; the callee just fills it in. */
void fill_counter(int *out) {
    *out = 0;            /* writes into storage the caller keeps alive */
}

int main(void) {
    int *p = make_counter_bug();
    printf("%d\n", *p);  /* undefined behavior: *p reads a dead local */

    int n;
    fill_counter(&n);    /* n lives in main's frame, valid throughout */
    printf("%d\n", n);   /* well-defined */
}
```

Two things about the buggy version deserve care. First, **the compiler warns you.** Built with `gcc -Wall` on this machine, `make_counter_bug` produces `warning: function returns address of local variable [-Wreturn-local-addr]`; clang emits the analogous `-Wreturn-stack-address`. This is a bug the toolchain actively flags — heed it. Second, **what happens at runtime is not guaranteed**, and this is the crux of undefined behavior. On this machine, compiling the buggy program (at both `-O0` and `-O2`) and running it produced a **segmentation fault** (the process crashed). But that outcome is *not* part of any contract: UB means the standard imposes no requirements at all, so a different compiler, optimization level, or run could instead print `0`, print stale garbage, or appear to "work"

— which is worse, because the latent bug hides until conditions change. Do *not* reason "it segfaulted, so at least it's detectable": you cannot rely on the crash any more than on the value. The correct fix, shown on the right, is the standard pattern — **the caller owns the storage and passes a pointer in** (`fill_counter(&n)`), so the object outlives the callee's frame; or, when the callee must create the object, allocate it on the *heap* and return that pointer (the caller then frees it — Ch 9). Either way the rule is the same: *never hand out a pointer whose target will die before the pointer does.*

TRAP: RETURNING (OR STORING) A POINTER TO A STACK LOCAL

The recurring lifetime bug: a function returns `&local`, or stores `&local` into a longer-lived structure (a global, an out-parameter's target, a caller's list), and something reads through that pointer after the function has returned. The local's frame is gone, so the pointer dangles and the access is undefined behavior — it may crash, may return stale or overwritten bytes, or may *appear* to work until the next call reuses the frame, which is the dangerous case because it hides in testing. The tells: returning the address of a local or parameter; taking `&` of a local and stashing it somewhere that outlives the function; a pointer that "worked yesterday" now returning garbage after unrelated code changed the call pattern. Heed `-Wreturn-local-addr` / `-Wreturn-stack-address` — the compiler catches the direct return. The fixes: return by value for small objects; take an out-pointer the caller owns; or heap-allocate and transfer ownership (documenting who frees). And do not trust a crash to reveal it — UB is not obligated to crash, so the discipline is to reason about lifetime, not to test-and-see.

QUICK CHECK

Two functions each need to give the caller an `int` the caller can keep: one returns `&local`; the other takes `int *out` and writes `*out`. Which one is correct and why? Where does the correct one's storage actually live? (Answer below the next section.)

8.5 Choosing where an object should live

The stack/heap choice, made well, is mostly mechanical once you ask two questions: *how long must this object live,* and *how big is it (and is that known at compile time)?* Prefer the **stack** when the object's lifetime fits within the function that creates it and its size is known and modest — this is the common case, and it is free, automatically cleaned up, and cache-friendly (a frame's locals are hot and contiguous). Reach for the **heap** when the object must *outlive* the function that creates it (you need to return it, or store it in a longer-lived structure), when its size is only known at runtime or is too large for the stack (§8.3), or when its lifetime genuinely does not nest with call structure (shared data structures, caches, anything with a dynamic graph of owners). The cost of the heap is not only the allocator's time (§8.2) but the obligation it creates: *someone must free it, exactly once, after the last use* — and getting that wrong is the entire subject of the next chapter. This is why the guidance leans toward the stack: it removes the question. The senior habit is to make lifetime an explicit, stated design decision — "this object lives on the stack in `f`," or "the heap owns this and `g` frees it" — rather than reaching for `malloc` reflexively or letting ownership stay implicit.

(Answer to the §8.4 quick check: the `int *out` version is correct — it writes into storage the caller already owns and keeps alive (a local in the caller's frame, or wherever the caller got the pointer), so the object outlives the callee's return. The `&local` version returns a dangling pointer to a frame that is popped on return. The correct one's storage lives in the caller's frame, not the callee's.)

> **SAY IT LIKE A SYSTEMS ENGINEER**
>
> *Try it first.* In code review, someone defends a helper that does `char *label(int id) { char buf[32]; sprintf(buf, "id-%d", id); return buf; }` with "it compiles and it worked in my test." Cover the paragraph and explain the problem in two or three sentences, using *stack frame*, *lifetime*, and *undefined behavior*.
>
> *Model answer.* "`buf` is an automatic array in `label`'s stack frame, so its lifetime ends the instant `label` returns — the returned pointer dangles, and any read through it is undefined behavior. It 'worked in your test' only by luck: UB isn't required to crash, so the frame's bytes happened to still hold the string until something reused them, which is exactly the bug that hides until the call pattern changes. The compiler will even warn with `-Wreturn-local-addr`. Fix it by having the caller pass in a buffer it owns — `label(id, char *out, size_t n)` — or by returning a heap allocation the caller frees." Naming the frame's lifetime and calling the crash *unreliable* — rather than "it segfaults sometimes" — is the senior register, and it points at the real fix: make ownership explicit.

8.6 What to carry forward

Objects have lifetimes, and a pointer is only valid while its target lives. **Automatic (stack) storage** is created and destroyed with function calls: a frame is pushed by bumping the stack pointer down and popped by bumping it back, which is why stack allocation is nearly free and why a local's lifetime ends exactly when its function returns. **Dynamic (heap) storage** is managed by a real allocator, costs real time, lives until you free it, and exists precisely for objects that must outlive their creating call or whose size or lifetime does not fit the stack's LIFO discipline. The stack's characteristic failure is **overflow** (runaway recursion or huge locals); the heap's failures are the next chapter's. And the through-line bug of this chapter — **returning or storing a pointer to a stack local** — is undefined behavior because the storage is reclaimed on return whether or not a pointer still refers to it; the compiler warns, the crash is *not* guaranteed, and the fix is to make the storage outlive the pointer (caller-owned or heap-owned).

Chapter 9 picks up the heap's controls in full: what `malloc`, `calloc`, `realloc`, and `free` actually promise; what the allocator is doing underneath (free lists, size classes, fragmentation); and the precise catalogue of manual-memory bugs — use-after-free, double-free, leak, buffer overflow, uninitialized read — each stated exactly as the undefined (or indeterminate) behavior it is, with the real tools that catch them. The dangling-pointer idea you just learned on the stack reappears there on the heap as *use-after-free*: the same bug — a pointer outliving its storage — in the region where *you* control the lifetime, and therefore *you* can get it wrong.

CAN YOU… WITHOUT LOOKING?

- Explain what a **stack frame** is, why a call pushing one and a return popping one is nearly **free**, and what a local's **lifetime** is tied to.
- Say why the **heap** must be **managed** (an allocator tracking free space in arbitrary allocate/free order) and how that makes it costlier than the stack.
- Define **stack overflow**, name its two common causes, and give the matching fixes.
- State precisely why **returning a pointer to a local** is a bug — in terms of frame lifetime — and why it is **undefined behavior** rather than "returns garbage."
- Explain why a crash from that bug is *not* guaranteed, and give the two correct patterns (caller-owned storage, or heap-allocate-and-transfer-ownership).
- Decide, for a given object, whether it belongs on the **stack** or the **heap**, from its lifetime and size.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is a stack frame, and in what sense is allocating one "free"? What reclaims a function's locals? (§8.1)
2. **R2.** Why must the heap be managed by an allocator, and name two costs that follow. (§8.2)
3. **R3.** What is stack overflow, what are its two usual causes, and how do you fix each? (§8.3)
4. **R4.** Precisely why is returning a pointer to a local undefined behavior, and why is a crash *not* guaranteed? (§8.4)
5. **R5.** Give two questions that decide whether an object belongs on the stack or the heap, and the correct fix for a callee that must hand the caller an object it created. (§8.4–8.5)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP)**, Chapter 3 (procedure calls, the stack frame, the call/return sequence and stack-pointer mechanics). The reference for how frames are pushed and popped; the free CMU 15-213 materials cover it.
- **Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP)**, the address-space and memory-API chapters (free full text). The OS view of the stack and heap regions, stack limits and guard pages, and the allocation interface.
- **cppreference.com** and the **C standard** — storage duration (§6.2.4) and object lifetime. The authority for when an automatic object's lifetime ends and why using a pointer past it is undefined.
- **Beej's Guide to C** (free). Practical treatment of the stack, the heap, and the common lifetime mistakes, with runnable examples.

Exercises

8.1 WARM-UP In one sentence each: (a) what is a stack frame? (b) what CPU operation allocates one? (c) when does an automatic (local) variable's lifetime end?

8.2 WARM-UP Name one reason to allocate on the heap rather than the stack, and one cost the heap imposes that the stack does not.

8.3 WARM-UP Give the two most common causes of stack overflow, and the matching fix for each.

8.4 CORE Explain, in terms of stack frames and the stack pointer, precisely why this is a bug: `int *f(void) { int x = 7; return &x; }`. State what happens to `x`'s storage on return, why using the returned pointer is undefined behavior, and what warning gcc/clang emit.

8.5 CORE Rewrite `int *f(void) { int x = 7; return &x; }` two correct ways: (a) so the *caller* owns the storage; (b) so the object lives on the heap. For (b), state who is responsible for freeing it and when.

8.6 CORE A colleague says "my function returns a pointer to a local and it works fine — I tested it." Explain why "it worked" is *not* evidence the code is correct, using the definition of undefined behavior. What is the danger of a bug that happens to appear to work?

8.7 COST Compare the cost of one stack allocation vs. one heap allocation. (a) In terms of operations, what does the machine do to allocate a stack frame's worth of locals? (b) What must an allocator potentially do to satisfy one `malloc`? (c) Given (a) and (b), state a rule of thumb for a hot loop that needs a small scratch buffer each iteration, and justify it.

8.8 FADED *Worked, with the last step for you.* Decide where a function's result should live for: a helper that builds a linked list of unknown length and returns it to its caller. Steps 1–3 given; complete step 4.
Step 1 (given): The list must *outlive* the helper that builds it — the caller uses it after the helper returns.
Step 2 (given): Its size (number of nodes) is only known at runtime.
Step 3 (given): Both facts rule out automatic (stack) storage: the nodes would die on return, and the count is not fixed at compile time.
Step 4 (you): State where the nodes must therefore be allocated, who becomes responsible for freeing them and when, and what bug (from the next chapter's preview) results if that responsibility is forgotten. Name the design principle: who owns the memory?

8.9 MIXED For each symptom, decide which mechanism from Chapters 4–8 is the primary cause and justify in a line: (a) a recursive function crashes with a segfault only on large inputs; (b) a function returns a pointer that gives correct values in unit tests but garbage in production after unrelated code was added; (c) a random-access loop over a huge array is slower than DRAM latency predicts and huge pages help; (d) a struct is larger than the sum of its fields.

8.10 MIXED Without running anything, classify each as *well-defined*, *undefined behavior*, or *a performance issue only*, and say why, drawing on Chapters 7–8: (a) taking `&local` and using it *within* the same function; (b) returning `&local` and dereferencing it in the caller; (c) declaring `int big[8000000];` as a local and using it; (d) storing `&local` into a global and reading it after the function returns.

8.11 STRETCH The chapter draws a parallel: a dangling pointer to a stack local (this chapter) and a use-after-free on the heap (Chapter 9) are "the same bug in different regions." (a) State the one property both violate. (b) Explain why the stack version is in some ways *easier* to catch (name the compiler warning) while the heap version generally is not. (c) Argue why "the storage is reclaimed whether or not a pointer still refers to it" is the root of *both*.

8.12 LAB On your own machine, compile (with `-Wall`) the buggy `int *f(void){ int x=7; return &x; }` and a `main` that dereferences the result. (a) Record the exact compiler warning. (b) Run it and report what happens on *your* machine (crash, a value, garbage) — and note that this outcome is not guaranteed by the standard. (c) Then write the caller-owned-storage fix and confirm it is warning-free and correct. Report only what you actually observed, and state why (b) must not be relied upon.

Chapter 9 — Manual Allocation: `malloc`/`free` and What Goes Wrong

Before you start: Chapter 8 (stack vs. heap; the dangling-pointer bug; lifetimes), Chapter 7 (pointers, arrays, bounds), Chapter 5 (locality). · **Closes the core of Part 2** and sets up **Rust** (Part 5), where the compiler enforces by rule what this chapter makes you do by hand. Here are the heap's controls — `malloc`, `calloc`, `realloc`, `free` — what the allocator does underneath, and the precise catalogue of ways manual memory management goes wrong, each stated as the undefined behavior it is.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 8 — why is a pointer to a stack local invalid after its function returns, and what one word names a pointer whose target's lifetime has ended? (2) Chapter 7 — pointer arithmetic and indexing are only defined *within* an array (plus one-past-the-end). What is stepping outside those bounds, in the language's terms? (3) *Predict:* you `free(p)` a heap block, then a moment later read `p[0]` anyway. What do you think happens — a guaranteed crash, the old value still there, or something the standard refuses to define? By the end you will be able to name this bug and say precisely why its behavior is not guaranteed.

The heap gives you storage whose lifetime you control — and hands you, in exchange, the entire responsibility for managing it correctly. No garbage collector will reclaim what you forget; no bounds checker will stop you writing past the end; no runtime will notice you using a block after you freed it. C gives you the raw controls and trusts you completely, which is exactly why this chapter matters: the bugs here are not "the program crashes," they are **undefined behavior**, and understanding that distinction precisely is the difference between a mysterious, occasional failure and a controllable one. We will cover the four allocation functions and what each actually promises; sketch what an allocator does underneath (free lists, size classes, fragmentation — at the level this book needs); then catalogue the canonical memory bugs — use-after-free, double-free, memory leak, buffer overflow, uninitialized read — each stated exactly, with why it is dangerous going beyond "it crashes." Finally we meet the real tools that catch them, and run one — AddressSanitizer — on this machine to see what it reports. This is the chapter that most directly motivates Rust: every rule here that you must hold in your head, Rust's type system checks for you.

9.1 The allocation functions and what they promise

Four functions from `<stdlib.h>` make up the manual-allocation interface. Their *documented* behavior (cppreference; the C standard §7.22.3) is precise, and precision matters because the failure modes live in the gaps:

- **`void *malloc(size_t size)`** — allocates `size` bytes and returns a pointer to the block, suitably aligned for any object type. **The contents are uninitialized** — indeterminate bytes, *not* zeroed. Returns `NULL` on failure (and if `size` is 0, returns either `NULL` or a unique pointer you may not dereference). You must check the return value before using it.

- **`void *calloc(size_t n, size_t size)`** — allocates space for `n` objects of `size` bytes each, **initialized to all-zero bytes**. Because it takes the count and element size separately, mainstream implementations guard the `n * size` multiplication against overflow (returning `NULL` rather than silently under-allocating) — which is a practical reason to prefer it over `malloc(n * size)` when allocating an array. Use it when you want zeroed memory or are allocating an array.
- **`void *realloc(void *ptr, size_t size)`** — resizes the block `ptr` previously returned to `size` bytes, preserving the contents up to the smaller of old and new sizes. It *may move* the block (returning a new address), so you must assign its result — `p = realloc(p, n)` loses the block on failure; the safe idiom uses a temporary. On failure it returns `NULL` and leaves the *original* block valid and unchanged. `realloc(NULL, size)` behaves like `malloc(size)`.
- **`void free(void *ptr)`** — returns a block previously obtained from `malloc`/`calloc`/`realloc` to the allocator. `free(NULL)` is explicitly a **no-op** (always safe). Passing anything else — a pointer not from the allocator, or one already freed — is **undefined behavior**.

The contract, stated as rules you must uphold: allocate, check for `NULL`, use only within the requested size, free exactly once when done, and never touch the block after freeing. Every bug in §9.3 below is a violation of one of these — and the language does not enforce a single one of them for you.

> **QUICK CHECK**
>
> Does `malloc` zero the memory it returns? Does `calloc`? And why is `p = realloc(p, n)` a dangerous idiom — what do you lose if the `realloc` fails? (Answer after the next section.)

9.2 What an allocator does, conceptually

Between your `malloc` call and the memory you get back sits the **allocator** — usually part of the C library — managing a large pool of memory it obtained from the OS. You do not need its internals to use it, but a conceptual picture explains the costs and one of the bugs (fragmentation). The allocator's job is to satisfy variable-sized requests, in arbitrary allocate/free order, out of that pool. To do so it tracks which regions are free, typically via **free lists**: linked lists of available blocks it searches to find one big enough for a request. To make that search fast, modern allocators group free blocks into **size classes** — separate lists for "16-byte blocks," "32-byte blocks," and so on — so a request goes straight to a list of right-sized blocks instead of scanning one giant list. When a block is freed, the allocator may **coalesce** it with adjacent free blocks into a larger one, and it records bookkeeping (typically a small header near each block) so `free` knows the block's size.

Two consequences matter for this book. First, **cost**: because a request may involve searching a list, splitting a block, or (when the pool is exhausted) asking the OS for more memory, a heap allocation is far more expensive than the stack's pointer-bump (Ch 8) —

which is why hot loops avoid per-iteration `malloc`. Second, **fragmentation**: after many allocations and frees of varying sizes, the free space can end up chopped into many small, non-adjacent pieces. You may have plenty of total free memory yet be unable to satisfy a large request because no single free block is big enough — this is *external* fragmentation. (There is also *internal* fragmentation: rounding a request up to a size class wastes the slack.) Fragmentation is not undefined behavior — it is a performance and capacity problem — but it is a real cost of manual allocation, and it is why allocation patterns (many small short-lived blocks vs. few large long-lived ones) affect a program's memory footprint. CS:APP (Chapter 9) develops allocator design in depth, including the free-list and coalescing mechanics; this book keeps the model conceptual.

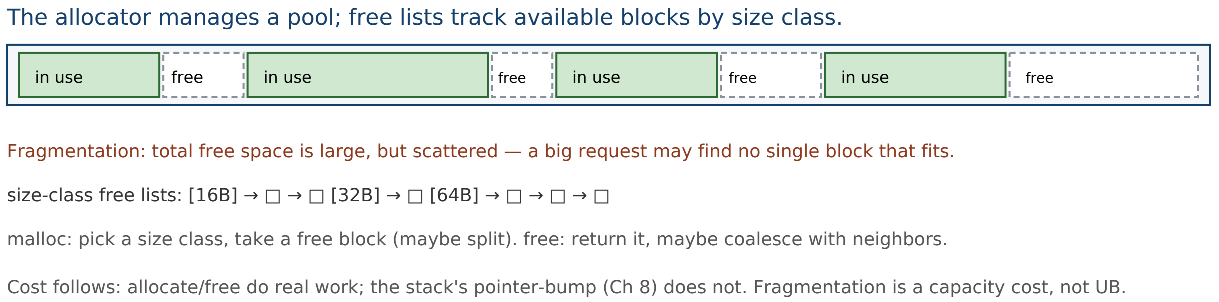

Figure 9.1: An allocator satisfies variable-sized requests from a pool using size-classed free lists, splitting and coalescing blocks. Interleaved allocate/free of varying sizes leaves external fragmentation — free space too scattered to satisfy a large request.

*(Answer to the §9.1 quick check: `malloc` does **not** zero its memory — the contents are indeterminate; `calloc` **does** zero it. `p = realloc(p, n)` is dangerous because if `realloc` returns `NULL` on failure, you have overwritten your only pointer to the still-valid original block — leaking it; assign to a temporary and check it first.)*

9.3 The canonical memory bugs — each stated precisely

Now the catalogue. The critical framing, which this whole book has been building toward: most of these are **undefined behavior (UB)**, a specific term from the C standard (§3.4.3) meaning behavior "for which this International Standard imposes *no requirements*." That is stronger and stranger than "it crashes." Under UB the program may crash, may produce wrong results, may appear to work, may corrupt unrelated data, may do different things on different runs or compilers — and, crucially, the optimizer is permitted to assume UB *never happens*, so code near a latent UB bug can be transformed in surprising ways. It is worth distinguishing three standard categories you will meet:

- **Undefined behavior** (§3.4.3): no requirements at all — anything may happen (e.g. use-after-free, out-of-bounds access, double-free).
- **Unspecified behavior** (§3.4.4): the standard offers two or more possibilities and does not say which you get (e.g. the order function arguments are evaluated) — but each possibility is a valid, non-catastrophic outcome.
- **Implementation-defined behavior** (§3.4.1): unspecified behavior that each implementation must *document* (e.g. `sizeof(int)`, byte order from Ch 7).

The bugs below are firmly in the first category (with uninitialized reads a closely related "indeterminate value" case). Do not read any of the realistic outcomes as guarantees — they are what *tends* to happen, offered only so you recognize the symptom.

Use-after-free. Reading or writing a block after you have `free`d it. The pointer still holds the old address, but the block's lifetime has ended (it may already be handed to another allocation). This is exactly Chapter 8's dangling-pointer bug, now on the heap, and it is **undefined behavior**. Realistically it may read stale data, read data that now belongs to an unrelated allocation, corrupt the allocator's bookkeeping, or crash — and it is a serious security vulnerability class, because an attacker who controls what gets allocated into the freed slot can influence what your dangling pointer reads or writes. This is the answer to the opener: reading `p[0]` after `free(p)` is not a guaranteed crash and not a guaranteed old value — it is UB.

Double-free. Calling `free(p)` twice on the same block (without a reallocation in between). Also **undefined behavior**: the second `free` corrupts the allocator's internal structures (it tries to reclaim a block already on a free list), which can crash immediately, silently corrupt the heap so a *later* unrelated allocation misbehaves, or — again — be exploited. Note `free(NULL)` is *not* a double-free and is always safe; the bug is freeing the same non-null block twice.

Memory leak. Losing the last pointer to a heap block without freeing it, so the memory can never be reclaimed. This one is *not* undefined behavior — the program stays well-defined — but it is a real defect: a long-running program that leaks steadily grows its memory footprint until it exhausts memory and is killed or drags the system into swap. Leaks are the mirror of use-after-free: use-after-free frees too *early* (while a pointer still needs it), a leak frees too *late* (never).

Buffer overflow (out-of-bounds access). Reading or writing outside the bounds of an allocation — `a[4]` on a 4-element array, or writing 20 bytes into a 16-byte block. Recall from Chapter 7 that pointer arithmetic and indexing are only defined within the array (plus one-past-the-end); going past that is **undefined behavior**. An out-of-bounds *write* is especially dangerous: it can overwrite adjacent data, allocator metadata, or (on the stack) a return address — the classic memory-corruption vulnerability behind a large fraction of security exploits. An out-of-bounds *read* can leak adjacent memory (the Heartbleed class of bug).

Uninitialized read. Reading memory before anything has been written to it — a `malloc`'d block (whose contents are indeterminate, §9.1) or an uninitialized local (Ch 8) — and using the value. Using such an **indeterminate value** is at best unspecified and, in general (for objects whose address is never taken, or trap representations), undefined; either way the value is meaningless and the behavior unreliable. The realistic symptom is a bug that changes with optimization level, unrelated code, or run-to-run — because the "value" is whatever bytes happened to be there. The fix is trivial and non-negotiable: initialize before you read (or use `calloc` for zeroed memory).

THE SAME IDEA THREE WAYS: THE MEMORY BUGS ARE LIFETIME/BOUNDS VIOLATIONS

1. In words. Every bug here breaks one of §9.1's rules: touch a block outside its *lifetime* (use-after-free, double-free), outside its *bounds* (buffer overflow), before it is *initialized* (uninitialized read), or never end its lifetime (leak). Most are undefined behavior — no guaranteed outcome.

2. As a picture. A timeline of one block: `malloc` (born, contents garbage) → initialize → use → `free` (dead). Reading before "initialize" = uninitialized read; touching after "dead" = use-after-free; freeing "dead" again = double-free; never reaching "dead" = leak; touching left/right of the block's extent = overflow.

3. As UB, not "a crash." Each undefined case may crash, corrupt, leak data, or seem fine — and may differ by compiler, optimization, or run. The tools in §9.4 turn these unreliable, sometimes-invisible faults into loud, located errors.

TRAP: ASSUMING UNDEFINED BEHAVIOR IS DETERMINISTIC ("IT DIDN'T CRASH, SO IT'S FINE")

The most dangerous misconception in manual memory management is treating a memory bug as if it had a reliable outcome — "the use-after-free returns the old value," "the overflow just writes the next byte," "it ran fine, so it's correct." **Undefined behavior means the standard imposes no requirements** (C §3.4.3): the outcome is not defined, may vary with compiler/optimization/run, and the optimizer may assume the bug cannot occur and transform your code accordingly — so a UB bug can even change behavior elsewhere in the function. A program that "works in testing" with a latent use-after-free or overflow is not correct; it is *unexploded*. The tells: relying on a specific value or crash from a bug; "it only fails in release builds" or "only under load" or "only on the other machine" — all classic UB signatures. The discipline: reason about lifetimes and bounds to *prevent* these (§9.1's rules), and use the sanitizers/Valgrind of §9.4 to *detect* them — never conclude "no crash = no bug." This is precisely the guarantee Rust's type system will provide by construction in Part 5.

9.4 Tools that catch these bugs — and one run on this machine

Because these bugs are unreliable by nature, you do not hunt them by staring; you use tools that instrument memory accesses and turn a silent UB into a loud, located report. Two real, widely used ones:

- **AddressSanitizer (ASan)** — a compiler feature (in gcc and clang) enabled with `-fsanitize=address`. It instruments the program to check every memory access against "shadow memory" and surrounds allocations with *redzones*, catching heap and stack buffer overflows, use-after-free, double-free, and (with its LeakSanitizer component) leaks — reporting the bug's type, address, and source location, with a modest runtime slowdown. It detects *addressability* bugs (touching memory you shouldn't); it does *not* catch reads of uninitialized-but-valid memory — that is MemorySanitizer's job (clang), or Valgrind's.
- **Valgrind (Memcheck)** — a runtime instrumentation tool that runs your unmodified binary on a synthetic CPU, checking each access. It detects use-after-free, invalid frees, leaks, out-of-bounds on heap blocks, and — unlike ASan — use of uninitialized

values, at a larger slowdown. (Valgrind was not installed on this machine, so the runs below use ASan.)

To make this concrete, four tiny buggy programs were compiled with `gcc -fsanitize=address -g` and run on this machine. ASan's actual reports (trimmed to the key lines):

```
/* use-after-free: read p[0] after free(p) */
==ERROR: AddressSanitizer: heap-use-after-free ... READ of size 4
  0x... is located 0 bytes inside of 16-byte region [freed by thread T0 here ...]
SUMMARY: AddressSanitizer: heap-use-after-free  uaf.c:6 in main

/* buffer overflow: a[4] = 7 on a 4-int (16-byte) block */
==ERROR: AddressSanitizer: heap-buffer-overflow ... WRITE of size 4
  0x... is located 0 bytes to the right of 16-byte region [0x..10,0x..20)
SUMMARY: AddressSanitizer: heap-buffer-overflow  oob.c:4 in main

/* double-free: free(p); free(p); */
==ERROR: AddressSanitizer: attempting double-free ...
  [freed by thread T0 here ...]

/* memory leak: malloc(1024), never freed */
==ERROR: LeakSanitizer: detected memory leaks
Direct leak of 1024 byte(s) in 1 object(s) allocated from: ... leak.c:3
```

Every bug was caught, named, and pinned to a source line — the whole point of the tools. For contrast, the **uninitialized read** (printing an uninitialized `int`) ran under ASan *without* an error, printing a garbage value — confirming ASan does not cover that class (you would reach for Valgrind or MemorySanitizer, or catch the simple case at compile time: `gcc -Wall` flags it with `-Wuninitialized`). The lesson is twofold: these tools convert probabilistic, sometimes-invisible UB into deterministic, located failures — run your tests under a sanitizer — and no single tool catches everything, so know which catches which. **Outputs above are from this specific machine and toolchain (gcc 11 on x86-64); exact addresses and wording vary by version.**

> **QUICK CHECK**
>
> Which of these does AddressSanitizer catch: use-after-free, heap buffer overflow, double-free, memory leak, uninitialized read? Which one is the odd one out, and what tool would you use for it? (Answer below the communicate box.)

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate closes a bug report with "I can't reproduce the crash — it runs fine for me, so I think it's a flaky test." The code has a use-after-free. Cover the paragraph and respond in two or three sentences, using *undefined behavior* and naming a tool.

Model answer. "A use-after-free is undefined behavior, so 'runs fine for me' is exactly what we'd expect some of the time — the standard makes no promise about the outcome, and it can depend on the build, the timing, or what got allocated into the freed slot, which is why it's intermittent rather than flaky. Instead of trying to reproduce by luck, let's build the tests with `-fsanitize=address` (or run under Valgrind); ASan will turn the use-after-free into a deterministic, located error pointing at the exact line. 'Can't reproduce' isn't evidence the bug is gone — with UB, no crash doesn't mean no bug." Naming the behavior as UB and reaching for a sanitizer — rather than accepting "flaky" — is the senior register, and it converts an intermittent mystery into a caught, fixable defect.

(Answer to the §9.4 quick check: AddressSanitizer catches use-after-free, heap buffer overflow, double-free, and (via LeakSanitizer) memory leaks. The odd one out is the ***uninitialized read*** *— ASan does not detect it; use Valgrind or MemorySanitizer, or catch simple cases with `gcc -Wall` (`-Wuninitialized`).)*

9.5 What to carry forward — and the bridge to Rust

Manual allocation is a contract you uphold by hand. `malloc` gives uninitialized bytes (check for `NULL`), `calloc` gives zeroed bytes (and, on mainstream implementations, checks its $n \times size$ multiply for overflow), `realloc` resizes and may move (assign to a temporary), and `free` returns a block exactly once (`free(NULL)` is a safe no-op). Underneath, the allocator manages a pool with free lists and size classes, splitting and coalescing blocks, at real cost, and can leave the pool fragmented. And the ways it goes wrong form a small, precise catalogue: **use-after-free** and **double-free** (lifetime violations), **buffer overflow** (bounds violation), **uninitialized read** (reading before writing), and **memory leak** (never freeing) — the first four undefined behavior, meaning *no guaranteed outcome*, not "a crash," and the last a defined-but-real defect. AddressSanitizer and Valgrind turn these unreliable faults into located errors, and each covers a different subset — which is why you run your tests under them rather than trusting that "it didn't crash."

Step back and notice the shape of the last three chapters. You learned that a pointer is an address into regioned memory (Ch 7), that objects have lifetimes and a pointer can outlive its target (Ch 8), and that on the heap *you* own every lifetime and every bound — and can violate them, silently, with the language offering no net (Ch 9). Every safe program in C is one where the programmer has, in their head, proven that no pointer is used outside its target's lifetime, no access falls outside its bounds, and every allocation is freed exactly once. That proof obligation is real and it is heavy — the bugs above are what happens when it fails. This is the exact setup for **Rust** (Part 5): its ownership, borrowing, and lifetime system is a way to make the compiler *check that proof* — to enforce, as type rules verified before the program ever runs, the very invariants you just had to maintain by hand. When you reach that Part, the motivation will already be in your hands: you will have felt the weight of the net that Rust removes by building the guarantees into the language.

CAN YOU… WITHOUT LOOKING?

- State what `malloc`, `calloc`, `realloc`, and `free` each do and promise — including which zeroes, which may move, and why `free(NULL)` is safe.
- Sketch what an **allocator** does (free lists, size classes, split/coalesce) and define **fragmentation** — and say why it is a capacity cost, not UB.
- Name and precisely define the five canonical bugs, and say which are **undefined behavior** and which is not.
- Explain why "undefined behavior" is stronger than "it crashes," and why "no crash" is *not* evidence of correctness.
- Say which bugs **AddressSanitizer** catches, which it does not, and what tool covers the gap.
- Explain, in one sentence, how this chapter motivates **Rust** — the compiler enforcing what you had to prove by hand.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What does each of `malloc/calloc/realloc/free` promise, and what is the safe `realloc` idiom? (§9.1)
2. **R2.** What does an allocator do underneath, and what is external fragmentation? (§9.2)
3. **R3.** Define use-after-free, double-free, buffer overflow, uninitialized read, and leak — which are UB? (§9.3)
4. **R4.** What precisely does "undefined behavior" mean, and why is "it ran fine" not evidence of correctness? (§9.3)
5. **R5.** Which bugs does AddressSanitizer catch and which does it miss, and how does this chapter set up Rust? (§9.4–9.5)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP)**, Chapter 9 (dynamic memory allocation: allocator design, free lists, coalescing, fragmentation, and the common memory bugs). The reference for §9.2's allocator model; the free CMU 15-213 materials cover it.
- **cppreference.com** and the **C standard** — `<stdlib.h>` memory functions (§7.22.3), and the definitions of undefined (§3.4.3), unspecified (§3.4.4), and implementation-defined (§3.4.1) behavior. The authority for what each function promises and what the language leaves undefined.
- **The AddressSanitizer and Valgrind documentation** — the tools' own manuals for what each detects and how to run them. Primary sources; verify flags against your compiler/tool version.
- **The Rust Programming Language ("the book") and the Rustonomicon** (free). Read ahead to ownership and borrowing to see how the invariants this chapter enforced by hand become compiler-checked rules — the payoff Part 5 develops.

Exercises

9.1 WARM-UP In one sentence each: (a) does `malloc` zero its memory? (b) does `calloc`? (c) is `free(NULL)` safe?

9.2 WARM-UP Write the *safe* `realloc` idiom (using a temporary), and explain in one sentence what goes wrong with `p = realloc(p, n);` when the allocation fails.

9.3 WARM-UP Match each bug to its one-line definition: use-after-free, double-free, memory leak, buffer overflow, uninitialized read. Then mark which are undefined behavior.

9.4 CORE Explain precisely why "the use-after-free returns the old value, so it's harmless" is wrong. Use the C standard's definition of undefined behavior, and give two realistic outcomes other than "returns the old value."

9.5 CORE For each snippet, name the bug and state whether it is undefined behavior: (a) `free(p); return p[0];` (b) `int *p = malloc(4*sizeof(int)); p[4] = 0;` (c) `free(p); free(p);` (d) `int *p = malloc(sizeof(int)); printf("%d", *p);` (e) `int *p = malloc(1024); p = NULL;`

9.6 CORE Explain the difference between a *memory leak* and a *use-after-free* in terms of freeing "too late" vs. "too early," and state why exactly one of the two is undefined behavior and the other is not.

9.7 COST (a) Why is a heap allocation much more expensive than a stack allocation (tie to §9.2 and Ch 8)? (b) What is external fragmentation, and why can it make a large `malloc` fail even when total free memory exceeds the request? (c) Is fragmentation undefined behavior? Classify it.

9.8 FADED *Worked, with the last step for you.* Diagnose and fix: `char *dup(const char *s) { char *r = malloc(strlen(s)); strcpy(r, s); return r; }`. Steps 1–3 given; complete step 4.
Step 1 (given): `strcpy` copies the string *including* its terminating `'\0'`, so it writes `strlen(s) + 1` bytes.
Step 2 (given): `malloc(strlen(s))` reserves only `strlen(s)` bytes — one too few.
Step 3 (given): So `strcpy` writes one byte past the end of the block: a heap buffer overflow, which is undefined behavior.
Step 4 (you): Give the corrected allocation size, add the one check the function is missing (what if `malloc` returns `NULL`?), and name the tool from §9.4 that would have caught the original overflow and the line it would point to.

9.9 MIXED For each observation, decide whether the cause is a *use-after-free*, a *buffer overflow*, an *uninitialized read*, a *leak*, or a *non-memory performance issue*, and justify in a line: (a) a long-running server's memory grows without bound until the OOM killer stops it; (b) a test prints a different number each run and only in release builds; (c) an out-of-bounds write corrupts an unrelated variable's value; (d) the program crashes inside `free` with heap-corruption after the same pointer is released twice; (e) a random-access loop over a huge array is slow and huge pages help (careful — one of these is not a memory *bug*).

9.10 MIXED Without running code, decide for each whether it is *undefined behavior*, *a defined-but-real defect*, or *well-defined and fine*, drawing on Chapters 7–9: (a) `free(NULL);` (b) allocating in a loop and never freeing; (c) `a[n]` where `a` has exactly `n` elements; (d) reading a `calloc`'d block before writing it; (e) reading a `malloc`'d block before writing it.

9.11 STRETCH The chapter says this material "motivates Rust." (a) State the three proof obligations a C programmer carries for heap memory (lifetime, bounds, free-exactly-once). (b) For each of use-after-free, double-free, and leak, name which obligation it violates. (c) In one or two sentences, describe what it would mean for a *compiler* to check these obligations before the program runs — and why that is more reliable than a sanitizer that checks at runtime.

9.12 LAB On your own machine (if a sanitizer is available), write four tiny programs that each contain exactly one of: a use-after-free, a heap buffer overflow, a double-free, and a leak. Compile each with `-fsanitize=address -g` and run it. (a) Record the exact error type and source line ASan reports for each. (b) Write a fifth program with an uninitialized read and confirm whether ASan flags it; if not, note what tool would. Report only what you actually observed, and label your compiler/tool versions, since the exact output varies.

Chapter 10 — Undefined Behavior & the Optimizing Compiler

Before you start: Chapter 9 (the memory bugs, most of them undefined behavior; the §3.4.3 definition), Chapter 7 (pointers, bounds, and one-past-the-end), Chapter 5 (what the compiler is allowed to reorder). · **Continues Part 2.** Chapter 9 named undefined behavior and told you it was "stronger than a crash." This chapter shows you *why* — how the optimizing compiler actively reasons from the assumption that your program contains no UB, so a latent bug does not merely misbehave where it sits: it lets the compiler transform code elsewhere in surprising, sometimes dangerous ways. We will compile real examples on this machine and read the assembly.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 9 — in the C standard's terms, what does **undefined behavior** mean, and how is it different from *unspecified* and *implementation-defined* behavior? (2) Chapter 7 — indexing a pointer is defined only within an array (plus one-past-the-end for the address, not the dereference); what is stepping outside that? (3) *Predict:* you write `int f(int x) { return x + 1 > x; }`. For *signed* `int`, is `x + 1 > x` always true? What do you think an optimizing compiler emits for this function — a comparison, or the constant `1`? By the end you will be able to read the assembly and say exactly why.

Here is the mental shift this chapter demands. It is tempting to think of C as a "portable assembler": you write operations, the compiler emits the corresponding instructions, and undefined behavior is what happens when those instructions hit a bad state at runtime. That model is wrong, and believing it is the source of a whole category of baffling bugs. The correct model is that **undefined behavior is a promise you make to the compiler** — a precondition it is entitled to assume you never violate — and the optimizer reasons *from* that promise. When you write code that can execute UB, you have not described "what the machine does in the bad case"; you have told the compiler the bad case cannot happen, and it will rewrite your program on that basis. We will make this concrete: the standard's definition and its two milder cousins (§10.1); the single assumption the optimizer runs on (§10.2); a gallery of UB that actually changes the compiled code on this machine, read as assembly (§10.3); and why the language is built this way, plus the tools and flags that defend you (§10.4). This is the intellectual core of why manual memory management (Ch 9) is so heavy, and — again — the exact weight Rust will lift in Part 5.

10.1 Undefined behavior, precisely — and its two milder cousins

Chapter 9 gave the definition; we now lean on it fully. The C standard (C11 §3.4.3) defines **undefined behavior** as behavior "for which this International Standard imposes *no requirements*," and notes that permissible responses range from "ignoring the situation completely with unpredictable results" to "behaving during translation or program execution in a documented manner characteristic of the environment" to "terminating a translation or execution." Read that carefully: "no requirements" and "during translation" together mean the license extends to *compile time* — the compiler, not just the running program, may do anything when UB is possible. Three standard categories, sharpened from §9.3, are worth keeping straight because only the first grants the compiler this license:

- **Undefined behavior** (§3.4.3): no requirements at all. The program has no meaning on the inputs that trigger it. Examples: signed integer overflow, dereferencing a null or dangling pointer, an out-of-bounds access, a data race, reading an object through a pointer of an incompatible type (a strict- aliasing violation).
- **Unspecified behavior** (§3.4.4): the standard gives two or more valid possibilities and does not say which you get — e.g. the order in which a function's arguments are evaluated. Each possibility is a normal, non-catastrophic outcome; the compiler need not document its choice.
- **Implementation-defined behavior** (§3.4.1): unspecified behavior that the implementation must *document* — e.g. the width of `int`, or the byte order (Ch 7). Portable code avoids depending on it; it is not dangerous, merely non-portable.

The distinction that matters for this chapter: unspecified and implementation-defined behavior still produce a *valid program* with *some* defined outcome — you just may not know which. Undefined behavior produces **no valid program at all** on the triggering input. There is no "the value you happen to get"; the standard has stopped describing your program entirely. Everything in §10.2 follows from that single fact.

QUICK CHECK

Which of these is undefined, which unspecified, which implementation-defined: (a) `sizeof(long)`; (b) the evaluation order of `f() + g()`; (c) signed integer overflow? (Answer after the next section.)

10.2 The optimizer's one assumption: UB never happens

An optimizing compiler transforms your code under the **as-if rule**: it may emit any instructions it likes as long as the observable behavior of a *well-defined* program is preserved. The crucial word is *well-defined*. For an input that triggers undefined behavior, the program has no observable behavior the compiler is required to preserve — so the optimizer is free to assume that input never occurs. It does not check; it does not warn (usually); it simply reasons: "this operation would be UB if condition *C* held, and the programmer has promised never to invoke UB, therefore *C* is false — and I may use that fact to simplify everything downstream (and, under the as-if rule, upstream too)."

Two consequences of that reasoning surprise people. First, **the optimizer propagates the assumption**: one operation that is only UB when `p` is null lets the compiler conclude `p` is non-null *everywhere* that value is used, deleting your later null check as provably-dead code. Second, **the effects can appear to "travel" in time**: because a program that executes UB has undefined behavior for the *whole* execution, an optimizer may hoist, sink, or delete work around the UB in ways that make observable effects *before* the buggy line also vanish or change. This is why "it printed the right thing up to the crash" is not something you can rely on. Neither of these is a compiler bug — each is the as-if rule applied to a program whose meaning you forfeited by writing the UB. Lattner's LLVM series and Regehr's guide (Further Reading) walk through the same reasoning at length; the sections below reproduce it on this machine.

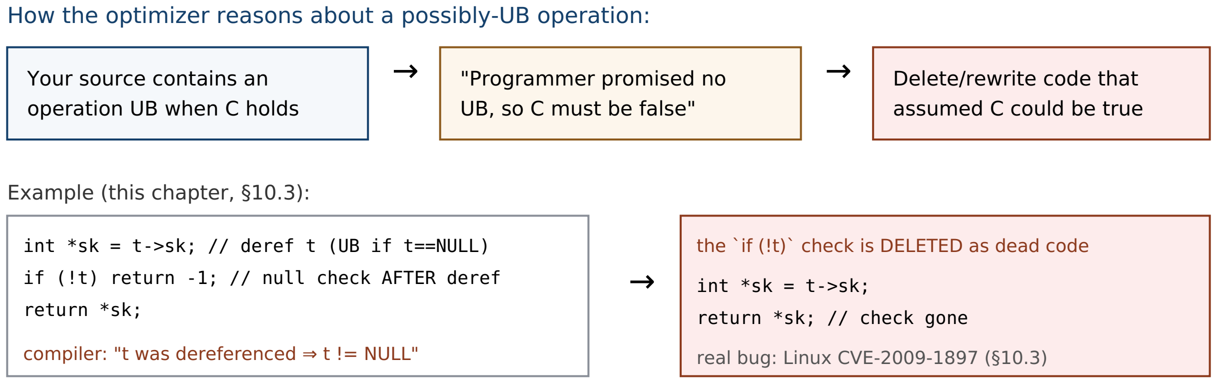

Figure 10.1: The optimizer treats a possibly-UB operation as a promise that the triggering condition never holds, then simplifies on that basis — here deleting a null check because the pointer was already dereferenced. The pattern is the real Linux kernel bug reproduced in §10.3.

*(Answer to the §10.1 quick check: (a) `sizeof(long)` is **implementation-defined** — it varies but must be documented; (b) the evaluation order of `f() + g()` is **unspecified** — one of two valid orders, undocumented; (c) signed integer overflow is **undefined behavior** — no requirements at all, the case the optimizer exploits.)*

10.3 A gallery of UB that changes the compiled code — read on this machine

Abstract claims about "the optimizer may assume" become believable only when you watch it happen. Each example below was compiled with `gcc 11.4.0` at `-O2` on this x86-64 machine and the relevant assembly read directly. **Exact output varies by compiler and version; these are from this specific toolchain.**

Signed integer overflow (the opener). In C, signed integer overflow is undefined (unsigned overflow is defined and wraps modulo 2^n). So in `int f(int x){ return x + 1 > x; }` the compiler may assume `x + 1` never overflows — and if it never overflows, `x + 1 > x` is *always* true. It compiles the whole function to a constant:

```
/* gcc -O2 -S : int f(int x){ return x + 1 > x; } */
f:
        movl    $1, %eax      ; return 1, unconditionally
        ret
```

No addition, no comparison — just `return 1`. This is correct per the standard and often useful (it lets loops with an `int` counter be optimized as if they cannot wrap). But if you were *relying* on wraparound to detect overflow — the classic `if (x + 1 < x) /* overflowed */` — the check is deleted and your overflow goes undetected. (Compile the same with `-fwrapv`, which *defines* signed overflow as two's-complement wraparound, and the comparison reappears.)

Null check deleted after a dereference (a real CVE). Dereferencing a null pointer is UB, so once a pointer has been dereferenced the compiler may assume it is non-null from then on. Consider the pattern below — a simplified form of the actual Linux kernel function `tun_chr_poll`:

```
struct T { int *sk; };
int poll(struct T *t) {
    int *sk = t->sk;      /* dereference t */
    if (!t) return -1;    /* null check AFTER the dereference */
    return *sk;
}
```

At `-O2`, gcc on this machine emits (the `if (!t)` is *gone* — no test of the pointer against zero):

```
poll:
        endbr64
        movq    (%rdi), %rax   ; sk = t->sk   (dereference t)
        movl    (%rax), %eax   ; return *sk
        ret                    ; no `testq %rdi,%rdi; je ...` -- check deleted
```

Compiled with `-fno-delete-null-pointer-checks`, the guard survives (`testq %rdi,%rdi; je .L3` reappears). This is not a contrived teaching example: it is the shape of **CVE-2009-1897** in the Linux kernel's `drivers/net/tun.c` (kernel 2.6.30). The pointer was dereferenced before the null check; gcc, entitled to assume the dereference could not have been of a null pointer, deleted the check; an attacker able to map memory at address zero could then reach the code that should have been guarded — a privilege-escalation hole. The kernel's fix moved the check before the dereference and, as belt-and-braces, the build adopted `-fno-delete-null-pointer-checks`. The lesson is exact: **a null check that comes after the dereference it is meant to guard is not a weak check — to the optimizer it is a statement that the pointer was already known non-null, and it may be deleted.**

The others you already met (Ch 9), now as UB the optimizer trusts. Out-of-bounds access, use-after-free, and uninitialized reads are all UB; the optimizer is likewise permitted to assume they never happen and to transform code accordingly (for instance, assuming an index stays in range, or that two pointers of incompatible types never alias — the **strict-aliasing** rule, §6.5, which lets the compiler keep a value in a register across a write through an `int*` when it "knows" a `float*` cannot touch the same object; `-fno-strict-aliasing` disables it). The through- line: every UB in Chapter 9's catalogue is not just a runtime hazard but a compile-time license.

THE SAME IDEA THREE WAYS: UB IS A PROMISE, NOT A BEHAVIOR

1. In words. Undefined behavior is a precondition you promise the compiler you will never violate. The compiler does not define "what happens in the bad case"; it assumes the bad case is impossible and optimizes on that assumption. So UB has no outcome to reason about — only consequences the optimizer derives from "this can't happen."

2. As a decision. For an operation that is UB exactly when condition *C* holds, the compiler branches: "either *C* is false (fine, proceed) or *C* is true (UB, undefined — I owe nothing)." Both branches let it act as if *C* is false. So it deletes anything that only mattered when *C* was true — like a null check after a dereference.

3. As compiled code. `x + 1 > x` becomes `return 1;` `if (!t)` after `t->sk` vanishes. The transform is visible in the assembly (§10.3) and correct per the standard — the "surprise" is entirely in the reader's wrong mental model of C as a portable assembler.

TRAP: "IT WORKED AT -O0, SO THE OPTIMIZER BROKE MY CODE"

When a program behaves at `-O0` and misbehaves at `-O2`, the instinct is "the optimizer introduced a bug." Almost always the truth is the reverse: **the undefined behavior was already in your program, and the optimizer merely exercised the license that UB granted it.** At `-O0` the compiler emits naive code that happens to do what you expected; at `-O2` it reasons from "no UB" and the latent bug surfaces. The tells: "only fails in release builds," "only with optimizations on," "adding a `printf` makes it go away" (the print perturbs what the optimizer can prove). The wrong fix is to compile at `-O0` or sprinkle `volatile` until it "works" — that hides UB, it does not remove it, and the next compiler version may re-expose it. The right response is to find and eliminate the undefined behavior: build with `-fsanitize=undefined` (UBSan) and `-Wall -Wextra`, and reason about which operation had no defined meaning. "Works at `-O0`" is not evidence of correctness, exactly as "didn't crash" was not in Chapter 9.

10.4 Why the language is built this way — and how to defend

It is fair to ask why the standard grants this license at all. The answer is a deliberate bargain. Leaving these situations undefined — rather than mandating a checked outcome — is what lets a C compiler generate fast code across wildly different hardware without inserting a bounds check on every array access, an overflow check on every addition, or a null check on every dereference. Signed-overflow UB, for example, lets the compiler promote loop counters, strength-reduce, and assume loops terminate; defined wraparound would forbid some of those. The cost of the bargain is precisely this chapter: the responsibility for never triggering UB shifts entirely to you, and the compiler will not (in general) tell you when you have. Different languages strike the bargain differently — Java defines integer overflow and mandates bounds checks (paying runtime cost for safety); Rust checks bounds and, in debug builds, overflow, and confines the UB-bearing operations to explicitly `unsafe` blocks (Part 5). C chose maximum latitude for the compiler, and hands you the bill.

Given that, your defenses are concrete and layered:

- **Sanitizers.** `-fsanitize=undefined` (UndefinedBehaviorSanitizer, in gcc and clang) instruments the program to detect many UB categories at runtime — signed overflow, out-of-bounds on known-size arrays, null dereference, misaligned access, and more — reporting the file and line. Combined with AddressSanitizer (Ch 9) for memory errors, it turns much silent UB into loud, located failures. Run your tests under both.

- **Warnings.** `-Wall -Wextra` catch a meaningful fraction statically (uninitialized reads, some overflows, format-string mismatches). Free; turn them on and treat them as errors (`-Werror`) where you can.
- **Defining-the-undefined flags.** When you deliberately depend on a behavior the standard leaves undefined, tell the compiler to define it: `-fwrapv` (signed overflow wraps), `-fno-strict-aliasing` (allow type-punning through pointers), `-fno-delete-null-pointer-checks` (keep null checks even after a dereference). These trade some optimization for predictability and are how the Linux kernel, among others, tames the sharpest edges.
- **Write UB-free code.** The durable fix. Check pointers before dereferencing, indices before indexing, and sizes before copying (Ch 11); use unsigned types or explicit overflow-checked arithmetic when you need wraparound or overflow detection; never read before initializing. Every rule from §9.1 is really a rule for staying inside the defined language.

QUICK CHECK

Your code does `if (x + 1 < x)` to detect signed overflow and the check keeps getting optimized away. Name two ways to make the detection actually work — one that changes the flag you compile with, one that changes the type or arithmetic you use. (Answer below the communicate box.)

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate files a compiler bug: "gcc deleted my null check! Look — I clearly write `if (!t) return -1;` and it's not in the assembly. This is a miscompilation." The code dereferences `t` one line above the check. Cover the paragraph and respond in three or four sentences, using *undefined behavior* and the as-if rule.

Model answer. "It's not a miscompilation — it's the as-if rule doing exactly what the standard permits. You dereference `t` with `t->sk` *before* the check, and dereferencing a null pointer is undefined behavior, so the compiler is entitled to assume `t` was non-null at that point — which makes the later `if (!t)` provably dead code it can delete. This is literally the shape of the Linux kernel bug CVE-2009-1897. The fix isn't to report gcc; it's to move the null check *before* the dereference (and, if we want a belt-and-braces guard, build with `-fno-delete-null-pointer-checks`). A check that comes after the dereference it guards isn't a weak check — to the optimizer it's a promise the pointer was already non-null." Naming the as-if rule and the CVE, rather than blaming the compiler, is the senior register — and it turns a "compiler bug" into a one-line code fix.

(Answer to the §10.4 quick check: (1) compile with `-fwrapv`, which defines signed overflow as two's-complement wraparound so the comparison is no longer optimized away; or (2) do the arithmetic in an ***unsigned*** *type (defined to wrap) or use a checked-arithmetic*

*builtin such as `__builtin_add_overflow`, which reports overflow without relying on UB. Detecting signed overflow by **letting it happen** and testing afterward is itself UB — that is why the naive check disappears.)*

10.5 What to carry forward

Undefined behavior is not "what the machine does in the bad case." It is a precondition you promise the compiler you will never violate, and the optimizer reasons from that promise: under the as-if rule it may assume every UB-triggering condition is false and rewrite your program accordingly — deleting a null check after a dereference, folding `x + 1 > x` to the constant `1`, keeping a value in a register across a write it "knows" cannot alias. We watched each of these happen in the assembly on this machine. The categories to keep separate are undefined (no requirements — the compiler's license), unspecified (a valid but unstated choice), and implementation-defined (unspecified but documented); only the first grants the license. The language makes this bargain on purpose — it is what lets C compile to fast code across every architecture without safety checks on every operation — and it hands you the whole cost. Your defenses are UBSan and ASan for detection, `-Wall -Wextra` for the static fraction, the define-the-undefined flags (`-fwrapv`, `-fno-strict-aliasing`, `-fno-delete-null-pointer-checks`) when you must depend on an edge, and — above all — writing code that never triggers UB.

This chapter closes the loop opened in Chapter 9. There we cataloged the manual-memory bugs and called most of them UB; here you saw why that word carries so much weight — UB is what makes those bugs unpredictable, build-dependent, and, as CVE-2009-1897 shows, exploitable. It also sets up the next chapter: the buffer overflow of Chapter 9 is UB, and Chapter 11 follows exactly what an out-of-bounds write can do to a program's control flow — the single most consequential UB in the history of security. And it is the sharpest possible motivation for Rust (Part 5): a language whose safe subset has *no* undefined behavior, confining every UB-bearing operation to code you must mark `unsafe`, so the promises this chapter made you keep by hand are enforced by the type system before the program ever runs.

CAN YOU… WITHOUT LOOKING?

- State the C standard's definition of **undefined behavior** and distinguish it from **unspecified** and **implementation-defined** behavior.
- Explain the **as-if rule** and why it lets the optimizer assume UB-triggering conditions are false — including how an assumption "propagates" and can affect code before the buggy line.
- Walk through why `int f(int x){ return x + 1 > x; }` can compile to `return 1`, and why a null check *after* a dereference can be deleted (CVE-2009-1897).
- Explain why "it worked at `-O0`" and "adding a `printf` fixed it" are UB signatures, not evidence of correctness.
- Name four defenses (UBSan, warnings, define-the-undefined flags, UB-free code) and say what each buys.
- Say, in one sentence, why the language leaves these behaviors undefined at all — the optimization/safety bargain — and how Rust strikes it differently.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** Define undefined, unspecified, and implementation-defined behavior, and say which grants the compiler its license. (§10.1)
2. **R2.** State the as-if rule and explain how "the optimizer assumes UB never happens" leads to deleting code. (§10.2)
3. **R3.** Why can `x + 1 > x` become `1`, and how does `-fwrapv` change that? (§10.3–10.4)
4. **R4.** Describe CVE-2009-1897: what pattern caused it, why the check was deleted, and the two-part fix. (§10.3)
5. **R5.** Name the bargain the language makes and four defenses against UB. (§10.4)

FURTHER READING

- **Chris Lattner, "What Every C Programmer Should Know About Undefined Behavior" (LLVM Project Blog, 2011, three parts).** The optimizer-writer's own account of how and why the compiler exploits UB, with worked examples (null checks, signed overflow, strict aliasing). Free.
- **John Regehr, "A Guide to Undefined Behavior in C and C++" (Embedded in Academia, 2010, three parts).** A researcher's catalogue of UB and its interaction with aggressive compilers, including the "UB can time-travel" framing. Free (`blog.regehr.org`).
- **The C standard** — §3.4.1/3.4.3/3.4.4 (the three behavior categories), §6.5 (aliasing), and the integer-overflow rules. The authority for what is undefined; **cppreference.com** summarizes it accessibly.
- **UndefinedBehaviorSanitizer (UBSan) documentation** (Clang/GCC). The runtime detector for many UB categories; verify the exact checks and flags against your compiler version.

Exercises

10.1 WARM-UP In one sentence each, define *undefined*, *unspecified*, and *implementation-defined* behavior, and state which one grants the optimizer license to assume the situation never happens.

10.2 WARM-UP Is *unsigned* integer overflow undefined behavior in C? Is *signed* integer overflow? Answer each and, for the defined one, say what value results.

10.3 WARM-UP A program prints correct output at `-O0` but wrong output at `-O2`. State the single most likely explanation, and name the two build flags you would add to hunt for it.

10.4 CORE Explain, step by step, why `int f(int x){ return x + 1 > x; }` can be compiled to `return 1`. Then explain what breaks if a programmer used `if (x + 1 < x)` to detect overflow, and give a correct way to detect it.

10.5 CORE Given `int *sk = t->sk; if (!t) return -1; return *sk;`: (a) name the UB, (b) explain why an optimizer may delete the `if (!t)` check, (c) give the one-line code fix, and (d) name the flag that would preserve the check without changing the code. This is a real CVE — name the pattern.

10.6 CORE "The compiler has a bug — my code is fine, it just miscompiled." A teammate says this about the §10.5 null- check example. Explain, using the as-if rule, why the compiler is behaving correctly and the code is not.

10.7 COST The standard *could* have defined all of these behaviors (mandating bounds checks, overflow checks, null checks). (a) What runtime cost would that impose? (b) Name one specific optimization that signed-overflow UB enables. (c) State the bargain in one sentence: what the programmer gives up and what the compiler gains.

10.8 FADED *Worked, with the last step for you.* A teammate reports: "My bounds check `if (i < 0 || i >= n)` works, but after I access `a[i]` *first* for logging and check `i` afterward, the check stopped firing." Diagnose and fix.
Step 1 (given): Accessing `a[i]` with `i` out of range is an out- of-bounds access — undefined behavior.
Step 2 (given): Once `a[i]` has executed, the compiler may assume `i` was in range (else UB), i.e. `0 <= i < n`.
Step 3 (given): Therefore the later `if (i < 0 || i >= n)` is provably false and can be deleted — same shape as the null-check CVE.
Step 4 (you): State the fix (what must come before what), name one sanitizer that would have flagged the original out-of-bounds access at runtime, and say why moving the log line does not merely "reorder output" but changes what the compiler is allowed to prove.

10.9 MIXED For each, decide whether it is *undefined*, *unspecified*, or *implementation-defined*, and justify in a line: (a) the value of `sizeof(int)`; (b) the order `a()` and `b()` are called in `a() + b()`; (c) `INT_MAX + 1`; (d) right-shifting a negative signed integer; (e) reading a `malloc`'d block before writing it. (One of these is a §9.3 memory bug; recognizing that is the point.)

10.10 MIXED A colleague proposes three "fixes" for an intermittent bug that only appears in release builds: (a) always compile at `-O0`; (b) mark the variables `volatile` until it stops; (c) build the tests with `-fsanitize=undefined,address` and fix whatever it reports. Rank them from least to most effective and say, for each, whether it removes the bug or merely hides it.

10.11 STRETCH The chapter says a UB assumption can "propagate" and even affect code *before* the buggy line. (a) Explain why, using the definition of UB as applying to the whole program execution. (b) Give a concrete scenario where deleting a downstream null check (because of an upstream dereference) turns a would-be crash into a silent security hole. (c) Argue why a *static* guarantee (Rust's safe subset having no UB) is qualitatively stronger than a runtime sanitizer here.

10.12 LAB On your own machine, compile `int f(int x){ return x + 1 > x; }` with `gcc -O2 -S` and read the assembly; then recompile with `-fwrapv` and compare. Do the same for the §10.5 `poll` function with and without `-fno-delete-null-pointer-checks`. (a) Record what changed in each case. (b) Write a tiny program with a signed-overflow bug and confirm whether `-fsanitize=undefined` reports it. Report only what you observed and label your compiler/version, since output varies.

Chapter 11 — Arrays, Strings & Buffer Safety

Before you start: Chapter 10 (a buffer overflow is undefined behavior — and the optimizer trusts it never happens), Chapter 9 (the buffer-overflow bug, stated precisely), Chapter 8 (the stack, frames, and the return address that sits on it), Chapter 7 (arrays, pointers, and array-to-pointer decay). · **Continues Part 2.** This chapter takes the buffer overflow — Chapter 9's most consequential bug — and follows it all the way to its worst outcome: overwriting a return address and seizing a program's control flow. Then it covers the C string model that makes overflows so easy, the string-function minefield, and the layered defenses real systems deploy — measuring each on this machine.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 8 — a function's local array and its return address both live in the same stack frame; which direction does the stack grow, and which of those two is at a *higher* address on a typical downward-growing stack? (2) Chapter 9 — an out-of- bounds *write* was called "especially dangerous." Why more dangerous than an out-of-bounds read? (3) *Predict:* you replace an unsafe `strcpy(dst, src)` with `strncpy(dst, src, n)` where `n` is exactly the size of `dst`, and `src` is `n` characters long. Do you think the result is a valid, null-terminated C string? We will compile exactly this and read the bytes.

Almost every catastrophic memory-corruption vulnerability in the history of software traces to the same root: a program wrote past the end of a buffer. Chapter 9 named it and called it undefined behavior; Chapter 10 showed why UB is dangerous in the abstract. This chapter makes it visceral, because the buffer overflow is not merely "corrupt some adjacent bytes" — on the stack, the bytes adjacent to a local array include the saved return address, and overwriting *that* hands an attacker control of what the CPU executes next. We will build up to it deliberately: why the C string is a fragile convention rather than a real type (§11.1); the anatomy of a stack buffer overflow and why it is exploitable (§11.2); the string-function minefield and the sized functions that replace it — including the trap that `strncpy` is *not* the safe `strcpy` (§11.3); the layered defenses (stack canaries, `_FORTIFY_SOURCE`, ASLR, NX/DEP) and precisely what each does and does not guarantee, run on this machine (§11.4); and the disciplines that actually prevent the bug (§11.5). Throughout, the refrain from Chapters 9 and 10 holds: the bug is UB, so a program that "runs fine in testing" may be one crafted input away from disaster.

11.1 The C string is a convention, not a type

C has no string type. What C calls a string is a **convention**: a contiguous array of `char` terminated by a null byte `'\0'` (a zero byte, value 0). The length is not stored anywhere — it is *implied* by the position of the terminator, which is why `strlen` is an O(n) scan that walks bytes until it finds the zero. This one design decision is the source of a startling share of C's danger. Because the length is not carried with the data, every function that touches a string must either be told the buffer's capacity or trust that a terminator exists within bounds — and when that trust is misplaced, the read or write runs off the end. Recall from Chapter 7 that an array decays to a pointer to its first element the moment you pass it to a function: the callee receives an address with *no length information at all*.

A `char *` tells you where a string starts; it tells you nothing about how much room there is. Bounds safety in C is therefore never automatic — it is something the programmer must reconstruct by hand, by tracking capacities alongside pointers.

Two consequences follow immediately. First, **a missing terminator is a bug waiting to happen**: a "string" with no `'\0'` in bounds makes `strlen`, `printf("%s")`, and every other string function read past the end — an out-of-bounds read (UB), which can leak adjacent memory (the Heartbleed class, Ch 9). Second, **a copy into an undersized buffer overflows it**: writing an *n*-plus-one-byte string (n characters plus the terminator) into an *n*-byte buffer writes one byte past the end — an out-of-bounds write (UB), the subject of the rest of this chapter. The C string's elegance (a string is just bytes) and its danger (a string is *just* bytes, with no length and no bounds) are the same fact.

> **QUICK CHECK**
>
> Why is `strlen` an O(n) operation rather than O(1)? And when you pass a `char buf[64]` to a function, how much does the function know about `buf`'s capacity? (Answer after the next section.)

11.2 The stack buffer overflow — anatomy and why it is exploitable

Recall the stack frame from Chapter 8. When a function runs, its local variables — a `char buf[8]`, say — live in its stack frame, and (on the common downward-growing stack of x86-64 and most architectures) the frame also holds the **saved return address**: the address the CPU will jump to when this function returns. Critically, the return address sits at a *higher* address than the local buffer, while a copy into the buffer writes toward *higher* addresses. So a write that overruns `buf` marches straight toward the saved return address. Overflow it by enough bytes and you overwrite the return address itself. Now the picture is stark: when the function returns, the CPU loads that overwritten value into the instruction pointer and jumps *wherever the overflowing bytes told it to*. An attacker who controls the input controls where the program goes next.

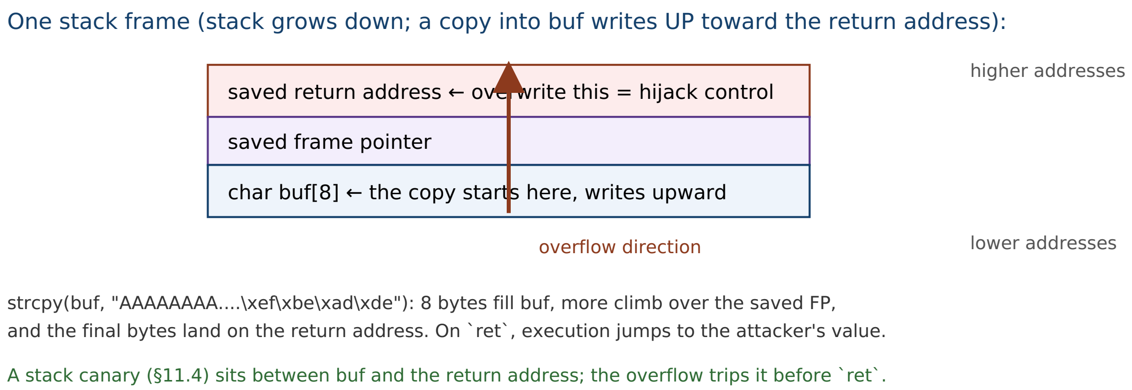

Figure 11.1: A stack buffer overflow. Because the saved return address sits above the local buffer and a copy writes upward, overrunning the buffer can overwrite the return address, redirecting control flow when the function returns. A stack canary is placed in the gap to detect exactly this.

This is not a hypothetical; it is the mechanism behind decades of exploits, laid out for the first time in a widely read form by **Aleph One's "Smashing the Stack for Fun and Profit"** (Phrack 49, 1996) and developed rigorously in CS:APP (§3.10). The classic exploit places the attacker's own machine code ("shellcode") in the buffer and overwrites the return address to point at it; modern mitigations (§11.4) have made that specific technique much harder, but the fundamental hazard — an out-of-bounds write reaching control data — remains, and newer techniques (return-oriented programming) route around the defenses. To make it concrete, this program overflows an 8-byte buffer:

```
#include <string.h>
int main(void){
    char buf[8];
    strcpy(buf, "this string is much too long for the buffer");  /* 44 bytes into 8 */
    return buf[0];
}
```

Compiled with stack protection on this machine (`gcc -O0 -fstack-protector-all`) and run, it does *not* silently corrupt and continue — the canary catches it (§11.4). gcc even warns at compile time before you run it:

```
warning: '__builtin_memcpy' writing 44 bytes into a region of size 8 overflows the
destination
         [-Wstringop-overflow=]
$ ./a.out
*** stack smashing detected ***: terminated
Aborted (core dumped)          # exit status 134 (SIGABRT)
```

Without those defenses — an older toolchain, or a copy whose length the compiler cannot see — the same overflow would have corrupted the return address silently. **Outputs are from this machine (gcc 11.4.0, x86-64); wording and status vary by toolchain.**

TRAP: "STRNCPY IS THE SAFE STRCPY"

The most common half-fix in C is to reach for `strncpy(dst, src, n)` believing the `n` makes it safe. It bounds the write, which prevents the overflow — but **`strncpy` does not guarantee a null terminator**. Its actual contract (cppreference; the C standard): it copies at most `n` bytes; if `src` is shorter than `n` it pads the rest with zeros, but if `src` is *n* or more bytes long it writes exactly `n` bytes and stops — leaving `dst` **unterminated**. The next `strlen` or `printf("%s")` then runs off the end: you traded a bounded overflow for an unbounded over-read. On this machine, `char dst[4]; strncpy(dst, "abcd", 4);` leaves the four bytes `97 98 99 100` (`'a' 'b' 'c' 'd'`) with *no* terminating zero — a "string" that is not a string. `strncpy` was designed in the 1970s for fixed-width records (like early UNIX directory entries), *not* as a safe string copy, and using it as one is a classic trap. If you use it, you must terminate by hand (`dst[n-1] = '\0';`) — or, better, use `snprintf` (§11.3), which always terminates.

*(Answer to the §11.1 quick check: `strlen` is O(n) because a C string stores no length — it must scan byte by byte until it finds the terminating `'\0'`. And a function receiving `char buf[64]` knows **nothing** about the capacity: the array decayed to a bare `char *`, an address with no length attached — you must pass the size separately.)*

11.3 The string-function minefield — and the sized alternatives

The classic C string functions are unsafe by construction because they take no destination size: they write until they hit a terminator in the *source*, with no regard for the *destination*'s capacity. The worst offenders and their safer replacements:

- **`gets(char *s)`** — reads a line into `s` with *no* bound whatsoever. It was so irredeemably unsafe that it was deprecated in C99 and **removed from the language in C11**. Never use it; use `fgets(s, size, stdin)`, which takes the buffer size.
- **`strcpy(dst, src)` / `strcat(dst, src)`** — copy/append with no destination bound; overflow `dst` whenever `src` is too long. Prefer `snprintf` (below), or bounded copies where you control termination.
- **`sprintf(dst, fmt, ...)`** — formats into `dst` with no bound; a long formatted result overflows. Replace with **`snprintf(dst, size, fmt, ...)`**, which writes at most `size` bytes *including* the terminator, always null-terminates (when `size > 0`), and returns the length it *would* have written — so you can even detect truncation.
- **`strncpy` / `strncat`** — bounded but treacherous: `strncpy` may not terminate (the §11.2 trap), and `strncat`'s size argument is the number of bytes to append, not the buffer size, which readers routinely get wrong. Usable, but only with care and a manual terminator.

The reliable pattern for building a string safely is `snprintf`: it is bounded, always terminates, and reports truncation. When you truly need a raw copy, carry the capacity and check it — `if (srclen + 1 <= dstcap) memcpy(dst, src, srclen + 1);` — the same "reconstruct the length by hand" discipline §11.1 demanded. (Some platforms offer `strlcpy`/`strlcat`, which always terminate and return the source length; they are widely available but are *not* in the C standard, so their portability must be checked. The standard's own `_s` "bounds-checking" functions, Annex K, exist but are optional and unevenly implemented — do not assume them.)

> **QUICK CHECK**
>
> Name the one standard string function that was *removed* from C11 for being unsafe, and its safe replacement. And what does `snprintf` guarantee that `strncpy` does not? (Answer below the communicate box.)

11.4 Defense in depth — canaries, FORTIFY, ASLR, NX — and what each buys

Because buffer overflows are so consequential, modern toolchains and operating systems layer several *mitigations*. It is essential to understand what each actually guarantees, because none of them *removes* the bug — they raise the cost of exploiting it, and each defeats a specific technique while leaving others open. Four you will meet, three demonstrated on this machine:

- **Compiler bounds warnings.** With optimization and `-Wall`, gcc's `-Wstringop-overflow` can catch overflows it can see at compile time. On this machine the §11.2 program drew `warning: '__builtin_memcpy' writing 44 bytes into a region of size 8 overflows the destination` — a static catch, but only when the sizes are visible to the compiler.

- **`_FORTIFY_SOURCE`.** Building with `-D_FORTIFY_SOURCE=2 -O2` makes glibc substitute checked variants (`__strcpy_chk`, `__memcpy_chk`, …) that know the destination size and abort on overflow — catching many cases at compile time *or* runtime. On this machine, fortifying `char buf[4]; strcpy(buf, "hello");` produced `warning: '__builtin___strcpy_chk' writing 6 bytes into a region of size 4`. It only helps where the size is known and requires optimization; it is not universal.
- **Stack canaries** (`-fstack-protector` / `-fstack-protector-all`). The compiler places a random "canary" value between the local buffers and the saved return address (Figure 11.1) and checks it just before returning; if an overflow overwrote it, the program aborts instead of returning to corrupted control data. On this machine, the §11.2 overflow tripped it: `*** stack smashing detected ***: terminated`, exit status 134. Canaries defeat the classic "overwrite the return address by linear overflow" attack — but not overwrites that skip the canary, or non-control-data corruption, and they act *after* the corruption (they convert exploitation into a crash, i.e. a denial of service, not into "no bug").
- **ASLR and NX/DEP** (OS-level). **Address Space Layout Randomization** randomizes the base addresses of the stack, heap, and libraries each run, so an attacker cannot easily predict where to jump. **NX/DEP** (the no-execute page permission, tied to virtual memory from Chapter 4) marks the stack and heap non-executable, so injected shellcode cannot run as code. Together they defeat the original "inject shellcode on the stack and jump to it" recipe — which is why real attacks moved to *return-oriented programming*, chaining existing code fragments rather than injecting new ones. These are runtime/OS defenses, not compiler ones, and again they raise cost rather than eliminate the bug.

The load-bearing point: **every one of these is a mitigation, not a fix.** They assume the overflow exists and try to contain its blast radius — turning a silent hijack into a crash, or a reliable exploit into an unreliable one. The bug is still there, still undefined behavior (Ch 10), and a determined attacker or an unlucky input may route around any single layer. The only true fix is to not write out of bounds in the first place (§11.5) — or to use a language that enforces bounds by construction (Part 5).

THE SAME IDEA THREE WAYS: A BUFFER OVERFLOW IS A BOUNDS VIOLATION WITH CONSEQUENCES

1. In words. Writing past a buffer's end (§9.3's bounds violation) is undefined behavior; on the stack the bytes just past a local array include control data — the saved return address — so the overflow can redirect execution, not merely corrupt data.

2. As a picture. Figure 11.1: `buf` low, saved frame pointer above it, return address above that; a copy writes upward, so an overrun climbs over the frame pointer onto the return address. A canary sits in the gap to catch the climb.

3. As UB the compiler and OS fight in layers. The write is UB (Ch 10) — no defined outcome. Compiler warnings and `_FORTIFY_SOURCE` catch some cases; canaries turn a hijack into a crash; ASLR and NX defeat specific exploit recipes. None removes the UB; only bounds-correct code does.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. In a review, a colleague writes: "I turned on `-fstack-protector`, so our buffer overflows are handled — we don't need to fix the `strcpy` calls." Cover the paragraph and respond in three or four sentences, distinguishing *mitigation* from *fix*.

Model answer. "A stack canary is a mitigation, not a fix: it detects a linear stack overflow just before the function returns and aborts, so it converts a control-flow hijack into a crash — a denial of service, still a real incident, and it does nothing for heap overflows, non-control-data corruption, or overwrites that skip the canary. The overflow is still there, still undefined behavior, so the compiler is free to do surprising things around it, and a canary that fires in production is an outage. We should keep the canary *and* ASLR *and* `_FORTIFY_SOURCE` as defense in depth, but the actual fix is to remove the unbounded copies — replace `strcpy` with `snprintf` or a capacity-checked `memcpy` — so the out-of-bounds write cannot happen." Naming the distinction — mitigation contains blast radius, only bounds-correct code removes the bug — is the senior register, and it keeps the defenses without treating them as a license to leave the bug in.

(Answer to the §11.3 quick check: `gets` was removed from C11; its safe replacement is `fgets`, which takes the buffer size. And `snprintf` always null-terminates its output (when the size is nonzero) and reports truncation, whereas `strncpy` may leave the destination with no terminator at all.)

11.5 What to carry forward

The C string is a convention — a null-terminated byte array with no stored length — and that single fact is why bounds safety in C is manual and why buffer overflows are so easy. An out-of-bounds write is undefined behavior (Ch 9, Ch 10), and on the stack it is the most consequential UB in security: the saved return address lives just past a local buffer, so an overrun can overwrite it and redirect control flow — the mechanism Aleph One documented and CS:APP develops. The classic string functions (`gets`, `strcpy`, `strcat`, `sprintf`) take no destination size and are unsafe by construction; `gets` was removed from C11 outright. The sized replacements — `fgets`, `snprintf`, capacity-checked `memcpy` — are safe *when you carry the capacity and check it*, and `strncpy` is a trap because it can leave a string unterminated. Layered defenses (compiler warnings, `_FORTIFY_SOURCE`, stack canaries, ASLR, NX/DEP) each raise the cost of exploitation and we watched three of them fire on this machine — but every one is a mitigation that contains the blast radius, not a fix that removes the bug.

The discipline that actually prevents the bug is the same one Chapters 7–10 kept circling: a bare pointer carries no bounds, so *you* must carry them — keep the capacity beside every buffer, check every copy against it, prefer sized APIs, and never trust that a terminator is present. That is a proof obligation, on every string operation, that the programmer discharges by hand and that no amount of mitigation discharges for you. It is also the cleanest possible statement of what a safer language buys: in Rust (Part 5) a string and a slice *carry their length*, indexing is bounds-checked, and an out-of-bounds access is a defined panic rather than undefined memory corruption — the "carry the length by hand" discipline of this chapter, promoted into the type system. Part 2 has now taken you from "a pointer is an address" (Ch 7) to "and you own every lifetime, every bound, and every promise to the compiler" (Ch 8–11). The final chapter turns the last of those responsibilities — allocation — from a hazard you survive into a tool you wield.

CAN YOU… WITHOUT LOOKING?

- Explain why a C string is "a convention, not a type," and why that makes `strlen` O(n) and bounds safety manual.
- Draw a stack frame and explain why a buffer overflow can overwrite the **saved return address** and hijack control flow.
- State why `strncpy` is *not* the safe `strcpy` — the exact contract that bites — and name a genuinely safe alternative.
- Name the four classic unsafe string functions and their sized replacements, and which one C11 removed.
- Describe stack canaries, `_FORTIFY_SOURCE`, ASLR, and NX/DEP, and say for each what it buys and why it is a mitigation, not a fix.
- State the one discipline that actually prevents overflows, and how Rust promotes it into the type system.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is a C string, physically, and why does it store no length? (§11.1)
2. **R2.** Why can a stack buffer overflow hijack control flow — what sits next to the buffer, and in which direction does a copy write? (§11.2)
3. **R3.** State `strncpy`'s exact contract and the trap it creates; give a safer function. (§11.2–11.3)
4. **R4.** For canaries, `_FORTIFY_SOURCE`, ASLR, and NX/DEP, what does each defeat, and why is each a mitigation not a fix? (§11.4)
5. **R5.** What single discipline prevents overflows, and how does Rust encode it? (§11.5)

FURTHER READING

- **Aleph One, "Smashing the Stack for Fun and Profit," Phrack 49 (1996).** The first widely read, step-by-step account of stack buffer overflow exploitation; historically pivotal. Free (`phrack.org`). Read it for the mechanism, not as current exploit practice (the mitigations of §11.4 postdate it).
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), §3.10.** Buffer overflows, stack smashing, and the mitigations (canaries, ASLR, NX) developed rigorously with the stack-frame model of Chapter 8. The reference for this chapter.
- **cppreference.com** and the **C standard** — the exact contracts of `strcpy`, `strncpy`, `strcat`, `snprintf`, `fgets`, and the removal of `gets` in C11. The authority for what each function promises.
- **Beej's Guide to C.** Free (`beej.us`). Practical, readable coverage of C strings and the safe idioms.

Exercises

11.1 WARM-UP In one sentence each: (a) what physically marks the end of a C string? (b) why is `strlen` O(n)? (c) when you pass an array to a function, what does the function learn about its capacity?

11.2 WARM-UP Which standard string function was *removed* from the language in C11 for being unsafe, and what should you use instead?

11.3 WARM-UP True or false, with a one-line reason each: (a) `strncpy(dst, src, n)` always produces a null- terminated string. (b) A stack canary removes the buffer-overflow bug. (c) `snprintf` always null- terminates when its size argument is nonzero.

11.4 CORE Explain, using a stack-frame diagram in words, why an out-of-bounds *write* into a local array can hijack a program's control flow but an out-of-bounds *read* (usually) cannot. What lies just past the buffer, and in which direction does a copy write?

11.5 CORE For each, state whether it can overflow the destination and why, and give a safe rewrite: (a) `strcpy(buf, user_input);` (b) `sprintf(buf, "%s-%s", a, b);` (c) `char d[4]; strncpy(d, "abcd", 4); printf("%s", d);`

11.6 CORE A colleague argues: "We enabled stack canaries and ASLR, so buffer overflows are handled." Give three distinct reasons this is wrong — each naming something a canary or ASLR does *not* prevent — and state what the actual fix is.

11.7 COST (a) Why is `strlen` O(n) while a length-carrying string type (as in Rust or C++ `std::string`) makes "get the length" O(1)? (b) What runtime cost do stack canaries and `_FORTIFY_SOURCE` add, and what do you get for it? (c) Is a canary check on the hot path free? Classify the trade-off (safety vs. speed).

11.8 FADED *Worked, with the last step for you.* Diagnose and fix: `void greet(const char *name){ char msg[16]; strcpy(msg, "Hello, "); strcat(msg, name); puts(msg); }`.
Step 1 (given): `"Hello, "` is 7 characters plus a terminator, using 8 of the 16 bytes.
Step 2 (given): `strcat` appends `name` with no bound; if `name` is longer than 7 characters the total exceeds 16 bytes.
Step 3 (given): So a long `name` overflows `msg` — an out-of- bounds write, undefined behavior, and (on the stack) a potential control-flow hijack.
Step 4 (you): Rewrite the function using `snprintf` so it can never overflow regardless of `name`'s length; state what `snprintf` guarantees that `strcat` does not; and name one §11.4 defense that would turn the original overflow into a crash rather than a silent hijack.

11.9 MIXED For each symptom, name the most likely cause — *unterminated string (over-read), buffer overflow (over-write), use-after-free* (Ch 9), or *a mitigation firing* — and justify in a line: (a) a program prints a corrupted string with garbage bytes appended after the expected text; (b) a service aborts with `*** stack smashing detected ***` on a long request; (c) reading a freed heap block returns another request's data; (d) `printf("%s", s)` reads far past where the data should end and leaks adjacent memory.

11.10 MIXED Classify each as *undefined behavior*, *a defined-but-real defect*, or *well-defined and fine*, drawing on Chapters 9–11: (a) `strncpy(d, s, sizeof d)` then reading `d` as a string, where `strlen(s) >= sizeof d`; (b) `snprintf(d, sizeof d, "%s", s)` then reading `d`; (c) writing one byte past a heap block; (d) a canary aborting the process on a detected overflow.

11.11 STRETCH The chapter calls the mitigations "defense in depth." (a) Explain what each of stack canaries, NX/DEP, and ASLR defeats, and name the exploit technique (return-oriented programming) that was developed to route around NX. (b) Argue why layering mitigations is still worthwhile even though none is a fix. (c) Explain what it means for Rust to make the bug "not happen" rather than "not exploit" — and why a bounds-checked panic is categorically different from undefined memory corruption.

11.12 LAB On your own machine, write the §11.2 8-byte-buffer overflow. (a) Compile with `-fstack-protector-all`, run it, and record the exact message and exit status. (b) Compile the same with `-D_FORTIFY_SOURCE=2 -O2` and record any compile-time diagnostic. (c) Write `char d[4]; strncpy(d, "abcd", 4);` and print the four bytes as integers to confirm whether a terminator is present. Report only what you observed and label your compiler/version, since output varies.

Chapter 12 — Custom Allocators: Arenas, Pools & Free Lists

Before you start: Chapter 9 (what the allocator does — free lists, size classes, fragmentation, cost), Chapter 10 (a use-after-free is undefined behavior, whoever owns the memory), Chapter 5 (mechanical sympathy: cost follows layout and access pattern). · **Closes Part 2.** Chapter 9 treated the general-purpose allocator as a fact of the machine and cataloged the ways manual memory goes wrong. This chapter turns allocation from a hazard you survive into a tool you wield: when your allocation *pattern* is known, a custom allocator — an arena, a pool, a free list — can be dramatically faster than `malloc`, and we will build and measure one on this machine. It also completes the arc of the part: even your own allocator does not relax a single rule from Chapters 9–11.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 9 — why is a heap `malloc` far more expensive than the stack's pointer-bump, and what two things (a search structure and a per-block record) does the allocator maintain? (2) Chapter 9 — what is *external fragmentation*, and why can it make a large allocation fail even when total free memory is ample? (3) *Predict:* suppose you allocate a million small objects that all die at the same moment. If you could replace a million individual `free` calls with a *single* operation that reclaims them all at once, roughly how much faster do you think freeing would be? We will measure exactly this.

Chapter 9's allocator is a marvel of generality: it satisfies any size, in any order, at any time, and it is the right default for the overwhelming majority of code. But generality has a price — every allocation may search a free list, split a block, and update bookkeeping; every free may coalesce; and interleaved requests fragment the pool. When you know something specific about *how* your program allocates — that many objects share a lifetime, or that they are all the same size — you can trade the general allocator's flexibility for speed and simplicity by writing an allocator specialized to that pattern. This chapter builds the two most useful specializations: the **arena** (also called a region or bump allocator), which makes allocation a pointer increment and frees everything at once (§12.2), and the **pool** or free-list allocator for fixed-size objects (§12.3). We start by asking precisely when the general allocator loses (§12.1), build and measure an arena on this machine, then close with an honest accounting of when a custom allocator is worth it — and the crucial fact that it does not lift the burden of Chapters 9–11 (§12.4–12.5). This is the capstone of "manual memory": having felt its hazards, you now use it deliberately.

12.1 Why not just `malloc`? — allocation patterns

The general-purpose allocator is optimized for the case where it knows *nothing* about your requests. That ignorance is what costs you. Three patterns where a specialized allocator can do far better, precisely because it exploits knowledge the general allocator lacks:

- **Many objects, one lifetime.** A request handler, a compiler pass, a frame of a game — all allocate a burst of objects that become garbage together at a known moment.

The general allocator makes you `free` each one individually (and track every pointer to do so); an allocator that knows they share a lifetime can free the whole batch in one operation.

- **Many objects, one size.** A pool of network connections, tree nodes, fixed-size buffers. The general allocator's size-class machinery and per-block headers are overhead you do not need if every object is identical; a fixed-size allocator can hand out and reclaim blocks in O(1) with almost no bookkeeping and no fragmentation *within* that size.
- **Hot allocation loops.** Per-iteration `malloc`/`free` in a tight loop pays the general allocator's cost every iteration. Amortizing that — allocating a big block once and sub-dividing it cheaply — can dominate.

The mechanism behind all three is the same idea as Chapter 5's mechanical sympathy: when you know the access or lifetime pattern, you can lay memory out and manage it to match, and the machine rewards you. But there is a discipline here that this chapter insists on and §12.4 returns to: custom allocators are a *measured* optimization, not a default. Berger, Zorn, and McKinley's study of eight real programs using custom allocators (OOPSLA 2002) found that for most of them a good general-purpose allocator matched or beat the custom one — the big wins came specifically from the region/arena pattern (the "one lifetime" case), where they measured improvements up to 44%. The lesson is not "always write your own"; it is "when the pattern is right, the win is real and large — and when it is not, you have added complexity for nothing."

> **QUICK CHECK**
>
> Name the allocation pattern an *arena* exploits and the one a *fixed-size pool* exploits. Which single piece of knowledge does each have that the general-purpose allocator does not? (Answer after the next section.)

12.2 The arena (region / bump) allocator — build and measure it

An **arena** (or region, or bump allocator) exploits the "many objects, one lifetime" pattern. The idea is minimal: reserve one large block up front, keep a single *offset* into it, and satisfy each allocation by returning the current offset and bumping it forward by the requested size (rounded up for alignment). There is no free list, no size class, no per-block header, and — most importantly — no per-object free: individual objects are *never* freed. Instead the entire arena is reclaimed at once, either by resetting the offset to zero (reuse the memory) or by freeing the backing block. Allocation is a pointer increment; bulk-free is a single assignment or one `free`. Here is the whole allocator:

```
typedef struct { unsigned char *base; size_t cap, off; } Arena;

Arena arena_new(size_t cap){ Arena a; a.base = malloc(cap); a.cap = cap; a.off = 0; return
a; }

void *arena_alloc(Arena *a, size_t n){
    size_t p = (a->off + 15) & ~(size_t)15;   /* round up to 16-byte alignment */
    if (p + n > a->cap) return NULL;           /* arena exhausted */
    a->off = p + n;                             /* bump the offset */
    return a->base + p;
}

void arena_reset(Arena *a){ a->off = 0; }       /* frees EVERYTHING in O(1) */
```

That is a complete, working allocator. Compiled and run on this machine, it hands out correctly aligned, independent blocks (`x=42`, `s="hello"`, `y=7`, each 16-byte aligned), and `arena_reset` returns the offset to 0, reclaiming all of them in one operation. The payoff is speed, and it is worth measuring honestly. The arena wins when many objects are live *simultaneously* and die together — so the fair comparison is: allocate N objects that all stay live, then free them. With the arena that is N pointer-bumps and one bulk free; with `malloc` it is N allocations and N individual frees. For N = 5,000,000 small objects on this machine (gcc 11.4.0, `-O2`, x86-64):

```
arena  (N live, 1 bulk free) :  ~50-90 ms
malloc (N live, N frees)     : ~240-370 ms
speedup                      :  ~4-5x
```

Roughly a **four-to-five-fold** speedup, which lines up with the region wins Berger et al. reported. **These figures are illustrative and from this specific machine/run — they vary run-to-run (the ranges above are across repeated runs) and with size, allocator, and hardware; measure your own.** The win has a clear source: the arena replaced a million bookkeeping-heavy `free` calls with a single offset reset, and made each allocation a pointer add instead of a free-list search.

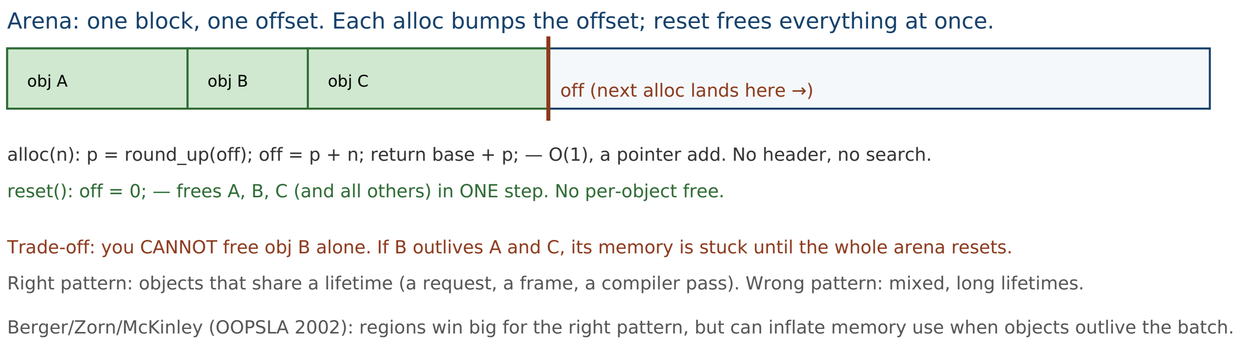

Figure 12.1: An arena allocates by bumping a single offset and frees everything at once by resetting it — O(1) allocation, O(1) bulk free, no fragmentation, no per-object free. The cost is inflexibility: you cannot free one object early, so memory is held until the whole region is reset.

The trade-off is exactly that inflexibility, and it is not free. Because you cannot free an individual object, any object that outlives the batch pins the *entire* arena's memory until reset — so an arena used with the wrong lifetime pattern can inflate memory consumption badly (Berger et al. note precisely this downside of regions). And — the theme of §12.5 — resetting an arena while a pointer into it is still in use is a use-after-free by another name: undefined behavior, exactly as in Chapter 10, now with *you* as the allocator.

*(Answer to the §12.1 quick check: an **arena** exploits "many objects share one lifetime" — it knows the batch dies together, so it can skip per-object frees and reclaim all at once. A **fixed-size pool** exploits "many objects, one size" — it knows every block is identical, so it needs no size classes, no split/coalesce, and no per-block size header.)*

12.3 The pool / free-list allocator — fixed-size, O(1) both ways

When objects share a *size* rather than a lifetime — a pool of tree nodes, connection structs, fixed buffers — a **pool allocator** (a fixed-size free-list allocator) is the specialization that fits. Carve one large block into an array of equal-size slots and thread the free slots onto a singly linked **free list**: because every slot is the same size and currently unused, you can store the "next free slot" pointer *inside the slot itself*, costing no extra memory. Allocation pops the head of the free list; freeing pushes the slot back onto the head. Both are O(1), a couple of pointer operations, with no search, no splitting, no coalescing, and — because all slots are identical — no external fragmentation within the pool at all. The sketch:

```
/* free_list points at the first free slot; each free slot's first bytes hold the next
pointer */
void *pool_alloc(Pool *p){
    if (!p->free_list) return NULL;            /* pool exhausted */
    void *slot = p->free_list;
    p->free_list = *(void **)slot;             /* pop: next free = this slot's stored next */
    return slot;
}
void pool_free(Pool *p, void *slot){
    *(void **)slot = p->free_list;             /* push: this slot's next = old head */
    p->free_list = slot;                        /* new head = this slot */
}
```

Unlike the arena, a pool *does* support freeing individual objects — that is its point — so it fits the "allocate and free repeatedly, but always the same size" pattern the arena cannot. What it gives up is generality: it serves exactly one size. This is not a toy idea. The Linux kernel's **slab allocator** — introduced in Jeff Bonwick's 1994 paper (USENIX) for SunOS and adopted widely since — is a production-grade elaboration: it maintains per-size caches of pre-initialized objects ("slabs") so that allocating a kernel object is a free-list pop and freeing it a push, retaining the object's initialized state between uses and adding a cache-coloring scheme to spread objects across cache lines (Ch 3). Production general allocators (glibc's `malloc`, jemalloc, tcmalloc) likewise use size-classed free lists internally — the pool is the general allocator's own core trick, exposed for you to specialize.

THE SAME IDEA THREE WAYS: SPECIALIZE THE ALLOCATOR TO THE PATTERN

1. In words. The general allocator is fast *because it knows nothing* is a contradiction — its generality costs search, headers, and fragmentation. If you know the pattern (shared lifetime → arena; shared size → pool), you drop the machinery you do not need and get O(1) with almost no bookkeeping.

2. As two pictures. Arena (Fig 12.1): one offset, bump to allocate, reset to free all. Pool: an array of equal slots with a free list threaded through the unused ones; pop to allocate, push to free — both O(1), no fragmentation within the size.

3. As numbers. Arena on this machine: ~4–5× faster than `malloc` for the one-lifetime batch (§12.2). Pool: allocation and free are each a couple of pointer writes versus the general allocator's free-list search and coalesce. The win is real *only* when the pattern holds (§12.4).

12.4 Choosing an allocator — an honest accounting

With three allocators in hand — general-purpose, arena, pool — the engineering question is when to reach for each. The honest answer starts with a warning that the research backs up: **the general-purpose allocator is the right default, and custom allocation is a measured optimization, not a reflex.** Berger, Zorn, and McKinley found that for six of eight real programs a state-of-the-art general allocator matched or beat the hand-written custom one — the programmers' custom allocators were often *slower* as well as buggier and harder to maintain. Custom allocation earned its keep in exactly the region/arena case. So the decision procedure is: profile first (Chapter 9's cost model, and the performance-engineering discipline Part 6 develops — measure before you optimize); confirm allocation is actually a bottleneck; then match the allocator to the pattern:

- **General-purpose `malloc`** — the default. Any size, any order, any lifetime, individual frees. Use it unless you have measured a reason not to.
- **Arena** — when a burst of objects shares a lifetime and dies together (a request, a frame, a parse). O(1) alloc, O(1) bulk free, no per-object free. Watch the memory downside: anything that outlives the batch pins the whole region.
- **Pool** — when objects share a fixed size and are allocated/freed repeatedly. O(1) both ways, no fragmentation within the size, individual frees supported. Serves exactly one size.

Each choice is a trade, not a free win. The arena trades the ability to free individually for speed and simplicity — and can waste memory if the lifetime assumption is wrong. The pool trades generality (one size only) for O(1) operations and zero fragmentation. And every custom allocator trades the maturity, debugging support, and hardening of the system allocator for code you now own and must get right. That last cost is the bridge to §12.5.

TRAP: "A CUSTOM ALLOCATOR MAKES THE MEMORY BUGS GO AWAY"

It is tempting to think that once *you* control allocation, the hazards of Chapters 9–11 are behind you. The opposite is true: **writing your own allocator makes you responsible for the very invariants the language and the system allocator's tooling used to help enforce** — and it introduces new ways to violate them. Reset an arena while a pointer into it is still live, and the next allocation reuses that memory under the old pointer: a *use-after-reset*, which is a use-after-free (Ch 9), undefined behavior (Ch 10), with you as the allocator. `pool_free` the same slot twice and you corrupt the free list — a double-free in your own code. Hand out a slot without checking the pool is non-empty and you return a null you may dereference. Worse, sanitizers (Ch 9) understand `malloc/free` but do *not*, by default, understand *your* arena's reset or *your* pool's recycling, so they may not catch a use-after-reset at all — you have moved the bug below the tool's visibility. (ASan offers manual "poisoning" hooks precisely so custom allocators can be made visible to it, but you must add them.) A custom allocator does not repeal lifetimes, bounds, and free-exactly-once; it makes you their implementer. Every rule from Chapters 9–11 still holds — you have just taken over enforcing them.

QUICK CHECK

You reset an arena to reclaim a request's memory, but one object from that request was stored in a cache that outlives the request. What bug have you just created, and what is its Chapter-9 name? (Answer below the communicate box.)

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate proposes: "Our service is slow — let's replace `malloc` with an arena allocator everywhere for a quick speed win." Cover the paragraph and respond in three or four sentences, drawing on §12.1 and §12.4.

Model answer. "An arena is a big win for a specific pattern — a burst of objects that share a lifetime and die together, like everything allocated while handling one request — but it's the wrong tool 'everywhere,' because it can't free individual objects, so any object that outlives its batch pins the whole region and can inflate our memory badly. The evidence backs the caution: Berger, Zorn, and McKinley found a good general allocator matched or beat hand-written custom allocators for most programs, with the arena/region case being the real exception. Let's profile first to confirm allocation is actually the bottleneck, then apply an arena precisely where the lifetime pattern fits — per-request scratch memory — and keep `malloc` for the mixed-lifetime long-lived state. And whatever we pick, we still own every lifetime and bound: resetting an arena with a live pointer into it is a use-after-free that our sanitizers won't see by default." Naming the pattern, citing the measured result, and insisting on profiling — rather than a blanket swap — is the senior register.

(Answer to the §12.4 quick check: you have created a ***use-after-free*** *— specifically a use-after-reset. Resetting the arena ended the lifetime of that object's storage; the cached pointer now dangles, and the next allocation from the arena will reuse those bytes under it. It is undefined behavior, and by default the sanitizers will not flag it, because they do not know your arena reset ended a lifetime.)*

12.5 What to carry forward — and out of Part 2

The general-purpose allocator is optimized for knowing nothing about your requests, and that ignorance is its cost — search, headers, coalescing, fragmentation (Ch 9). When you *do* know the pattern, a custom allocator trades generality for speed. An **arena** exploits shared lifetime: allocate by bumping an offset, free everything at once by resetting it — O(1) both ways, no per-object free, measured at roughly four-to-five times faster than `malloc` for the batch pattern on this machine, at the cost of being unable to free one object early. A **pool** exploits shared size: an array of equal slots with a free list threaded through the unused ones, O(1) allocation and free with no fragmentation within the size — the same trick the kernel's slab allocator and production `malloc`s use internally. But custom allocation is a *measured* optimization, not a default: the research (Berger et al.) shows a good general allocator usually matches hand-rolled ones, with the arena/region pattern the clear exception — so profile, confirm the bottleneck, and match the allocator to the pattern. And, decisively, writing your own allocator does not lift one obligation from Chapters 9–11: you become the implementer of lifetimes, bounds, and free-exactly-once, with new ways to violate them (use-after-reset, pool double-free) that your tools may not see.

That last point closes Part 2. Across six chapters you went from "a pointer is an address into regioned memory" (Ch 7), through lifetimes and the stack/heap (Ch 8), manual allocation and its bug catalogue (Ch 9), undefined behavior and the optimizer that trusts it (Ch 10), the buffer overflow that hijacks control flow (Ch 11), to writing the allocator itself (Ch 12) — and the through-line never changed: **in C you hold, by hand, a proof that no pointer is used outside its lifetime, no access outside its bounds, every block freed exactly once, and no undefined behavior ever triggered.** That proof is real, it is heavy, and every bug in this part is what happens when it fails. You now understand the machine to the metal and the net that is absent. Two paths lead out. The operating system (Part 3) is what sits beneath all of this — the processes, threads, and virtual memory that gave you the address space and the heap in the first place; you will see the system-call boundary from the other side. And Rust (Part 5) is the language that makes the compiler *check* the proof you have been carrying — ownership, borrowing, and lifetimes turning the invariants of this entire part into type rules verified before the program runs. You have felt the weight; the rest of the book is about the machine underneath it and the tools that lift it.

CAN YOU… WITHOUT LOOKING?

- State the three allocation patterns where a custom allocator beats general-purpose `malloc`, and the knowledge each exploits.
- Write an **arena** allocator from memory (base, capacity, offset; bump to allocate, reset to free all) and state its one big trade-off.
- Write a **pool**/free-list allocator's alloc and free (pop/push a free list threaded through the slots) and say why both are O(1) with no fragmentation within the size.
- Give the honest decision rule (general by default; profile; arena for shared lifetime; pool for shared size) and cite what the Berger et al. study found.
- Explain why a custom allocator does *not* remove the Chapter 9–11 obligations — and name a bug (use-after-reset) your sanitizers may miss.
- Summarize Part 2's through-line and how it sets up the OS (Part 3) and Rust (Part 5).

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** Name three allocation patterns where a custom allocator wins, and what each exploits. (§12.1)
2. **R2.** Describe an arena's allocate and free, its complexity, and its trade-off. (§12.2)
3. **R3.** Describe a pool/free-list allocator and why it has no fragmentation within its size. (§12.3)
4. **R4.** What did Berger et al. find about custom vs. general allocators, and what is the decision rule? (§12.1, §12.4)
5. **R5.** Why does a custom allocator not remove the Chapter 9–11 obligations? Name a bug it introduces. (§12.4–12.5)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), Chapter 9.** Dynamic memory allocation: allocator design, free lists, coalescing, and the mechanics an arena and pool specialize. The reference for §12.1's cost model.
- **Berger, Zorn & McKinley, "Reconsidering Custom Memory Allocation," OOPSLA 2002.** The empirical study behind §12.4: a good general allocator matches most hand-written custom allocators, with regions (arenas) the clear exception (improvements up to 44%, but a memory-consumption downside). Free from the authors.
- **Bonwick, "The Slab Allocator: An Object-Caching Kernel Memory Allocator," USENIX 1994.** The production pool/slab allocator behind §12.3, still used in the Linux kernel. Free.
- **The jemalloc / tcmalloc documentation.** Real, widely deployed general allocators whose internals (size-classed free lists, thread caches) are the pool idea at scale. Documented behavior only; verify against the version you use.

Exercises

12.1 WARM-UP In one sentence each: (a) what does an arena exploit? (b) what does a pool exploit? (c) which of the two can free an individual object?

12.2 WARM-UP What is the time complexity of an arena's `alloc` and its bulk `reset`? What is the one thing an arena cannot do that `malloc` can?

12.3 WARM-UP In a fixed-size pool, where is the "next free slot" pointer stored, and why does that cost no extra memory?

12.4 CORE Write, from memory, an arena allocator (a struct with base/capacity/offset, plus `arena_alloc` with 16-byte alignment and `arena_reset`). Explain why `arena_alloc` is O(1) and why `arena_reset` frees everything without touching individual objects.

12.5 CORE Write a pool allocator's `pool_alloc` and `pool_free` using a free list threaded through the slots. Explain why both are O(1) and why the pool has no external fragmentation within its size class.

12.6 CORE A teammate wants to replace `malloc` with an arena "everywhere for speed." Give two concrete reasons this is a bad blanket policy — one about the arena's inability to free individually, one citing the Berger et al. finding — and state the correct decision procedure.

12.7 COST (a) Why is an arena's per-allocation cost lower than `malloc`'s — name two things the general allocator does that the arena skips (tie to Ch 9). (b) The chapter measured ~4–5× on this machine for the "N live, then free" pattern; why does that comparison favor the arena, and what different pattern would narrow or erase the gap? (c) What memory-consumption risk does an arena carry, and when does it bite?

12.8 FADED *Worked, with the last step for you.* A per-request arena is reset at the end of each request. A developer adds a cache that stores, across requests, a pointer to a string that was allocated *in* the arena. Diagnose the bug.
Step 1 (given): `arena_reset` ends the lifetime of everything in the arena by returning the offset to 0.
Step 2 (given): The cached pointer still holds the string's old address, but that storage is now considered free.
Step 3 (given): The next request's allocations will reuse those bytes, so a later read through the cached pointer sees another request's data or garbage.
Step 4 (you): Name the bug using Chapter 9's vocabulary, state whether it is undefined behavior, explain why the ASan/Valgrind of §9.4 will likely *not* catch it by default, and give a fix (what must be copied where, so the cache does not point into the arena).

12.9 MIXED For each workload, choose *general* `malloc`, *arena*, or *pool*, and justify in a line: (a) a web handler that allocates many small objects while serving one request, all discardable when the response is sent; (b) a graph engine that constantly creates and destroys same-size nodes throughout its run; (c) a general-purpose library that cannot predict its callers' allocation patterns; (d) a compiler pass that allocates a burst of AST nodes freed together when the pass ends.

12.10 MIXED Classify each as *undefined behavior*, *a defined-but-real defect*, or *well-defined and fine*, drawing on Chapters 9–12: (a) reading through a pointer into an arena after `arena_reset`; (b) an arena holding a long-lived object that pins the whole region, growing memory use; (c) `pool_free`-ing the same slot twice; (d) resetting an arena when no live pointers into it remain.

12.11 STRETCH Part 2 has argued that in C the programmer carries, by hand, a proof about lifetimes, bounds, and freeing. (a) State that proof obligation in one sentence. (b) Explain how a custom allocator *relocates* rather than *removes* that obligation — who now enforces "free exactly once" for arena and pool memory? (c) In one or two sentences, describe what it would mean for a language (Rust, Part 5) to tie an allocation's lifetime to a scope so that "use after reset" becomes a compile-time error — and why that is stronger than a runtime sanitizer that does not understand your allocator.

12.12 LAB On your own machine, implement the arena of §12.2. (a) Verify correctness: allocate three objects of different sizes, confirm they are independent and 16-byte aligned, and that `arena_reset` returns the offset to 0. (b) Time allocating N objects that stay live then freeing them, arena (one bulk free) vs. `malloc` (N frees), for a large N, and report the ratio. (c) Repeat with a pattern where each object is freed immediately after allocation and explain why the arena's advantage shrinks. Report only what you measured and label your compiler/hardware, since numbers vary.

VOL 1 · SYSTEMS PROGRAMMING & PERFORMANCE · PART III

The Operating System

Processes, threads, scheduling, virtual memory, and the system-call boundary — how the OS multiplexes the machine.

Chapter 13 — The System-Call Boundary

Before you start: Chapter 4 (virtual addresses; the hardware translates every pointer and can refuse the access), Chapter 11 (a buffer overflow corrupts *this* process — but only this one), Chapter 12 (you became the allocator; the backing memory came from somewhere). · **Opens Part 3.** Part 2 left you holding, by hand, a proof about the memory *inside* one program. This part is about the operating system underneath all of it — the code that gave you an address space, a heap, and the illusion of owning the machine. We start at the seam where your program meets that code: the **system call**, the single controlled doorway from your program into the kernel.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 11 — a stack buffer overflow can overwrite the saved return address and hijack *this* program's control flow. Can it, by the same mechanism, overwrite memory belonging to a *different* running program? What stops it? (2) Chapter 4 — when you dereference a pointer, the address is virtual and the hardware translates it; who set up the translation tables, and could your code just rewrite them to point anywhere in physical RAM? (3) *Predict:* when your C program calls `write()` to print to the screen, does *your* code push the bytes out to the terminal device — or does something else do it on your behalf? Guess what stops an ordinary program from writing straight to the disk controller. By the end you will be able to trace exactly what happens on that call.

Every useful program eventually needs to do something it is not allowed to do. Reading a file, sending a network packet, getting more memory from the OS, creating a process, drawing on the screen — all of these touch hardware or shared state that a single process must not control directly, because the whole point of an operating system is that one buggy or malicious program cannot seize the machine or read another program's secrets. So the machine runs your code in a restricted mode, and the only way to get privileged work done is to *ask*: to make a **system call**, a controlled transfer into the kernel that switches into privileged mode, runs a specific pre-approved handler, and returns. This chapter is about that boundary. We establish the two privilege modes and why they exist (§13.1), trace exactly how a system call crosses the boundary on this machine, register by register (§13.2), separate the C library's friendly wrappers from the kernel underneath them (§13.3), measure what a crossing actually costs and how to avoid paying it (§13.4), and carry the boundary forward into processes and threads (§13.5). This is the seam Chapter 12 promised you would see "from the other side."

13.1 Two modes: user and kernel

The CPU itself enforces the boundary. A modern processor runs in one of (at least) two **privilege modes**: **user mode**, in which most of your code runs, and **kernel mode** (also called supervisor or privileged mode), in which the operating system runs. On x86 these are implemented as *rings* — ring 3 for user code, ring 0 for the kernel. The difference is not speed; it is *authority*. In user mode a set of instructions simply will not execute: you cannot halt the CPU, disable interrupts, load the page tables that define address translation (Chapter 4), or read and write device registers. Attempt one and the hardware traps into the kernel instead of obeying you.

That single mechanism is what makes the isolation of Chapter 11 real. Your buffer overflow can wreck *this* process because it runs with this process's authority — but it cannot reach another process's memory, because the memory-management unit (Chapter 4) translates every address through *this* process's page tables and faults on anything outside them, and it cannot rewrite those page tables to escape, because installing page tables is a privileged instruction it is not allowed to run. Two hardware mechanisms, working together: the MMU draws the boundary around each process's *memory*, and the user/kernel mode bit draws the boundary around the machine's *instructions and devices*. Everything you have written in this book so far has run in ring 3, with no direct access to any of it. The kernel is the only software that runs privileged, and it is the referee that hands out access under its own rules.

This is the answer to the opener's first two questions. A process cannot corrupt another process's memory because it is confined to its own translated address space and cannot touch the translation machinery; it cannot rewrite the page tables because that instruction traps. The confinement is not politeness — it is enforced by silicon on every instruction.

> **QUICK CHECK**
>
> Name one privileged operation that a program in user mode is *not* allowed to perform directly, and say what the hardware does if it tries anyway. (Answer after the next section.)

13.2 Crossing the boundary: the system-call mechanism

If user code cannot touch devices or memory tables, how does it ever read a file? It asks the kernel to do it, through a deliberately narrow door. The mechanism is a **trap**: a special instruction that simultaneously switches the CPU into kernel mode *and* jumps to a single, fixed entry point that the kernel installed at boot. This is the crucial safety property — user code does not get to switch into kernel mode and then run wherever it likes; the trap lands only at the kernel's chosen handler, which decides what to do. On 64-bit x86 Linux that instruction is `syscall` (the older 32-bit path used the software interrupt `int 0x80`).

Because the boundary is narrow, there is a strict convention — an **application binary interface (ABI)** — for how you name the call and pass its arguments. On x86-64 Linux (verified against the `syscall(2)` manual page): the **system-call number** goes in register `rax`; the first six arguments go in `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9` (note `r10`, not `rcx` — the `syscall` instruction itself clobbers `rcx` and `r11`); you execute `syscall`; and the kernel returns the result in `rax`. A negative return in the range of a small error code means failure — the raw kernel interface returns `-errno` (§13.3 turns that into the friendly `errno` you know). The kernel also *validates* what you pass: hand it a pointer to write into, and it checks that the pointer really belongs to your address space, so you cannot trick it into scribbling on kernel memory on your behalf.

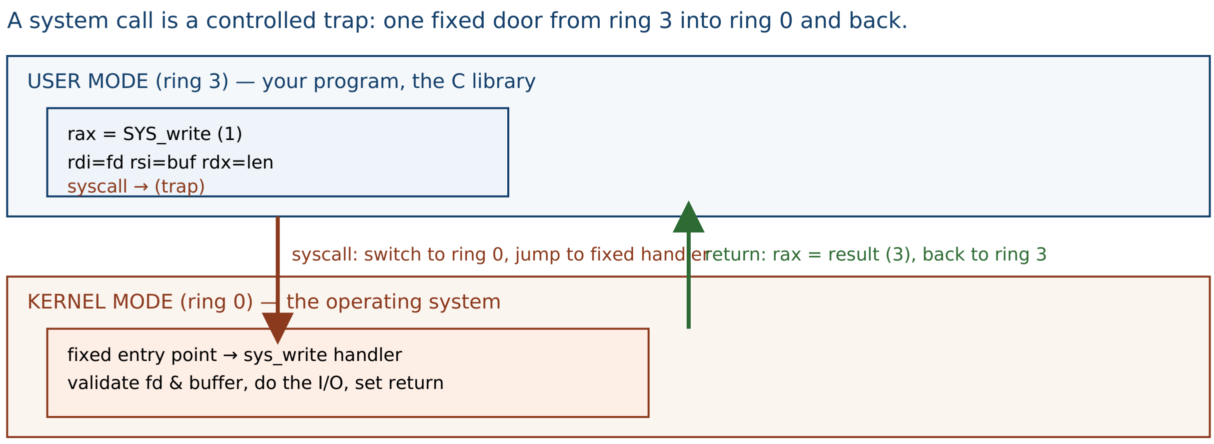

Figure 13.1: A system call. User code loads the call number and arguments into the ABI's registers and executes `syscall`; the CPU switches to kernel mode and jumps to one fixed entry point (never an address of the caller's choosing); the kernel validates, does the privileged work, and returns the result in `rax`. The boundary is narrow by design — that narrowness is the security.

Here is the whole path made concrete. When a program runs `write(1, "hi\n", 3)` — write three bytes to file descriptor 1, standard output — the sequence is: put the number for `write` in `rax`, put `1` in `rdi`, the buffer address in `rsi`, and `3` in `rdx`; execute `syscall`; the CPU traps to the kernel, which writes the bytes and returns `3` (the count written) in `rax`. We can watch this happen with `strace`, which intercepts and prints every system call a program makes. Compiling a three-line C program that does exactly that and running it under `strace` on this machine (gcc 11.4.0, x86-64 Linux) shows the crossing directly:

```
$ strace -e trace=write ./hello
write(1, "hi\n", 3)                     = 3
+++ exited with 0 +++
```

That one line *is* the boundary: the arguments the program marshaled, and the `= 3` the kernel returned. Everything else the program did — computing, moving data in its own memory — never appears, because none of it crossed into the kernel.

(Answer to the §13.1 quick check: a user-mode program may not execute privileged instructions such as halting the CPU, disabling interrupts, loading the page tables, or accessing device registers directly. If it tries, the hardware does not obey — it traps into the kernel, which typically terminates the offending process.)

13.3 The library wrapper vs. the raw system call

You have almost certainly never written `syscall` by hand, and you rarely will. The C library (glibc on most Linux systems) provides an ordinary-looking function for each system call — `write`, `read`, `open`, `fork` — and that **wrapper** does the register marshaling for you: it moves your arguments into `rdi`/`rsi`/`rdx`, sets `rax`, executes `syscall`, and inspects the result. On the negative-return failure convention from §13.2, the wrapper does the translation you already know: it stores the error code in the global `errno` and returns `-1`. So the raw kernel interface returns `-EBADF`; the wrapper you call returns `-1` and sets `errno == EBADF`. The wrapper is a thin shell around one `syscall`.

The distinction that trips people up is that *not every library function is a wrapper*, and the ones that matter most are not. `printf` is **not** a system call. It is a substantial C library routine that parses your format string, converts numbers to text, and — critically — *buffers* the result in user-space memory, only calling the `write` system call later, when the buffer fills or is flushed. Similarly `malloc` (Chapters 9 and 12) is a library function that manages a heap in user space; it crosses into the kernel only occasionally, calling the `brk` or `mmap` system calls when it needs to grow the heap — which is exactly where your arena's backing block came from in Chapter 12. This is the "other side" that chapter promised: your allocator handed out bytes that the C library had obtained, in bulk, from the kernel across this very boundary. The layering is real and worth holding precisely: *your code* calls a *library function*, which may buffer and compute in user space, and which crosses into the *kernel* only when it must, through a *system-call wrapper*.

> **QUICK CHECK**
>
> You run a program that prints with `printf`, then crashes before returning from `main` — and the printed line never appears in your terminal. Nothing was wrong with the format string. What most likely happened to those bytes? (Answer below the communicate box.)

13.4 The cost of crossing — and how to avoid paying it

Crossing the boundary is not free, and the cost is the reason a great deal of systems performance work exists. A system call must switch privilege mode, save and later restore the caller's state, enter the kernel, validate arguments, and eventually switch back — and along the way it can disturb the CPU's caches and TLB (Chapters 3–4). Compared with an ordinary function call, which on a modern core is a couple of instructions, that is enormously more expensive. Measured on this machine (gcc 11.4.0, `-O2`, x86-64), calling a trivial non-inlined function ten million times versus making the `getpid` system call the same number of times:

```
plain function call :  ~1-2 ns each
getpid system call  :  ~150-190 ns each
ratio               :  roughly two orders of magnitude
```

These figures are illustrative, from this specific machine and run — they vary with hardware, kernel, and mitigations (some CPU-vulnerability mitigations make the boundary crossing markedly more expensive), and you should measure your own. But the shape is robust and the lesson is durable: a system call costs on the order of a hundred function calls or more. Three consequences follow, and each is a technique you will use:

- **Batch — do not cross per item.** Reading a megabyte one byte at a time is a million crossings; reading it in large blocks is a handful. This is exactly why the C library buffers: `printf` and `fread` accumulate in user space and cross the boundary rarely. The single most common I/O performance bug is an unbuffered per-item system call in a hot loop.

- **Let the kernel share read-only data without a crossing — the vDSO.** A few "system calls" that only *read* kernel-maintained data, such as `clock_gettime` and `gettimeofday`, are served from the **vDSO** (virtual dynamic shared object): a page the kernel maps read-only into every process, containing the data and the code to read it, so the call runs entirely in user mode with no trap. On this machine, a program that calls `clock_gettime` one million times produces, under `strace -c`, *zero* `clock_gettime` system calls — the crossings simply are not there, because the vDSO answered in user space. (You can see the vDSO mapped into any process: `ldd` on a dynamically linked program lists `linux-vdso.so.1`.)
- **Observe the boundary with `strace`.** Because every crossing is visible, `strace` (and `strace -c` for a counted summary) is the first tool to reach for when a program is mysteriously slow or stuck: it shows you exactly which system calls it is making, how many, and where it is blocked.

THE SAME IDEA THREE WAYS: A SYSTEM CALL IS A CONTROLLED TRAP

1. In words. User code cannot touch devices or memory tables, so it *asks* the kernel by executing a trap instruction that switches to kernel mode and jumps to one fixed handler — never an address of the caller's choosing. The kernel validates, does the privileged work, and returns. Narrowness is the security.

2. As a picture. Figure 13.1: two stacked bands (ring 3, ring 0) with a single arrow down (`syscall` → fixed entry) and a single arrow up (return in `rax`).

3. As numbers. The ABI: number in `rax`; args in `rdi`,`rsi`,`rdx`,`r10`,`r8`,`r9`; result in `rax`; `-errno` on failure. The cost: ~150-190 ns for `getpid` versus ~1-2 ns for a function call on this machine — roughly 100×, which is why you batch and why the vDSO exists.

TRAP: "`PRINTF` IS A SYSTEM CALL" — CONFUSING THE LIBRARY WITH THE KERNEL

It is tempting to treat every function that produces output as a direct line to the operating system. **It is not: `printf` is a C library function that formats and *buffers* in user-space memory, and calls the `write` system call only when that buffer flushes.** Conflating the two produces real, confusing bugs. A program that `printf`s a line and then crashes may show *nothing* in the terminal, because the bytes were sitting in the library's buffer and the crash skipped the flush — the boundary was never crossed (this is the §13.3 quick check, and it returns in Chapter 14 when `fork` *duplicates* that unflushed buffer and a line prints twice). The fix when you need output to be real *now* is to force the crossing: `fflush(stdout)`, or configure line buffering, or call `write` directly — which is unbuffered and goes straight across. The mental model that prevents a dozen bugs: *your code → a library function (may buffer) → a system-call wrapper → the kernel*. Know which layer you are on. The library's job is often to *avoid* the kernel; the kernel is only reached deliberately.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate reports: "Our service writes one log line per request with a bare `write()` call, and profiling says it is spending huge time in the kernel. Let's just delete the logging." Cover the paragraph and respond in three or four sentences, drawing on §13.3–13.4.

Model answer. "The problem is almost certainly not *that* we log, but *how* — one unbuffered `write` per line is one system-call crossing per line, and a crossing costs on the order of a hundred function calls or more (on our box `getpid` was ~150 ns versus ~1-2 ns for a call), so at high request rates those crossings dominate. The fix is to batch: buffer log lines in user space and flush in large blocks, which is exactly why the C library buffers `printf` in the first place; we cross the boundary a few times instead of thousands. Let's confirm with `strace -c` that `write` is the hot call, switch to a buffered logger, and re-measure — deleting logging throws away observability to fix a batching bug." Naming the crossing cost, reaching for batching rather than removal, and confirming with `strace` is the senior register.

(Answer to the §13.3 quick check: the bytes were almost certainly still in `printf`'s user-space buffer. Because `printf` buffers and only crosses into the kernel via `write` on a flush, a crash before the flush loses them — they never reached the terminal. Force the crossing with `fflush`, line buffering, or a direct `write`.)

13.5 What to carry forward

The CPU runs your code in **user mode** (ring 3), where privileged instructions — touching devices, loading page tables, disabling interrupts — simply will not execute, and the operating system in **kernel mode** (ring 0) is the only software with that authority. Together the MMU (Chapter 4) and this mode bit are what make process isolation real: a bug confined to your address space cannot reach another process or the machine. To get privileged work done, user code makes a **system call** — a controlled trap (`syscall` on x86-64) that switches mode and jumps to *one fixed kernel entry point*, passing a call number in `rax` and arguments in `rdi,rsi,rdx,r10,r8,r9`, receiving a result in `rax` (with `-errno` on failure). The C library wraps each call in an ordinary function and translates the error convention into `errno` and `-1` — but many library functions (`printf`, `malloc`) are *not* system calls; they compute and buffer in user space and cross only when they must. And the crossing is expensive — on this machine roughly two orders of magnitude more than a function call — so you batch, you lean on the vDSO for cheap read-only data like the time, and you observe the boundary with `strace`.

This is the seam the rest of Part 3 lives on. A **process** (Chapter 14) is the isolated container this boundary protects — its own address space, created and destroyed through system calls (`fork`, `execve`, `exit`, `wait`) that cross exactly this line. A **thread** (Chapter 15) is a second flow of control that shares one side of the boundary — the same address space — trading isolation for cheap sharing. Every abstraction the OS gives you, from the heap that fed your allocator to the files and sockets you will read later, is reached through the doorway you have just learned to trace.

CAN YOU… WITHOUT LOOKING?

- Name the two CPU privilege modes and give two operations user mode is forbidden to do — and say what enforces the ban.
- Explain why a system call jumps to a *fixed* kernel entry point rather than an address the caller chooses, and why that matters.
- State the x86-64 Linux system-call ABI: where the call number, the arguments, and the return value go.
- Explain how `printf` differs from the `write` system call, and why a crash can lose a `printf`ed line.
- Give the rough cost of a system call versus a function call on a machine you have measured, and name two ways to cross the boundary less (batching, the vDSO).
- Say how process isolation follows from the MMU plus the user/kernel mode bit.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is the difference between user mode and kernel mode, and what enforces it? (§13.1)
2. **R2.** Describe the system-call mechanism on x86-64 Linux: instruction, where the number and arguments go, where the result comes back. (§13.2)
3. **R3.** Is `printf` a system call? What actually crosses the boundary, and when? (§13.3)
4. **R4.** Roughly how much more expensive is a system call than a function call, and why? (§13.4)
5. **R5.** What is the vDSO, and why does calling `clock_gettime` often *not* show up as a system call? (§13.4)

FURTHER READING

- **Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP), "Mechanism: Limited Direct Execution."** The free chapter that develops exactly this boundary — user vs. kernel mode, traps, and the system-call mechanism. The reference for §13.1–13.2.
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), Chapter 8.** Exceptional control flow: how the hardware and OS handle traps and system calls from the programmer's view.
- **The Linux `syscall(2)` and `vdso(7)` manual pages.** The primary sources for the per-architecture calling convention of §13.2 and the vDSO of §13.4. Verify against your kernel and architecture.
- **`strace` documentation and man page.** The tool used throughout this chapter to make the boundary visible; `strace -c` for a counted summary. Free.

Exercises

13.1 WARM-UP Name the two CPU privilege modes, and give one operation a program in user mode is not allowed to perform directly.

13.2 WARM-UP On x86-64 Linux, which register holds the system-call number, and which register holds the return value?

13.3 WARM-UP Is `printf` a system call? If not, what does it do first, and which system call does it eventually use to produce output?

13.4 CORE Trace, step by step, what happens when a program executes `write(1, buf, 5)`: what does the C library put in which registers, what instruction crosses the boundary, and what comes back on success? (Use the ABI from §13.2.)

13.5 CORE Explain the relationship between the kernel's raw error convention and the `errno`/`-1` you see from a C library call. If a raw system call returns `-EBADF`, what does the wrapper return and what does it set?

13.6 CORE You run `strace` on a program and see, among other lines: `openat(...) = 3`, `read(3, ..., 4096) = 512`, and then thousands of `read(3, ..., 1) = 1` lines. In one or two sentences, say what the last pattern tells you is wrong and what to change.

13.7 COST (a) Name two reasons a system call costs far more than an ordinary function call. (b) A program reads a 1 MB file by calling `read` one byte at a time; explain why this is slow in terms of boundary crossings, and give the fix. (c) Why does calling `clock_gettime` in a tight loop often cost *no* system calls at all?

13.8 FADED *Worked, with the last step for you.* A program prints a diagnostic line with `printf` just before dereferencing a bad pointer and crashing. The line never appears in the terminal, even though the program definitely reached the `printf`.
Step 1 (given): `printf` writes into the C library's user-space output buffer, not directly to the terminal.
Step 2 (given): The buffer is flushed (crossed to the kernel via `write`) only when it fills, at a newline for a line-buffered terminal, or at normal program exit.
Step 3 (given): A crash from an invalid dereference terminates the process abnormally, skipping the normal flush.
Step 4 (you): State where the bytes actually were when the program died, explain why the line was lost (referencing the boundary), and give two ways to make sure the line appears before a possible crash.

13.9 MIXED For each, decide whether it requires crossing the system-call boundary or is pure user-space computation, and say why in a few words: (a) adding two integers in registers; (b) opening a file; (c) sorting an in-memory array; (d) sending a network packet; (e) formatting a number into a string with `sprintf`.

13.10 MIXED Drawing on Chapters 10–13, classify each as *undefined behavior*, *a boundary crossing (system call)*, or *neither*: (a) reading one past the end of a stack array; (b) calling `read` on a valid file descriptor; (c) signed integer overflow in a loop counter; (d) a buffered `printf` that has not yet flushed.

13.11 STRETCH The kernel validates every pointer you pass to a system call such as `write`, checking it belongs to your address space. (a) Relate this to Chapter 4's address translation: why can a process not simply pass a pointer to another process's memory or to kernel memory and have the kernel read it? (b) Sketch what could go wrong for security if the kernel skipped this validation. (c) Explain how the two mechanisms — per-process address translation and the user/kernel mode bit — together make one process unable to spy on another.

13.12 LAB On your own machine: (a) write a program that prints with `write` and run it under `strace -e trace=write`; confirm you see the `write` call and its return value. (b) With `strace -c`, count the total system calls of a trivial "hello world" and note how many are startup overhead (dynamic linking, etc.) versus your own output. (c) Measure the cost of the `getpid` system call versus a trivial function call in a loop, and (d) call `clock_gettime` a million times and confirm with `strace -c` that it does *not* appear as a million system calls. Report your numbers and label your compiler, kernel, and hardware, since figures vary.

Chapter 14 — Processes: fork, exec, and wait

Before you start: Chapter 13 (the system-call boundary — `fork`, `exec`, and `wait` are all system calls), Chapter 4 (each process has its own virtual address space, translated through its own page tables), Chapter 11 (a memory bug is confined to *one* process's address space). · Pairs with Chapter 15 (threads share an address space instead of copying it). Chapter 13 showed the boundary that protects a process; this chapter is about the process itself — what it is, how the OS makes a new one, how one program becomes another, and how a parent collects a finished child.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 13 — to do something privileged, user code must do what, and through which instruction on x86-64? (2) Chapter 11 — a buffer overflow can corrupt the process it runs in; can it corrupt a *different* running process's memory, and what two mechanisms decide the answer? (3) *Predict:* `fork()` makes a new process that is a copy of the calling one. If your program is holding a 1 GB array in memory and calls `fork()`, do you think the operating system immediately copies all 1 GB? Guess, and note why the answer matters for how often you can afford to `fork`. We measure the idea behind it shortly.

A running program is not the same thing as the file on disk. The file is inert bytes; the running instance — with its own memory, its own place in the CPU, its own open files — is a **process**, and the process is the operating system's fundamental unit of isolation and accounting. Everything Chapter 13 protected was protecting a process: its private address space, its authority level, its resources. This chapter opens the process up. We define precisely what a process bundles together (§14.1); we build a new one with the Unix system call `fork`, and see why it returns *twice* and why copying a process can be cheap (§14.2); we turn a copied process into a *different* program with `exec`, and meet the fork-then-exec pattern that every shell uses (§14.3); we collect a finished child with `wait`, and meet the zombie and the orphan (§14.4); and we carry the model into threads (§14.5). By the end you will be able to write, and reason about, the create-run-reap lifecycle that underlies every command you have ever typed.

14.1 What a process is — the unit of isolation

A process is three things bundled and protected together. First, an **address space**: the private virtual memory of Chapters 4, 7, and 8 — text, initialized data, bss, heap, and stack — that the MMU translates through this process's own page tables so that its pointers name only its own memory. Second, one or more **threads of execution**: the actual running context — the program counter, the stack pointer, the register values — that says *where* in the code the process currently is (this chapter's processes have exactly one; Chapter 15 adds more). Third, a bundle of **kernel-managed resources**: the open file descriptors, the process ID (PID) and parent PID, the current working directory, the user and group credentials, and more, all tracked by the kernel on the process's behalf.

The reason these are bundled is **isolation**. The kernel gives each process the illusion of owning the machine: its own address space (so one process's pointers are meaningless in another's — the answer to Chapter 11's confinement), its own turn on the CPU, its own resource handles. A fault in one process — a wild pointer, a crash, even a security compromise — is contained to that process's address space and cannot, by the mechanisms of Chapter 13, corrupt another process or the kernel. This is why the process is the natural boundary for a sandbox, a multi-tenant service, or any place you need one failure not to become everyone's failure. The process is precisely the thing the system-call boundary was built to protect.

> **QUICK CHECK**
>
> Name the three things a process bundles together. Which one makes two processes' identical-looking pointer values refer to completely different memory? (Answer after the next section.)

14.2 Creating a process: `fork` and copy-on-write

Unix creates a new process in a way that surprises everyone the first time: the `fork()` system call is called *once* but returns *twice*. It makes the calling process (the **parent**) into two nearly identical processes — the parent and a new **child** — and then both continue from the point of the return. The child is a copy: same code, same current values of every variable, copies of the open file descriptors. The one difference is the return value, which is how each copy knows who it is: `fork` returns **0 in the child** and **the child's PID in the parent** (or `-1` on failure). You branch on it. Here is the whole pattern, compiled and run on this machine (gcc 11.4.0, x86-64 Linux):

```
int x = 100;
pid_t pid = fork();          /* one call, returns TWICE */
if (pid == 0) {              /* child sees 0 */
    x += 1;
    printf("child : pid=%d ppid=%d x=%d  (fork returned %d)\n", getpid(), getppid(), x, pid);
} else {                     /* parent sees the child's pid */
    int st; waitpid(pid, &st, 0);
    x += 10;
    printf("parent: pid=%d x=%d  (fork returned %d; child exit=%d)\n", getpid(), x, pid,
WEXITSTATUS(st));
}
```

```
child : pid=1341934 ppid=1341933 x=101  (fork returned 0)
parent: pid=1341933 x=110  (fork returned 1341934; child exit=0)
```

Read that output closely, because it shows every essential fact. The two processes have different PIDs (1341934 and 1341933), and the child's parent PID equals the parent's PID — the family tree is real. Both started with `x == 100`, but the child's `x += 1` and the parent's `x += 10` did not interfere: the child ends at 101 and the parent at 110. Their memory is *independent* — the same variable name, at what is even the same virtual address, but two separate physical copies, because each process has its own address space (§14.1). And `fork` returned 0 down one path and the child's PID down the other, which is the only thing distinguishing the twins.

Now the predict from the opener: if the parent held a 1 GB array, did `fork` copy a gigabyte? No — and it could not afford to, or forking would be unusable. The kernel uses **copy-on-write (COW)**: rather than duplicating the parent's pages, it maps the *same* physical pages into both processes and marks them read-only. Both share the physical memory as long as they only read it. The instant *either* one writes to a page, the hardware faults, the kernel makes a private copy of just that one page for the writer, and both continue. So `fork`'s cost is proportional not to the size of the address space but to the number of pages actually modified afterward — which is why forking a huge process is cheap, and why the child's `x += 1` above triggered a copy of exactly the one page holding `x` and nothing else. Copy-on-write is Chapter 4's page tables doing double duty: the same translation-and-fault machinery that isolates processes also lets them share until the moment sharing becomes incorrect.

THE SAME IDEA THREE WAYS: `FORK` RETURNS TWICE INTO TWO ADDRESS SPACES

1. In words. One call, two returns: the parent gets the child's PID, the child gets 0, and from then on they are separate processes with separate (copy-on-write) memory. You branch on the return value to run different code in each.

2. As a picture. Figure 14.1: a single arrow entering `fork` and two arrows leaving, into two boxes that start as copies and diverge as each writes its own pages.

3. As a trace. Before: one process, `x = 100`. After `fork`: two processes, both `x = 100`, distinguished by return value (child 0, parent child-PID). After each runs: child `x = 101`, parent `x = 110` — independent, exactly as the measured output shows.

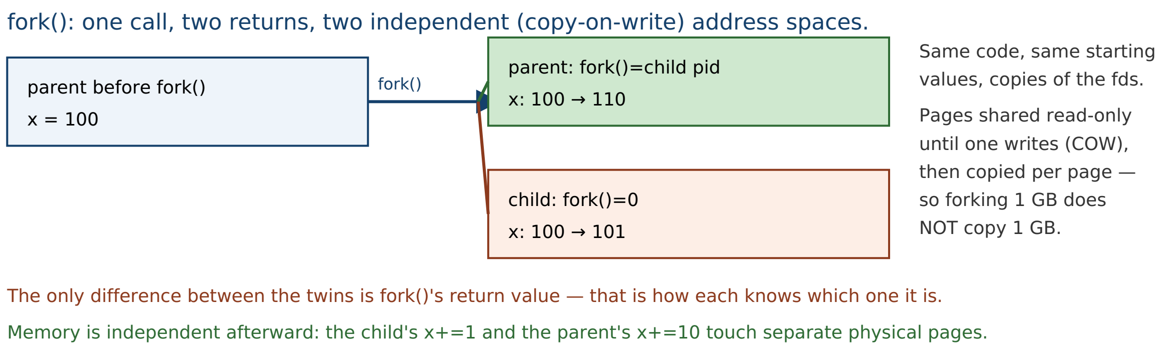

Figure 14.1: `fork` returns twice — 0 in the child, the child's PID in the parent — producing two processes that start as copies and, thanks to copy-on-write, share physical pages until one writes. The branch on the return value is how each copy runs different code.

*(Answer to the §14.1 quick check: a process bundles an **address space**, one or more **threads of execution**, and a set of **kernel resources** such as open file descriptors and its PID. The **address space** — separate per process and translated through its own page tables — is what makes two processes' identical-looking pointer values refer to different memory.)*

14.3 Running a new program: `exec` and the fork-exec pattern

`fork` gives you a copy of the *same* program. To run a *different* program you need `exec` — the `execve` system call and its convenience wrappers. `exec` does something drastic: it **replaces the current process image** — text, data, heap, stack — with a brand-new program loaded from disk, and jumps to its entry point. Everything the process was running is gone; the new program starts fresh. Two things survive the replacement, and they are the important ones: the **PID stays the same** (it is the same process, now running different code), and the **open file descriptors stay open** (unless marked close-on-exec). On success, `exec` *does not return* — there is nothing to return to, because the code that called it no longer exists.

Put the two together and you get the pattern behind every command you run: **fork, then exec, then wait**. To launch a program, a process `fork`s; the child `exec`s the new program (replacing itself with it); and the parent `wait`s for the child to finish. This is exactly what a shell does for every command you type — `fork` a child, have it `exec /bin/ls`, and wait for it to complete before printing the next prompt — and it is how your terminal launched the demo program in §14.2. The pattern is also why the parent in that demo called `waitpid` before continuing.

Why split creation (`fork`) from loading (`exec`) into two calls, when almost every use runs them back to back? Because the *gap between them* is where the elegance lives. After the child is forked but before it execs, it is still running the parent's code and can adjust its inherited resources: it can redirect standard output to a file, wire its input or output to a pipe connected to a sibling, drop privileges, or change directory — and then `exec` the target program, which starts up already plumbed into that environment without knowing anything about it. Unix I/O redirection and pipelines (`ls | grep foo > out.txt`) are built entirely in that fork-to-exec gap. The two-call design is not clumsiness; it is the seam that makes composition possible.

> **QUICK CHECK**
>
> A shell runs the command `ls`. In order, which of `fork`, `exec`, and `wait` does the shell process perform, which does the new child process perform, and which call does *not* return on success? (Answer below the communicate box.)

14.4 Reaping: `wait`, exit status, zombies, and orphans

A process ends by calling `exit` (or returning from `main`, or being killed). But it does not vanish the instant it ends. The kernel must keep a small amount of information about it — most importantly its **exit status**, the number it passed to `exit` — so that the parent can find out how the child fared. A process that has terminated but whose exit status has not yet been collected is a **zombie** (shown as `<defunct>` in `ps`): it holds no memory or CPU, just a slot in the process table and its exit status, waiting to be read.

The parent collects it by calling `wait` or `waitpid`, which blocks until a child finishes (or returns immediately if one already has), hands back the child's PID and its exit status (decode it with macros like `WEXITSTATUS`), and lets the kernel finally free the zombie's last slot. This is **reaping**. It is the parent's responsibility, and forgetting it is a real defect: a long-running parent that `fork`s children and never `wait`s accumulates zombies, slowly filling the process table. Note the category carefully — a leaked zombie is a *defined* resource leak, not undefined behavior; the program is not doing anything the language forbids, it is just failing to clean up, exactly as a memory leak is defined-but-bad (Chapter 9).

The mirror-image case is the **orphan**: a child whose parent exits *first*. An orphan is not left dangling — the kernel **reparents** it to `init` (PID 1, or a subreaper), which `wait`s for it when it finishes, so orphans are reaped automatically. The lifecycle, then, is: a process is `fork`ed, runs, `exit`s (becoming a zombie holding its status), and is reaped by a `wait`ing parent (or by `init` if orphaned). Create, run, exit, reap.

TRAP: `FORK` DUPLICATES YOUR UNFLUSHED OUTPUT BUFFER (THE DOUBLE-PRINT)

Chapter 13 warned that `printf` buffers in user-space memory and crosses to the kernel only on a flush. `fork` copies the *entire* address space — **including that buffer with its unflushed contents.** So if you `printf` a line and *then* `fork`, the un-crossed bytes sit in both the parent's and the child's copy of the buffer, and when each process later flushes (at exit), the line is written *twice* — once per process — even though your code printed it once. This is not hypothetical: constructing exactly this (a `printf` before the fork, with output going to a pipe so buffering is block-mode) reproduces the doubled line every time. The trap is subtle because it disappears when output goes to a terminal (line-buffered, so the newline already flushed before the fork) and reappears when output is redirected to a file or pipe. The fix is the same as Chapter 13's: force the crossing before you branch — `fflush(stdout)` before `fork` (or use unbuffered `write`). The deeper lesson: `fork` copies *everything*, including the parts of your program's state you had forgotten were state.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate proposes: "Let's handle each incoming request by `fork`ing a fresh process — that way one bad request can't take down the others." Cover the paragraph and respond in three or four sentences, drawing on §14.1–14.2.

Model answer. "That is a genuinely good instinct for *isolation*: a separate process has its own address space, so a crash or memory corruption in one request is contained and can't touch a sibling — it's the classic robustness pattern, and `fork` is cheaper than it looks because copy-on-write means we don't duplicate the whole address space, only the pages a child actually writes. The costs to weigh are that processes don't share memory, so passing data between them needs explicit IPC (pipes, shared memory) rather than a pointer, and each request pays process-creation and context-switch overhead, which can hurt at very high rates. So process-per-request is the right call when fault-containment or a security boundary is the priority; if instead we need cheap shared-memory concurrency and can enforce discipline, threads (Chapter 15) share the address space and skip the copy — at the cost of exactly that isolation. Let's decide based on whether containment or shared-state throughput is what this service needs, and measure." Naming copy-on-write, the isolation-vs-sharing trade, and deferring to a measurement is the senior register.

(Answer to the §14.3 quick check: the shell process performs `fork` and then `wait`; the new child process performs `exec` (replacing itself with `ls`). `exec` is the one that does not return on success — on success there is no original program left to return to.)

14.5 What to carry forward

A **process** is the operating system's unit of isolation: an **address space** (private virtual memory, Chapter 4), one or more **threads of execution** (the running context), and a bundle of **kernel resources** (open files, PID, credentials), all protected by the boundary of Chapter 13. You create one with `fork`, which is called once and **returns twice** — 0 in the child, the child's PID in the parent — producing two processes that start as copies and diverge; copying is cheap because of **copy-on-write**, which shares physical pages until one side writes, so forking a huge process costs only the pages later modified. You turn a copied process into a different program with `exec`, which **replaces the image** (keeping the PID and open descriptors) and does not return on success; the universal **fork-exec-wait** pattern — with the fork-to-exec gap used to set up redirection and pipes — is how every shell runs every command. And you clean up with `wait`, which reaps a finished child's exit status; an unreaped terminated child is a **zombie** (a defined leak), while an orphaned child is reparented to `init` and reaped for you.

Every process in this chapter had a single thread of execution — one program counter walking through the code. Chapter 15 relaxes that. A **thread** is a second (third, hundredth) flow of control *within one process*, sharing its address space rather than copying it as `fork` does. That one change — share instead of copy — buys cheap concurrency and drops the isolation this chapter spent its length building: the very memory that `fork` kept separate, threads deliberately share, and the proof burden of Part 2 returns, now with multiple threads reaching for the same bytes at once.

CAN YOU… WITHOUT LOOKING?

- State the three things a process bundles together, and which one enforces memory isolation between processes.
- Explain why `fork` returns twice, what it returns to each side, and how a program uses that to run different code in parent and child.
- Explain copy-on-write and why forking a 1 GB process does not copy 1 GB.
- Describe what `exec` replaces and what it keeps, and why it does not return on success.
- Lay out the fork-exec-wait pattern and say what the gap between fork and exec is used for.
- Define a zombie and an orphan, say who reaps each, and classify a leaked zombie (UB or defined defect?).

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What three things make up a process, and what does the address space give it? (§14.1)
2. **R2.** What does `fork` return in the parent and in the child, and why is that the only difference between them? (§14.2)
3. **R3.** What is copy-on-write, and why does it make `fork` cheap? (§14.2)
4. **R4.** What does `exec` do, what survives it, and does it return on success? (§14.3)
5. **R5.** What is a zombie, what is an orphan, and who reaps each? (§14.4)

FURTHER READING

- **Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP), "Abstraction: The Process" and "Interlude: Process API."** The free chapters that build exactly this material — the process abstraction and the `fork/exec/wait` API. The reference for §14.1–14.4.
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), Chapter 8 (Exceptional Control Flow).** Processes, process creation, and reaping children, from the programmer's view.
- **The Linux `fork(2)`, `execve(2)`, and `wait(2)` manual pages.** Primary sources for the exact semantics, including copy-on-write, the fate of file descriptors across `exec`, and zombie/orphan behavior. Verify per system.
- **Beej's Guide to Unix IPC / C.** Free, practical walkthroughs of `fork/exec` and inter-process communication (pipes) — the plumbing that lives in the fork-to-exec gap.

Exercises

14.1 WARM-UP Name the three things a process bundles together.

14.2 WARM-UP What does `fork` return in the parent process, and what does it return in the child?

14.3 WARM-UP Does `exec` return on success? What does it replace, and name one thing it keeps.

14.4 CORE Write, from memory, the `fork/if-else` skeleton that distinguishes child from parent, with the parent calling `waitpid`. Explain in a sentence what each branch is and how each process knows which it is.

14.5 CORE Explain copy-on-write: what does the kernel actually do to the parent's pages at `fork` time, what triggers a real copy, and why does this make `fork`ing a 1 GB process fast?

14.6 CORE Describe the fork-exec-wait pattern a shell uses to run a command. Then explain what useful work can happen in the gap between `fork` and `exec`, and give one concrete example (e.g. I/O redirection).

14.7 COST (a) Explain why `fork` is cheaper than the size of the address space would suggest — name the mechanism. (b) Describe a workload for which copy-on-write buys you almost nothing (the child writes to nearly all inherited pages) and say why. (c) Compare process-per-request and thread-per-request on one cost each pays that the other does not.

14.8 FADED *Worked, with the last step for you.* A long-running server `fork`s a worker for each connection and never calls `wait`. After hours, `ps` shows hundreds of `<defunct>` entries and the server eventually cannot create new processes.
Step 1 (given): When a child exits, the kernel keeps its exit status until the parent collects it, leaving the child as a zombie.
Step 2 (given): This server never calls `wait/waitpid`, so no child's status is ever collected.
Step 3 (given): Each zombie holds a process-table slot, and the table is finite.
Step 4 (you): Name the accumulating state, explain why it eventually stops new `fork`s from succeeding, say whether this is undefined behavior or a defined resource leak, and give a fix (how the parent should reap, mentioning `wait/waitpid` or handling `SIGCHLD`).

14.9 MIXED For each goal, decide whether you need `fork`, `exec`, or `fork` then `exec`, and say why in a line: (a) run `/bin/ls` from inside your program and return to your program afterward; (b) create a second process that runs the *same* code on the second half of an array; (c) a shell running a command the user just typed.

14.10 MIXED Drawing on Chapters 9–14, classify each as *undefined behavior*, *a defined-but-real defect*, or *well-defined and fine*: (a) after `fork`, the parent expects to see changes the child made to `x` in the child's memory; (b) a server that never reaps its children; (c) both parent and child printing a line that was `printf`ed (unflushed) before the `fork`; (d) a child that `exec`s a new program while the parent keeps running.

14.11 STRETCH Tie process isolation back to Chapters 11 and 13. (a) Explain why a buffer overflow in process A cannot corrupt process B's memory, naming the two hardware mechanisms (Chapters 4 and 13) that enforce it. (b) Explain why this makes a separate process, not a thread, the right unit for sandboxing untrusted code. (c) In one or two sentences, describe what a multi-tenant service gains by putting each tenant's work in its own process rather than sharing one.

14.12 LAB On your own machine: (a) write the `fork` demo of §14.2 and confirm from its output that `fork` returned twice, that the child's `ppid` equals the parent's `pid`, and that the two processes' copies of a variable are independent. (b) Add a `printf` of some line *before* the `fork`, without an `fflush`, and run the program with its output redirected to a file; observe whether the line appears once or twice and explain why. (c) Add `fflush(stdout)` before the `fork` and confirm the fix. Report your output and your explanation; note your compiler and system.

Chapter 15 — Threads: Shared Memory and Its Hazards

Before you start: Chapter 14 (a process copies its address space; threads will share it), Chapter 13 (the system-call boundary — `clone` is the syscall underneath thread creation), Chapter 12 (in C you carry, by hand, a proof about memory — that proof is about to get a new clause). · Chapter 14's processes each had one thread of execution. This chapter adds more threads to a single process — a cheaper concurrency than `fork`, bought by giving up the isolation that made processes safe.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 13 — a system call is a controlled trap; name the instruction on x86-64 and say where the system-call number goes. (2) Chapter 14 — `fork` gives the child its own address space (via copy-on-write), so when the parent and child both hold a variable `x`, are they the same storage or two copies? (3) *Predict:* two threads share a single counter and each executes `counter++` one million times, with no locking. When both finish, is the final value exactly two million, less than two million, or possibly more? Write down your guess and your reasoning — we will run this exact program and measure the answer.

Chapter 14 gave you a way to do two things at once — `fork` a second process — but at the price of a second, isolated address space: the processes cannot see each other's memory, so sharing data means explicit inter-process communication. Often that isolation is exactly what you want. But often you want the opposite: several flows of control working on the *same* data structures in the *same* memory, cheaply. That is a **thread**. This chapter defines a thread as a second flow of control inside one address space (§15.1), creates threads with the POSIX `pthreads` API (§15.2), explains how the kernel actually schedules them and why Linux uses a one-to-one model (§15.3), and then confronts the hazard that sharing buys you — the **data race**, which we will reproduce and measure losing more than half of a program's work (§15.4). We close by carrying the new proof burden forward toward the concurrency part that makes shared memory correct (§15.5). Threads make sharing trivial; this chapter is about why that is both the point and the danger.

15.1 A thread is a second flow of control in one address space

A process can contain more than one **thread** of execution, and the defining fact of threads is *what they share and what they don't*. All threads in a process **share** one address space: the same code (text), the same heap, the same global and static variables, and the same open file descriptors. What each thread keeps **private** is only what it needs to be an independent flow of control: its own **stack**, its own **registers** (including the program counter and stack pointer), and its own thread-local storage. That is the whole model. Two threads in one process see the same `malloc`'d object at the same address and can both read and write it — directly, through an ordinary pointer, with no copy and no IPC.

Set this against Chapter 14 precisely, because the contrast *is* the concept. When you `fork`, the child gets a *copy* of the address space (copy-on-write), so the parent's and child's variables are separate storage — writes in one are invisible to the other, and that isolation is what contains a crash. When you create a thread, the new thread gets the *same* address space, so a write by one thread is immediately visible to the others — and a wild pointer or a crash in one thread corrupts or kills the *whole* process, all threads with it. Process is copy-and-isolate; thread is share-and-expose. The power of threads (cheap communication through shared memory) and their danger (no isolation, and the races of §15.4) are the same fact viewed twice.

One process, two threads: shared address space, private stacks and registers.

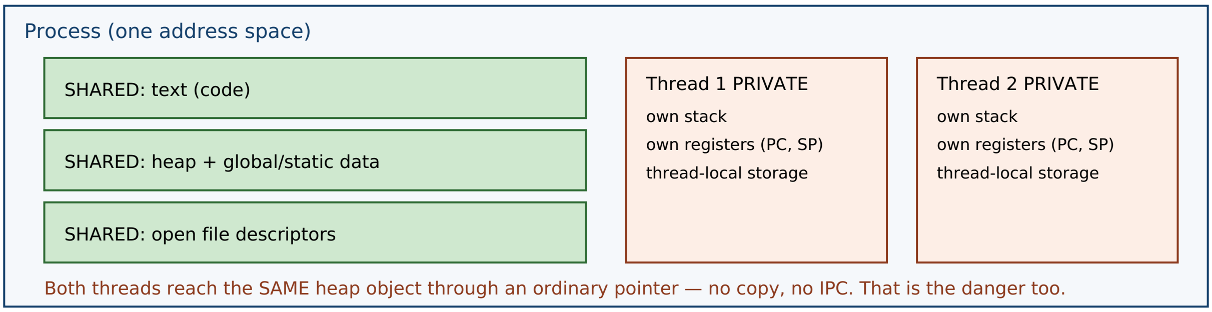

Figure 15.1: Threads in one process share the address space — code, heap, globals, file descriptors — while each keeps a private stack, registers, and thread-local storage. Compare Chapter 14's `fork`, which *copies* the address space. Sharing is why threads communicate cheaply and why they race.

> **QUICK CHECK**
>
> Name two things all threads in a process share, and two things each thread keeps private. If thread A stores 7 into a global variable, can thread B see it — and how does that differ from a parent and child after `fork`? (Answer after the next section.)

15.2 Creating threads: the `pthreads` model

On Unix systems the portable interface is **POSIX threads** (`pthreads`). Two calls carry most of the weight. `pthread_create(&tid, attr, fn, arg)` starts a new thread that begins by running `fn(arg)` concurrently with the caller, and writes the new thread's identifier into `tid`. `pthread_join(tid, &ret)` blocks until that thread finishes and collects its return value — it is the thread analog of Chapter 14's `wait`: create is to fork as join is to wait. The thread function has the fixed signature `void *fn(void *)`; a thread ends by returning from it (or calling `pthread_exit`). You compile such a program with `-pthread`. A minimal two-thread program:

```
void *worker(void *arg){ /* runs concurrently */ return NULL; }

int main(void){
    pthread_t t1, t2;
    pthread_create(&t1, NULL, worker, NULL);   /* start thread 1 */
    pthread_create(&t2, NULL, worker, NULL);   /* start thread 2 */
    pthread_join(t1, NULL);                     /* wait for it, like wait() for a child */
    pthread_join(t2, NULL);
    return 0;
}
```

After the two `pthread_create` calls, three flows of control exist in one address space — the main thread and the two workers — all sharing the heap and globals, each on its own stack. The two `pthread_join` calls make the main thread wait for both to finish before returning, so the process does not exit (and tear down the shared memory the workers are using) out from under them. That is the whole lifecycle: create, run concurrently, join.

15.3 Kernel threads, user threads, and the one-to-one model

Who actually schedules a thread onto a CPU? There are two classic models. In a **user-level** (or **M:N**) model, a runtime library multiplexes many user threads onto a smaller number of kernel-scheduled entities, switching between them in user space; this makes creating and switching threads very cheap, but it has a sharp edge — if one user thread makes a blocking system call, it can stall the underlying kernel thread and thus every user thread mapped onto it. In a **kernel-level** (or **one-to-one**, **1:1**) model, each user thread corresponds to its own kernel-scheduled thread, so the kernel schedules them independently and a blocking call in one blocks only that one, at the cost of a kernel-managed object per thread.

Linux uses the **one-to-one** model, implemented by the **NPTL** (Native POSIX Thread Library, which replaced the older LinuxThreads around the 2.6 kernel). Under the hood, `pthread_create` calls the `clone` system call with a set of flags that ask for sharing — `CLONE_VM` (share the address space), `CLONE_FILES` (share the file-descriptor table), `CLONE_THREAD`, `CLONE_SIGHAND`, and others — so the new thread is a full kernel-scheduled entity that happens to share memory and resources with its creator. This reveals a clean underlying truth: `clone` is the general primitive, and `fork` and thread-creation are two points on its spectrum. `fork` (Chapter 14) is `clone` asking to *copy* the address space; thread creation is `clone` asking to *share* it. A thread and a process are the same kind of schedulable thing; they differ only in how much they share. (User-space schedulers do still exist above the kernel — language runtimes such as Go schedule many lightweight tasks onto a pool of OS threads — but on Linux the OS threads underneath are 1:1 kernel threads.)

> **QUICK CHECK**
>
> Does Linux map each `pthread` to its own kernel-scheduled thread (1:1) or multiplex many onto few (M:N)? Which system call does `pthread_create` use underneath, and which flag makes the new thread share the caller's memory? (Answer below the communicate box.)

15.4 The shared-memory hazard: the data race

Now the predict. Threads share memory, so two threads can touch the same variable at the same time — and that is where correctness goes to die. Take the simplest possible shared operation, `counter++` on a shared integer. It looks atomic in C, but it is not: at the machine level it is a **read-modify-write** — load the current value into a register, add one, store it back — three steps that can interleave with another thread's three steps. If thread A loads 5, thread B loads 5, both add one, both store 6, then two increments produced a net change of *one*: an update was lost. Run this at scale and the losses pile up. Here is the exact program from the opener — four threads, each incrementing a shared counter one million times, no locking — compiled and run on this machine (gcc 11.4.0, `-O2`, x86-64), five times:

```
void *bump(void *a){ for (int i = 0; i < 1000000; i++) counter++; return NULL; }
/* 4 threads all run bump() on the same shared `counter`, then we join and print it */

expected = 4000000, got = 1092939
expected = 4000000, got = 1170537
expected = 4000000, got = 1372703
expected = 4000000, got = 1810815
expected = 4000000, got = 1324249
```

The program should have counted to four million. It counted to between roughly 1.1 and 1.8 million — **more than half of the four million increments simply vanished**, and the answer was different every run. This is a **data race**: concurrent accesses to the same memory location, at least one of them a write, with no synchronization ordering them. In C and C++ a data race is **undefined behavior** — Chapter 10's UB returns, now arising from concurrency rather than from a bad pointer or an overflow — and the nondeterministic, mostly-wrong output above is exactly what UB looks like when it manifests. The lost updates come straight from the interleaved read-modify-writes: whenever two threads read the same value before either stores, one increment is overwritten and lost.

THE SAME IDEA THREE WAYS: A DATA RACE LOSES UPDATES

1. In words. `counter++` is not atomic — it is load, add, store. Two threads running it on the same variable can both load the same value, both add one, and both store, so their two increments net only one. Nothing orders them, so results are nondeterministic. That unordered, conflicting access is a data race, and in C/C++ it is undefined behavior.

2. As a picture. An interleaving timeline: A loads 5 · B loads 5 · A adds → 6 · B adds → 6 · A stores 6 · B stores 6. Two increments, one net change.

3. As numbers. Four threads, one million increments each, no lock: expected 4,000,000; measured on this machine ~1.1–1.8 million across five runs — over half the work lost, and different every time. Adding synchronization (a mutex or an atomic increment) restores exactly 4,000,000.

TRAP: "IT PRINTED THE RIGHT ANSWER ON MY MACHINE, SO THE CODE IS CORRECT"

A data race is undefined behavior *whether or not it visibly misbehaves on a given run*, and it can easily produce the right answer while being thoroughly broken — which is exactly what makes it dangerous. **Correct-looking output is not evidence of correctness for racing code.** Writing this very chapter, an earlier version of the counter program — compiled at `-O2` without the variable marked `volatile` — printed `4000000` on every single run, because the optimizer had collapsed each thread's million-iteration loop into a single add, shrinking the race window to almost nothing so the threads happened to serialize. The bug was fully present (four threads doing unsynchronized read-modify-writes on shared memory); it simply did not *manifest*, just as Chapter 10's undefined behavior need not misbehave to be undefined. A race that surfaces only under a particular compiler, optimization level, CPU load, or core count will pass every test on your laptop and corrupt data in production. The lesson: reason about shared-memory correctness from the *rules* (is there an unsynchronized conflicting access? then it is a race, full stop), never from whether a test run looked fine. The fix is real synchronization — a mutex around the update, or an atomic increment — not a comforting green test.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "Threads are just cheaper processes. Let's convert our process-per-request server to threads across the board for the speed win." Cover the paragraph and respond in three or four sentences, drawing on §15.1 and §15.4.

Model answer. "Threads *are* cheaper — they share the address space instead of copying it, so creation skips the copy-on-write setup and they can communicate through shared memory with no IPC — but 'cheaper processes' misses the crucial difference: threads drop the isolation that processes gave us. A memory bug or crash in one thread takes down the whole process — every in-flight request — where a bad process only took itself down, and every piece of mutable state we now share becomes a potential data race, which is undefined behavior. That is not theoretical: an unsynchronized shared counter under four threads lost over half its increments in our measurement and gave a different wrong answer each run, and a race can even pass tests by printing the right number. So threads are right where we want cheap shared-memory concurrency and can enforce synchronization discipline; keep processes where fault containment or a security boundary matters. Let's not convert blindly — decide per component and protect every shared write." Naming the isolation trade, the data-race hazard, and refusing the blanket swap is the senior register.

(Answer to the §15.3 quick check: Linux uses the one-to-one (1:1) model via NPTL — each `pthread` is its own kernel-scheduled thread. `pthread_create` uses the `clone` system call underneath, and `CLONE_VM` is the flag that makes the new thread share the caller's address space.)

15.5 What to carry forward — and toward concurrency

A **thread** is a second flow of control inside one process: all threads **share** the address space — code, heap, globals, file descriptors — while each keeps a **private** stack, registers, and thread-local storage. You create and reap them with `pthreads` — `pthread_create` starts a thread running a `void *fn(void *)`, `pthread_join` waits for it and collects its result (create/join is the thread's fork/wait). Linux schedules threads **one-to-**

one with kernel threads via NPTL, and `pthread_create` is really the `clone` system call asking to share memory (`CLONE_VM`) — the same primitive `fork` uses to *copy* it, so a thread and a process differ only in how much they share. The price of sharing is the **data race**: unsynchronized conflicting access to the same memory, which is undefined behavior in C/C++ and which we measured losing more than half of a four-million-increment workload, nondeterministically — and which can hide by printing the right answer, so correctness must be reasoned from the rules, not from a passing run.

That last point reopens the theme of Part 2 at a new level. Chapter 12 said that in C you carry, by hand, a proof about lifetimes, bounds, and freeing. Threads add a clause: *no unsynchronized concurrent access to shared mutable memory* — and violating it is undefined behavior just like the rest. Making shared-memory concurrency correct is a large subject of its own: the synchronization primitives that order accesses (locks, condition variables, atomics), the lock-free techniques that avoid them, and the memory model that defines precisely what a multithreaded program means. That is the concurrency-and-parallelism part of this volume, which the data race you just measured is the doorway into. And it is one more place where a language can help: just as Rust's ownership makes use-after-free a compile error, its type system also encodes which data may cross between threads and which must be synchronized — turning "don't race" from a discipline you enforce by hand into an invariant the compiler checks, the payoff this whole part has been building toward.

CAN YOU… WITHOUT LOOKING?

- State what threads in a process share and what each keeps private, and contrast that with parent/child after `fork`.
- Write a minimal two-thread `pthreads` program and explain `pthread_create`/ `pthread_join` as the thread analog of `fork`/`wait`.
- Explain the difference between 1:1 and M:N threading, say which Linux uses, and name the system call and flag underneath `pthread_create`.
- Explain why `counter++` from two threads can lose updates, decomposing it into load/add/store with an interleaving.
- State what a data race is, why it is undefined behavior, and why a passing test does not prove racing code correct.
- Give the new clause threads add to Part 2's hand-carried proof, and name the fix category (synchronization).

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What does a thread share with its process's other threads, and what does it keep private? (§15.1)
2. **R2.** What do `pthread_create` and `pthread_join` do, and what earlier pair are they analogous to? (§15.2)
3. **R3.** What is the 1:1 threading model, which library implements it on Linux, and what system call/flag underlie a thread? (§15.3)
4. **R4.** Why can two threads doing `counter++` lose updates, and what did the measurement show? (§15.4)
5. **R5.** What is a data race, why is it undefined behavior, and why can it pass a test yet be wrong? (§15.4)

FURTHER READING

- **Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP), "Concurrency: An Introduction" and "Interlude: Thread API."** The free chapters that introduce threads, the shared/private split, the `pthreads` API, and the race that motivates all of concurrency. The reference for §15.1–15.4.
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), Chapter 12 (Concurrent Programming).** Threads, shared variables, and races from the programmer's view.
- **The Linux `pthreads(7)` and `clone(2)` manual pages.** Primary sources for the NPTL one-to-one model of §15.3 and the exact `clone` flags (`CLONE_VM`, `CLONE_FILES`, `CLONE_THREAD`, …) that distinguish a thread from a `fork`. Verify per system.
- **The C11/C++11 standard's memory model (see cppreference.com).** The authority for why a data race is undefined behavior and what "synchronizes-with" means — the formal ground the concurrency part builds on.

Exercises

15.1 WARM-UP Name two things all threads in a process share, and two things each thread keeps private.

15.2 WARM-UP What does `pthread_create` take as arguments, and what does `pthread_join` do?

15.3 WARM-UP Does Linux use a 1:1 or an M:N threading model, and which system call does `pthread_create` use underneath?

15.4 CORE Write, from memory, a minimal program that starts two threads running the same function and waits for both. Explain how `pthread_create`/`pthread_join` mirror Chapter 14's `fork`/`wait`, and why the joins are necessary before `main` returns.

15.5 CORE Explain why two threads each doing `counter++` on a shared variable can lose updates. Decompose `counter++` into its machine-level steps and give a concrete interleaving in which two increments produce a net change of one.

15.6 CORE Contrast a process and a thread on four axes: address space, isolation, cost to create, and what data sharing costs you. For each, say which is cheaper/safer and why.

15.7 COST (a) Why is creating a thread cheaper than `fork`ing a process — what work does thread creation skip? (b) What new cost or hazard do you take on in exchange for sharing memory? (c) The measured demo lost more than half of its increments and gave a different total each run; is that loss deterministic, and what does its nondeterminism imply for whether tests can be trusted to catch it?

15.8 FADED *Worked, with the last step for you.* A web service keeps a shared `long hits` counter that each request thread increments with `hits++`. Under load, the reported hit count is noticeably lower than the true number of requests — but every unit test passes.
Step 1 (given): `hits++` is a non-atomic read-modify-write on memory shared by all request threads.
Step 2 (given): The unit tests exercise one request at a time, so no two increments ever overlap and nothing races.
Step 3 (given): Under real load, many threads increment concurrently and their read-modify-writes interleave, so some increments overwrite others.
Step 4 (you): Name the bug precisely, say whether it is undefined behavior, explain why the passing tests failed to catch it, and name the category of fix (and one concrete mechanism) that makes the count correct.

15.9 MIXED For each need, choose a separate *process* (via `fork`) or a *thread*, and justify in a line: (a) run an untrusted plugin so that if it crashes or corrupts memory it cannot take down the host; (b) compute a parallel sum over a large in-memory array that all workers read; (c) run a command the user typed at a shell.

15.10 MIXED Drawing on Chapters 9–15, classify each as *undefined behavior*, *a defined-but-real defect*, or *well-defined and fine*: (a) two threads writing the same non-atomic `long` with no synchronization; (b) two threads each writing a *different* variable with no synchronization; (c) a thread reading through a pointer whose target was already `free`d; (d) two threads reading (never writing) the same shared value.

15.11 STRETCH Chapter 12 said that in C the programmer carries, by hand, a proof about lifetimes, bounds, and freeing. (a) State the additional clause that threads add to that proof. (b) Explain why violating it is undefined behavior and why, unlike a use-after-free, it may never reproduce in testing. (c) In one or two sentences, describe how a language could move this from a runtime hope to a compile-time guarantee — for example, a type system that tracks which data may be shared across threads and which accesses must be synchronized (Rust's `Send`/`Sync` and borrow checking; forward reference, prose only).

15.12 LAB On your own machine: (a) run the shared-counter race — four threads, one million increments each of one shared counter, no lock — five times, and report the wrong totals and how much they vary run to run. (b) Add a mutex around the increment (or use an atomic increment) and confirm the total is now exactly four million every time. (c) Try building the unsynchronized version at `-O0` versus `-O2`, and with the counter marked `volatile` or not; report which combinations make the bug visible and which hide it — and explain why a "correct" run does not mean the code is correct. Label your compiler, core count, and hardware.

Chapter 16 — CPU Scheduling

Before you start: Chapter 15 (Linux threads are 1:1 kernel-scheduled entities — *these* are what the scheduler picks among), Chapter 14 (a process blocks in `wait` or on I/O — a blocked task is off the CPU and off the scheduler's radar), Chapter 13 (a system call is a trap into the kernel — where the scheduler lives). · Chapters 14 and 15 gave you more runnable things than you have CPUs. This chapter is about the OS's answer to the question that creates: *who runs next, and for how long?*

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 14 — when a process calls `read` on a socket with no data, it *blocks*; is a blocked process using the CPU while it waits? (2) Chapter 15 — Linux schedules each `pthread` one-to-one with a kernel thread; so is the unit the scheduler places on a CPU the process or the thread? (3) *Predict:* three jobs arrive at almost the same instant — one needs 100 ms of CPU, the other two need 10 ms each. If the scheduler runs them strictly in arrival order (the 100 ms job first), what is the *average* time from arrival to completion across the three? What if it ran the two short ones first instead? Write both numbers down — we will compute them exactly in §16.2.

A modern machine has a handful of CPUs and, at any moment, dozens or hundreds of runnable threads. Something must decide which runnable thread occupies each CPU and when to take it away and give the CPU to another — that something is the **scheduler**, and the decision is **scheduling**. Chapter 13 gave the mechanism the scheduler rides on (a timer interrupt traps into the kernel, which can switch the running thread); this chapter is about the **policy** — the algorithm that chooses. We start by naming what a scheduler optimizes and the metrics that make "good" precise (§16.1); build up the classic policies from the simplest, FIFO, through shortest-job-first and its convoy trap (§16.2); add preemption and the time slice with round-robin, exposing the response-vs-turnaround trade-off (§16.3); and then meet the schedulers real systems ship — the multi-level feedback queue and Linux's proportional-share scheduler — which approximate the ideal without an oracle (§16.4). We close by carrying forward what "the OS multiplexes the CPU" really costs and buys (§16.5).

16.1 The scheduling problem, and how to measure a scheduler

At any instant each thread is in one of a few states. A **running** thread is on a CPU right now. A **ready** (runnable) thread could run but no CPU is free for it. A **blocked** thread is waiting for something — I/O to complete, a child to exit, a lock to free — and *cannot* run until that event arrives, so it is not the scheduler's concern until it wakes. The scheduler's job is to pick, from the ready threads, which one each CPU runs, and a **context switch** (Chapter 13's mechanism) is how it hands a CPU from one thread to another: save the outgoing thread's registers, load the incoming thread's, resume. The switch is not free — it costs the direct register save/restore plus the indirect cost of a cold cache and TLB (Chapter 3) for the new thread — which is why the scheduler cannot simply switch on every instruction.

"Good" needs a definition, and there are several, often in tension. The two that matter most here are **turnaround time** — completion time minus arrival time, how long the whole job took to finish, the metric a batch workload cares about — and **response time** — first-run time minus arrival time, how long until the job *started* getting served, the metric an interactive user feels as lag. A scheduler also cares about **throughput** (jobs completed per unit time), **fairness** (each job gets a defensible share), and avoiding **starvation** (no ready job waits forever). The reason scheduling is hard is that these goals conflict: as we will see, the policy that minimizes average turnaround is not the one that minimizes response time, and the one that is "fair" may not be either.

> **QUICK CHECK**
>
> Define turnaround time and response time in one line each. A job arrives at t=0, first gets the CPU at t=5, and finishes at t=30. What is its response time and its turnaround time? (Answer after the next section.)

16.2 FIFO, shortest-job-first, and the convoy effect

The simplest possible policy is **FIFO** (first-in, first-out, also called FCFS): run jobs to completion in arrival order, no preemption. It is obviously fair in one sense and trivial to implement — and it has a sharp failure mode. Take the opener's workload: three jobs, A needs 100 ms and B and C need 10 ms each, all arriving at essentially t=0 with A first. FIFO runs A to completion, then B, then C. A finishes at 100, B at 110, C at 120, so average turnaround is (100 + 110 + 120) / 3 = **110 ms**. The two short jobs sat behind the long one for almost their entire wait. This is the **convoy effect**: short jobs pile up behind a long one like cars behind a truck, and average turnaround balloons because everyone inherits the long job's runtime.

The fix, if you knew the run times, is to run the short jobs first. **Shortest-Job-First (SJF)** schedules the ready jobs in increasing order of length. On the same workload it runs B, then C, then A: B finishes at 10, C at 20, A at 120, and average turnaround drops to (10 + 20 + 120) / 3 = **50 ms** — less than half of FIFO's, for the exact same jobs. That is not a coincidence: when all jobs arrive together, SJF is *provably optimal* for average turnaround (any swap that puts a longer job earlier only adds its length to more jobs' waits). When jobs arrive over time, the preemptive version — **Shortest-Time-to-Completion-First (STCF)**, which preempts the running job if a newly arrived one has less work left — extends the same idea. These are the numbers you predicted; here they are, computed by a scheduling simulator:

```
workload: A=100ms, B=10ms, C=10ms, all arrive ~t=0
FIFO (order A,B,C): finish A=100 B=110 C=120  -> avg turnaround 110.0 ms
SJF  (order B,C,A): finish B=10  C=20  A=120  -> avg turnaround  50.0 ms
```

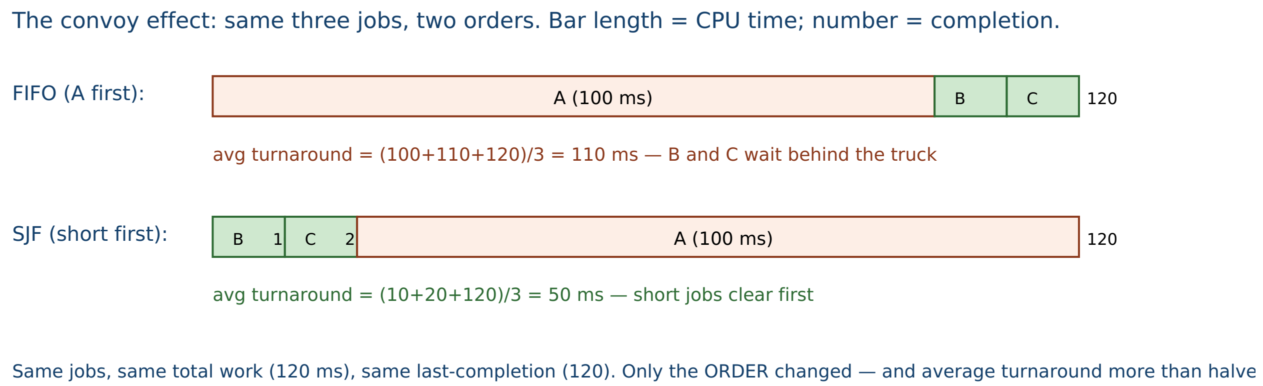

Figure 16.1: FIFO vs SJF on A=100, B=10, C=10 (arriving together). Running the long job first makes every short job inherit its runtime (the convoy effect); running short jobs first minimizes average turnaround — which SJF provably does when all jobs arrive at once.

> **THE SAME IDEA THREE WAYS: ORDER DOMINATES AVERAGE TURNAROUND**
>
> **1. In words.** Turnaround is completion minus arrival. When jobs share a CPU, a job's turnaround includes the runtime of everything scheduled before it. So putting a long job first adds its length to every later job's turnaround; putting short jobs first minimizes the total. That is why SJF beats FIFO on average turnaround.
>
> **2. As a picture.** Figure 16.1: the same three bars, packed truck-first versus short-first. The sum of the completion marks — not the total width — is what average turnaround measures, and reordering shrinks it.
>
> **3. As numbers.** A=100, B=10, C=10: FIFO averages (100+110+120)/3 = 110 ms; SJF averages (10+20+120)/3 = 50 ms. Identical work, identical finish of the last job (120 ms), less than half the average turnaround — purely from order.

(Answer to the §16.1 quick check: response time = first-run − arrival = 5 − 0 = 5; turnaround = completion − arrival = 30 − 0 = 30.)

16.3 Preemption, the time slice, and round-robin

SJF minimizes turnaround, but it has two problems. First, it needs an **oracle** — you must know each job's length in advance, which for interactive and server workloads you never do. Second, even with the oracle its *response* time is bad: a job that arrives just after a long one starts must wait for it to finish (or, with STCF, only if it is shorter). Interactive workloads care about response time above all — you want the cursor to move the instant you type — so we need a policy that shares the CPU in *small slices* rather than running each job to completion. That policy is **round-robin (RR)**: run each ready job for a fixed **time slice** (quantum), then preempt it (via the timer interrupt) and move to the next, cycling forever until jobs finish.

Round-robin's whole point is response time. On the A/B/C workload with a 1 ms slice, all three jobs get their first turn within the first few milliseconds, so every job's response time is essentially immediate (about 1 ms on average) rather than SJF's — where C waited 10 ms and A waited 20. But that responsiveness has a price: RR's average *turnaround* is worse than SJF's, because it stretches every job out by interleaving. Here are all three policies on the same workload from the simulator:

```
workload: A=100ms, B=10ms, C=10ms
FIFO:        avg turnaround 110.0 ms,  avg response  70.0 ms  (B waits 100, C waits 110)
SJF:         avg turnaround  50.0 ms,  avg response  10.0 ms  (C waits 10, A waits 20)
RR (q=1ms):  avg turnaround  59.7 ms,  avg response  ~1.0 ms  (every job starts almost at
once)
```

That is the fundamental trade-off in one table: **SJF wins turnaround, round-robin wins response, and you cannot have both**. The **size** of the time slice is the knob. A tiny slice gives excellent response (jobs cycle fast) but pays a context-switch cost on every single switch — and if the slice approaches the switch cost itself, the CPU spends a large fraction of its time switching rather than working. A large slice amortizes the switch cost (better throughput) but degrades response time toward FIFO's. Real schedulers pick a slice large enough that the switch overhead is a small percentage, and small enough that interactivity stays crisp — typically a few milliseconds.

> **QUICK CHECK**
>
> Round-robin makes the time slice small to improve one metric and worsen another — which is which? And what goes wrong if you make the slice as small as the cost of a single context switch? (Answer below the communicate box.)

16.4 What real schedulers do: MLFQ and proportional share

Real systems cannot use pure SJF (no oracle) or pure round-robin (ignores that jobs differ in importance and behavior). Two ideas dominate real schedulers, and both aim to get SJF-like behavior — favor short, interactive work — *without* knowing job lengths in advance, by **learning from the past**.

The first is the **multi-level feedback queue (MLFQ)**. Keep several ready queues, each a priority level; always run a job from the highest non-empty level, round-robin within a level. The trick is how a job's priority *changes*: a new job enters at the top; if it uses its whole time slice (CPU-bound behavior) it is demoted a level; if it gives up the CPU before the slice ends (blocking on I/O — interactive behavior) it stays high. Over a few slices this *learns* each job's character: interactive jobs, which block often, float near the top and get scheduled quickly (great response time); CPU-bound jobs sink to the bottom and share the leftovers. That approximates "short jobs first" using observed behavior instead of an oracle. Two refinements make it work in practice: to prevent a long job from **starving** at the bottom forever, MLFQ periodically **boosts** every job back to the top; and to stop a job from **gaming** the rule (yielding just before each slice ends to keep its priority), it accounts for total CPU used at a level rather than per-slice behavior.

The second idea is **proportional share** (fair-share): rather than juggle priorities, give each job a **weight** and hand out CPU time in proportion to weight, so a job with twice the weight gets twice the CPU over time. This is the family Linux's normal scheduler belongs to. For over a decade Linux used the **Completely Fair Scheduler (CFS)**, which tracks each task's accumulated virtual runtime and always runs the task that has had the least, weighted by priority; kernel 6.6 (2023) replaced CFS with **EEVDF** (Earliest Eligible Virtual Deadline First), a proportional-share scheduler with better latency guarantees. In both, a process's **nice** value (−20 to +19, set with `nice`/`renice`; lower is "less nice," i.e. higher priority) sets its weight: a lower nice value claims a larger share. You can see this directly. Two identical CPU-bound loops pinned to the *same* core so they must compete, one at nice 0 and one at nice 19, run for five wall-clock seconds on this machine (Linux 6.18, which uses EEVDF; numbers illustrative):

```
taskset -c 0 nice -n 0  ./hog    ->  nice0:  cpu-seconds used = 4.66
taskset -c 0 nice -n 19 ./hog    ->  nice19: cpu-seconds used = 0.09
```

The nice-0 loop got roughly fifty times the CPU of the nice-19 loop over the same wall-clock window — not because nice 19 was "paused," but because the proportional-share scheduler weighted its slice far smaller. That is the whole idea made visible: `nice` does not change what a program computes, only what *fraction of the CPU* it is entitled to when there is contention. (Linux also offers strict real-time classes — `SCHED_FIFO` and `SCHED_RR` — that sit above the normal class and always preempt it; those are for bounded-latency work and are a foot-gun for everything else, as the warning below explains.)

TRAP: REACHING FOR REAL-TIME PRIORITY (`SCHED_FIFO`) TO "MAKE IT FASTER"

When a latency-sensitive service is being starved by a background job, the tempting fix is to promote the service to a real-time scheduling class — `SCHED_FIFO` or `SCHED_RR` — so it always wins. This is *strict fixed-priority* scheduling, and it does not mean "a bit faster": a runnable `SCHED_FIFO` thread preempts **every** normal task and runs until it blocks or a higher real-time thread appears. If that thread is CPU-bound and never blocks — a busy loop, a spin-wait, a bug — it will monopolize its core and **starve everything else on it, including kernel threads**, and can wedge the machine hard enough to require a reboot. (Linux mitigates this with real-time throttling — `sched_rt_runtime_us` caps RT time per period — precisely because the default is so dangerous.) Real-time priority is the right tool only for work that is both genuinely latency-critical *and* bounded in CPU (it blocks regularly). To bias the normal scheduler toward an important service, reach first for `nice` or cgroup CPU weights, which change the *share* under contention without the power to starve the system. The lesson: "higher priority" is not a speed dial; strict priority is a loaded gun pointed at every other task on the CPU.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "Our request service and the nightly compaction job share a box, and compaction is starving the service's tail latency. Let's just give the service `SCHED_FIFO` real-time priority so it always wins." Cover the paragraph and respond in three or four sentences, drawing on §16.4.

Model answer. "The instinct is right — the service should win against a background batch job — but `SCHED_FIFO` is a much bigger hammer than we want: it is strict fixed priority, so if any of the service's threads ever spins or busy-loops without blocking, it will preempt everything on that core, including kernel threads, and can hang the box, which is why the kernel even throttles real-time time by default. What we actually want is a bigger *share* under contention, not absolute priority — so lower the service's `nice` value or, better, put the two workloads in separate cgroups and give the service a larger CPU weight (and cap or de-prioritize compaction). That fixes the starvation through the fair scheduler, keeps the system stable, and leaves headroom if compaction ever needs to burst. Save real-time scheduling for work that is both latency-critical and provably bounded in CPU." Distinguishing "bigger share" from "strict priority," and naming the starvation and hang risk, is the senior register.

*(Answer to the §16.3 quick check: a small time slice improves **response** time (jobs cycle onto the CPU quickly) and worsens **turnaround** and throughput (more interleaving and more context switches). If the slice approaches the cost of one context switch, the CPU spends a large fraction of its time switching rather than doing useful work — overhead dominates.)*

16.5 What to carry forward

The **scheduler** decides which ready thread each CPU runs and for how long; a **context switch** is the (non-free) mechanism that hands a CPU from one thread to another. We measure schedulers by **turnaround** (completion − arrival) and **response** (first-run − arrival) time, among others, and the central fact is that these goals conflict. FIFO is simple but suffers the **convoy effect**; **SJF/STCF** minimize average turnaround but need an oracle and give poor response; **round-robin** with a small time slice wins response at the cost of turnaround, with the slice size trading responsiveness against switch overhead. Real schedulers approximate "favor short, interactive work" without an oracle by learning from behavior — the **multi-level feedback queue** (priorities that adapt, with boosting to prevent starvation) — or by handing out CPU in proportion to a **weight** — Linux's proportional-share scheduler (CFS, then EEVDF from 6.6), where the **nice** value sets the share, which we watched split one core roughly fifty to one between a nice-0 and a nice-19 loop.

Step back and see what this part is assembling. Chapter 14 gave you many processes and Chapter 15 many threads; this chapter is how one machine *pretends* to run them all at once — by slicing the CPU in time so fast that each thread perceives a CPU (nearly) to itself. That is the CPU half of the operating system's core illusion. The other half is the same trick played on *memory*: giving each process the illusion of a large private address space on a machine whose physical memory is smaller and shared. Chapter 4 showed the hardware that makes an address virtual; the next chapter is the OS policy that makes the illusion hold when memory runs short — demand paging, page faults as a mechanism, and the replacement decisions that are this chapter's turnaround-versus-response trade-off played again on pages instead of milliseconds.

CAN YOU… WITHOUT LOOKING?

- Name the three scheduling-relevant states of a thread (running, ready, blocked) and say which ones the scheduler chooses among.
- Define turnaround time and response time, and compute both for a job given its arrival, first-run, and completion times.
- Explain the convoy effect with the A=100/B=10/C=10 example, and give FIFO's and SJF's average turnaround (110 vs 50 ms).
- State why SJF is optimal for average turnaround yet unusable in practice, and what round-robin trades to win response time.
- Explain how a multi-level feedback queue approximates SJF without an oracle, and why it needs priority boosting.
- Explain what a `nice` value does under Linux's proportional-share scheduler, and why `SCHED_FIFO` is dangerous.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What are the running/ready/blocked states, and which does the scheduler pick among? (§16.1)
2. **R2.** Define turnaround and response time. (§16.1)
3. **R3.** What is the convoy effect, and what were FIFO's and SJF's average turnarounds on A/B/C? (§16.2)
4. **R4.** Why does round-robin improve response time but hurt turnaround, and what does the time-slice size trade? (§16.3)
5. **R5.** How does an MLFQ learn to favor interactive jobs without knowing their lengths, and what does a `nice` value control? (§16.4)

FURTHER READING

- **Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP), "Scheduling: Introduction" and "Scheduling: The Multi-Level Feedback Queue."** The free chapters that develop turnaround/response metrics, FIFO/SJF/STCF/round-robin, and MLFQ (including boosting and anti-gaming). The reference for §16.1–16.4.
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), Chapter 8 (Exceptional Control Flow).** Context switches and the timer interrupt that preemption rides on, from the programmer's view.
- **The Linux `sched(7)` and `nice(2)` / `sched_setscheduler(2)` manual pages.** Primary sources for the scheduling classes (`SCHED_OTHER`/normal, `SCHED_FIFO`, `SCHED_RR`), the nice range, and real-time throttling. Verify per kernel version.
- **The Linux kernel scheduler documentation (`docs.kernel.org/scheduler/`), including the EEVDF scheduler page.** The authority for the CFS-to-EEVDF change in 6.6 and how proportional share is implemented. Fast-moving; treat details as version-specific.

Exercises

16.1 WARM-UP Name the three scheduling-relevant states of a thread, and say which state(s) the scheduler chooses the next runner from.

16.2 WARM-UP Define turnaround time and response time. For a job that arrives at t=0, first runs at t=8, and completes at t=40, give both.

16.3 WARM-UP In one sentence, what is the convoy effect, and which simple scheduling policy suffers from it?

16.4 CORE Three jobs arrive at t=0: X needs 8 ms, Y needs 4 ms, Z needs 4 ms. Compute the average turnaround time under FIFO (order X, Y, Z) and under SJF, showing each job's completion time. Which is lower, and why is that no accident?

16.5 CORE Explain why SJF minimizes average turnaround when all jobs arrive together, and give the two reasons it is nonetheless unusable as a general-purpose scheduler.

16.6 CORE Describe how a multi-level feedback queue decides a job's priority over time, and explain how this makes interactive (I/O-bound) jobs get scheduled quickly and CPU-bound jobs sink — without the scheduler ever being told which is which.

16.7 COST (a) What does shrinking the round-robin time slice buy you, and what does it cost? (b) A context switch on a machine costs roughly a few microseconds; if you set the time slice equal to that, what fraction of the CPU is wasted on switching, and why? (c) Give the qualitative reason real schedulers use a slice of a few milliseconds rather than a few microseconds or a few seconds.

16.8 FADED *Worked, with the last step for you.* A batch analytics job and an interactive dashboard API share one server. Users complain the dashboard is laggy whenever the batch job runs, even though the box has spare capacity at night.
Step 1 (given): The dashboard is I/O-bound and latency-sensitive (response time matters); the batch job is CPU-bound and only turnaround matters.
Step 2 (given): Under a pure round-robin or a naive equal-share policy, the CPU-heavy batch job gets an equal turn each cycle, so the dashboard waits behind it and its response time suffers.
Step 3 (given): The two workloads should not get equal CPU: the dashboard should be favored under contention, but the batch job must still make progress (not starve).
Step 4 (you): Name a concrete Linux mechanism (not real-time priority) that gives the dashboard a larger share under contention while letting the batch job run when the dashboard is idle, and say in one line why this is safer than `SCHED_FIFO`.

16.9 MIXED For each situation, name the scheduling metric that matters most (turnaround, response time, or throughput), and justify in a line: (a) a text editor reacting to keystrokes; (b) a nightly batch job that must finish before the business day; (c) a build farm maximizing jobs completed per hour across many machines.

16.10 MIXED Drawing on Chapters 13–16, classify each claim as *true* or *false* and fix the false ones: (a) "A process blocked on `read` is consuming CPU while it waits." (b) "On Linux the scheduler picks among processes, not threads." (c) "A context switch is free, so the time-slice size does not matter." (d) "Lowering a process's `nice` value increases the CPU share it gets under contention."

16.11 STRETCH Starvation is a scheduler's cardinal sin. (a) Show how strict shortest-job-first can starve a long job indefinitely if short jobs keep arriving. (b) Explain how a multi-level feedback queue's periodic priority *boost* prevents this, and what would go wrong without it. (c) Explain why a proportional-share scheduler (weight-based) does not starve a low-weight job even under load — contrast "smaller share" with "no share," and connect this to why `nice` cannot starve a task but `SCHED_FIFO` can.

16.12 LAB On your own machine: (a) write a CPU-bound loop that runs for a fixed number of wall-clock seconds and reports the CPU-seconds it actually accumulated (via `CLOCK_THREAD_CPUTIME_ID`). (b) Run two copies pinned to the *same* core with `taskset -c 0` so they must compete, one at `nice 0` and one at `nice 19`, and report the CPU-seconds each got — you should see the nice-0 copy dominate by a large ratio. (c) Now run them on *different* cores (or without pinning on a multi-core box) and explain why the nice difference nearly disappears. Label your kernel version, core count, and scheduler (CFS vs EEVDF). Explain what `nice` did and did not change.

Chapter 17 — Virtual Memory: Demand Paging and Page Replacement

Before you start: Chapter 4 (the hardware that makes an address virtual — the MMU, page tables, and the TLB translate a virtual page to a physical frame; *this* chapter is the OS policy that fills those tables), Chapter 16 (the OS multiplexes the CPU in time; it multiplexes memory in space the same way), Chapter 13 (a page fault is exceptional control flow — a trap into the kernel). · Chapter 4 showed *how* a virtual address becomes a physical one. This chapter is what the OS does when the answer is "that page isn't in memory."

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 4 — an address is *virtual*; what hardware structure translates a virtual page number to a physical frame, and what small cache speeds that translation up? (2) Chapter 16 — the scheduler faced conflicting goals (turnaround vs response); name the sin a scheduler must avoid (a job that never runs). (3) *Predict:* a program calls `mmap` to reserve 512 MiB of memory and immediately checks its resident memory (RSS) *before touching any of it*. Is its RSS about 512 MiB, about 0, or somewhere in between? Then it writes to every byte and checks again. Write down both guesses — we will measure them in §17.2.

Chapter 4 gave a process the illusion of a large, private, contiguous address space, and showed the hardware — the MMU walking a page table, the TLB caching recent translations — that turns each virtual address into a physical one on every memory access. But that chapter assumed the page was *there*: that every virtual page a process uses has a physical frame behind it. It usually does not, and cannot: the sum of every process's address space is far larger than physical RAM. The operating system sustains the illusion anyway, and this chapter is how. We reframe memory as the OS sees it — physical frames it hands out and page tables it owns (§17.1); introduce **demand paging**, where a page gets a frame only when first touched, and the **page fault** that makes it happen — measured (§17.2); go beyond physical memory to **swapping** and the replacement question it forces (§17.3); and work through the replacement policies — OPT, FIFO, LRU — and the anomaly that makes FIFO treacherous (§17.4). We close by carrying forward the mechanisms this unlocks — `mmap`, copy-on-write, the page cache — that the rest of the book runs on (§17.5).

17.1 Memory as the OS sees it: frames, page tables, and the illusion

Recall the two vocabularies. A process sees **virtual pages**: its address space, chopped into fixed-size pages (4 KiB on x86-64 by default). Physical memory is chopped into equal-size **frames**. A **page table**, one per process and owned by the OS, maps each virtual page to a frame — or records that it has none. Each page-table entry (PTE) holds the frame number plus control bits, and the one that matters here is the **present bit**: set if the page currently lives in a frame, clear if it does not. On every access the MMU consults the page table (via the TLB); if the translation says present, the access proceeds at hardware speed. This is the machinery of Chapter 4, now seen from the OS's chair: the OS is the party that *fills in* those tables, hands out frames, and decides what to do when a PTE's present bit is clear.

That last case is the whole subject of this chapter. Because every process has its own page table, each gets its own private virtual address space — the same virtual address in two processes maps to different frames, which is the isolation Chapter 14 relied on. And because the OS controls the present bit, it can lie productively: it can promise a process a page it has not actually backed with a frame yet, and make good on the promise only when the process reaches for it. When the process touches a page whose PTE is not present, the hardware cannot proceed — so it traps.

> **QUICK CHECK**
>
> What does the *present bit* in a page-table entry mean, and who sets it — the hardware or the OS? If a process accesses a virtual page whose PTE has the present bit clear, what happens next? (Answer after the next section.)

17.2 Demand paging and the page fault, measured

When a running instruction accesses a virtual page whose PTE is not present, the MMU raises a **page fault** — a hardware exception, Chapter 13's exceptional control flow, that traps into the kernel's page-fault handler. The handler inspects why the page is absent and chooses: if the access is *legal* but the page simply was never materialized (a freshly `mmap`'d region, the first touch of newly allocated heap), it finds a free frame, fills it appropriately — zeroed for anonymous memory, read from a file for a file mapping — sets the PTE's present bit, and restarts the faulting instruction, which now succeeds. If the access is *illegal* (a wild pointer into an unmapped region), the handler delivers `SIGSEGV` instead — the segmentation fault of Chapters 7–11, now revealed as "the page-fault handler found no valid mapping." This lazy strategy — give a page a frame only when it is first used — is **demand paging**, and it is why allocating memory is cheap: `malloc` or `mmap` mostly just records a promise in the page tables; the physical frames are spent only on the pages you actually touch.

You can watch it directly. A program that `malloc`s 100 MiB and writes one byte to every 4 KiB page should take exactly one page fault per page — 100 MiB / 4 KiB = 25,600 of them — and, because anonymous pages are served from zeroed frames with no disk I/O, they should all be **minor** faults (frame available without I/O), never **major** (required disk I/O). Running it under `/usr/bin/time -v` (gcc 11.4.0, Linux 6.18, x86-64):

```
Maximum resident set size (kbytes): 103852        # ~100 MiB actually resident after touching
Minor (reclaiming a frame) page faults: 25672     # ~one per 4 KiB page touched (25,600) + a
little
Major (requiring I/O) page faults: 0              # no disk I/O — anonymous demand-zero pages
```

And the reservation-versus-use split from the opener: a program that `mmap`s 512 MiB and checks its resident memory before and after touching it:

```
after mmap 512 MiB: VmRSS = 1364 kB       # the mapping exists, but almost nothing is
resident
after touching all: VmRSS = 525976 kB     # now every page has been faulted in (~512 MiB)
```

The 512 MiB mapping cost essentially *no* physical memory until it was used: RSS was about 1.3 MiB (just the program itself), not 512 MiB, right after the `mmap` succeeded. Touching every page faulted each one in, and only then did RSS climb to ~512 MiB. That is demand paging in two numbers: **allocation is a promise; residency is paid page by page, on first touch**.

THE SAME IDEA THREE WAYS: DEMAND PAGING PAYS PER TOUCHED PAGE

1. In words. Reserving address space (via `mmap`/`malloc`) only records mappings in the page table with the present bit clear; no frame is spent. The first access to each page faults, the kernel gives it a frame and sets present, and the instruction restarts. So physical memory is consumed in proportion to pages *touched*, not pages *reserved*.

2. As a picture. Figure 17.1: a virtual page with present=0 → access → page fault trap → handler grabs a free frame, zeroes/loads it, sets present=1 → instruction restarts and succeeds. Later faults on other pages repeat the path.

3. As numbers. `mmap` 512 MiB: RSS 1,364 kB before touching, 525,976 kB after touching all. Writing one byte per page across 100 MiB took 25,672 minor faults (≈ one per 4 KiB page) and *zero* major faults — anonymous pages are served from zeroed frames, no disk I/O.

*(Answer to the §17.1 quick check: the present bit means the page currently occupies a physical frame; the **OS** sets it (the hardware only reads it during translation). Accessing a not-present page raises a page fault — a trap into the kernel's fault handler, which either materializes the page or delivers `SIGSEGV`.)*

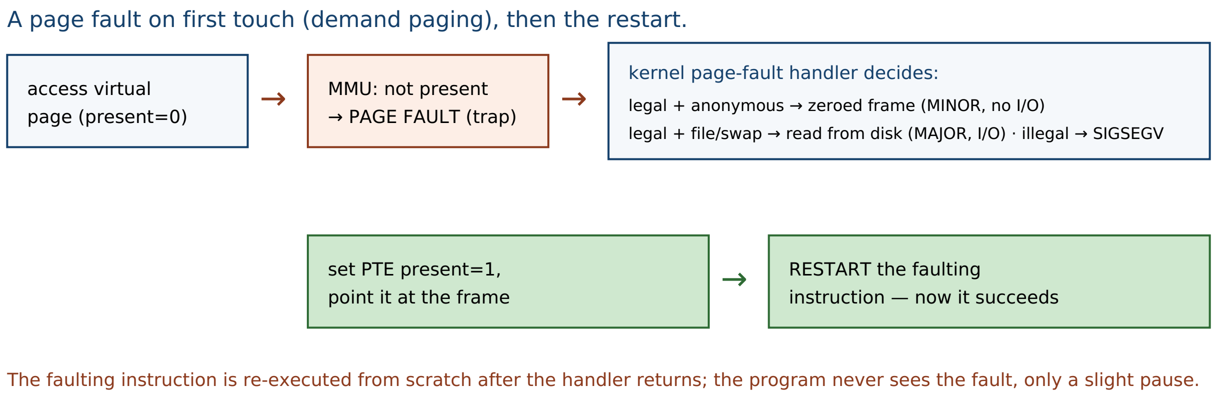

Figure 17.1: The demand-paging fault path. A first touch of a not-present page traps into the kernel, which materializes the page (a minor fault if no disk I/O is needed, a major fault if it must read from a file or swap), sets the present bit, and restarts the instruction. Illegal accesses become `SIGSEGV` instead.

17.3 Beyond physical memory: swapping and the replacement question

Demand paging is easy while free frames last. The hard case is when physical memory fills and a new page must be brought in with no frame to spare. The OS's answer is

swapping (paging to a backing store): it *evicts* some resident page — writing it out to a swap area on disk if it has been modified since it was loaded (a "dirty" page; a clean page backed by a file can just be dropped and re-read later) — freeing that frame for the incoming page. Later, if the evicted page is touched again, it faults back in from disk. This is where the minor/major distinction bites: a fault that can be satisfied from a free or droppable frame is a **minor** fault (fast, no I/O); a fault that must *read the page back from disk or swap* is a **major** fault, and it costs a disk access — Chapter 3's numbers make the gap vivid, since a disk (even an SSD) is orders of magnitude slower than DRAM, and a spinning disk millions of times slower than a cache hit.

Swapping keeps programs running past the size of RAM, but it turns the choice of *which page to evict* into a performance decision of the first order: every eviction of a page that is about to be needed again forces an expensive major fault to bring it back. This is the **page-replacement problem**, and it is Chapter 16's dilemma in a new guise — a policy question about a scarce, multiplexed resource, where a wrong choice is not incorrect, merely slow. The metric here is the **page-fault rate** (equivalently, the miss rate against the frames you have), and the goal is a replacement policy that minimizes it.

> **QUICK CHECK**
>
> What is the difference between a *minor* and a *major* page fault, and which one involves disk I/O? When the OS evicts a page to make room, why does it matter whether that page is "dirty" (modified since it was loaded)? (Answer below the communicate box.)

17.4 Replacement policies: OPT, FIFO, LRU, and Belady's anomaly

What should the OS evict? The unbeatable baseline is **OPT** (Belady's MIN, 1966): evict the page whose *next* use is farthest in the future. It provably minimizes faults — but it needs to see the future, so it is only a yardstick, not an implementable policy. The simplest real policy is **FIFO**: evict the page that has been resident longest, regardless of use. It is trivial but ignores *how* pages are used. The workhorse approximation is **LRU** (least-recently-used): evict the page that has gone longest without being accessed, betting that the recent past predicts the near future — which, thanks to **locality of reference** (Chapter 3), it usually does. On a short reference string, simulated exactly:

```
reference string: 0 1 2 0 1 3 0 3 1 2 1   (3 frames)
FIFO: 7 page faults
LRU : 5 page faults
OPT : 5 page faults        # the lower bound; no policy can do better on this string
```

FIFO takes 7 faults here where LRU and OPT take 5, because FIFO evicts pages by age even when they are about to be reused. But FIFO's real hazard is subtler and worth naming, because it violates an intuition everyone has: that giving a program *more* memory can only help. For FIFO, it can hurt. On the classic reference string of Belady, Nelson & Shedler (1969), FIFO takes *more* faults with four frames than with three:

```
reference string: 1 2 3 4 1 2 5 1 2 3 4 5
FIFO, 3 frames: 9 page faults
FIFO, 4 frames: 10 page faults      # MORE memory, MORE faults
```

This is **Belady's anomaly**: for FIFO, adding a frame can increase the fault count, because a larger FIFO queue changes the eviction order in a way that can evict a soon-to-be-reused page it would have kept with fewer frames. LRU and OPT are immune — they belong to a class (stack algorithms) whose resident set with N frames is always a superset of its resident set with N−1, so more memory never adds faults. The anomaly is the sharpest argument against FIFO and for use-based policies like LRU: a policy that ignores usage can be not just suboptimal but *non-monotonic*, punishing you for adding memory. (Real kernels do not implement true LRU — tracking exact recency on every access is too expensive — but approximate it cheaply with a per-page reference bit and a clock/second- chance sweep, keeping LRU's monotonic, locality-exploiting behavior without its bookkeeping cost.)

TRAP: THRASHING — WHEN THE WORKING SET DOESN'T FIT

Replacement policy only helps while the pages a program actively uses — its **working set** — fit in the frames it has. When the sum of all running processes' working sets exceeds physical memory, *no* replacement policy can win: every page the OS evicts is one that is about to be needed, so the system generates major fault after major fault, and the CPU sits idle waiting on disk while throughput collapses. This is **thrashing**, and its signature is a machine that is nearly idle on CPU yet grinding — high major-fault and swap-I/O rates, everything slow. The trap is diagnosing it as a CPU or code problem and "optimizing the hot loop," when the real fix is to reduce the memory footprint (smaller working set, fewer concurrent jobs) or add RAM so the working set fits. Modern Linux often does not let you thrash gently to a halt: when memory (plus swap) is truly exhausted, the **OOM killer** picks a process and kills it outright. Either way the lesson is the same — paging is a graceful extension of memory only up to the working-set boundary; past it, performance does not degrade linearly, it falls off a cliff. Watch major-fault and swap rates, not just CPU.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "Our service `malloc`ed a 40 GB buffer on a 16 GB box and the allocation *succeeded* — so we clearly have enough memory, and we can fill it up. Great." Cover the paragraph and respond in three or four sentences, drawing on §17.2 and the thrashing trap.

Model answer. "A successful `malloc` doesn't mean the memory exists — it means the kernel recorded the mapping; under demand paging (and overcommit) no physical frames are spent until you *touch* the pages, so a 40 GB allocation on a 16 GB box can succeed while backing almost nothing. The moment we actually write across all 40 GB, each page faults in, resident memory climbs past 16 GB, and the OS has to swap — if our working set genuinely exceeds RAM we'll thrash (near-idle CPU, endless major faults, everything crawling) or get OOM-killed outright. So allocation success proves nothing about capacity; what matters is the *resident working set*. Let's size the buffer to what fits with headroom, stream instead of buffering everything, and watch RSS and major-fault rates under load — not just whether `malloc` returned non-NULL." Separating "reserved" from "resident," and naming thrashing and the OOM killer, is the senior register.

*(Answer to the §17.3 quick check: a **minor** fault is satisfied without disk I/O — the page is served from a zeroed or already-cached frame; a **major** fault must read the page from a file or swap, so it costs a disk access. A "dirty" (modified) page must be **written out** to swap before its frame can be reused, whereas a clean page — an unmodified copy of file data — can be dropped and re-read later, so evicting clean pages is cheaper.)*

17.5 What to carry forward

The OS owns physical **frames** and each process's **page table**, and the **present bit** lets it back a virtual page with a frame lazily. **Demand paging** materializes a page only on first touch: the access to a not-present page raises a **page fault** (a trap), and the handler either supplies the page — a **minor** fault if no I/O is needed, a **major** fault if it must read from disk or swap — or delivers `SIGSEGV`. Allocation is therefore a promise (we watched a 512 MiB mapping cost ~1 MiB of RSS until touched), and residency is paid page by page (25,672 minor faults to touch 100 MiB). When RAM fills, **swapping** evicts pages to disk, making the choice of victim — the **page-replacement problem** — a performance decision: OPT is the unbeatable but unimplementable yardstick, FIFO is simple but suffers **Belady's anomaly** (more memory, more faults), and LRU (approximated in real kernels) exploits locality and is monotonic. Past the working-set boundary, no policy saves you: **thrashing**, then the OOM killer.

With this the operating system's core illusion is complete: Chapter 16 sliced the CPU in time, this chapter backs each address space with a smaller physical memory, and together they let many processes share one machine as if each owned it. The mechanisms this chapter unlocked are the ones the rest of the book leans on. The same page table that holds the present bit also holds protection and sharing bits, which give us **memory-mapped files** (`mmap` a file and read/write it as memory, the page cache faulting data in on demand) and **copy-on-write** — the trick from Chapter 14's `fork`, now seen fully: parent and child share every frame read-only, and only a write faults and copies the one page touched. Copy-on-write and shared mappings are also how separate processes can deliberately share a region of memory — the fast path for the inter-process communication that a later chapter takes up, and the reason the isolation of processes need not mean the cost of copying. Virtual memory is not only protection; it is the substrate for sharing, laziness, and speed.

CAN YOU… WITHOUT LOOKING?

- Explain, using the present bit, how the OS can promise a process a page it has not yet backed with a frame.
- Trace the page-fault path for a first touch of anonymous memory, and say why it is a minor fault, not a major one.
- State what demand paging makes cheap, and give the two measured numbers (RSS before/after touch; minor faults per page).
- Distinguish minor and major faults, and explain why evicting a dirty page costs more than evicting a clean one.
- Rank OPT, FIFO, and LRU, and explain Belady's anomaly and why LRU/OPT are immune to it.
- Define thrashing and the working set, and explain why a successful `malloc` does not prove you have the memory.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What does the present bit mean, and what happens on an access to a not-present page? (§17.1, §17.2)
2. **R2.** What is demand paging, and what did the `mmap`-512-MiB measurement show about reserved vs resident memory? (§17.2)
3. **R3.** Distinguish minor and major page faults; which involves disk I/O? (§17.2, §17.3)
4. **R4.** What is the page-replacement problem, and how do OPT, FIFO, and LRU differ? (§17.4)
5. **R5.** What is Belady's anomaly, and what is thrashing? (§17.4)

FURTHER READING

- **Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (OSTEP), "Paging: Introduction," "Beyond Physical Memory: Mechanisms," and "Beyond Physical Memory: Policies."** The free chapters on the page as the unit, the page fault and swapping mechanism, and the replacement policies (OPT/FIFO/LRU, the anomaly, clock). The reference for §17.1–17.4.
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), Chapter 9 (Virtual Memory).** Address spaces, page tables, demand paging, and memory mapping from the programmer's view; the companion to Chapter 4's hardware treatment.
- **Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal* 5(2):78–101 (1966);** and **Belady, Nelson & Shedler, "An anomaly in space-time characteristics of certain programs running in a paging machine," *Communications of the ACM* 12(6):349–353 (1969).** The primary sources for the optimal (MIN) policy and for Belady's anomaly, respectively.
- **The Linux `mmap(2)`, `madvise(2)`, and `proc(5)` manual pages.** Primary sources for memory mapping, demand-paging hints, and the `/proc/<pid>/status` fields (`VmRSS`) used to measure residency. Verify per kernel version.

Exercises

17.1 WARM-UP Define a virtual page and a physical frame, and say what a page-table entry's present bit records.

17.2 WARM-UP What is a page fault, and what two broad outcomes can the kernel's fault handler produce for it?

17.3 WARM-UP In one line each, distinguish a minor page fault from a major page fault.

17.4 CORE A program `mmap`s 1 GiB and its RSS is a few megabytes immediately afterward, then climbs toward 1 GiB as it works. Explain what happened at each stage in terms of demand paging, and why the allocation did not consume 1 GiB of physical memory up front.

17.5 CORE Simulate FIFO and LRU on the reference string 1 2 3 1 4 2 5 with 3 frames. Give the number of page faults for each and show the resident set after each reference. Which policy does better here, and why?

17.6 CORE Explain Belady's anomaly: state what it is, which policy exhibits it, and why LRU and OPT cannot. Use the phrase "resident set is a superset with more frames" in your explanation of the immune policies.

17.7 COST (a) Why is a major page fault dramatically more expensive than a minor one — put rough numbers on it using Chapter 3's memory hierarchy. (b) Why does evicting a *clean* page cost less than evicting a *dirty* one? (c) A service's p99 latency has sudden 50 ms spikes that correlate with high major-fault counts; what is the likely cause, and is the fix in the CPU-bound code?

17.8 FADED *Worked, with the last step for you.* A batch job processes a 30 GB dataset on a 32 GB machine by `mmap`ing the whole file and iterating over it once, sequentially. It runs fine. A colleague "optimizes" it to make ten concurrent passes over the same mapping from ten threads to "use all the cores," and now the machine crawls with near-zero CPU utilization.
Step 1 (given): The single sequential pass has a small working set at any instant (a sliding window of recently touched pages), so it fits comfortably in RAM and pages stream in and out cleanly.
Step 2 (given): Ten concurrent passes at different file offsets touch ten far-apart regions at once, so the combined working set is large and scattered.
Step 3 (given): That combined working set exceeds physical memory, so every page the OS evicts is soon needed by another thread, generating major fault after major fault.
Step 4 (you): Name the phenomenon precisely, explain why CPU utilization is near *zero* during it (what are the cores waiting on?), and give the direction of the fix (not "optimize the loop").

17.9 MIXED For each observation, name the most likely cause and one thing to measure to confirm it: (a) allocating a huge array succeeds instantly but the program slows down later as it fills the array; (b) a program that fit in RAM last week now runs 100× slower after the dataset grew slightly; (c) reading a memory-mapped file is fast the second time through but slow the first.

17.10 MIXED Drawing on Chapters 4, 14, and 17, classify each statement as *true* or *false* and fix the false ones: (a) "After a successful 8 GB `malloc`, 8 GB of physical RAM is now in use." (b) "A segmentation fault is just a page fault the handler could not satisfy legally." (c) "`fork` copies the parent's entire address space immediately." (d) "Two processes with the same virtual address are reading the same physical frame."

17.11 STRETCH Copy-on-write ties this chapter to Chapter 14. (a) Explain how the page table lets `fork` give the child a "copy" of the address space while physically copying no data at first. (b) Describe exactly what happens, page by page, when the child later writes to one page — which fault fires, what the handler does, and how many pages get copied. (c) Explain why this makes the common `fork`-then-`exec` pattern (Chapter 14) nearly free even for a large parent, and what a single write to a shared page costs.

17.12 LAB On your own machine: (a) write a program that `malloc`s (or `mmap`s) a few hundred MiB and touches one byte per 4 KiB page; run it under `/usr/bin/time -v` and report the minor-fault count — confirm it is about one per page. (b) Read `VmRSS` from `/proc/self/status` before and after touching a large mapping, and report both values to show reservation vs residency. (c) Optional and careful: on a machine you can afford to stress (or a VM), allocate and touch more than physical RAM and observe swap I/O / major faults climbing — describe the thrashing behavior and stop before you wedge the box. Label your kernel version, page size, and RAM.

Chapter 18 — Signals: Asynchronous Control Flow

Before you start: Chapter 14 (when a child exits, the kernel notifies the parent with `SIGCHLD` — that notification *is* a signal; and `wait` reaps the child), Chapter 13 (a blocking system call can be interrupted — this chapter is what interrupts it), Chapter 11 (a bad memory access became a segmentation fault — `SIGSEGV` is a signal too). · Every earlier chapter's control flow was *synchronous*: the program ran its own instructions in order. This chapter adds a second, *asynchronous* flow the kernel can force on a process at almost any instant.

BEFORE YOU READ ON

From memory, reaching back: (1) Chapter 14 — after a child process exits but before the parent `waits` for it, what is the child called, and how did the parent learn the child had finished? (2) Chapter 13 — a process blocked in a `read` that never completes is off the CPU; name one thing that could wake it besides the data arriving. (3) *Predict:* a program installs a handler for `SIGUSR1`, then blocks that signal, sends it to itself *five* times in a tight loop, and finally unblocks it. How many times does the handler run — five, one, or zero? Write down your guess and why — we will measure it in §18.4.

So far a program's control flow has been its own: it executes its instructions, calls functions, makes system calls, and returns — a single thread of cause and effect (threads in Chapter 15 added more such flows, but each was still running *its own* code). A **signal** breaks that assumption. It is an **asynchronous notification** the kernel delivers to a process to tell it that an event happened — a software analog of a hardware interrupt — and when one arrives, the process's normal flow is suspended, a designated **handler** runs, and then the normal flow resumes where it left off. You have already met signals without naming them: the segmentation fault (`SIGSEGV`) of Chapter 11, the `Ctrl-C` that kills a program (`SIGINT`), the `SIGCHLD` that told a parent its child had exited (Chapter 14). This chapter makes the mechanism precise: what a signal is and its default dispositions (§18.1), how to catch one with a handler (§18.2), how signals interrupt blocking system calls — the `EINTR` you must handle (§18.3) — and the sharp hazards of running code asynchronously, namely async-signal-safety and the fact that standard signals do not queue (§18.4). We close by placing signals among the OS's communication mechanisms (§18.5). Signals are the kernel's way of interrupting your program to tell it something; the danger is that "at almost any instant" means the handler can run in the middle of anything.

18.1 What a signal is, and its default disposition

A **signal** is a small integer, identified by name, delivered to a process to indicate an event. Some are sent by the kernel in response to hardware or program conditions — `SIGSEGV` (invalid memory access), `SIGFPE` (arithmetic error like divide-by-zero), `SIGCHLD` (a child changed state). Some are sent by users or programs to control a process — `SIGINT` (from `Ctrl-C`), `SIGTERM` (the polite "please exit," what `kill` sends by default), `SIGKILL` (the un-catchable "die now"), `SIGUSR1`/`SIGUSR2` (application-defined). A process can send a signal to another (subject to permissions) with the `kill` system call — poorly named, since it sends *any* signal, not only lethal ones.

Every signal has a **default disposition** — what happens if the process does nothing about it. The common defaults are: **Terminate** the process (`SIGTERM`, `SIGINT`); **Terminate and dump core** (`SIGSEGV`, `SIGABRT` — the core file for a debugger); **Ignore** the signal (`SIGCHLD`'s default is to be ignored — which is why an un-reaped child still becomes a zombie, per Chapter 14; the notification is delivered but discarded); and **Stop / Continue** the process (`SIGSTOP`/`SIGCONT`, job control). A process can *change* the disposition of most signals — catch them with a handler, or ask to ignore them — with two crucial exceptions: **`SIGKILL` and `SIGSTOP` can neither be caught nor ignored.** That is by design: the OS must retain an unblockable way to kill or halt any process, so that a program cannot make itself immortal by trapping every signal.

QUICK CHECK

Name the four broad default dispositions a signal can have. Which two signals can a process *not* catch or ignore, and why does the OS insist on that? (Answer after the next section.)

18.2 Catching a signal: handlers and `sigaction`

To catch a signal, a process installs a **signal handler**: a function the kernel will call when that signal is delivered. The portable, well-defined way to install one is the `sigaction` system call (the older `signal()` function has historically ambiguous semantics across systems; prefer `sigaction`). You fill in a `struct sigaction` with your handler function and flags, and register it for a given signal number:

```
void handler(int signum){ /* runs asynchronously when the signal arrives */ }

struct sigaction sa;
memset(&sa, 0, sizeof sa);
sa.sa_handler = handler;
sigaction(SIGINT, &sa, NULL);   /* from now on, Ctrl-C runs handler() instead of terminating
*/
```

Once installed, the handler runs *asynchronously*: when `SIGINT` arrives, the kernel suspends whatever the process was doing, calls `handler`, and — when the handler returns — resumes the interrupted code exactly where it left off. The program did not *call* the handler; the kernel injected it into the control flow. This is the defining strangeness of signals and the source of every hazard in §18.4: the handler can begin executing between any two instructions of your main code, including in the middle of a library call. The handler's signature is fixed (`void handler(int)`, receiving the signal number so one handler can serve several signals), and while it runs, the same signal is (by default) blocked so the handler does not interrupt itself.

QUICK CHECK

Which system call installs a signal handler the well-defined way, and why is the older `signal()` discouraged? In what sense does the program *not* call its own handler? (Answer below the communicate box.)

18.3 Signals interrupt system calls: `EINTR`

Here is the consequence that bites every real program. Suppose a process is blocked in a slow system call — `read` waiting for input that has not arrived, `wait` for a child, `sleep` — and a signal arrives that the process catches. The kernel cannot both run the handler and leave the process cleanly blocked, so it interrupts the system call: it runs the handler, and then the blocked call returns early with the error **`EINTR`** ("interrupted system call"). The call did not fail for any reason of its own — the data may still be coming — it was simply cut short to deliver the signal. Here is a program that blocks in `read` on an empty pipe, arranges `SIGALRM` to fire after one second (handler installed *without* the restart flag), and reports what `read` returns (gcc 11.4.0, Linux 6.18):

```
alarm(1);                                  /* deliver SIGALRM in 1 second */
ssize_t n = read(fd[0], buf, sizeof buf);  /* blocks; nobody writes to the pipe */
/* ... one second later the signal fires, the handler runs, and read returns: */

read returned -1, errno=4 (Interrupted system call)
```

The `read` returned `-1` with `errno == EINTR`, not because the pipe failed but because the signal interrupted it. A correct program must expect this: any slow system call that can be interrupted by a caught signal may return `EINTR`, and the usual fix is to **retry the call** (the classic `while ((n = read(...)) < 0 && errno == EINTR);` loop). Alternatively you can install the handler with the **`SA_RESTART`** flag, which asks the kernel to *automatically restart* many interrupted system calls instead of failing them with `EINTR` — but the coverage is incomplete (some calls still return `EINTR` even with `SA_RESTART`), so robust code handles `EINTR` regardless. The naive assumption that a system call either blocks until it succeeds or fails for a real reason is simply wrong once signals are in play.

THE SAME IDEA THREE WAYS: A CAUGHT SIGNAL CUTS A BLOCKING CALL SHORT

1. In words. A process blocked in a slow system call cannot stay cleanly blocked while a handler runs, so the kernel interrupts the call: it runs the handler, then the call returns `-1` with `errno == EINTR`. The operation did not truly fail; it was cut short to deliver the signal, so the program must retry it (or use `SA_RESTART`, with caveats).

2. As a picture. Figure 18.1: main thread blocked in `read` → signal arrives → handler runs → control returns to `read`, which unwinds and returns `EINTR` rather than resuming the block.

3. As numbers. A `read` on an empty pipe with a `SIGALRM` after one second (no `SA_RESTART`): `read returned -1, errno=4 (Interrupted system call)`. With a correct retry loop it would resume waiting; with `SA_RESTART` the kernel would restart it for you.

(Answer to the §18.1 quick check: the four default dispositions are Terminate, Terminate-and-dump-core, Ignore, and Stop/Continue. `SIGKILL` and `SIGSTOP` cannot be caught or ignored, so the OS always retains a way to kill or halt any process — a program cannot trap its way to immortality.)

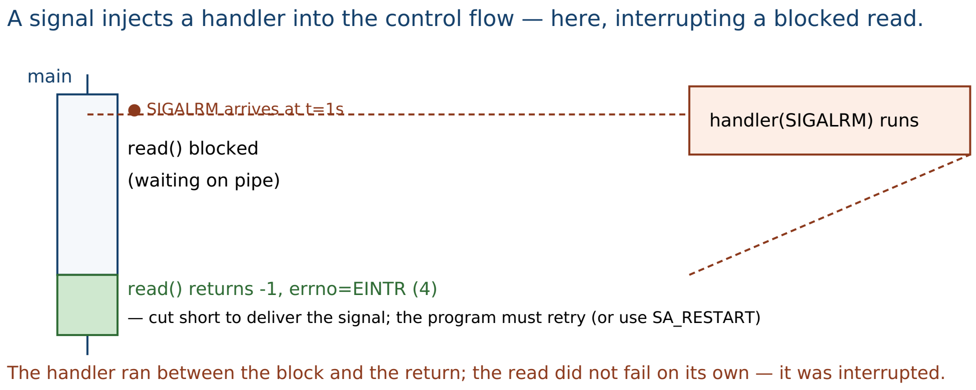

Figure 18.1: A caught signal interrupts a blocking system call. The kernel suspends the block, runs the handler, then the interrupted call returns `-1` with `errno == EINTR` instead of continuing to wait — which the program must handle (retry) or opt out of with `SA_RESTART`.

18.4 The hazards: async-signal-safety and non-queuing

Because a handler can run between any two instructions of your program — including inside a library function that is halfway through updating its own state — the code a handler may safely execute is sharply restricted. A function is **async-signal-safe** if it is safe to call from a signal handler, which essentially means it is reentrant: it does not rely on global state that a signal could catch mid-update. Most of the standard library is *not* async-signal-safe. The classic trap is calling `printf` (or `malloc`) in a handler: if the signal arrives while the main program is already inside `printf` or `malloc` — holding an internal lock or partway through a data structure — the handler's second call to the same function re-enters it and can deadlock (waiting on a lock the interrupted call holds) or corrupt the heap. The kernel documents the exact set of functions guaranteed safe (the `signal-safety(7)` man page); it is a short list — `write`, `_exit`, and a handful of others — and `printf`/`malloc`/most of libc are not on it. The safe idiom is to do almost nothing in a handler: set a flag of type `volatile sig_atomic_t` and return, then let the main loop notice the flag and do the real work synchronously.

The second hazard is delivery semantics, and it settles the opener's prediction. Standard signals (numbers 1–31) are **not queued**: a signal is either pending or not, a single bit, so if the same signal is generated several times while it is blocked, the process still sees it *once* when unblocked — the extra instances coalesce and are lost. Here is the opener's exact program — install a `SIGUSR1` handler that counts calls, block `SIGUSR1`, send it to ourselves five times, then unblock:

```
sigprocmask(SIG_BLOCK, ...);       /* block SIGUSR1 */
for (int i=0;i<5;i++) raise(SIGUSR1);   /* send it 5 times while blocked */
/* handler count so far = 0  (blocked, so not delivered yet) */
sigprocmask(SIG_UNBLOCK, ...);     /* unblock: the pending signal is delivered */

sent 5 while blocked; handler count so far = 0
after unblock, handler ran 1 time(s)
```

The handler ran **once**, not five times: the five sends collapsed into a single pending bit. This is why you must never use a standard signal as a counter or a reliable event stream — if two children exit at nearly the same moment, the parent may get a single `SIGCHLD`, so a correct `SIGCHLD` handler reaps *all* available children in a loop (`while (waitpid(-1, ..., WNOHANG) > 0)`), not one per signal. (POSIX real-time signals, `SIGRTMIN` through `SIGRTMAX`, *do* queue and can carry a small payload, at the cost of more machinery — the exception that proves the rule.) The robust modern pattern for turning asynchronous signals into something a normal event loop can consume is the **self-pipe trick** (the handler does one async-signal-safe `write` to a pipe the main loop reads) or Linux's `signalfd`, both of which move the real work out of the handler and back into the synchronous main flow.

TRAP: DOING REAL WORK IN A SIGNAL HANDLER

The natural instinct on catching a signal — log a message, update a data structure, free some resources, flush state — is exactly what you must not do, because a handler runs *asynchronously, in the middle of arbitrary code*. Calling anything not on the async-signal-safe list (which is nearly all of libc, including `printf`, `malloc/free`, and most of the standard library) risks a deadlock or heap corruption if the signal interrupted that very function. The bug is vicious because it is *rare*: it only manifests when the signal happens to land inside the unsafe call, so the program runs fine in testing and hangs or crashes in production under load — the same "passes the test, broken anyway" pathology as the data race of Chapter 15, and for the same reason (a schedule-dependent window). The discipline: a handler should do the minimum async-signal-safe thing — set `volatile sig_atomic_t got_signal = 1;` (or `write` one byte down a self-pipe) and return — and the main loop does the real work when it is safe. If you find yourself wanting to `printf` from a handler to debug it, that itch is the bug.

SAY IT LIKE A SYSTEMS ENGINEER

Try it first. A teammate says: "For graceful shutdown I added a `SIGTERM` handler that logs 'shutting down,' flushes the in-memory cache to disk, closes the DB connections, and calls `exit` — all right there in the handler. Clean, right?" Cover the paragraph and respond in three or four sentences, drawing on §18.4.

Model answer. "The intent is right but the handler is doing far too much: it runs asynchronously, possibly in the middle of a `malloc` or an `fprintf`, and logging, flushing, and closing all call functions that are not async-signal-safe — so if `SIGTERM` lands at the wrong instant, the handler deadlocks on a lock the interrupted code holds or corrupts the heap, and your 'graceful' shutdown hangs the process precisely when you're trying to stop it cleanly. The safe pattern is to make the handler trivial — set a `volatile sig_atomic_t shutdown_requested = 1;` (or write a byte to a self-pipe) and return — and let the main loop, on seeing the flag, do the flush, close, and exit synchronously where all of libc is safe to call. That also lets you bound and order the shutdown steps instead of racing them inside an interrupt. Keep handlers to a flag; do the work in the main flow." Naming async-signal-safety and moving the work out of the handler is the senior register.

(Answer to the §18.2 quick check: `sigaction` is the well-defined way to install a handler; the older `signal()` is discouraged because its semantics — handler persistence, whether the signal is blocked during its own handler, and system-call restart behavior — historically vary across systems. The program does not call its handler: the kernel ***injects*** *it into the control flow at delivery time, suspending whatever was running and resuming it when the handler returns.)*

18.5 What to carry forward

A **signal** is an asynchronous notification the kernel delivers to a process — a software interrupt — and each has a **default disposition** (terminate, terminate-and-core, ignore, stop/continue), with `SIGKILL` and `SIGSTOP` the two that can never be caught or ignored. A process catches a signal by installing a handler with `sigaction`; the handler runs asynchronously, injected into the control flow, and returns to where the program was interrupted. Two consequences dominate correct use. First, a caught signal **interrupts blocking system calls**, which return `-1` with `errno == EINTR` (we measured a `read` cut short to `errno 4`) — you must retry, or use `SA_RESTART` with care. Second, a handler runs in the middle of arbitrary code, so it may call only **async-signal-safe** functions (not `printf`/`malloc`/most of libc), and standard signals **do not queue** — five rapid `SIGUSR1`s coalesced into a single delivery in our measurement — so the safe pattern is a handler that sets a `volatile sig_atomic_t` flag (or writes a self-pipe) and defers the real work to the synchronous main loop.

Signals complete the picture of how the operating system talks *to* a process: the system-call boundary (Chapter 13) is how a process asks the kernel for things synchronously; signals are how the kernel (or another process) interrupts a process asynchronously to tell it something happened. But a signal carries almost no information — just its number (real-time signals add a small payload) — so it is a doorbell, not a conversation. When processes need to exchange real *data* rather than notifications, they need genuine inter-process communication: pipes and FIFOs for byte streams, shared memory (built, as Chapter 17 showed, on shared mappings in the page tables) for the fastest sharing, and sockets for talking across machines. That IPC toolkit — the channels that carry data between the isolated address spaces this part has spent five chapters building and protecting — is where the operating system part of this volume finishes, and it is the bridge to the low-level I/O and networking that the data systems in later volumes are made of.

CAN YOU… WITHOUT LOOKING?

- Define a signal as asynchronous control flow, and name its four default dispositions.
- Say which two signals can never be caught or ignored, and why the OS insists on that.
- Install a handler with `sigaction` from memory, and explain in what sense the kernel "injects" the handler.
- Explain `EINTR`: why a caught signal makes a blocking call return it, and the two ways to cope.
- Explain async-signal-safety and why `printf` in a handler is a bug, and give the safe handler pattern.
- Explain why standard signals do not queue, and what that means for a `SIGCHLD` handler reaping children.

RETRIEVAL — CLOSED BOOK

Cover the chapter. Answer from memory; rate confidence first.

1. **R1.** What is a signal, and what are the four default dispositions? (§18.1)
2. **R2.** Which two signals cannot be caught or ignored, and why? (§18.1)
3. **R3.** What does `sigaction` do, and in what sense does a handler run asynchronously? (§18.2)
4. **R4.** What is `EINTR`, when does it occur, and how do you handle it? (§18.3)
5. **R5.** What does async-signal-safe mean, and why do standard signals not queue? (§18.4)

FURTHER READING

- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CS:APP), Chapter 8 (Exceptional Control Flow), §8.5 (Signals).** Signals as exceptional control flow, handlers, blocking, and the concurrency hazards of handlers — the reference for §18.1–18.4.
- **Stevens & Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. (2013), Chapter 10 (Signals).** The canonical, thorough treatment of signal semantics, `sigaction`, reliable signals, and interrupted system calls. `[⚠ reference — not free]`
- **The Linux `signal(7)`, `sigaction(2)`, and `signal-safety(7)` manual pages.** Primary sources for the signal list and default dispositions, the `SA_RESTART`/EINTR behavior, and the exact set of async-signal-safe functions. Verify per system.
- **The Linux `signalfd(2)` manual page and the "self-pipe trick" (D. J. Bernstein).** The documented ways to turn asynchronous signal delivery into a synchronous, event-loop-friendly readable stream — the robust alternative to doing work in a handler.

Exercises

18.1 WARM-UP In one sentence, what is a signal, and how does it differ from a normal function call in how it enters the program?

18.2 WARM-UP Name the four broad default dispositions a signal can have, with one example signal for each if you can.

18.3 WARM-UP Which two signals can a process neither catch nor ignore, and what would go wrong if it could?

18.4 CORE A program blocks in `read` waiting for input and catches `SIGINT` with a handler. Explain, step by step, what happens when the user presses `Ctrl-C`: what the `read` returns, what `errno` is, and why the call did not simply resume waiting. Give the two standard ways to make the program robust to this.

18.5 CORE Explain why calling `printf` (or `malloc`) inside a signal handler is unsafe. What property must a function have to be callable from a handler, and what is the safe pattern for a handler that needs to trigger real work?

18.6 CORE Explain why standard signals do not queue, and why this means a `SIGCHLD` handler must reap children in a loop (with `WNOHANG`) rather than reaping exactly one child per delivered signal. Tie this back to Chapter 14's zombies.

18.7 COST (a) Why is a handler that only sets a `volatile sig_atomic_t` flag both cheap and safe, compared with one that does the work directly? (b) What is the latency cost of the self-pipe / `signalfd` approach versus a direct handler, and what correctness benefit do you buy with it? (c) Why can a signal-based design never be a reliable *counter* of events under load?

18.8 FADED *Worked, with the last step for you.* A long-running server installs a `SIGHUP` handler to reload its config. The handler re-reads the config file, rebuilds several in-memory structures with `malloc`, and logs progress with `fprintf`. In testing it works; in production, under heavy request load, the server occasionally hangs completely when config is reloaded.
Step 1 (given): The handler runs asynchronously and can interrupt the main code at any instant, including while the main code is inside `malloc` or `fprintf`.
Step 2 (given): `malloc` and stdio hold internal locks / mutable state; they are not async-signal-safe.
Step 3 (given): Under heavy load the odds rise that `SIGHUP` lands while a request thread is inside one of those functions, so the handler's own call to it re-enters and deadlocks.
Step 4 (you): Name the bug precisely, explain why it appears only under load (not in tests), and describe the redesign that fixes it — what the handler should do and where the reload work should actually run.

18.9 MIXED For each need, say whether a *signal* is the right mechanism or whether you want real IPC (a pipe, shared memory, or a socket), and justify in a line: (a) tell a running daemon to re-read its configuration; (b) stream a megabyte of data from one process to another on the same host; (c) notify a parent that a child has exited; (d) send a structured request and get a structured reply between two processes.

18.10 MIXED Drawing on Chapters 13–18, classify each statement as *true* or *false* and fix the false ones: (a) "A signal handler is called by the program the way any other function is." (b) "If you install a handler with `SA_RESTART`, you never have to worry about `EINTR`." (c) "Sending a process `SIGKILL` lets it run cleanup code first." (d) "A segmentation fault is the delivery of the `SIGSEGV` signal."

18.11 STRETCH A signal handler and the main program both touch a shared `int running` flag. (a) Why must that flag be declared `volatile sig_atomic_t` rather than a plain `int` — what could the compiler or the hardware otherwise do? (b) Compare this hazard with the data race of Chapter 15: how is a handler-vs-main-flow conflict similar to a thread-vs-thread race, and how is it different (single thread, asynchronous interruption)? (c) Explain why this makes signals a form of concurrency even in a single-threaded program.

18.12 LAB On your own machine: (a) write a program that blocks in `read` on an empty pipe, sets an `alarm`, and prints what `read` returns and `errno` when `SIGALRM` fires — confirm you see `EINTR` (errno 4); then reinstall the handler with `SA_RESTART` and observe the difference. (b) Write a program that installs a counting handler for `SIGUSR1`, blocks it, `raise`s it five times, unblocks, and prints how many times the handler ran — confirm it is once, not five. (c) Explain both results in terms of interrupted system calls and non-queuing standard signals. Label your compiler and kernel version.

VOL 1 · SYSTEMS PROGRAMMING & PERFORMANCE · PART IV

Concurrency & Parallelism

Threads, locks and their hazards, lock-free techniques, atomics, and the memory model that makes concurrent code correct (or not).

VOL 1 · SYSTEMS PROGRAMMING & PERFORMANCE · PART V

Rust for Systems

Ownership, borrowing, and lifetimes — memory safety without a GC, and how the type system encodes the invariants the previous parts made you prove by hand.

VOL 1 · SYSTEMS PROGRAMMING & PERFORMANCE · PART VI

Performance Engineering

Measure before you optimize: profiling, benchmarking, cache-aware and data-oriented design, SIMD, and reasoning about where time actually goes.

VOL 1 · SYSTEMS PROGRAMMING & PERFORMANCE · PART VII

I/O & Networking, Low-Level

Files, sockets, blocking vs async, epoll/io_uring, zero-copy — the systems substrate under every data pipeline and service.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART VIII

Storage & the Disk

Pages, files, the buffer pool, and the storage manager — the foundation you build a database on.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART IX

Storage Engines

B-trees and LSM-trees implemented and analyzed — the two families and when each wins.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART X

Durability & Recovery

Write-ahead logging, ARIES, checkpoints, and crash recovery — how a database keeps its promises across failures.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART XI

Transactions & Concurrency Control

ACID, isolation levels and their anomalies, 2PL, MVCC, and OCC — correctness under concurrency, implemented.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART XII

Query Processing & Execution

Parser, planner, physical operators, and execution models (Volcano/vectorized/codegen) — turning SQL into work.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART XIII

Query Optimization

Statistics, cardinality estimation, cost models, and join ordering — the optimizer, honestly.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART XIV

Indexing & Access Methods

Secondary/clustered/covering indexes, and specialized structures — the access-method layer.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART XV

[BUILD] A Working Database

Assemble the pieces into a small but real relational engine — storage → txn → query → execute — with tests.

VOL 2 · DATABASE INTERNALS (BUILD A DATABASE) · PART XVI

Toward Distributed Databases

Sharding, replication, and the distributed-transaction problem — the bridge to Volume 3.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XVII

Models & Foundations

System and failure models, the FLP impossibility, time and clocks, causality and logical time — the formal ground.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XVIII

Replication & Consistency

Consistency models, linearizability, CAP/PACELC, and quorums — what 'consistent' actually means.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XIX

Consensus

Paxos, Raft, and view-stamped replication (with a note on Byzantine fault tolerance) — agreement under failure.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XX

Coordination & Transactions

Leader election, distributed transactions, 2PC/3PC, and sagas — coordinating across nodes.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XXI

Replicated Data & Convergence

Eventual consistency, anti-entropy, gossip, and CRDTs — converging without coordination.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XXII

Partitioning & Scale

Consistent hashing, rebalancing, and hot-spot handling — spreading state and load.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XXIII

Distributed Algorithms & Patterns

Broadcast/ordering, failure detectors, distributed snapshots, and the patterns real systems use.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XXIV

Formal Methods

Specifying and model-checking a protocol with TLA+ — finding the bug before production does.

VOL 3 · DISTRIBUTED SYSTEMS, DEEPLY · PART XXV

Building & Operating Distributed Systems

From spec to a real, observable, testable distributed system — and operating it.

VOL 4 · SECURITY & PRIVACY ENGINEERING · PART XXVI

Security Foundations & Threat Modeling

The security mindset, threat modeling, trust boundaries, and reasoning about adversaries.

VOL 4 · SECURITY & PRIVACY ENGINEERING · PART XXVII

Applied Cryptography

Symmetric/asymmetric crypto, hashing, MACs, signatures, TLS, and key management — used correctly, not invented.

VOL 4 · SECURITY & PRIVACY ENGINEERING · PART XXVIII

Authentication & Authorization

Identity, sessions, OAuth2/OIDC, access-control models, and zero-trust — who can do what.

VOL 4 · SECURITY & PRIVACY ENGINEERING · PART XXIX

Software & Systems Security

Memory-safety bugs, injection, the supply chain, and secure-by-design coding — the common failure modes and their defenses.

VOL 4 · SECURITY & PRIVACY ENGINEERING · PART XXX

Data Security

Encryption at rest and in transit, secrets management, tokenization, and secure key handling for data platforms.

VOL 4 · SECURITY & PRIVACY ENGINEERING · PART XXXI

Privacy Engineering

Differential privacy, anonymization vs pseudonymization, GDPR/CCPA, and data governance — privacy as an engineering discipline.

VOL 4 · SECURITY & PRIVACY ENGINEERING · PART XXXII

Secure Data Pipelines & Platforms

Applying the above to real data systems: lineage, access, multi-tenancy, and compliance-by-design.

VOL 4 · SECURITY & PRIVACY ENGINEERING · PART XXXIII

Detection, Response & Operations

Logging, monitoring, detection, and incident response — assuming breach and limiting blast radius.

VOL 5 · THE STAFF/PRINCIPAL ENGINEER & TECH LEAD · PART XXXIV

The Staff+ Role & Scope

Archetypes (tech lead, architect, solver, right-hand), what 'impact' means above senior, and the shift from output to leverage.

VOL 5 · THE STAFF/PRINCIPAL ENGINEER & TECH LEAD · PART XXXV

Technical Leadership & Strategy

Technical vision, strategy, and roadmaps — setting direction others can follow.

VOL 5 · THE STAFF/PRINCIPAL ENGINEER & TECH LEAD · PART XXXVI

Architecture & Design at Scale

Design review, ADRs, trade-off analysis, and evolving systems without a rewrite — the architect's judgment, applied.

VOL 5 · THE STAFF/PRINCIPAL ENGINEER & TECH LEAD · PART XXXVII

Influence Without Authority

Building alignment and buy-in, driving decisions across teams, and disagree-and-commit.

VOL 5 · THE STAFF/PRINCIPAL ENGINEER & TECH LEAD · PART XXXVIII

Communication & Writing

The written word as a staff engineer's main tool: design docs, RFCs, and making complex ideas land.

VOL 5 · THE STAFF/PRINCIPAL ENGINEER & TECH LEAD · PART XXXIX

Mentoring & Growing Engineers

Mentorship, sponsorship, code/design review as teaching, and multiplying the people around you.

VOL 5 · THE STAFF/PRINCIPAL ENGINEER & TECH LEAD · PART XXXX

Execution & Judgment

Leading projects and incidents, making decisions under uncertainty, and knowing what NOT to do.

VOL 5 · THE STAFF/PRINCIPAL ENGINEER & TECH LEAD · PART XXXXI

Career & the Organization

Navigating the org, the promotion case, sustainability, and a durable definition of technical leadership.

BACK MATTER

Solutions to Exercises

Worked solutions to the end-of-chapter exercises. Try each before reading its solution.

Chapter 1 — Solutions

1.1 **Uniform cost:** every basic operation — arithmetic, comparison, and any memory read or write — costs the same fixed unit of time, independent of which address is accessed. The three false assertions about real hardware: (1) all memory accesses cost the same (they vary by orders of magnitude with which tier of the hierarchy holds the data); (2) one instruction takes one unit of time (real CPUs overlap, reorder, and speculate, retiring several per cycle or stalling for hundreds — Chapter 2); (3) operations are independent and equally priced (some ops cost more than others, and a dependent instruction must wait for its input). *Method note:* the trap is listing symptoms ("caches exist") without naming the underlying assumption they violate — always tie the fact back to *uniform cost*.

1.2 Fastest to slowest: **register** (<1 ns) < **L1** (~1 ns) < **L2** (a few ns) < **L3** (~10 ns) < **DRAM** (~100 ns) < **NVMe SSD** (tens of µs) < **same-datacenter network round trip** (hundreds of µs to ~1 ms). All figures are orders of magnitude, machine- dependent (§1.3).

1.3 The memory wall is the large and historically widening gap between how fast a CPU can execute instructions and how long DRAM takes to return a requested byte, because processor throughput improved much faster than DRAM latency for decades (Wulf & McKee, 1995). It makes uniform cost worse over time because the ratio between "one arithmetic op" and "one DRAM access" grew, so pricing a DRAM access at the same "one unit" as an add gets more wrong every hardware generation, not less.

1.4 (a) A few billion ops/second is $\sim 3\times10^9$/s, so in 100 ns = 10^{-7} s the core could do on the order of **a few hundred operations** ($3\times10^9 \times 10^{-7} = 300$). (b) A program dominated by random pointer-following spends most of its time *waiting* for DRAM, leaving the arithmetic units idle for hundreds of potential operations per miss — so it can run at a small fraction of the core's peak throughput even though its big-O looks fine. *Order-of-magnitude flags:* both the 100 ns and the few-billion-ops/s are approximate and machine-dependent; the point is the ratio ($\sim 10^2$), not the exact count. **Feed-forward:** if this surprised you, revisit §1.2.

1.5 Two explanations: (1) **Constant factors dominate at this size.** Big-O ignores the constant, and the O(n) bucketing pass may do far more *slow* memory traffic per element (e.g. scattering writes across buckets spread through DRAM) than the O(n log n) sort, which may stream memory in a cache-friendly order (§1.4). (2) **Wrong regime.** The asymptotic advantage only shows up once n is large enough; on this dataset n may be small enough that the log n factor is tiny and the bucketing's larger constant wins for the wrong reasons (§1.4). The single settling action: **measure both on the real data** — big-O cannot answer a "how fast" question. *Tempting wrong answer:* "the O(n) version must be faster, the benchmark is flawed" — this is trusting the RAM model for a speed question, exactly the §1.4 trap. **Feed-forward:** §1.4 and the warning box.

1.6 **Step 4:** The binary search over a small contiguous sorted array is likely to make *fewer trips to slow memory* per lookup — its handful of accesses are clustered and cache-friendly and the small array may live largely in cache, whereas the hash map's single lookup typically lands on a random heap address and pays a full cache/DRAM miss. So the asymptotically worse structure can win on real hardware because *the metric that matters here is costly-memory-accesses per lookup, not operation count.* The one thing to do before trusting the argument: **benchmark both on the real data and access pattern.** *Why this final step and not "pick the lower big-O":* big-O measures scaling and is silent about the constant factor — where the data lives — which is the entire basis of the argument; only measurement can confirm the memory-access reasoning holds for your actual sizes and hardware. **Feed-forward:** §1.4; the array-vs-pointer effect is measured in Chapter 3.

1.7 (a) **RAM/big-O** — the tables grow without bound, so the deciding factor is scaling (quadratic vs linear), which is exactly what big-O captures. (b) **Real cost model** — same size, same O(n); the 8× gap can only come from constant factors, i.e. memory-access patterns. (c) **RAM/big-O** — "1000× bigger" is a scaling question. (d) **Real cost model** — identical length and O(n); the difference is contiguous streaming vs random pointer-chasing through the hierarchy. *Method-selection cue:* if the question mentions "grows / bigger / scales," reach for big-O; if it fixes the size and asks "why is one faster," reach for the real cost model.

1.8 The claims are **not in contradiction**. Claim (i) is about the RAM/big-O model (optimal *scaling*); claim (ii) is about the real cost model (*speed* at a given size, set by constant factors the memory hierarchy dominates). Both can be true simultaneously: two O(n) algorithms are equally optimal asymptotically yet differ 10× in wall-clock time. To choose for production you still need: the actual input sizes you will run at, and a **measurement** of both on representative data and hardware. *Tempting wrong answer:* "'optimal' means fastest, so (i) and (ii) conflict" — conflating asymptotic optimality with speed, the core confusion of §1.4.

1.9 **Mechanical sympathy** is writing code that works *with* the grain of the hardware — its memory hierarchy, pipeline, and I/O costs — rather than against it, by understanding enough of how the machine works to get the most from it. The ordering rule: choose a good *algorithm* first (a bad big-O cannot be rescued by micro-optimization — no cache trick fixes an accidental quadratic), *then* apply mechanical sympathy to win the constant factor. Failure modes: knowing only big-O → asymptotically optimal code that needlessly runs 10× slow; knowing only micro-optimization → a lovingly hand-tuned O(n^2) that dies at scale.

1.10 Three reasons the absolute numbers can't be trusted: (1) they are **microarchitecture-specific** — cache sizes, latencies, and cycle counts differ across CPU models and generations; (2) they **drift with hardware** over time, so any printed figure is dated; (3) measured latency depends on **context** — contention, frequency scaling, what else is resident in cache, and access pattern — so there is no single "the" latency. The ratios are more stable because they reflect the *structural* reason each tier exists (each is a deliberate size/speed tradeoff a step down from the last), and that layering persists even as every absolute number moves. To get trustworthy numbers: **measure on the actual target machine** under representative load (a microbenchmark or a profiler), which is the discipline the performance-engineering chapters build. **Feed-forward:** §1.3; measurement technique in Chapter 3.

1.11 Answers are machine-specific by design; a correct write-up (i) reports real cache-line size (commonly 64 bytes on x86-64) and your actual L1/L2/L3 sizes from `getconf`/ `lscpu`, (ii) reports two measured per-element times — the in-L1 loop should be markedly faster per element than the exceeds-L3 loop, which is bottlenecked on DRAM bandwidth — and (iii) states the ratio you observed. The learning goal is the habit: on a "how fast" question you *generated your own numbers* rather than quoting the book's orders of magnitude. Exact values depend entirely on your hardware; the *shape* (in-cache faster than out-of-cache) is what must appear. **Feed-forward:** the mechanism behind the ratio is §1.2–1.3 and all of Chapter 3.

Chapter 2 — Solutions

2.1 In order: (1) **pipelining** — overlap the stages so different instructions occupy different units in the same cycle; (2) **superscalar** — duplicate execution units and issue several instructions per cycle; (3) **out-of-order** — reorder execution to run later independent instructions while an earlier one is stalled; (4) **speculation / branch prediction** — guess a branch's direction and execute down the predicted path so the pipeline need not wait for the branch to resolve.

2.2 **Latency** is the number of cycles for one instruction to pass through all stages; **throughput** is how often a new instruction *completes*. Because the stages overlap — while one instruction is in "execute," the next is in "decode," and so on — a new instruction can finish every cycle even though each individually spends several cycles traversing the pipeline. The two numbers measure different things: time-through-one vs. completions-per-time.

2.3 A **data hazard**: an instruction needs a result that an earlier, not-yet-finished instruction will produce, so it cannot proceed until that value is ready. A **control hazard**: the CPU does not yet know which instruction comes next because an unresolved branch determines it. **Branch prediction addresses the control hazard** — it guesses the branch direction so fetching can continue instead of stalling.

2.4 In loop A each add depends on the previous add's result (they all target one accumulator), forming a single dependency chain: the out-of-order engine can never have more than one add in flight, so the superscalar units sit mostly idle and the loop runs at roughly one add per dependency latency. In loop B the four partial sums are *independent*, so up to four adds can be in flight at once, keeping multiple execution units busy and finishing in a fraction of the time — same number of additions, more ILP. The enabling condition: floating-point addition is not associative, so B only computes the "same" result if you accept that the grouping (and thus rounding) differs, or the compiler is allowed to reassociate — B's parallelism is possible precisely because its adds have no data dependency between them. *Method-selection note:* the tell that ILP is the lever is "same operation count, one version serializes them." **Feed-forward:** §2.3 and the threeways box.

2.5 On P the data-dependent branch resolves in long predictable runs, so the branch predictor is almost always right, speculation is nearly free, and the pipeline stays full. On Q the branch is a patternless coin flip: the predictor is wrong about half the time, and each misprediction flushes the speculatively executed work and refills the pipeline — a penalty on the order of tens of cycles (machine-specific) paid on ~50% of iterations. The extra time on Q is spent flushing and refilling the pipeline, not doing arithmetic. *Tempting wrong answer:* "Q must be doing more comparisons" — no, the arithmetic is identical; the cost is misprediction, not extra work. **Feed-forward:** §2.4.

2.6 (a) Average cost per iteration ≈ $b + p \cdot k$ cycles (caveat: k is an order-of-magnitude, microarchitecture-specific figure). (b) The misprediction term dominates when $p \cdot k > b$, i.e. roughly $p > b/k$; for a cheap body and a ~tens-of-cycles penalty, even a modest p (a branch that misses a nontrivial fraction of the time) makes misprediction the largest term. (c) Making the branch predictable lowers p (the predictor is right more often); a branchless rewrite removes the branch so there is no k to pay at all. You measure first because on an already-predictable branch p is already tiny, so a branchless rewrite adds complexity for no gain — and may even be slower if it does more arithmetic unconditionally. **Feed-forward:** §2.4 warning box.

2.7 **Step 4:** The miss stalls the CPU *only if there is no independent work to do while it is serviced.* The deciding property is the amount of **instruction-level parallelism in the surrounding code** — whether there are nearby instructions whose inputs are ready and do not depend on the missing load. If there is enough such work, the out-of-order engine hides the miss latency behind it; if the code is a tight dependency chain that all waits on the loaded value, the machine has nothing else to do and stalls for the full miss. It is that property, not the miss latency alone, that decides the outcome, because the latency is a fixed cost of reaching DRAM — what varies, and what you control by how you write the code, is whether the CPU can *overlap* useful work with it. **Feed-forward:** §2.3 (out-of-order) and Chapter 1 §1.2 (miss latency).

2.8 (a) **Branch prediction** — sorting changed only predictability, not arithmetic. (b) **Dependency chain / ILP** — splitting an accumulator exposes independent work. (c) **Memory hierarchy** — a linked list scatters accesses to random addresses (cache misses, Chapter 1) where an array streams contiguously. (d) None of the three is a bottleneck — no branches, no memory pressure, presumably enough ILP — so the loop is already near peak; the lesson is that when none of the mechanisms is being violated, there is little constant-factor slack left to recover. *Method-selection cue:* match the symptom — "sorting helped" → branches; "extra accumulators helped" → ILP; "layout/pointers hurt" → memory.

2.9 The **real cost model** (Chapter 1 §1.4) resolves it: equal big-O guarantees equal *scaling*, never equal *speed*; a 5× gap lives entirely in the constant factor big-O discards. Three independent mechanisms that could each cause it: (1) **memory access pattern** — one version streams cache-friendly memory, the other chases pointers into DRAM (Chapter 1, Chapter 3); (2) **branch misprediction** — one has an unpredictable data-dependent branch flushing the pipeline (§2.4); (3) **dependency chains / low ILP** — one serializes its work and starves the superscalar units (§2.3). The single identifying action: **profile / measure** (e.g. with a profiler reporting cache misses and branch mispredictions) — the mechanism is an empirical question, not one big-O can answer. *Tempting wrong answer:* "it's impossible / the benchmark is wrong," which is the Chapter 1 trap of trusting big-O for a speed question. **Feed-forward:** §2.5 and Chapter 1 §1.4.

2.10 (a) On typical code branches are highly predictable, so speculation is almost always correct and the CPU keeps working instead of stalling several cycles at every branch — the occasional wasted work is far cheaper than stalling on every branch. (b) On an unpredictable branch the predictor is wrong a large fraction of the time, so much speculative work is discarded and the misprediction penalty dominates — a net loss versus not having to speculate at all. (c) At a high level: because speculatively executed instructions can affect microarchitectural state (such as what ends up in the cache) even when their results are later discarded, that residual state can, in principle, be observed and used to infer data that should not have been accessible — this is the basis of the real speculative-execution vulnerability class. The precise mechanisms and mitigations are a genuine, separate topic (touched on in Volume 4); describing them accurately requires primary sources, and this answer deliberately does not invent specifics. *No-fabrication note:* full credit requires the high-level reasoning *and* the acknowledgement that the details need real sources, not invented ones.

2.11 Machine- and compiler-specific by design. A correct write-up reports two measured times (single vs. four accumulators) and their ratio, with the four-accumulator version typically faster when the compiler has not already reassociated or vectorized the single-accumulator loop. It must state which numbers are the reader's own and note the ratio depends on CPU and compiler. Crucially, if optimization collapses the difference (the compiler reassociates or auto-vectorizes both), the correct response is to *report that* and describe what was observed — never to quote a ratio that was not actually measured. *Method note:* the exercise is as much about honest measurement discipline (no fabricated numbers) as about ILP. **Feed-forward:** §2.3 threeways box.

Chapter 3 — Solutions

3.1 64 / 8 = **8 doubles** per line. Reading one from a cold line brings in the whole line, so the **7 neighbors** become cheap hits. This rewards **spatial locality** — you get the nearby elements for free because memory arrives a full line at a time.

3.2 **Temporal:** a loop accumulator `sum += x[i]` — `sum` is reused every iteration, so after its first access it stays resident (retention of recently used lines). **Spatial:** walking `x[0], x[1], x[2], …` — consecutive elements share lines (line fill) and the sequential pattern lets the *prefetcher* fetch ahead. Line fill and prefetching exploit spatial locality; keeping recent lines exploits temporal.

3.3 (a) **Compulsory (cold)** — first-ever reference to those lines. (b) **Capacity** — the 4 GB working set vastly exceeds the 8 MB cache, so lines are evicted before reuse. (c) **Conflict** — a specific power-of-two row length aligns many accesses onto the same cache sets, so they evict each other despite free space elsewhere.

3.4 (a) **Inner loop over columns (row-major order) is faster**: consecutive accesses are adjacent in memory (stride 1), so each 64-byte line is fully used and the prefetcher runs ahead. Inner loop over rows strides by N elements, touching a new line almost every access. (b) Same big-O and same additions govern *scaling*, not *speed*; the constant factor — memory moved and misses incurred — differs by roughly an order of magnitude because only one version has spatial locality (this is the Chapter 1 §1.4 point, now concrete). (c) The slow version suffers mostly **capacity misses** (the whole matrix far exceeds cache, and its poor locality means it re-fetches lines it effectively wasted) — with a real **conflict** component too, since the power-of-two dimension (N = 8192) gives a power-of-two stride that aliases into a few cache sets (the very symptom §3.4 described); its lack of spatial reuse is what turns each fetch into wasted bandwidth. *Tempting wrong answer:* "they're both O(N^2) so the difference must be a measurement error" — the §3.6 trap. **Feed-forward:** §3.6.

3.5 Replace the array-of-pointers with a contiguous **array of the objects themselves** (array of structs, or a struct-of-arrays if you only touch some fields), laid out in traversal order. Why it reduces misses: the objects now share cache lines, so walking them in order has spatial locality — each fetched line yields several objects instead of one random-heap object per pointer (fewer misses). Why it helps hide the rest: sequential, contiguous access is prefetchable and, unlike pointer-chasing, does not force a dependency chain where each object's address depends on reading the previous one — so the out-of-order engine (Chapter 2 §2.3) can overlap independent accesses and hide the misses that remain. *Method note:* the two wins are distinct — locality (fewer misses) and independence (hide the misses) — and the pointer layout loses on both. **Feed-forward:** §3.5.

3.6 (a) 8 bytes used of 64 fetched = **1/8** of each line. (b) A stride-1 traversal uses all 8 values per line, so the stride-8 traversal moves roughly **8× more memory** for the same values. (c) That ~8× is a floor on the slowdown from wasted bandwidth alone; the measured row-major/column-major ratio was larger (~24–26×) because real effects compound it — two that push the measured number above the naive estimate: (i) the hardware **prefetcher** helps the sequential (row-major) loop even more, widening the gap; (ii) **loss of memory-level parallelism / stalling** on the strided loop, and possibly **TLB misses** from touching many pages, add cost the simple bytes-moved model ignores. *No-fabrication note:* the 8× is arithmetic; the ~25× is measured — do not present the estimate as the measurement. **Feed-forward:** §3.6.

3.7 **Step 4:** In this particular experiment the dominant lever is the **number of misses (locality)**, not the inability to hide each one. The column-major loop touches a new line almost every iteration and uses only 1 of 8 values per line, so it issues roughly 8× as many line fetches and moves many times more memory than the row-major loop; that sheer volume of DRAM traffic is the main cost. The dependency argument is secondary here because both versions are simple predictable summations with similar per-iteration dependency structure — what differs overwhelmingly is how many expensive lines each one demands. The number of misses is primary *because the two loops differ in locality but not meaningfully in their dependency chains* — so the variable that changed is misses. To test the claim: measure cache misses and memory traffic for both (e.g. with a profiler's cache-miss counters) and confirm the column-major loop's miss count is the multiple that tracks its slowdown. **Feed-forward:** §3.5–3.6, Chapter 2 §2.3.

3.8 (a) **Capacity miss fixed by blocking** — tiling keeps each block's working set inside cache so it is reused before eviction. (b) **Branch prediction** — sorting makes the filter's data-dependent branch predictable (Chapter 2 §2.4). (c) **Spatial locality / cache lines** — consuming each line fully raises useful-bytes-per-line. (d) **ILP / dependency chain** — independent accumulators expose parallelism (Chapter 2 §2.3). *Method-selection cue:* "tiling/ blocking" → capacity; "sorting helped a branch" → prediction; "layout/stride" → spatial locality; "extra accumulators" → ILP.

3.9 Candidate causes across Chapters 1–3 (at least three): **poor spatial locality / high miss rate** (§3.1–3.5); **capacity misses** from a working set exceeding cache, fixable by blocking (§3.3); **branch mispredictions** from a data-dependent branch (Ch 2 §2.4); **a serial dependency chain / low ILP** (Ch 2 §2.3); pointer-chasing combining bad locality *and* dependencies (§3.5). The first measurement: **run a profiler that reports cache-miss rate and branch-misprediction rate** — it is most informative because it directly distinguishes the two biggest mechanism families (memory vs. branches) in one shot, telling you which chapter's toolkit to reach for before you change any code. *Tempting wrong answer:* "start rewriting the layout" — that guesses the cause; measure first (the discipline of this whole Part). **Feed-forward:** §3.6 and Chapter 1 §1.4.

3.10 (a) Three properties and their effect on the ratio: **cache/last-level-cache size** — a much larger cache (or a smaller matrix) shrinks the gap by keeping more of the working set resident; **memory bandwidth and prefetcher quality** — a stronger prefetcher and higher bandwidth help the sequential loop more, tending to *raise* the ratio; **compiler behavior** — if the compiler vectorizes or reorders one loop, the ratio can move either way. (b) The effect's existence and rough size are robust because they follow from a structural fact true of essentially all cache-based machines: memory moves in lines, and a strided traversal wastes most of each line — so a large (not small) multiplier is guaranteed even though its exact value tracks the specific hardware. (c) Reporting "about 25× on this specific CPU, reproduce it yourself" is honest because the number was measured on one machine and genuinely varies; it is more useful because it teaches the reader the *method* (measure on your own hardware) rather than a false universal constant they might quote as fact — which is exactly the no-fabrication discipline of this book. **Feed-forward:** §3.6 and the warning box.

3.11 Machine-specific by design. A correct write-up records the reader's real cache-line size (commonly 64 bytes) and L1/L2/L3 sizes; confirms both loops produce the identical sum (proving equal work); reports two measured times and the observed ratio; and compares that ratio to the book's ~25× and to the ~8× stride estimate from §3.6, expecting it to sit somewhere around or above the estimate and to vary with hardware. The essential discipline: report only measured numbers, and if the compiler elides a loop, change the code (consume the result / use a `volatile` sink) and say what was done — never quote a number you did not observe. **Feed-forward:** §3.6; deeper measurement in the performance-engineering chapters.

Chapter 4 — Solutions

4.1 (a) **12 bits** (2^{12} = 4096 = 4 KiB). (b) **One translation**: `a[0]` and `a[1]` sit on the same 4 KiB page (unless `a[0]` happens to be the last element of a page), so the same page number translates both. (c) A 4 KiB page holds 4096/8 = **512 `longs`**, so up to ~512 consecutive elements share one translation before you cross into the next page. This is the TLB analogue of the cache-line reuse from §3.1 — one translation amortized over many accesses.

4.2 **Isolation** — each process names only its own frames, so it cannot read or corrupt another's memory (this is the security/isolation boundary). **A simple, uniform programming/layout model** — every process sees the same clean virtual layout regardless of where it lands in physical RAM. **OS flexibility** — a virtual page can map to a frame, a file, nothing yet (lazy allocation), or be shared read-only. The isolation one is the protection boundary.

4.3 A data cache stores recently used **cache lines** so a repeated data access is a cheap hit; the TLB stores recently used **translations (virtual-page→physical-frame mappings)** so a repeated access to the same **page** skips the **page-table walk**. Same hit/miss/eviction logic (§3.3), different key and value: address→line for the data cache, page→frame for the TLB.

4.4 A multi-level walk reads one table to learn the address of the next table, reads that to learn the next, and so on — **each read's address depends on the value returned by the previous read**, so the reads cannot be issued in parallel or overlapped; they are strictly serial. That matters because a dependency chain defeats the CPU's latency-hiding (Chapter 2 §2.3): there is no independent work to run during each memory access, so the walk pays the full latency of every step in sequence. This is exactly the **pointer-chasing** structure of a linked-list traversal (§3.5): in both, you cannot compute the next address until the current access completes, so both are serial-latency-bound rather than bandwidth-bound. *Method note:* recognizing "the next address depends on this read" is the general tell for a latency-bound pattern. **Feed-forward:** §3.5, Chapter 2 §2.3.

4.5 (a) A random probe pays (i) a **data-cache miss** to fetch the line holding the bucket (~100 ns to DRAM) and (ii) a **TLB miss / page-table walk** to translate the bucket's address before the fetch — several dependent memory accesses. (b) With 4 KiB pages, TLB reach is (entries × 4 KiB), typically only a few megabytes; a 400 MiB table spans far more pages than the TLB can hold, so random probes keep landing on pages whose translations were evicted — heavy TLB misses (a capacity miss at the translation level, §4.4). (c) To cut translation cost: map the table with **huge pages** (raises TLB reach ~512×), assuming a profiler confirms page-walk cost. To cut the data-miss cost: improve **locality** of the buckets (e.g. keys and values stored together, open addressing with good probe locality, or a more cache-friendly layout) so more probes hit. *Tempting wrong answer:* "it's all just DRAM latency, nothing to do" — the §4.4 trap; the surplus over ~100 ns is the translation cost. **Feed-forward:** §4.4–4.5.

4.6 (a) **reach = E × 4 KiB**. (b) Expect heavy thrashing when the randomly-touched footprint exceeds reach: **W > E × 4 KiB** (more precisely, when the number of distinct pages touched, W / 4 KiB, far exceeds E), so translations are evicted before reuse. (c) Switching to 2 MiB pages multiplies reach by **2 MiB / 4 KiB = 512×**, so the threshold in (b) becomes W > E × 2 MiB — a working set 512× larger now fits within reach, which is exactly why huge pages relieve TLB thrashing. *No-fabrication note:* the 512× is arithmetic from real, documented page sizes; the actual E is a chip-specific number this book does not invent. **Feed-forward:** §4.5.

4.7 **Step 4:** The random-by-page walker needs a *new* translation on essentially **every jump**, because each jump targets a different page and it does not stay on any page long enough to reuse a translation. Since the TLB's reach (a few MiB) is far smaller than the 512 MiB footprint, the translation for a page it jumps to was almost certainly evicted since the last visit — so nearly every random access **misses the TLB and pays a page-table walk**, on top of the data-cache miss. The sequential walker, by contrast, amortizes one translation over ~512 accesses (Step 3), so its TLB-miss *rate* is ~1 per 512 accesses versus ~1 per access for the random walker. To confirm the TLB — not just the data cache — is the culprit, measure the **dTLB-load-miss (or page-walk) count/rate** for both runs (e.g. `perf stat -e dTLB-load-misses`): the random run should show dramatically more, isolating translation cost from data-fetch cost. **Feed-forward:** §4.4, and the Chapter 6 capstone.

4.8 (a) **TLB miss / page-walk** — huge pages helping is the signature; they raise reach, not data-cache capacity. (b) **Data-cache capacity miss** — 64 MiB exceeds the 24 MiB L3 so it lives in DRAM, while 1 MiB fits in L2; same accesses, different tier (Ch 3 §3.3). (c) **Dependency chain / low ILP** combined with poor locality — pointer-chasing a list is serial and scattered (§3.5); the standout distinguishing feature here is the serial dependency. (d) **Branch misprediction** — sorting makes the data-dependent branch predictable (Ch 2 §2.4). *Method-selection cue:* "huge pages helped" → TLB; "bigger set, re-read" → capacity; "linked list" → dependency/locality; "sorting fixed a branch" → prediction.

4.9 (a) Huge pages help specifically when **TLB misses / page-walk cost are a measured bottleneck** — i.e. a large, scattered working set whose page footprint exceeds TLB reach. (b) Downsides: internal **fragmentation** / wasted memory (a 2 MiB page backing a few live KiB); difficulty **finding contiguous physical memory** once RAM is fragmented (allocation can fail or stall); and no benefit — pure overhead — for workloads that were never TLB-bound. (c) First measurement: the **TLB-miss / page-walk rate** (e.g. `perf stat -e dTLB-load-misses`); if it is low, huge pages will not help. *Discipline:* this is the "measure before you optimize" rule of Part 1 — don't flip huge pages on faith. **Feed-forward:** §4.5 and the Chapter 6 lab.

4.10 (a) **Compulsory**: yes — the first access to a page's translation always misses the TLB (it was never cached), just as the first touch of a line is a cold miss. **Capacity**: yes — when the page footprint exceeds TLB reach, translations are evicted before reuse; this is TLB thrashing (§4.4). **Conflict**: an analogue exists for set-associative TLBs (translations colliding on the same TLB set), though it is secondary and hardware-specific; the dominant effect for large working sets is capacity. (b) Huge pages attack the *capacity* analogue: they do not add TLB entries but make each entry cover far more memory, so the same footprint needs fewer translations and fits within reach. (c) Larger base pages are not a free system-wide win because programs with small, sparse footprints would waste memory to internal fragmentation (rounding every allocation up to a huge page) and the OS would find it harder to place them; the 4 KiB default is a compromise, and huge pages are opt-in for the workloads that benefit. *Tempting wrong answer:* "just make all pages 2 MiB" — ignores fragmentation and allocation cost. **Feed-forward:** §4.5.

4.11 Machine-specific by design. A correct write-up records the real page size (commonly 4 KiB) and cites the architecture's documented page-table depth and huge-page sizes (do not guess — for x86-64, a four- level table with 2 MiB and 1 GiB huge pages); confirms both loops touch the same data the same number of times; and reports two measured ns-per-access figures showing the random-by-page run slower. Where TLB counters are available, the random run should show far more dTLB-load-misses / page-walks than the sequential run, isolating translation cost. The essential discipline: report only measured numbers, attribute the architecture facts to real documentation, and if the compiler elides a loop, consume the result (a `volatile` sink) and say so. **Feed-forward:** §4.4, and the Chapter 6 capstone which measures exactly this latency-bound gap.

Chapter 5 — Solutions

5.1 **Array-of-structs (AoS):** one array whose elements are whole records, so a record's own fields are adjacent in memory. **Struct-of-arrays (SoA):** one array per field, so the same field across different records is contiguous. **SoA** stores "the same field across different records" contiguously.

5.2 (a) **8 of 64 bytes** — one `double` per fetched line. (b) SoA uses all 8 values per line, so AoS moves roughly **8× more memory** for the same field. (c) SoA restores **spatial locality**: consecutive values of the scanned field are adjacent, so each line is full of values the loop uses (§3.1–3.2).

5.3 A hot loop reads only a couple of a record's many fields, so most of every fetched cache line is fields the loop never touches — dead weight that lowers useful-bytes-per-line. (Split the few hot fields into their own compact record; move the rest to a cold side-structure.)

5.4 (a) **AoS** — reads/writes all fields of one particle before moving on, so a record's adjacent fields are all used from one or two fetched lines. (b) **SoA** — summing one column over ten million rows is a stride-1 scan of that field; contiguous storage uses every byte of every line. (c) **SoA** — a SIMD kernel over one field wants that field contiguous so a vector load grabs consecutive values. (d) **AoS** — position, color, and size of one object are used together, so keeping a record's fields adjacent brings them in one fetch. *Method-selection cue:* "one field across records / SIMD one channel" → SoA; "whole record at a time" → AoS.

5.5 (a) The `double` needs 8-byte alignment, so the compiler inserts **padding** after `flag` to place `value` at an 8-byte offset, and adds tail padding so an array keeps every element aligned — pushing the size above 13 bytes (commonly to 24). (b) Order large/strictly-aligned first: `struct S { double value; int id; char flag; };` — `value` at offset 0, `id` at 8, `flag` at 12, then a little tail padding to the struct's alignment; this minimizes internal gaps. (c) A smaller struct means **more records per cache line and per page**, so a scan over ten million of them fetches fewer lines and touches fewer pages — better spatial locality and less TLB pressure, for free, from field ordering alone. *Tempting wrong answer:* "field order doesn't affect layout in C" — it does, through alignment padding. **Feed-forward:** §5.4.

5.6 (a) The scan uses **8 useful bytes of each 64-byte line** (assuming one field per record on a line), a fraction of **8/64 = 1/8** when $S \geq 64$; for smaller S a line may hold more than one record's copies but you still touch one field's worth per record. (b) As S grows past the line size, each record's scanned field sits on its own line, so the AoS scan moves ~S/8 lines' worth where SoA moves 1 — the bandwidth ratio grows with S (bounded by one line fetched per record). (c) The measured speedup can be smaller than the bandwidth ratio because the SoA loop may be limited by something other than bandwidth — e.g. a **single-accumulator dependency chain** (Ch 2 §2.3) that caps how fast it can consume values, so it cannot fully cash in the saved bandwidth. That is exactly why this chapter's demo measured ~4×, not the naive 8×; removing the accumulator ceiling widens it. *No-fabrication note:* 8× is the arithmetic ceiling; ~4× is what was measured — keep them distinct. **Feed-forward:** §5.2 and Chapter 6.

5.7 **Step 4:** Decide empirically. **Measure** where the runtime actually goes — profile both loops (or time them separately) to learn which dominates and by how much; the layout should favor whichever loop owns the most wall-clock time. If loop A (scan) dominates, lean SoA; if loop B (whole-record) dominates, lean AoS. A **hybrid** can serve both partly: hot/cold-split so the field loop A scans lives in its own contiguous array (SoA for that field) while the fields loop B updates together stay grouped — or duplicate/derive the scanned field into a packed side-array refreshed when needed. The posture "pick the layout that helps the hotter loop, then re-measure" is right because the interaction of layout with the prefetcher, the accumulator dependency, and the two loops' real proportions is hard to predict on paper (the demo's ~4×-not-8× is proof), so the paper argument only picks the *candidate* and the measurement decides. This reflects the **measure-before-you-optimize** discipline of Part 1. **Feed-forward:** §5.2–5.3, Chapter 6.

5.8 (a) **Hot/cold splitting** — moving a cold field out raises useful-bytes-per-line for the hot loop. (b) **SoA / whole-line use** — column arrays make the column scan stride-1. (c) **Capacity miss fixed by blocking** — tiling keeps each block resident (Ch 3 §3.3). (d) **TLB / huge-page effect** — 2 MiB pages raise TLB reach for the random index (Ch 4 §4.5). (e) **ILP / multiple accumulators** — independent accumulators break the dependency chain (Ch 2 §2.3). *Method-selection cue:* "cold field moved" → hot/cold; "rows→columns" → SoA; "tiling" → capacity/blocking; "huge pages" → TLB; "more accumulators" → ILP.

5.9 Candidate causes across Chapters 2–5 (at least three, incl. a layout cause and an earlier one): **AoS-when-scanning-one-field / poor useful-bytes-per-line** (§5.2); **cold fields bloating hot records**, fixable by hot/cold splitting (§5.3); **a capacity miss** from a working set exceeding cache (Ch 3 §3.3); **TLB misses** on a large scattered footprint (Ch 4); **a single-accumulator dependency chain / low ILP** (Ch 2 §2.3); a **data-dependent branch** misprediction (Ch 2 §2.4). First measurement: run a **profiler reporting cache-miss rate (and, if available, TLB-miss and branch-miss rates)** — it distinguishes the big mechanism families in one shot, telling you whether to reach for a layout change, a blocking change, or a branch/ILP fix before touching code. *Tempting wrong answer:* "rewrite the layout first" — that guesses; measure first. **Feed-forward:** §5.2 and Chapter 6.

5.10 (a) Both follow from one fact: **a whole cache line moves as a unit**. Single-threaded, you want the bytes you use together on one line (packing) so one fetch serves them all. Multi-threaded, if two cores write different variables on one line, that same all-or-nothing line movement forces the line to bounce between caches on every write (false sharing). Same mechanism, opposite consequence depending on who writes. (b) Deliberately pad a struct up to a full line for **per-thread counters/state written by different threads** — pad so each thread's variable sits on its own line, eliminating false sharing; you trade away memory (and some single-thread locality) to stop the coherence ping-pong. (c) The choice depends on core count because false sharing only occurs when *multiple cores write* the shared line; with one writer, packing is pure win — so "how many cores write this?" decides whether to pack or spread. *Tempting wrong answer:* "packing is always good for cache performance" — true single-threaded, false under concurrent writers. **Feed-forward:** §5.5 and the concurrency Part.

5.11 Machine-specific by design. A correct write-up records the cache-line size, defines a line-sized record, confirms both sums are identical (equal work), reports two measured times and the observed SoA/AoS ratio, and compares it to the book's ~3.7–4× and to the ~8× bandwidth ceiling — expecting a several-fold win that varies with hardware. The second part should show the ratio widening toward 8× once the accumulator dependency is removed (multiple accumulators / vectorization), confirming the demo's explanation for why it measured ~4×. Essential discipline: report only measured numbers, and if a loop is elided, consume the result and say what was done. **Feed-forward:** §5.2, §5.6, and the Chapter 6 capstone.

Chapter 6 — Solutions

6.1 **Latency** is the time to complete *one* access you must wait for; **bandwidth** is the throughput when streaming *many* contiguous items. Use **latency** for (a) fetching one random element (a single wait) and **bandwidth** for (b) streaming a huge array in order (the prefetcher keeps many accesses in flight).

6.2 (a) **Memory-bound** — `memcpy` does no arithmetic per byte; it is pure data movement at memory bandwidth. (b) **Compute-bound** — SHA-256 does much arithmetic per byte and the data is already in L1, so the execution units are the limit. (c) **I/O-bound** — the time is dominated by per-row network/SSD latency, not CPU or memory. *Method-selection cue:* "work per byte" — near zero → memory; high → compute; waiting on storage/network → I/O.

6.3 **Estimate** (predict the class and rough cost from the cost model), **measure** (run it and observe the real time / counters), **reconcile** (explain any gap and update the model). A gap is useful because it usually reveals a wrong assumption — most often using latency where bandwidth applies (or vice versa) — so the discrepancy points directly at the true bottleneck.

6.4 (a) 512 MiB of `long` is 2^{26} = ~67 million elements. Latency estimate: 67M × 100 ns ≈ **~6.7 s**. Bandwidth estimate: 512 MiB / 6 GB/s ≈ **~0.09 s**. (b) The **bandwidth** estimate is right for a sequential sum: the prefetcher streams the data, so accesses overlap rather than each incurring the full latency (this matches the ~0.08–0.09 s measured in §6.3). (c) They differ by roughly **~70–100×** — the cost of using the latency model for a bandwidth-bound workload is being wrong by about two orders of magnitude, exactly the §6.3 surprise. *Tempting wrong answer:* the ~6.7 s latency figure — tempting because ~100 ns is the "known" DRAM number, but it is the wrong number for streaming. **Feed-forward:** §6.3.

6.5 (a) The sum is stride-1, so the hardware prefetcher recognizes the pattern and fetches lines ahead of the loop, keeping many independent accesses in flight; the loop is then limited by how fast memory can *stream* (bandwidth), not by any single access's latency. (b) Even the L2 sum is capped by the **single-accumulator dependency chain** (§2.3): each `sum +=` depends on the previous, so the additions serialize at roughly one add-latency apart, limiting throughput below what the cache could supply. (c) Sum into **several independent accumulators** (or let the compiler vectorize), breaking the dependency chain so more values are consumed per cycle and the loop can approach memory bandwidth. *Method note:* the two limiters — prefetchable bandwidth and the add dependency — are distinct; naming both is the full explanation. **Feed-forward:** §6.3 and Chapter 2 §2.3.

6.6 (a) **compute-time ≈ F / 10^{10} s**; **memory-time ≈ B / 10^{10} s**; the runtime floor is the larger of the two. (b) Compute-bound when compute-time > memory-time, i.e. **F / B > 1** op per byte for these rates (more generally, when work per byte exceeds the machine's ops-per-byte balance point). (c) A sum does 1 add per 8 bytes, so F/B = 1/8 < 1 → **memory-bound** by this test. §6.3 agrees: the sum's speed tracked memory (GB/s), not the add rate — though the single-accumulator dependency kept it from reaching full bandwidth, which is why the measured GB/s was modest. *No-fabrication note:* the 10^{10} rates are round orders of magnitude for estimation, not a spec of any particular chip. **Feed-forward:** §6.2–6.3.

6.7 **Step 4:** ~150 ns/step exceeds the ~100 ns bare DRAM latency because the random chase over 512 MiB also **thrashes the TLB** (Chapter 4): each jump lands on a new page whose translation has been evicted, so most steps pay a **page-table walk** — extra dependent memory accesses to translate the address — *on top of* the data miss. So the ~150 ns is roughly the DRAM data latency plus the translation cost. To confirm the surplus is translation and not data, measure the **dTLB-load-miss / page-walk rate** (e.g. `perf stat -e dTLB-load-misses`) and check it is high for the random run and negligible for the sequential one. The pair teaches the rule: estimate with **bandwidth** for prefetchable sequential access (§6.3's ~1.7×) and with **latency** — plus translation cost — for scattered, dependent access (this ~25×); the access pattern, not just the data's tier, chooses the model. **Feed-forward:** §6.3–6.4, Chapter 4 §4.4.

6.8 (a) **Memory-bound**, memory bandwidth saturated — a single core's sequential scan already uses much of the available bandwidth, so more threads add little (the bottleneck is the memory pipe, not cores). (b) **Compute-bound-ish / control**, branch misprediction — sorting made the data-dependent branch predictable (Ch 2 §2.4), cutting flushes. (c) **Memory-bound**, TLB miss / page-walk — the ~160 ns over ~100 ns and the huge-page fix are the TLB signature (Ch 4). (d) **Compute-bound**, execution throughput — the multiply-add issue rate limits an in-cache matmul (Ch 2). *Method-selection cue:* "adding threads doesn't help a scan" → bandwidth-bound; "sorting fixed it" → branch; "huge pages fixed it" → TLB; "issue rate limited" → compute.

6.9 (a) The 100× is a *latency* ratio, but a sequential scan is prefetchable and therefore **bandwidth-bound**, so latency is the wrong model — the actual slowdown vs. cache is a small factor, not 100× (§6.3). (b) Rough class: **memory-bound (streaming)**; rough slowdown vs. an in-cache version: on the order of a **~2×**, since DRAM streaming bandwidth is within a small factor of cache (and it may be further gated by the file/I-O layer for a file-backed array). (c) Confirm by measuring (i) the scan's **effective GB/s** (is it near memory bandwidth?) and (ii) its **cache-miss / prefetch behavior** (or simply compare to the same scan on an in-cache slice) before optimizing. *Tempting wrong answer:* accept the 100× and build a cache — that optimizes a bottleneck that isn't there. **Feed-forward:** §6.3 and the warning box.

6.10 (a) The **random** ratio is more portable because it is bounded by DRAM *latency* (plus translation), which is set by physics and the memory subsystem and varies modestly across machines; the **sequential** ratio is bounded by the *bandwidth* gap between L2 and DRAM and by loop details (accumulators, vectorization, prefetcher quality), which vary a lot — so the sequential number is the fragile one. (b) Raise the sequential ratio: a stronger prefetcher / higher L2 bandwidth relative to DRAM (widens the streaming gap), or removing the accumulator dependency so the L2 loop speeds up more than the DRAM loop. Lower the random ns/step: faster DRAM / lower memory latency, a larger TLB or huge pages (cuts page-walk cost), or a bigger last-level cache that keeps more of the footprint on-chip. (c) Reporting both ratios with their access patterns is honest because there is no single "DRAM is N× slower" — the answer depends entirely on whether access is streaming or scattered; a lone headline number invites readers to apply it to the wrong pattern and mis-estimate by orders of magnitude, which is exactly the §6.3 trap. *Tempting wrong answer:* "just quote the bigger, scarier ratio" — that mispredicts every bandwidth-bound workload. **Feed-forward:** §6.3–6.4.

6.11 Machine-specific by design, and the write-up matters as much as the numbers. A correct submission (1) records real L2/L3 and cache-line sizes; (2) states a *written-first* prediction with the expected bound (sequential → bandwidth-bound, small ratio; DRAM chase → latency-bound, large ratio); (3) reports measured ns/element and GB/s for both sums (identical sums verified); (4) reports ns/step for both pointer-chases; and (5) reconciles each measurement against the prediction — expecting a modest sequential ratio (bandwidth) and a large random one (latency plus TLB), ideally backed by `perf` cache-miss and dTLB-miss counters showing the random DRAM run dominated by misses and page-walks. The essential discipline: predict before measuring, report only observed numbers, explain surprises with latency-vs-bandwidth and the TLB rather than hand-waving, and if a loop is elided, consume the result and say so. *Tempting wrong answer:* skipping the written prediction and only reporting measurements — you lose the reconciliation that is the whole point. **Feed-forward:** §6.3–6.4 and the performance-engineering Part.

Chapter 7 — Solutions

7.1 (a) A pointer variable stores a **memory address** (a virtual address, per Ch 4). (b) `*p` **dereferences** — goes to the address in `p` and reads or writes the object there, using `p`'s type to know how many bytes and how to interpret them. (c) `&x` produces the **address of** `x` (a pointer to `x`).

7.2 As a `double *`: `q + 4` = 2000 + 4×`sizeof(double)` = 2000 + 4×8 = **2032**. As an `int *`: 2000 + 4×`sizeof(int)` = 2000 + 4×4 = **2016**. The rule: `p + n` advances by `n × sizeof(*p)` bytes, so the byte step is the element size. *Method note:* always multiply the integer by the pointee's `sizeof`, never treat it as a byte offset. **Feed-forward:** §7.2.

7.3 `sizeof(a)` = **40** (10 ints × 4 bytes — the whole array); `sizeof(p)` = **8** (just the pointer, on this x86-64 machine). Difference: `a` names the array object, so `sizeof` measures all its storage; `p` is a separate pointer variable, so `sizeof` measures the pointer, not what it points at. *Tempting wrong answer:* "both are 40 because `p` points at the array" — no, `sizeof` looks at the operand's *type*, and `p`'s type is `int *`. **Feed-forward:** §7.3.

7.4 **Array-to-pointer decay:** in most expression contexts an array name is automatically converted to a pointer to its first element (C §6.3.2.1), which is why `arr` can be assigned to an `int *`, indexed, or passed to a function taking `int *`. In `size_t len(int a[]) { return sizeof(a)/sizeof(a[0]); }` the parameter `a` is *already a pointer* (a function parameter of "array" type is adjusted to pointer type; the array decayed at the call), so `sizeof(a)` is the **pointer size** (8 here), not the array size. The returned value is `8 / sizeof(int)` = **2** on this machine, regardless of the caller's real length. Correct approach: pass the length explicitly. *Tempting wrong answer:* "it returns the element count" — it cannot; the length was lost at the call boundary. **Feed-forward:** §7.3.

7.5 `struct S { char a; short b; char c; int d; };` with align = size (char 1, short 2, int 4): `a` at offset 0 (byte 0); `b` (align 2) needs an even offset, next is 2, so 1 byte padding at 1, `b` occupies 2–3; `c` at offset 4 (byte 4); `d` (align 4) needs a multiple of 4, next after byte 4 is 8, so 3 bytes padding at 5–7, `d` occupies 8–11. Struct alignment = max(1,2,1,4) = 4; running size 12 is already a multiple of 4, so no tail padding. **`sizeof(struct S) = 12`**; real data = 1+2+1+4 = 8, so **4 bytes padding**. Verify with `offsetof(struct S, member)` for each member and `sizeof(struct S)`. *Tempting wrong answer:* "1+2+1+4 = 8" — ignores alignment padding. **Feed-forward:** §7.5.

7.6 Order largest-alignment first: `struct S { int d; short b; char a; char c; };` — `d` at 0 (0–3), `b` at 4 (4–5), `a` at 6, `c` at 7; running size 8, alignment 4, 8 is a multiple of 4, so no tail padding. **`sizeof = 8`** — the size of the real data, zero padding. Rule applied: placing members in decreasing alignment order minimizes the internal gaps needed to re-align each successive member (and packs the small fields into what would otherwise be padding). *Tempting wrong answer:* "field order can't matter in C" — it does, via alignment padding. **Feed-forward:** §7.5, and §5.4.

7.7 (a) 10 M × 24 bytes = **240 MB**. (b) 10 M × 16 = **160 MB**. A 64-byte line holds `64/24` = 2 whole 24-byte records vs. `64/16` = 4 whole 16-byte records — **2 more per line**; a 4 KiB page holds 4096/24 = 170 vs. 4096/16 = 256 — **86 more per page**. (c) The smaller record raises useful-bytes-per-line and records-per-page, so a scan fetches fewer cache lines and touches fewer pages — better spatial locality and less TLB pressure (Ch 3–5), for free, from field ordering. *Tempting wrong answer:* ignoring that a line/page holds a *whole* number of records (partial records straddle boundaries), which is why we floor. **Feed-forward:** §7.5, §5.4.

7.8 **Step 4:** The struct's alignment is max(8,1,4) = **8**. After `b` the running size is 16, which *is* a multiple of 8, so **no tail padding** is needed. Final **`sizeof(struct T) = 16`**; real data = 8+1+4 = 13, so **3 padding bytes** (the ones at offsets 9–11). Confirm each member's offset with `offsetof(struct T, member)` (and `sizeof` for the total). *Method note:* always finish by rounding the size up to the struct's alignment — here it happened to already be aligned. **Feed-forward:** §7.5.

7.9 (a) **Array-to-pointer decay** (Ch 7) — the parameter is a pointer, so `sizeof` is 8; length was lost at the call. (b) **TLB miss / page-walk** (Ch 4) — huge pages helping is the signature; they raise TLB reach. (c) **Struct padding/alignment** (Ch 7/5) — the compiler inserts padding to align members. (d) **Endianness** (Ch 7) — the other machine's byte order differs, so integers appear byte-swapped. *Method-selection cue:* "sizeof of a parameter is 8" → decay; "huge pages helped" → TLB; "bigger than the fields" → padding; "byte-swapped across machines" → endianness. **Feed-forward:** §7.3–7.6.

7.10 (a) A **virtual** address (Ch 4) — the hardware translates it to physical on access. (b) `&arr[5] - &arr[2]` = **3** (an element count of type `ptrdiff_t`; pointer subtraction divides out the element size). (c) **Not stable** — a PIE build is relocated by ASLR each run, so absolute addresses change; the relative layout is stable. (d) **No** — the C standard guarantees only `sizeof(char)==1` and minimum ranges (`int` ≥ 16 bits); 4 bytes is this ABI's implementation-defined value, verified, not guaranteed. *Tempting wrong answer:* on (b), answering "12 bytes" — subtraction yields elements, not bytes. **Feed-forward:** §7.2, §7.4–7.5, and Ch 4.

7.11 (a) For `int arr[10]`, `&arr[10]` is the special "one past the last element" address, which the standard explicitly permits you to *form* and compare against (useful as a loop end), but the object there does not exist, so *dereferencing* it is undefined (C §6.5.6). (b) Making it undefined (rather than "wraps" or "reads adjacent memory") lets the optimizer assume in-bounds access — enabling optimizations and diagnostics — and reflects that beyond the array there may be padding, another object, an unmapped page, or a segment boundary (Ch 4), so no single "reasonable" behavior exists to define. (c) This is exactly the **buffer overflow** bug of Ch 9: an out-of-bounds access is UB, and an out-of-bounds write is the classic memory-corruption vulnerability. *Tempting wrong answer:* "one-past-the-end is illegal to compute" — it is legal to compute and compare, only illegal to dereference. **Feed-forward:** §7.2 and §9.3.

7.12 Machine-specific by design. A correct write-up reports measured `sizeof`/`_Alignof` for the scalar types (on x86-64 LP64: char 1, short 2, int 4, long 8, double 8, pointer 8, alignment = size), a struct whose hand-computed offsets and total `sizeof` match `offsetof`/`sizeof` exactly, and an endianness check (x86-64 prints the low byte first — little-endian). The essential discipline: label the machine/ABI, report only measured numbers, and note that a different platform (16-bit, or big-endian, or a different ABI) would legitimately differ — which is the whole reason to measure rather than assume. *Tempting wrong answer:* hard-coding "int is 4 bytes" as universal — it is implementation-defined. **Feed-forward:** §7.5–7.6.

Chapter 8 — Solutions

8.1 (a) A **stack frame** (activation record) is the block of storage for one function call — its locals, arguments, saved return address, and bookkeeping. (b) Subtracting the frame's size from the **stack-pointer register** (moving it down) allocates it — one arithmetic operation. (c) A local's lifetime ends when execution **leaves its block/function** and the frame is popped.

8.2 Heap reason (any one): the object must **outlive the function that creates it**, its size is only known at runtime, or it is too large for the stack. Cost the heap imposes: the allocator does real work (searching/splitting free lists), and — the big one — **you must free it exactly once**, an obligation the stack does not have (frames free automatically on return). **Feed-forward:** §8.2, §8.5.

8.3 (1) **Runaway/unbounded recursion** (missing base case or too deep for the input) — fix by bounding the recursion or converting to iteration with an explicit (heap-backed) stack. (2) **Very large automatic arrays/locals** — fix by allocating large or runtime-sized buffers on the **heap** instead. *Tempting wrong answer:* "increase the stack limit" — a band-aid that does not fix unbounded recursion. **Feed-forward:** §8.3.

8.4 `int *f(void){ int x=7; return &x; }` returns the address of the local `x`, which lives in `f`'s stack frame. On `return`, that frame is **popped** — the stack pointer moves back — so `x`'s storage is no longer reserved for it and can be reused by the next call. The returned pointer therefore **dangles**, and reading/writing through it is **undefined behavior** (C §6.2.4: using a pointer to an automatic object past its lifetime). gcc emits `warning: function returns address of local variable [-Wreturn-local-addr]`; clang emits `-Wreturn-stack-address`. *Tempting wrong answer:* "it returns 7 because the value is still in memory" — not guaranteed; UB imposes no such promise. **Feed-forward:** §8.4.

8.5 (a) **Caller owns the storage:** `void f(int *out){ *out = 7; }`, called `int x; f(&x);` — `x` lives in the caller's frame and outlives `f`. (b) **Heap:** `int *f(void){ int *p = malloc(sizeof(int)); if(!p) return NULL; *p = 7; return p; }` — the block outlives the call; **the caller must `free` it** exactly once, after its last use (ownership transfers to the caller). *Method note:* the choice is "who owns the memory?" — make it explicit. **Feed-forward:** §8.4–8.5.

8.6 "It worked" is not evidence of correctness because the bug is **undefined behavior**: the C standard imposes no requirements, so the program is free to *appear* to work — the dead frame's bytes may simply still hold the old value until something reuses them. Correctness is a property of the code against the standard, not of one run's observed output. The danger of a bug that appears to work is that it is **latent**: it hides through testing and surfaces later when the optimization level, compiler, or call pattern changes — often in production, and possibly as data corruption rather than an obvious crash. *Tempting wrong answer:* "if it passes tests it's fine" — UB can pass tests and still be a defect. **Feed-forward:** §8.4.

8.7 (a) A stack frame's locals are reserved by **subtracting the frame size from the stack pointer** — a single register adjustment, with no per-variable work and nothing to track. (b) One `malloc` may **search a free list** for a fitting block, **split** it, update bookkeeping, and, if the pool is exhausted, **ask the OS for more memory** — orders of magnitude more work. (c) Rule of thumb: for a small scratch buffer needed each iteration, use a **stack local (or one buffer reused across iterations)**, not a per-iteration `malloc`/`free` — the stack version is nearly free and avoids allocator traffic and fragmentation in the hot loop. *Tempting wrong answer:* "malloc a fresh buffer each iteration for cleanliness" — needless cost in a hot loop. **Feed-forward:** §8.1–8.2.

8.8 **Step 4:** The nodes must be allocated on the **heap** (they outlive the helper and their count is dynamic). The **caller becomes responsible for freeing** every node — exactly once each, after it is done traversing the list (walking and freeing node by node). If that responsibility is forgotten, the result is a **memory leak** (Ch 9 preview): the nodes are never reclaimed. The design principle is **ownership** — decide and document who owns the memory and therefore who frees it. *Method note:* "outlives its creator" + "size known only at runtime" is the standard signal for heap allocation with transferred ownership. **Feed-forward:** §8.5 and §9.1, §9.3.

8.9 (a) **Stack overflow** (Ch 8) — deep recursion only on large inputs exhausts the bounded stack. (b) **Dangling pointer to a local** (Ch 8) — correct in tests, garbage after unrelated code changes the call pattern that reuses the frame; classic UB signature. (c) **TLB miss / page-walk** (Ch 4) — huge pages helping points at translation reach. (d) **Struct padding/alignment** (Ch 7) — the compiler pads to align members. *Method-selection cue:* "deep recursion, large input" → overflow; "works then garbage after unrelated change" → dangling/UB; "huge pages helped" → TLB; "bigger than its fields" → padding. **Feed-forward:** §8.3–8.4.

8.10 (a) **Well-defined** — using `&local` within the same function is fine; the object is alive. (b) **Undefined behavior** — the local's frame is popped on return, so the pointer dangles (§8.4). (c) **Performance/robustness issue tending to undefined behavior** — `int big[8000000]` is ~32 MB, likely overflowing a few-MB stack; if it overflows, the access faults (typically a crash). (d) **Undefined behavior** — storing `&local` in a global and reading it after return is the same dangling-pointer bug as (b), just via a longer-lived stash. *Tempting wrong answer:* calling (d) safe "because a global is permanent" — the global outlives the object, which is exactly the problem. **Feed-forward:** §8.3–8.4.

8.11 (a) Both violate the property that **a pointer must not be used outside its target's lifetime**. (b) The stack version is easier to catch because the direct pattern — returning `&local` — is statically visible, and compilers warn (`-Wreturn-local-addr` / `-Wreturn-stack-address`); the heap version (use-after-free) generally is not statically detectable, since the `free` and the later use can be far apart and flow-dependent, so it usually needs a runtime tool (Ch 9's sanitizers). (c) Both stem from "the storage is reclaimed whether or not a pointer still refers to it" — the frame pop (stack) and `free` (heap) both end the object's lifetime without invalidating the pointers that name it, leaving a dangling handle. *Tempting wrong answer:* "they're unrelated bugs" — they are the same lifetime violation in different regions. **Feed-forward:** §8.4 and §9.3.

8.12 Machine-specific by design. A correct write-up records the exact warning (on gcc, `-Wreturn-local-addr`; on clang, `-Wreturn-stack-address`); reports what the run actually did on that machine (on the book's machine the plain build **segfaulted**, but the write-up must note this is *not* guaranteed by the standard — a different build could print a value or garbage); and shows the caller-owned fix (`void f(int *out){ *out = 7; }`) compiling warning-free and behaving correctly. Essential discipline: never rely on the observed (b) outcome — UB is not obligated to crash — and reason about lifetime instead. *Tempting wrong answer:* "it segfaults, so it's at least safe to detect" — the crash is not reliable. **Feed-forward:** §8.4, and Ch 9's tools.

Chapter 9 — Solutions

9.1 (a) **No** — `malloc` returns uninitialized (indeterminate) bytes. (b) **Yes** — `calloc` returns all-zero bytes. (c) **Yes** — `free(NULL)` is an explicit no-op, always safe.

9.2 Safe idiom: `void *tmp = realloc(p, n); if (tmp) p = tmp; else { /* handle failure; p still valid */ }`. With `p = realloc(p, n);`, if `realloc` fails it returns `NULL` and leaves the original block *valid but unreferenced* — you have overwritten your only pointer to it, so it is **leaked** (and you have lost the data). *Tempting wrong answer:* "assign directly, it's cleaner" — cleaner and leaky. **Feed-forward:** §9.1.

9.3 **Use-after-free:** accessing a block after `free` — *UB*. **Double-free:** freeing the same block twice — *UB*. **Memory leak:** losing the last pointer without freeing — *not UB* (a defined defect). **Buffer overflow:** accessing outside an allocation's bounds — *UB*. **Uninitialized read:** using memory before writing it — *indeterminate value; unreliable, UB in general*. Four of five are undefined behavior; the leak is the exception. **Feed-forward:** §9.3.

9.4 "Returns the old value, so it's harmless" is wrong because use-after-free is **undefined behavior** (C §3.4.3): the standard imposes *no requirements*, so there is no guaranteed value and no guarantee it is harmless. Two realistic other outcomes: (1) the freed slot has been handed to another allocation, so you read (or clobber) **unrelated live data** — a correctness and security hole; (2) the access **corrupts allocator metadata or crashes**, possibly far from the buggy line. It can also differ by build or run, and be exploitable. *Tempting wrong answer:* the premise itself — treating UB as having a defined outcome. **Feed-forward:** §9.3.

9.5 (a) **Use-after-free** — reading `p[0]` after `free(p)`; UB. (b) **Buffer overflow** — `p[4]` on a 4-element (indices 0–3) block; UB. (c) **Double-free** — freeing the same block twice; UB. (d) **Uninitialized read** — reading a `malloc`'d `int` before writing it; indeterminate value, unreliable (UB in general). (e) **Memory leak** — overwriting the only pointer to a 1024-byte block without freeing; a defect, but *not* UB. *Method-selection cue:* "touched after free" → UAF; "index past the end" → overflow; "freed twice" → double-free; "read before write" → uninitialized; "pointer lost, not freed" → leak. **Feed-forward:** §9.3.

9.6 A **leak** frees *too late* — never; the block stays allocated but unreachable, so memory is wasted but no invalid access occurs, hence **not undefined behavior** (the program stays well-defined, just wasteful). A **use-after-free** frees *too early* — while a pointer still needs the block; the later access touches storage whose lifetime has ended, which **is undefined behavior**. The asymmetry: accessing outside a lifetime is undefined; merely retaining allocated memory forever is defined (if undesirable). *Tempting wrong answer:* "both are UB because both are memory bugs" — a leak is a defect but not UB. **Feed-forward:** §9.3.

9.7 (a) A heap allocation may search a free list, split a block, update bookkeeping, and sometimes call the OS for more memory — real work — whereas a stack allocation is a single stack-pointer adjustment (Ch 8). (b) **External fragmentation** is free space chopped into many small, non-adjacent pieces by interleaved allocate/free of varying sizes; a large request can fail because *no single free block* is big enough, even though the *total* free memory exceeds the request. (c) **Not undefined behavior** — fragmentation is a *performance/capacity* problem, not a language-rule violation. *Tempting wrong answer:* "fragmentation corrupts memory" — it does not; it just wastes/scatters it. **Feed-forward:** §9.2.

9.8 **Step 4:** Correct allocation size: `malloc(strlen(s) + 1)` — room for the string *and* its terminating `'\0'`. Missing check: after `malloc`, verify it did not return `NULL` before writing (`if (!r) return NULL;`). Corrected function: `char *dup(const char *s){ char *r = malloc(strlen(s)+1); if(!r) return NULL; strcpy(r,s); return r; }`. The tool from §9.4 that catches the original one-byte overflow is **AddressSanitizer** (`-fsanitize=address`), which would report a `heap-buffer-overflow` on the `strcpy` line (Valgrind would too). *Tempting wrong answer:* "`strlen` already counts the null terminator" — it does not; it counts the characters before it. **Feed-forward:** §9.3–9.4.

9.9 (a) **Leak** — steadily growing memory until the OOM killer acts is the leak signature. (b) **Uninitialized read** — a different value each run, only in release builds, is the classic "garbage bytes" symptom (build-dependent). (c) **Buffer overflow** — an out-of-bounds write corrupting an adjacent variable. (d) **Use-after-free / double-free** — crashing inside `free` with heap corruption after releasing the same pointer twice is a double-free. (e) **Non-memory performance issue** (a TLB/huge-page effect, Ch 4) — *not* a memory *bug*; the trap in this item. *Method-selection cue:* "unbounded growth" → leak; "differs per run/release" → uninitialized; "adjacent corruption" → overflow; "crash in free after two frees" → double-free; "huge pages help" → TLB, not a bug. **Feed-forward:** §9.3, Ch 4.

9.10 (a) **Well-defined and fine** — `free(NULL)` is a guaranteed no-op. (b) **A defined-but-real defect** — a memory leak; wasteful but not UB. (c) **Undefined behavior** — `a[n]` on an `n`-element array is one past the last valid index (valid indices 0..n-1); dereferencing it is out of bounds. (d) **Well-defined and fine** — a `calloc`'d block is zeroed, so reading it yields 0, a defined value. (e) **Reads an indeterminate value (at best unspecified, in general undefined behavior)** — a `malloc`'d block is uninitialized, so reading before writing uses an *indeterminate value*: on this ABI an `int` has no trap representation, so you get an unspecified (garbage) value rather than a crash, but the standard permits it to be full UB in general (matching §9.3), so never rely on it. *Tempting wrong answer:* conflating (d) and (e) — `calloc` zeroes, `malloc` does not. **Feed-forward:** §9.1, §9.3.

9.11 (a) The three obligations: use a pointer only within its target's **lifetime**; access only within the allocation's **bounds**; and **free each block exactly once** (not zero, not twice). (b) **Use-after-free** violates *lifetime*; **double-free** violates *free-exactly-once*; a **leak** also violates *free-exactly-once* (frees zero times) — while use-after-free additionally implies the free happened too early. (c) A compiler that checks these before the program runs would **reject**, at compile time, any program that could use a pointer past its target's lifetime, index out of bounds, or mismanage a free — turning a whole class of runtime UB into type errors. That is more reliable than a runtime sanitizer because a sanitizer only catches a bug on *inputs/paths you actually execute* during testing, whereas a static guarantee holds for *all* executions — which is exactly Rust's ownership/borrowing promise. *Tempting wrong answer:* "a sanitizer is just as good" — it is path-dependent, not a proof. **Feed-forward:** §9.5 and Part 5 (Rust).

9.12 Machine-specific by design. A correct write-up reports, for each of the four bugs, ASan's exact error type and the source line — on the book's machine (gcc 11, x86-64): `heap-use-after-free` (a READ on the line reading the freed block), `heap-buffer-overflow` (a WRITE "0 bytes to the right of" the allocated region), `attempting double-free`, and (via LeakSanitizer) `Direct leak of N byte(s)` pinned to the allocation line. The fifth program (uninitialized read) should show that ASan does **not** flag it — reach for Valgrind or MemorySanitizer, or catch the simple case with `gcc -Wall` (`-Wuninitialized`). Essential discipline: label compiler/tool versions, report only observed output (addresses and wording vary), and note that ASan detects addressability bugs, not uninitialized-value bugs. *Tempting wrong answer:* "ASan catches everything" — it misses uninitialized reads. **Feed-forward:** §9.4.

Chapter 10 — Solutions

10.1 Undefined behavior: the standard imposes *no requirements* (C §3.4.3) — the program has no meaning on the triggering input. **Unspecified behavior:** the standard offers two or more valid possibilities and does not say which you get (e.g. argument evaluation order). **Implementation- defined:** unspecified but the implementation must *document* its choice (e.g. `sizeof(int)`). Only **undefined** behavior grants the optimizer license to assume the situation never happens. **Feed-forward:** §10.1.

10.2 Unsigned integer overflow is *not* UB — it is defined to wrap modulo 2^n. **Signed** integer overflow *is* undefined behavior. The defined case (unsigned) yields the mathematically correct value reduced modulo 2^n (i.e. the low *n* bits). *Tempting wrong answer:* "both wrap, it's two's complement" — the hardware may wrap, but the *language* leaves signed overflow undefined, which is what the optimizer exploits. **Feed-forward:** §10.3.

10.3 Most likely explanation: the program contains **undefined behavior** that was harmless-looking at `-O0` but that the optimizer exploited at `-O2` (it assumed the UB never happens and transformed the code). The two flags to hunt it: `-fsanitize=undefined` (UBSan) and `-fsanitize=address` (ASan), ideally with `-Wall -Wextra`. *Tempting wrong answer:* "the optimizer has a bug" — far more often the UB was already there. **Feed-forward:** §10.2, §10.4.

10.4 Signed overflow is UB, so the compiler may assume `x + 1` never overflows; if it never overflows, `x + 1 > x` is always true, so the function folds to `return 1` (on this machine, `movl $1, %eax; ret`). What breaks: a programmer using `if (x + 1 < x)` to detect overflow relies on the overflow *happening* and wrapping — but that is UB, so the compiler assumes it cannot happen and deletes the check. Correct detection: compile with `-fwrapv` (defines signed overflow as wraparound), or do the arithmetic in an unsigned type, or use `__builtin_add_overflow`, which reports overflow without invoking UB. *Tempting wrong answer:* "test after the add" — testing *after* signed overflow is testing after UB. **Feed-forward:** §10.3–10.4.

10.5 (a) The UB is a **null-pointer dereference**: `t->sk` dereferences `t` before it is checked. (b) Because dereferencing null is UB, the compiler may assume `t` was non-null at the dereference, so `if (!t)` is provably false and is deleted as dead code. (c) Fix: move the check before the dereference — `if (!t) return -1; int *sk = t->sk; return *sk;`. (d) The flag that preserves the check without moving code: `-fno-delete-null-pointer-checks`. The pattern is the real Linux kernel bug **CVE-2009-1897** (`tun_chr_poll`). *Tempting wrong answer:* "the check is there, so it's fine" — it is there in the source, not in the binary. **Feed-forward:** §10.3.

10.6 The compiler is behaving correctly under the **as-if rule**: it must preserve the observable behavior of a *well-defined* program, but a program that dereferences a possibly-null pointer has undefined behavior on the null input, so there is no observable behavior it must preserve there. Given the dereference `t->sk`, the compiler infers `t != NULL` and legally removes the redundant `if (!t)`. The code, not the compiler, is wrong: the check comes after the dereference it is meant to guard. *Tempting wrong answer:* "the source clearly has the check, so the compiler dropped correct code" — to the optimizer, a check after a dereference is a statement the pointer was already non-null. **Feed-forward:** §10.2.

10.7 (a) Mandating checks would insert a bounds check on every array access, an overflow check on every signed add, and a null check on every dereference — runtime cost on the hot path of essentially every program. (b) One optimization signed-overflow UB enables: treating a loop counter as unable to wrap, so the compiler can prove loop termination, promote the counter to a wider type, or strength-reduce — none guaranteed if overflow were defined to wrap. (c) The bargain: the programmer gives up "defined behavior in the bad case" (and takes on the duty never to trigger UB); the compiler gains the freedom to generate fast code across all hardware without pervasive safety checks. *Tempting wrong answer:* "UB is just a mistake in the standard" — it is a deliberate performance/portability choice. **Feed-forward:** §10.4.

10.8 **Step 4:** Fix: the bounds check must come *before* the access — check `if (i < 0 || i >= n) return;` (or handle the error) *then* touch `a[i]`, including for logging. A sanitizer that would have flagged the original out-of-bounds access at runtime: **AddressSanitizer** (`-fsanitize=address`) for a heap/stack array, or UBSan's bounds check for a known-size array. Moving the log line is not "just reordering output": once `a[i]` executes, the compiler may *prove* `0 <= i < n` (else UB) and delete the later check — so the position of the access changes what the compiler is allowed to assume about `i`, not merely when a byte is printed. *Tempting wrong answer:* "it's just a logging line, order doesn't matter" — the access is what licenses the deletion. **Feed-forward:** §10.2–10.3.

10.9 (a) **Implementation-defined** — `sizeof(int)` varies but must be documented. (b) **Unspecified** — the order of evaluating `a()` and `b()` is one of two valid, undocumented choices. (c) **Undefined** — `INT_MAX + 1` is signed overflow. (d) **Implementation-defined** — right-shift of a negative signed integer is implementation-defined (the standard lets the implementation choose arithmetic vs. logical shift and document it). (e) **Undefined behavior** (in general) — reading a `malloc`'d block before writing it uses an indeterminate value (the §9.3 memory bug). *Method-selection cue:* "varies but documented" → impl-defined; "two valid orders" → unspecified; "no meaning at all" → undefined. **Feed-forward:** §10.1, §9.3.

10.10 Least to most effective: (a) `-O0` — *hides* the bug (the UB remains; a future build or a different path re-exposes it); (b) `volatile` — also *hides* it (it defeats some optimizer assumptions but does not remove the UB, and is fragile and slow); (c) `-fsanitize=undefined,address` plus fixing what it reports — *removes* the bug by finding and eliminating the actual UB. So the order is (a) < (b) < (c), and only (c) removes rather than hides. *Tempting wrong answer:* "compile at `-O0` in production to be safe" — you keep the bug and lose performance. **Feed-forward:** §10.4.

10.11 (a) The standard says a program that executes UB has undefined behavior for that *execution* — not just from the buggy line onward — so the compiler need not preserve observable effects that occur before the UB either; it may hoist, sink, or delete surrounding work ("UB can time-travel," in Regehr's phrasing). (b) Concrete scenario: an upstream `t->sk` lets the compiler delete a downstream `if (!t) return ERR;`; a caller that would have safely returned an error on a null `t` now falls through into code that trusts `t`, and an attacker who maps memory at address 0 reaches it — a would-be crash becomes a silent privilege escalation (the CVE-2009-1897 shape). (c) A static guarantee is stronger because a runtime sanitizer only catches UB on the *inputs and paths you actually execute* during testing, whereas "the safe subset has no UB" holds for *all* executions — a proof, not a spot check. *Tempting wrong answer:* "sanitizers make UB safe" — they detect it on exercised paths only. **Feed-forward:** §10.2, §10.5, Part 5.

10.12 Machine-specific by design. A correct write-up reports: at `-O2`, `x + 1 > x` folds to `return 1` (on the book's machine, `movl $1, %eax; ret`), and adding `-fwrapv` makes the addition and comparison reappear; for `poll`, the default `-O2` omits the `testq %rdi,%rdi` null check while `-fno-delete-null-pointer-checks` restores it. The signed-overflow program should be flagged by `-fsanitize=undefined` as a runtime error (`signed integer overflow`). Essential discipline: report only observed assembly/output and label the compiler/version (gcc 11.4.0, x86-64 here), since exact instructions and wording vary. *Tempting wrong answer:* "the assembly will be the same as mine" — it depends on compiler and version. **Feed-forward:** §10.3–10.4.

Chapter 11 — Solutions

11.1 (a) A single **null byte** `'\0'` (value 0) marks the end. (b) `strlen` is O(n) because the length is *not* stored — it must scan byte by byte until it finds the terminator. (c) The function learns **nothing** about capacity: the array decays to a bare `char *`, an address with no length attached. **Feed-forward:** §11.1.

11.2 `gets` — it read a line with no bound at all and was **removed from the language in C11** (deprecated in C99). Use `fgets(s, size, stdin)`, which takes the buffer size. *Tempting wrong answer:* "`scanf("%s")` is fine" — unbounded `%s` has the same overflow problem; use a width, e.g. `%63s`. **Feed-forward:** §11.3.

11.3 (a) **False** — `strncpy` does not terminate when the source is at least `n` bytes long; it leaves the destination unterminated. (b) **False** — a canary *detects* a linear stack overflow and aborts; the bug (the out-of-bounds write) is still there. (c) **True** — `snprintf` always null-terminates when its size argument is nonzero. **Feed-forward:** §11.2–11.4.

11.4 On a downward-growing stack, a local array sits *below* the saved frame pointer and saved **return address**, while a copy into the array writes toward *higher* addresses — so an overrun climbs over the frame pointer onto the return address. Overwriting it means that when the function executes `ret`, the CPU jumps to the attacker-supplied value: control-flow hijack. An out-of-bounds *read* (usually) cannot redirect execution — it leaks adjacent bytes but does not write control data. *Tempting wrong answer:* "reads are as dangerous as writes" — reads leak (serious, e.g. Heartbleed) but do not seize control. **Feed-forward:** §11.2.

11.5 (a) **Overflows** whenever `user_input` is longer than `buf` — no destination bound; rewrite `snprintf(buf, sizeof buf, "%s", user_input);`. (b) **Overflows** if the formatted result exceeds `buf` — `sprintf` is unbounded; rewrite `snprintf(buf, sizeof buf, "%s-%s", a, b);`. (c) **Over-reads** — `strncpy(d, "abcd", 4)` fills all 4 bytes with no terminator, so `printf("%s", d)` runs off the end; fix by terminating (`d[3] = '\0';`, losing a char) or, better, `snprintf(d, sizeof d, "%s", "abcd")`. *Method-selection cue:* "no size argument" → can overflow; "size argument but no guaranteed terminator" → can over-read. **Feed-forward:** §11.2–11.3.

11.6 Three reasons a canary + ASLR do not "handle" overflows: (1) a **canary only catches a linear stack overflow and only at return** — it does nothing for heap overflows or overwrites of non-control data, and it acts after the corruption (turning a hijack into a crash/DoS); (2) **ASLR only randomizes addresses** — it raises the difficulty of a working exploit but is defeated by info-leaks or techniques that do not need fixed addresses; (3) both are **mitigations, not fixes** — the out-of-bounds write is still undefined behavior, still present. The actual fix: remove the unbounded writes (use `snprintf`/bounded copies) so the overflow cannot occur. *Tempting wrong answer:* "mitigations on = bug fixed" — they contain blast radius, they do not remove the bug. **Feed-forward:** §11.4.

11.7 (a) `strlen` is O(n) because a C string stores no length — it scans to the terminator — whereas a length-carrying string stores the count, so reading the length is O(1). (b) Stack canaries add a random-value write on entry and a compare on return per protected frame; `_FORTIFY_SOURCE` adds size-checked variants of the string/memory calls — small overhead in exchange for turning many overflows into detected aborts or compile errors. (c) Not free: the canary compare is on the return path of every protected function, a real (if usually small) cost. Trade-off: **safety vs. speed** — you pay a little runtime to detect a class of memory corruption. *Tempting wrong answer:* "canaries are free" — they add work on every protected return. **Feed-forward:** §11.1, §11.4.

11.8 **Step 4:** Rewrite with a single bounded format: `void greet(const char *name){ char msg[16]; snprintf(msg, sizeof msg, "Hello, %s", name); puts(msg); }`. `snprintf` writes at most `sizeof msg` bytes *including* the terminator and always null-terminates (when the size is nonzero), so no `name`, however long, can overflow `msg` — it truncates instead; `strcat` guarantees neither a bound nor (relative to the destination capacity) safety. A §11.4 defense that turns the original overflow into a crash rather than a silent hijack: a **stack canary** (`-fstack-protector`), which aborts with "stack smashing detected" instead of returning through a corrupted return address (`_FORTIFY_SOURCE` would also catch it, at compile or run time). *Tempting wrong answer:* "use `strncat` with `sizeof msg`" — `strncat`'s bound is the bytes to *append*, not the buffer size, and it is easy to miscompute. **Feed-forward:** §11.3–11.4.

11.9 (a) **Buffer overflow (over-write)** is possible, but "garbage appended *after* the expected text" most cleanly matches an **unterminated string (over-read)** — the print continues past the intended end until it hits a stray zero. (b) **A mitigation firing** — `*** stack smashing detected ***` is the stack canary aborting on a detected overflow. (c) **Use-after-free** — reading a freed block that was reallocated to another request (Ch 9). (d) **Unterminated string (over- read)** — `printf("%s")` reading far past the data and leaking adjacent memory (Heartbleed class). *Method-selection cue:* "garbage after expected text / reads too far" → missing terminator; "stack smashing detected" → canary; "freed block reused" → UAF. **Feed-forward:** §11.1–11.4, §9.3.

11.10 (a) **Undefined behavior** — `strncpy` with `strlen(s) >= sizeof d` leaves `d` unterminated, so reading it as a string over-reads out of bounds. (b) **Well-defined and fine** — `snprintf` always terminates, so reading `d` is safe (possibly truncated, but valid). (c) **Undefined behavior** — a one-byte heap overflow is an out-of-bounds write. (d) **Well-defined** (a defined defensive action) — a canary deliberately aborting the process on a detected overflow is defined behavior, not UB (it is the mitigation working). *Tempting wrong answer:* conflating (a) and (b) — the difference is the guaranteed terminator. **Feed-forward:** §11.2–11.4.

11.11 (a) A **stack canary** defeats a linear overflow that reaches the return address (detected at return); **NX/DEP** marks the stack/heap non-executable, defeating "inject shellcode and jump to it"; **ASLR** randomizes segment base addresses, defeating attacks that need to predict them. **Return-oriented programming (ROP)** was developed to route around NX by chaining fragments of existing executable code ("gadgets") instead of injecting new code. (b) Layering is worthwhile because each mitigation raises the cost and reliability barrier for a different technique; an attacker must defeat all of them, and many real exploits fail against the stack. (c) Rust makes the bug "not happen" by carrying the length with every slice/string and bounds-checking indexing: an out-of-bounds access is a defined **panic**, not undefined memory corruption — categorically different because a panic has a defined, contained outcome (the program stops safely) whereas UB has no defined outcome and can corrupt control flow. *Tempting wrong answer:* "a panic is just a crash, same as UB" — a panic is defined and cannot be leveraged into memory corruption. **Feed-forward:** §11.4–11.5, Part 5.

11.12 Machine-specific by design. A correct write-up reports: (a) with `-fstack-protector-all` the 8-byte overflow aborts with `*** stack smashing detected ***` (on the book's machine, exit status 134, SIGABRT); (b) with `-D_FORTIFY_SOURCE=2 -O2` a compile-time diagnostic like `__builtin___strcpy_chk / __memcpy_chk writing N bytes into a region of size M` (and gcc's `-Wstringop-overflow` may also fire); (c) `strncpy(d, "abcd", 4)` leaves the four bytes `97 98 99 100` with *no* terminating zero. Essential discipline: report only observed output and label compiler/version (gcc 11.4.0, x86-64 here), since exact messages and status vary. *Tempting wrong answer:* "the canary means the overflow was harmless" — it converted a hijack into a crash; the bug is the same. **Feed-forward:** §11.2–11.4.

Chapter 12 — Solutions

12.1 (a) An arena exploits **shared lifetime** — many objects that die together. (b) A pool exploits **shared size** — many objects of one fixed size. (c) The **pool** can free an individual object (push its slot back on the free list); the arena cannot. **Feed-forward:** §12.1–12.3.

12.2 `arena_alloc` is **O(1)** (round up the offset, bump it, return the address); `arena_reset` is **O(1)** (set the offset to 0). The one thing an arena cannot do that `malloc` can: **free an individual object** — it frees only the whole region at once. *Tempting wrong answer:* "reset is O(n) because it frees n objects" — it frees them all with a single offset assignment, touching none of them. **Feed-forward:** §12.2.

12.3 The "next free slot" pointer is stored **inside the free slot itself** (its first bytes). It costs no extra memory because a free slot holds no live object — its space is unused — so the free list is threaded through the slots rather than kept in a separate structure. *Tempting wrong answer:* "you need a separate array of next-pointers" — unnecessary; the free slots' own bytes hold them. **Feed-forward:** §12.3.

12.4 `typedef struct { unsigned char *base; size_t cap, off; } Arena;` with `void *arena_alloc(Arena *a, size_t n){ size_t p = (a->off + 15) & ~(size_t)15; if (p + n > a->cap) return NULL; a->off = p + n; return a->base + p; }` and `void arena_reset(Arena *a){ a->off = 0; }`. `arena_alloc` is O(1) because it does a fixed amount of work — align, bounds-check, bump — with no search or coalescing. `arena_reset` frees everything without touching individual objects because "free" here just means "the offset no longer points past them," so resetting the offset makes all of that space available again in one step. *Tempting wrong answer:* forgetting the alignment round-up — unaligned returns are UB for some types (Ch 7). **Feed-forward:** §12.2.

12.5 `void *pool_alloc(Pool *p){ if (!p->free_list) return NULL; void *slot = p->free_list; p->free_list = *(void **)slot; return slot; }` and `void pool_free(Pool *p, void *slot) { *(void **)slot = p->free_list; p->free_list = slot; }`. Both are O(1): allocation pops the free-list head and freeing pushes onto it — a couple of pointer writes, no search. There is no external fragmentation within the size class because every slot is identical and interchangeable, so any free slot satisfies any request — free space is never "too scattered to fit." *Tempting wrong answer:* "you still need to coalesce" — with one fixed size there is nothing to coalesce. **Feed-forward:** §12.3.

12.6 Reason 1 (inflexibility): an arena cannot free individual objects, so any object that outlives its batch pins the *whole* region until reset — used "everywhere," including for long-lived mixed-lifetime state, it can inflate memory badly. Reason 2 (evidence): Berger, Zorn & McKinley (OOPSLA 2002) found a good general- purpose allocator matched or beat hand-written custom allocators for most programs; the arena/region case was the exception, not the rule. Correct procedure: **profile first** to confirm allocation is the bottleneck, then apply an arena only where the shared-lifetime pattern fits (e.g. per-request scratch), keeping `malloc` elsewhere. *Tempting wrong answer:* "arenas are always faster, so use them everywhere" — faster only for the right pattern, and costly in memory otherwise. **Feed-forward:** §12.1, §12.4.

12.7 (a) The arena skips, per allocation: the **free-list search** (it just bumps an offset) and the **per-block header / coalescing bookkeeping** (it keeps none) — both of which the general allocator does (Ch 9). (b) The ~4–5× measurement used "N objects live at once, then freed": the arena frees all N with one reset while `malloc` pays N individual `free` calls, so the comparison rewards the arena's bulk free. A pattern that narrows or erases the gap: freeing each object immediately after allocating it, where glibc's `malloc` reuses one hot block and the per-object cost nearly vanishes (as the chapter's first, discarded measurement showed). (c) Memory risk: because it never frees individual objects, an arena holding a long-lived object pins the entire region — it bites when object lifetimes are mixed rather than shared. *Tempting wrong answer:* "the arena is always ~5× faster" — only for its pattern. **Feed-forward:** §12.2, §12.4.

12.8 **Step 4:** The bug is a **use-after-free** (a use-after-reset): the cached pointer dangles into arena storage whose lifetime `arena_reset` ended, and the next request reuses those bytes under it. It **is undefined behavior** (Ch 10). ASan/Valgrind will likely *not* catch it by default because they instrument `malloc/free`, not *your* arena's reset — to them the arena's backing block is still one big live allocation, so a read into it looks legal (you would have to add ASan's manual poisoning hooks for the tool to see the reset). Fix: do not let the cache point into the arena — **copy** the string into memory with a lifetime that matches the cache (e.g. `strdup/malloc` owned by the cache), so nothing the cache holds lives in per-request arena storage. *Tempting wrong answer:* "the sanitizer would have caught it" — not for a custom allocator's reset, by default. **Feed-forward:** §12.4–12.5, §9.3–9.4.

12.9 (a) **Arena** — many small objects sharing one lifetime (the request), freed together at response time. (b) **Pool** — constant create/destroy of same-size nodes throughout the run; O(1) both ways, no fragmentation, individual frees. (c) **General `malloc`** — a library cannot predict its callers' patterns, so the general allocator is the safe default. (d) **Arena** — a burst of AST nodes freed together when the pass ends (the classic region/compiler-pass case Berger et al. highlighted). *Method-selection cue:* "share a lifetime, die together" → arena; "same size, repeated alloc/free" → pool; "unknown pattern" → general. **Feed-forward:** §12.1, §12.4.

12.10 (a) **Undefined behavior** — reading through a pointer into an arena after `arena_reset` is a use-after-reset (a use-after-free). (b) **A defined-but-real defect** — a long-lived object pinning the whole region wastes memory but is not a language-rule violation (analogous to a leak: defined, undesirable). (c) **Undefined behavior** — `pool_free`-ing the same slot twice corrupts the free list (a double-free in your allocator). (d) **Well-defined and fine** — resetting an arena with no live pointers into it is exactly how it is meant to be used. *Tempting wrong answer:* calling (b) UB — over-retaining memory is a defect, not undefined behavior. **Feed-forward:** §12.2–12.5, §9.3.

12.11 (a) In C the programmer must guarantee, by hand, that no pointer is used outside its target's lifetime, no access falls outside its bounds, and every block is freed exactly once — with no undefined behavior triggered. (b) A custom allocator *relocates* the obligation: it does not free individual objects for you (arena) or it recycles slots you must not double-free (pool), so *you* — the allocator's author and its caller — now enforce "free exactly once" and "no use after the lifetime ends" for arena/pool memory, and the system allocator's tooling no longer helps. (c) A language that ties an allocation's lifetime to a scope (Rust's ownership/borrowing) can make "use after reset" a **compile-time error** — the borrow checker refuses to let a reference outlive the region it points into — which is stronger than a runtime sanitizer because it holds for all executions and needs no test input to trigger, and because the sanitizer does not even understand your custom allocator's reset. *Tempting wrong answer:* "custom allocators remove the obligation" — they move it onto you. **Feed-forward:** §12.5, Part 5.

12.12 Machine-specific by design. A correct write-up reports: (a) three allocations return independent, 16-byte- aligned pointers with the expected values, and `arena_reset` sets the offset back to 0; (b) for large *N* with all objects live then freed, the arena (one bulk free) beats `malloc` (N frees) by a several-fold ratio — on the book's machine roughly 4–5×, varying run-to-run; (c) when each object is freed immediately after allocation, the arena's advantage shrinks or disappears, because the general allocator reuses one hot block and per-object cost nearly vanishes — showing the win depends entirely on the lifetime pattern. Essential discipline: report only measured numbers and label compiler/hardware (gcc 11.4.0, x86-64 here), since figures vary. *Tempting wrong answer:* "the arena is faster in every test" — not for the free-immediately pattern. **Feed-forward:** §12.2, §12.4.

Chapter 13 — Solutions

13.1 The two modes are **user mode** (ring 3, where your code runs) and **kernel mode** (ring 0, where the OS runs). A user-mode program may not, for example, execute privileged instructions such as halting the CPU, disabling interrupts, loading the page tables (Ch 4), or accessing device registers directly. *Tempting wrong answer:* "user mode is just slower than kernel mode" — the difference is authority, not speed; the same instruction runs at the same rate, but privileged ones simply trap in user mode. **Feed-forward:** §13.1.

13.2 On x86-64 Linux the system-call number goes in **`rax`**, and the return value comes back in **`rax`** as well (a negative small value being `-errno`). *Tempting wrong answer:* "the fourth argument goes in `rcx`" — it goes in `r10`, because the `syscall` instruction clobbers `rcx` (and `r11`). **Feed-forward:** §13.2.

13.3 No — `printf` is **not** a system call; it is a C library function that parses the format string, converts values to text, and **buffers** the result in user-space memory. It eventually produces output through the **`write`** system call, but only when the buffer flushes (fills, hits a newline for a line-buffered terminal, or the program exits normally). *Tempting wrong answer:* "every output function is a system call" — most output is buffered in user space precisely to avoid crossing the boundary per call. **Feed-forward:** §13.3.

13.4 The C library's `write` wrapper puts the `write` system-call number in `rax`, the file descriptor `1` in `rdi`, the buffer's address in `rsi`, and the length `5` in `rdx`; it executes the `syscall` instruction, which switches to kernel mode and jumps to the fixed kernel entry point; the kernel validates the arguments, does the I/O, and returns the number of bytes written (5 on success) in `rax`. *Tempting wrong answer:* "the program jumps directly into the kernel function" — it cannot choose the target; the trap lands only at the kernel's fixed entry point. **Feed-forward:** §13.2.

13.5 The raw kernel interface signals failure by returning a **negative error code** (e.g. `-EBADF`) in `rax`. The C library wrapper detects that, stores the code in the global **`errno`**, and returns **`-1`** to the caller. So a raw `-EBADF` becomes: the wrapper returns `-1` and sets `errno == EBADF`. *Tempting wrong answer:* "`errno` is set by the kernel" — the kernel returns a negative number; the library *translates* it into `errno` and `-1`. **Feed-forward:** §13.3.

13.6 The thousands of `read(3, ..., 1) = 1` lines mean the program is reading the file **one byte per system call** — one boundary crossing per byte, which is the most expensive way possible. The fix is to **read in large blocks** (a big buffer, or buffered I/O such as `fread`/a `FILE*`), turning thousands of crossings into a handful. *Tempting wrong answer:* "nothing is wrong; it's reading the file" — it is, but at ~100× the necessary cost. **Feed-forward:** §13.4.

13.7 (a) A system call must **switch privilege mode** (save/restore state, enter and leave the kernel) and it **disturbs caches/TLB** and runs argument validation — none of which a plain function call does. (b) Reading 1 MB one byte at a time is ~1,000,000 boundary crossings, and each costs ~100× a function call; buffering (read a large block, or use a `FILE*`) cuts it to a handful of crossings. (c) `clock_gettime` is served by the **vDSO** — a page the kernel maps read-only into every process — so it runs entirely in user mode with no trap, and the "call" makes no system call at all. *Tempting wrong answer:* "reading byte-by-byte is slow because the disk is slow" — the dominant cost here is the boundary crossings, not the device (the data may already be cached). **Feed-forward:** §13.4.

13.8 **Step 4:** When the program died, the diagnostic bytes were still in the **C library's user-space output buffer** — they had never crossed the boundary, because `printf` buffers and only calls `write` on a flush. The abnormal termination skipped the normal exit-time flush, so the line was lost: it never reached the terminal. Two ways to guarantee it appears before a possible crash: call **`fflush(stdout)`** immediately after the `printf` (force the crossing), or use **unbuffered output** — write with the `write` system call directly, or set the stream unbuffered / line-buffered. *Tempting wrong answer:* "the `printf` never executed" — it did; its output was simply stranded in the buffer. **Feed-forward:** §13.3, §13.4.

13.9 (a) **Neither / user space** — an integer add is a plain instruction. (b) **Boundary crossing** — opening a file needs the kernel (`openat`). (c) **User space** — sorting an in-memory array is pure computation. (d) **Boundary crossing** — sending a packet touches the network device, so it needs the kernel. (e) **User space** — `sprintf` formats into a memory buffer; no I/O, no crossing (contrast `printf`, which will eventually `write`). *Method-selection cue:* touch a device, another process, or kernel-managed state → crossing; compute in your own memory → user space. **Feed-forward:** §13.2–13.4.

13.10 (a) **Undefined behavior** — reading one past a stack array is an out-of-bounds access (Ch 11). (b) **A boundary crossing** — `read` on a valid descriptor is a system call. (c) **Undefined behavior** — signed integer overflow is UB (Ch 10). (d) **Neither** — a buffered `printf` that has not flushed is well-defined; the bytes are simply still in user space and no crossing has happened yet. *Tempting wrong answer:* calling (d) a system call — the crossing happens only at the flush, not at the `printf`. **Feed-forward:** §13.3, and Ch 10–11.

13.11 (a) A process's pointers are **virtual addresses translated through its own page tables** (Ch 4); a pointer value that names another process's or the kernel's memory does not map to anything in *this* process's address space, so the kernel's validation rejects it (or it simply is not this process's memory to read). (b) Without validation, a process could pass the kernel a pointer into *kernel* memory as the "buffer" for a `read`, tricking privileged code into copying secrets out, or into *another* process's memory — a confused-deputy attack; validation is what keeps the kernel from acting on addresses the caller has no right to. (c) Per-process address translation confines each process to its own memory (it cannot even name another's), and the user/kernel mode bit prevents it from rewriting the translation tables or reading devices to get around that — together, one process can neither address nor reach another's memory. *Tempting wrong answer:* "the kernel trusts the pointer because the process passed it" — the kernel must never trust a user pointer; validating it is essential to isolation. **Feed-forward:** §13.1–13.2, Ch 4.

13.12 Machine-specific by design. A correct write-up reports: (a) `strace` shows the `write` call with its arguments and return value (e.g. `write(1, "...", n) = n`); (b) a trivial hello-world makes a couple dozen system calls, most of them process startup (dynamic linker `mmap`/`openat`/ `mprotect`, `brk`, etc.) with only one or two being the program's own output; (c) the `getpid` system call costs on the order of ~100× a trivial function call (on the book's machine ~150–190 ns vs. ~1–2 ns), varying run to run and with mitigations; (d) one million `clock_gettime` calls produce *zero* `clock_gettime` system calls under `strace -c`, because the vDSO answers in user space. Essential discipline: report only measured numbers and label compiler/kernel/ hardware, since figures vary. *Tempting wrong answer:* "the syscall and the function call cost about the same" — the whole point is that they differ by roughly two orders of magnitude. **Feed-forward:** §13.2, §13.4.

Chapter 14 — Solutions

14.1 A process bundles (1) an **address space** (its private virtual memory — text, data, bss, heap, stack), (2) one or more **threads of execution** (the running context: program counter, stack pointer, registers), and (3) a set of **kernel resources** (open file descriptors, PID/parent PID, credentials, working directory). *Tempting wrong answer:* "a process is just the program's code" — the code on disk is inert; the process is the running instance with memory, execution state, and resources. **Feed-forward:** §14.1.

14.2 `fork` returns the **child's PID in the parent** and **0 in the child** (or `-1` on failure). That difference is how each of the two otherwise-identical processes learns which one it is, so it can branch to different code. *Tempting wrong answer:* "`fork` returns 0 to whichever called it" — both continue from the return; the caller becomes the parent and gets the child's PID, while the new child gets 0. **Feed-forward:** §14.2.

14.3 No — on success `exec` does **not return** (there is no original program to return to). It **replaces the entire process image** — text, data, heap, stack — with a new program, while **keeping the PID** (and the open file descriptors, unless marked close-on-exec). *Tempting wrong answer:* "`exec` starts a new process" — it does not; it transforms the *current* process into a different program, same PID. **Feed-forward:** §14.3.

14.4 `pid_t pid = fork(); if (pid == 0) { /* child */ } else if (pid > 0) { int st; waitpid(pid, &st, 0); /* parent, after child exits */ } else { /* fork failed */ }`. The `pid == 0` branch is the child; the `pid > 0` branch is the parent, which waits for the child; each knows which it is solely from `fork`'s return value. *Tempting wrong answer:* forgetting the parent must `wait` — without it the finished child becomes a zombie (§14.4). **Feed-forward:** §14.2, §14.4.

14.5 At `fork`, the kernel does **not** copy the parent's pages; it maps the same physical pages into both processes and marks them **read-only**. A real copy is triggered only when *either* process **writes** a page: the hardware faults, the kernel makes a private copy of that one page for the writer, and both continue. So the cost is proportional to the number of pages actually modified, not to the size of the address space — forking a 1 GB process is fast because almost none of that gigabyte is written before use. *Tempting wrong answer:* "`fork` duplicates all of memory immediately" — that would make forking large processes prohibitively slow; copy-on-write avoids it. **Feed-forward:** §14.2, Ch 4.

14.6 The shell `fork`s a child; the child `exec`s the requested program (replacing itself with it); the parent (shell) `wait`s for the child to finish before prompting again. The gap between `fork` and `exec` is where the child, still running the shell's code, adjusts its inherited resources — e.g. **redirecting standard output to a file** or wiring its input/output to a **pipe** — so that the program it then `exec`s starts up already plumbed into that environment. *Tempting wrong answer:* "the shell `exec`s the command directly" — then the shell would replace *itself* and never return to the prompt; it forks first so a child does the `exec`. **Feed-forward:** §14.3.

14.7 (a) `fork` is cheap because of **copy-on-write**: it shares the parent's physical pages read-only rather than copying them, so it does work proportional to pages later modified, not to address-space size. (b) A workload where the child promptly **writes to nearly every inherited page** (e.g. it mutates a large data structure it inherited) gets little from COW — almost every page faults and is copied anyway, so you pay close to a full copy. (c) Process-per-request pays **process-creation and context-switch overhead** and needs IPC to share data; thread-per-request pays the **loss of isolation** (a crash or memory bug in one thread takes down all requests in the process) and the burden of synchronizing shared state. *Tempting wrong answer:* "COW makes `fork` free regardless of workload" — a write-heavy child erodes the benefit. **Feed-forward:** §14.2, and Ch 15.

14.8 **Step 4:** The accumulating state is **zombies** — terminated children whose exit status the kernel is still holding because the parent never collected it. They eventually stop new `fork`s because each zombie occupies a **process-table slot**, and once the table (a finite resource) fills, the kernel cannot allocate a slot for a new process, so `fork` fails. This is a **defined resource leak, not undefined behavior** — the program violates no language rule; it simply fails to clean up (like a memory leak). Fix: the parent must **reap** its children — call `wait`/`waitpid` (e.g. from a `SIGCHLD` handler, or a reaping loop) so each child's status is collected and its slot freed. *Tempting wrong answer:* "the zombies are using up memory/CPU" — a zombie holds essentially no memory or CPU, only a table slot and its exit status; the leak is of slots. **Feed-forward:** §14.4.

14.9 (a) **`fork` then `exec`** — fork a child, have it `exec /bin/ls`, and the parent waits and continues (a lone `exec` would replace your program permanently). (b) **`fork`** alone — you want a second process running the *same* code, which is exactly what fork gives (no `exec` needed). (c) **`fork` then `exec`** — the shell forks and the child execs the typed command, while the shell waits. *Method-selection cue:* same program, new process → `fork`; different program, replace this one → `exec`; new process running a different program → both. **Feed-forward:** §14.2–14.3.

14.10 (a) **A defined-but-real defect** — the expectation is simply wrong: after `fork` the two processes have independent address spaces, so the child's changes are invisible to the parent (not UB, just a mistaken mental model that will produce bugs). (b) **A defined-but-real defect** — never reaping children leaks zombies (a resource leak, defined). (c) **Well-defined and fine (though usually unintended)** — both flushing a buffer copied across `fork` is defined behavior; it prints the line twice, which is a correctness surprise to fix with `fflush` before `fork`, but not UB. (d) **Well-defined and fine** — a child `exec`ing a new program while the parent runs is the normal pattern. *Tempting wrong answer:* calling (a) or (c) undefined behavior — both are defined; the issues are a wrong assumption and a duplicated buffer, not language-rule violations. **Feed-forward:** §14.2–14.4.

14.11 (a) A buffer overflow in process A writes through A's pointers, which are virtual addresses translated by **A's page tables** (Ch 4); they cannot name B's physical memory, and A cannot rewrite the translation tables to escape because installing page tables is a **privileged instruction** that traps in user mode (Ch 13). So the two mechanisms — per-process address translation and the user/kernel mode bit — confine the corruption to A. (b) Because a thread shares its process's address space, a compromised or crashing thread can corrupt the whole process; a separate *process* has its own address space and authority, so untrusted code in it cannot reach the host's memory — making the process the right sandbox unit. (c) Putting each tenant in its own process means one tenant's crash, memory corruption, or compromise cannot read or damage another tenant's data — fault and security isolation come for free from the process boundary. *Tempting wrong answer:* "a thread is isolated enough because it has its own stack" — a private stack does not isolate memory; all threads share the heap and globals and one bug corrupts them all. **Feed-forward:** §14.1, Ch 11, Ch 13, and Ch 15.

14.12 Machine-specific by design. A correct write-up reports: (a) the demo prints two lines with different PIDs, the child's `ppid` equal to the parent's `pid`, and independent values of the variable (e.g. child 101, parent 110) — confirming `fork` returned twice into two separate address spaces; (b) with a `printf`ed line *before* the `fork` and output redirected to a file (block buffering), the line appears **twice**, because `fork` copied the unflushed buffer into both processes and both flushed it at exit; (c) adding `fflush(stdout)` before the `fork` makes the line appear once, because the bytes crossed to the kernel before the buffer was duplicated. Essential discipline: report exactly what you observed and label your compiler/system. *Tempting wrong answer:* "the double line means `fork` ran the `printf` twice" — it ran once; `fork` duplicated the *buffer*, not the print. **Feed-forward:** §14.2, §14.4, and Ch 13.

Chapter 15 — Solutions

15.1 Threads in a process **share** the address space — code (text), the heap, global/static data, and open file descriptors. Each thread keeps **private** its own stack, its own registers (program counter and stack pointer), and its thread-local storage. *Tempting wrong answer:* "each thread has its own heap" — no; the heap is shared, which is exactly why threads can pass data by pointer and also why they race. **Feed-forward:** §15.1.

15.2 `pthread_create(&tid, attr, fn, arg)` takes a pointer to a `pthread_t` to fill in, optional attributes, the function `fn` (of type `void *(*)(void *)`) the new thread will run, and the `arg` passed to it; it starts `fn(arg)` running concurrently. `pthread_join(tid, &ret)` blocks until that thread finishes and collects its return value. *Tempting wrong answer:* "`pthread_create` runs the function and returns its result" — it starts the function on a *new* thread and returns immediately; `pthread_join` is what waits. **Feed-forward:** §15.2.

15.3 Linux uses the **one-to-one (1:1)** model — each `pthread` is its own kernel-scheduled thread (implemented by NPTL). `pthread_create` uses the **`clone`** system call underneath, with flags including `CLONE_VM` to share the address space. *Tempting wrong answer:* "Linux threads are M:N green threads scheduled in user space" — that was never NPTL; Linux threads are 1:1 kernel threads (language runtimes may add their own user-space scheduling above them). **Feed-forward:** §15.3.

15.4 `pthread_t t1, t2; pthread_create(&t1, NULL, worker, NULL); pthread_create(&t2, NULL, worker, NULL); pthread_join(t1, NULL); pthread_join(t2, NULL);` with `void *worker(void *a) { ... return NULL; }`. `pthread_create` is the thread analog of `fork` (start a new flow of control) and `pthread_join` of `wait` (wait for it and collect its result). The joins are necessary because if `main` returned first, the process would exit and tear down the shared address space the workers are still using. *Tempting wrong answer:* skipping the joins "because the threads will finish anyway" — `main` returning ends the process and can kill still-running threads. **Feed-forward:** §15.2, and Ch 14.

15.5 `counter++` compiles to a **read-modify-write**: (1) load `counter` into a register, (2) add one, (3) store it back. With two threads and no synchronization, an interleaving such as — A loads 5, B loads 5, A computes 6, B computes 6, A stores 6, B stores 6 — has both threads increment from the same starting value, so two increments produce a net change of only **one**; one update is lost. Repeated across millions of iterations, many updates vanish. *Tempting wrong answer:* "`++` is a single atomic operation" — in C it is not; it is three separate steps that can interleave. **Feed-forward:** §15.4.

15.6 **Address space:** processes have separate address spaces; threads share one. **Isolation:** processes are isolated (a crash is contained); threads are not (a crash or memory bug kills the whole process). **Cost to create:** a thread is cheaper (no address-space copy / copy-on-write setup); a process is heavier. **Cost of sharing data:** processes need explicit IPC (pipes, shared memory); threads share by pointer for free — but that "free" sharing costs you the obligation to synchronize (data races). *Tempting wrong answer:* "threads are strictly better because they're cheaper" — they trade away isolation, which is sometimes exactly what you need. **Feed-forward:** §15.1, and Ch 14.

15.7 (a) Thread creation skips duplicating the address space — no copy-on-write page setup, no new page tables — because the new thread *shares* the existing address space; it just needs a stack and a kernel-scheduled context. (b) In exchange you take on the **data race** hazard (and the loss of isolation): every piece of shared mutable state must now be synchronized or it is undefined behavior. (c) The loss is **not deterministic** — it depends on how the threads' read-modify-writes happen to interleave, which varies with scheduling, load, core count, and compiler, so the demo gave a different wrong total each run. Its nondeterminism means **tests cannot be trusted to catch it**: a racing program can pass repeatedly (even print the exactly-right answer) and still be broken. *Tempting wrong answer:* "if the test passes, there's no race" — a race is defined by the rules (unsynchronized conflicting access), not by whether a run misbehaved. **Feed-forward:** §15.4.

15.8 **Step 4:** The bug is a **data race** on `hits` — multiple threads performing unsynchronized read-modify-writes (`hits++`) on shared memory — and in C/C++ it is **undefined behavior**. The unit tests missed it because they exercised requests **one at a time**, so no two increments ever overlapped and nothing raced; the bug only manifests under genuine concurrency, which the tests never created. The fix category is **synchronization**: make the update atomic — e.g. protect it with a **mutex** (`pthread_mutex_lock/unlock` around `hits++`) or use an **atomic** increment (a `_Atomic`/ `std::atomic` counter, or an atomic fetch-add). *Tempting wrong answer:* "add more tests" — single-threaded tests, however many, cannot exercise a race; you need concurrency plus a race detector, and the real fix is synchronization. **Feed-forward:** §15.4.

15.9 (a) **Separate process** — isolation is the whole point; an untrusted plugin in its own address space cannot corrupt or crash the host, whereas a thread shares memory and could take everything down. (b) **Thread** — the workers all read one large in-memory array, so shared-address-space threads let them access it directly with no copy or IPC (and reads alone do not race). (c) **Separate process** (via `fork`+`exec`) — running a typed command means launching a different program, which is a process, not a thread. *Method-selection cue:* need containment/a different program → process; need cheap shared-memory work → thread. **Feed-forward:** §15.1, and Ch 14.

15.10 (a) **Undefined behavior** — two threads writing the same non-atomic variable with no synchronization is a data race. (b) **Well-defined and fine** — writing two *different* variables with no shared location is not a race (barring false-sharing performance effects, which are a cost, not UB). (c) **Undefined behavior** — reading through a freed pointer is a use-after-free (Ch 9), regardless of threads. (d) **Well-defined and fine** — concurrent *reads* of a shared value, with no writer, do not race. *Tempting wrong answer:* calling (b) or (d) a race — a data race requires a conflicting access (at least one write) to the *same* location; disjoint writes and read-only sharing are fine. **Feed-forward:** §15.4, and Ch 9.

15.11 (a) The added clause: **no unsynchronized concurrent access to shared mutable memory** (any two threads' accesses to the same location, at least one a write, must be ordered by synchronization). (b) Violating it is a data race, which is **undefined behavior** in C/C++; unlike a use-after-free — which reliably touches freed memory every run — a race depends on interleaving, so it **may never reproduce** under testing even though the program is incorrect for some schedule. (c) A language can move this to compile time with a type system that tracks sharing and mutation: Rust's `Send`/ `Sync` traits mark which data may cross or be shared between threads, and the borrow checker forbids aliasing a mutable reference — so an unsynchronized shared mutation is a *compile error*, not a runtime gamble. *Tempting wrong answer:* "a thorough test suite proves there's no race" — no finite set of runs can, because the offending interleaving may simply never occur; only reasoning from the rules (or a checker/ detector) settles it. **Feed-forward:** §15.4, §15.5, and Part 5 (Rust).

15.12 Machine-specific by design. A correct write-up reports: (a) the four-thread, one-million-each unsynchronized counter prints totals well below 4,000,000 that differ every run (on the book's machine ~1.1–1.8 million across five runs) — lost updates from interleaved read-modify-writes; (b) adding a mutex around the increment (or using an atomic increment) restores exactly 4,000,000 every run; (c) the bug's visibility depends on build and variable: at `-O2` without `volatile` the optimizer may collapse each loop into a single add, shrinking the race window so the "wrong" answer hides (it printed 4,000,000 for the author), while `-O0` or a `volatile` counter forces per-iteration memory writes and exposes the loss — demonstrating that a correct-looking run does not mean the code is correct. Essential discipline: report exactly what you saw and label compiler/core count/ hardware. *Tempting wrong answer:* "the version that printed 4,000,000 is the correct one" — it is the same racy code; it merely failed to manifest. **Feed-forward:** §15.4.

Chapter 16 — Solutions

16.1 The states are **running** (on a CPU now), **ready** (runnable but no free CPU), and **blocked** (waiting for an event — I/O, a child, a lock — and unable to run until it arrives). The scheduler chooses the next runner from the **ready** threads only; a blocked thread is not a candidate until the event wakes it. *Tempting wrong answer:* "the scheduler picks among all threads" — it cannot run a blocked thread; picking one would accomplish nothing until its event fires. **Feed-forward:** §16.1.

16.2 **Turnaround time** = completion − arrival; **response time** = first-run − arrival. For the job: response = 8 − 0 = **8**; turnaround = 40 − 0 = **32**. *Tempting wrong answer:* reporting response as the time it *finished* (40) — response measures when the job first got the CPU, not when it completed. **Feed-forward:** §16.1.

16.3 The **convoy effect**: short jobs pile up behind a long one (like cars behind a truck), so they inherit its runtime and average turnaround balloons. It afflicts **FIFO** (first-come-first-served, no preemption). *Tempting wrong answer:* attributing it to round-robin — RR's small slices are exactly what break the convoy. **Feed-forward:** §16.2.

16.4 **FIFO (X, Y, Z):** X completes at 8, Y at 12, Z at 16 → average turnaround (8+12+16)/3 = **12 ms**. **SJF (Y, Z, X):** Y at 4, Z at 8, X at 16 → average (4+8+16)/3 = **9.33 ms**. SJF is lower, and it is no accident: when all jobs arrive together, running shorter jobs first minimizes the sum of completion times, which is exactly what average turnaround measures — SJF is provably optimal for it. *Tempting wrong answer:* assuming the last-completion time (16 ms in both) decides the average — it is the same, yet the averages differ because ordering changes the *earlier* completions. **Feed-forward:** §16.2.

16.5 SJF minimizes average turnaround because, with simultaneous arrivals, a job's turnaround includes the runtime of everything before it; ordering shortest-first makes each long job's runtime fall on the fewest later jobs (any swap putting a longer job earlier only adds its length to more waits). It is unusable in general because (1) it needs an **oracle** — you must know each job's length in advance, which for interactive and server work you never do — and (2) even given the oracle it gives poor **response time**: a job arriving after a long one starts must wait (or, with STCF, only if shorter). *Tempting wrong answer:* "just estimate the length" — MLFQ does something like this by observing behavior, but a pure SJF that mis-estimates loses its optimality and can starve. **Feed-forward:** §16.2, §16.3.

16.6 An MLFQ keeps several priority queues and always runs from the highest non-empty one (round-robin within a level). A job's priority *adapts*: it enters at the top; if it uses its entire time slice (CPU-bound), it is demoted a level; if it yields before the slice ends (blocks on I/O — interactive), it keeps its level. So interactive jobs, which block frequently, stay near the top and are scheduled quickly (low response time), while CPU-bound jobs sink and share the leftovers — the scheduler *learns* each job's character from behavior rather than being told it. *Tempting wrong answer:* "the programmer sets the priority" — MLFQ derives it from observed CPU/blocking behavior; a boost and anti-gaming accounting keep it honest. **Feed-forward:** §16.4.

16.7 (a) Shrinking the slice improves **response time** (jobs cycle onto the CPU sooner) and hurts **turnaround/throughput** (more interleaving, more context switches). (b) If the slice equals the switch cost, then for every unit of switching the CPU does one unit of useful work — roughly **half** the CPU is wasted on switching (and it only gets worse as the slice drops below the switch cost). (c) A few milliseconds is large enough that a few-microsecond switch is a tiny percentage overhead, yet small enough that interactive jobs still feel instant; seconds would wreck response time, microseconds would drown in switch overhead. *Tempting wrong answer:* "smaller is always more responsive, so make it tiny" — below a threshold, switch overhead dominates and total throughput collapses. **Feed-forward:** §16.3.

16.8 **Step 4:** Lower the dashboard's `nice` value (or, better, put the two workloads in separate **cgroups** and give the dashboard a larger **CPU weight**). Under Linux's proportional-share scheduler this grants the dashboard a bigger *share* of the CPU when the two contend, while the batch job still runs freely whenever the dashboard is idle. It is safer than `SCHED_FIFO` because it changes the *proportion*, not the absolute priority: a low-weight job still gets *some* CPU, so nothing starves and a runaway loop cannot freeze the machine, whereas a CPU-bound `SCHED_FIFO` thread can starve everything on its core, kernel threads included. *Tempting wrong answer:* "give the dashboard real-time priority" — that fixes the starvation by creating a worse one and risks wedging the box. **Feed-forward:** §16.4.

16.9 (a) **Response time** — the editor must react to each keystroke immediately; the user feels lag directly. (b) **Turnaround time** — the batch job's only requirement is that the whole thing finishes by a deadline; no user is watching it start. (c) **Throughput** — a build farm is judged by jobs completed per hour in aggregate, not any one job's latency. *Method-selection cue:* a human waiting on each action → response; a job that must complete → turnaround; a fleet maximizing aggregate work → throughput. **Feed-forward:** §16.1.

16.10 (a) **False** — a process blocked on `read` is off the CPU entirely; it consumes no CPU until the data arrives and it becomes ready. (b) **False** — Linux threads are 1:1 kernel-scheduled entities (Chapter 15), so the scheduler picks among *threads*. (c) **False** — a context switch costs register save/restore plus cold cache/TLB; the slice size trades responsiveness against that overhead. (d) **True** — a lower `nice` value means a larger weight and thus a larger CPU share under contention. *Tempting wrong answer:* calling (d) false because "nice" sounds like it should lower priority — the naming is inverted: *lower* nice is *higher* priority. **Feed-forward:** §16.1, §16.4, and Ch 15.

16.11 (a) Strict SJF always runs the shortest ready job; if short jobs keep arriving, a long job is never the shortest and **never runs** — indefinite starvation. (b) An MLFQ periodically **boosts** every job back to the top priority, guaranteeing even a long, demoted job eventually gets CPU; without the boost, CPU-bound jobs stuck at the bottom could starve behind a steady stream of interactive work. (c) A proportional-share scheduler gives every job CPU in proportion to its weight, so a low-weight job gets a *small* share, never *zero* — "smaller share" is not "no share." That is exactly why `nice` cannot starve a task (it only shrinks the proportion) while `SCHED_FIFO`, being strict priority, can (a higher-priority runnable thread gets *all* the CPU). *Tempting wrong answer:* "a low nice value can starve a process" — no; only strict-priority classes can starve, which is the whole safety argument for using nice/weights instead. **Feed-forward:** §16.4.

16.12 Machine-specific by design. A correct write-up reports: (a) a loop that spins for N wall-seconds and prints the CPU-seconds it accrued; (b) pinned to one shared core, the `nice 0` copy gets far more CPU than the `nice 19` copy — on the book's machine (Linux 6.18, EEVDF) roughly 4.66 vs 0.09 CPU-seconds over a five-second window, about fifty to one; (c) on separate cores the two no longer compete for the same CPU, so each gets a full core and the nice difference nearly vanishes — `nice` only matters *under contention for the same CPU*. The discipline: report exactly what you saw and label kernel/core count/scheduler. *Tempting wrong answer:* "nice 19 was paused/slept" — it was not; it ran continuously but was granted a much smaller *share* of the contended core. **Feed-forward:** §16.4.

Chapter 17 — Solutions

17.1 A **virtual page** is a fixed-size chunk (4 KiB on x86-64) of a process's virtual address space; a **physical frame** is an equal-size chunk of physical RAM. A page-table entry's **present bit** records whether that virtual page currently occupies a frame (present) or does not (not present). *Tempting wrong answer:* "pages and frames are the same thing" — they are the same size, but a page is virtual and a frame is physical; the page table is exactly the mapping between them. **Feed-forward:** §17.1.

17.2 A **page fault** is a hardware exception raised when a process accesses a virtual page whose PTE is not present; it traps into the kernel's fault handler. The two broad outcomes: (1) the access is **legal**, so the handler materializes the page — finds a frame, zeroes it (anonymous) or reads it from a file/swap, sets present, and restarts the instruction; or (2) the access is **illegal**, so the handler delivers `SIGSEGV`. *Tempting wrong answer:* "a page fault means a crash" — most page faults are the normal, successful mechanism of demand paging; only the illegal case becomes a fault the program sees. **Feed-forward:** §17.2.

17.3 A **minor** fault is satisfied without disk I/O — the page comes from a zeroed frame or one already in memory (e.g. the page cache). A **major** fault must read the page in from a file or swap, costing a disk access. *Tempting wrong answer:* "all page faults hit the disk" — anonymous demand-zero and cache-hit faults are minor and never touch the disk. **Feed-forward:** §17.2, §17.3.

17.4 The `mmap` only recorded a 1 GiB mapping in the page table with the present bits clear — no frames spent — so RSS was just the program itself immediately afterward. As the program touched pages, each first touch raised a page fault, the handler gave that page a frame and set present, and RSS climbed one page at a time toward 1 GiB. The allocation did not consume 1 GiB up front because **demand paging** makes allocation a promise; *residency* is paid per touched page. *Tempting wrong answer:* "the OS was slow to account for the memory" — nothing was resident to account for; the frames genuinely did not exist until the touches. **Feed- forward:** §17.2.

17.5 Reference 1 2 3 1 4 2 5, 3 frames. **FIFO** = **5** faults: [1]→[1,2]→[1,2,3]→hit 1→evict 1 for 4 [2,3,4]→hit 2→evict 2 for 5 [3,4,5]. **LRU** = **6** faults: [1]→[1,2]→[1,2,3]→hit 1 (1 now MRU) [2,3,1]→evict LRU 2 for 4 [3,1,4]→evict LRU 3 for 2 [1,4,2]→evict LRU 1 for 5 [4,2,5]. Here **FIFO does better** (5 vs 6) — a deliberate reminder that LRU is a *heuristic* that pays off when the reference stream has locality; on a short string engineered without it, LRU can lose. Only OPT is guaranteed optimal. *Tempting wrong answer:* "LRU always beats FIFO" — LRU usually wins on real (local) workloads, but not on every string; this one is a counterexample. **Feed-forward:** §17.4.

17.6 **Belady's anomaly** is that giving a program more page frames can *increase* its fault count. It is exhibited by **FIFO** (on 1 2 3 4 1 2 5 1 2 3 4 5, FIFO takes 9 faults with 3 frames but 10 with 4). LRU and OPT cannot exhibit it because they are *stack algorithms*: their **resident set with more frames is a superset of the resident set with fewer**, so any page that would have been a hit with N frames is still resident with N+1 — adding frames can only remove faults, never add them. FIFO lacks this property because a larger queue reorders evictions and can eject a soon-to-be-reused page it would have kept. *Tempting wrong answer:* "more memory always means fewer faults" — true for LRU/OPT, false for FIFO; that surprise is the whole point of the anomaly. **Feed-forward:** §17.4.

17.7 (a) A minor fault costs a handful of instructions plus a zeroed or already-resident frame — sub-microsecond; a major fault waits on a disk. Using Chapter 3's hierarchy, DRAM access is tens of nanoseconds while an SSD read is tens of microseconds and a spinning disk seek is milliseconds — so a major fault is roughly a thousand to a million times slower than a minor one. (b) A *clean* page is an unmodified copy of data still on disk (or a file), so the OS can drop it and re-read later; a *dirty* page has been modified, so it must be **written out** to swap before its frame is reused — an extra disk write. (c) The likely cause is paging/swap pressure: the working set no longer fits, so the service takes major faults that stall it for disk-access times. The fix is *not* in the CPU-bound code — it is to shrink the memory footprint or add RAM so the working set fits. *Tempting wrong answer:* "profile and optimize the hot function" — the CPU is not the bottleneck; the cores are idle waiting on disk. **Feed-forward:** §17.3, §17.4, and Ch 3.

17.8 **Step 4:** The phenomenon is **thrashing** — the combined working set exceeds physical memory, so nearly every eviction removes a page another thread is about to need, producing a storm of **major** page faults. CPU utilization is near *zero* because the cores are almost always *blocked waiting on disk I/O* (paging pages in and out), not computing — the machine is I/O-bound on the swap device, not CPU-bound. The fix is to shrink the working set, not "optimize the loop": go back to one (or a few) sequential passes so the active window fits in RAM, or add memory. *Tempting wrong answer:* "add more threads / cores to speed it up" — more concurrent passes enlarge the working set and make the thrashing worse; the original single sequential pass was faster precisely because its working set fit. **Feed-forward:** §17.4.

17.9 (a) **Demand paging / overcommit** — the allocation only reserved mappings; the slowdown is the pages faulting in as they are first touched. Measure: RSS climbing over time (vs the reserved VSZ). (b) **Thrashing** — the dataset just crossed the size where the working set stopped fitting in RAM, so the system is now swapping. Measure: major-fault rate and swap I/O. (c) **Page cache / demand paging of a file mapping** — the first pass takes major faults reading the file from disk; the second pass hits the now-cached pages (minor or no faults). Measure: major faults on first pass vs near-zero on the second. *Method-selection cue:* "allocate huge/fast then slow" → demand paging; "small growth, huge slowdown" → crossed the working-set cliff (thrashing); "slow first, fast second" → cache warming. **Feed-forward:** §17.2, §17.3, §17.4.

17.10 (a) **False** — a successful `malloc` records mappings; physical RAM is consumed only as pages are touched (demand paging/overcommit). (b) **True** — a segmentation fault is a page fault the handler found no legal mapping for, so it delivered `SIGSEGV`. (c) **False** — `fork` uses *copy-on-write*: it copies page tables, not data; frames are shared read-only until a write faults and copies one page. (d) **False** — each process has its own page table, so the same virtual address in two processes normally maps to *different* frames (that is the isolation); they share a frame only through explicit shared memory or a shared file mapping. *Tempting wrong answer:* calling (d) true by analogy to threads — threads share one page table; distinct processes do not. **Feed-forward:** §17.1, §17.2, and Ch 4, Ch 14.

17.11 (a) `fork` copies the parent's *page tables*, mapping the child's pages to the same physical frames as the parent's, but marks every shared writable page **read-only** — so the "copy" is instantaneous and shares all data. (b) When the child (or parent) writes to a read-only shared page, the write raises a **protection page fault**; the handler sees a copy-on-write page, allocates a fresh frame, copies *that one page*'s contents into it, remaps the writer's PTE to the new frame with write permission, and restarts the instruction — exactly **one** page is copied per first write. (c) This makes `fork`-then-`exec` nearly free even for a large parent: `exec` immediately replaces the address space, so almost none of the shared pages are ever written before being discarded — copying them eagerly would be wasted work. A single write to a shared page costs one page fault plus one page copy. *Tempting wrong answer:* "the first write copies the whole address space" — copy-on-write copies only the single page touched, on demand. **Feed-forward:** §17.5, and Ch 14.

17.12 Machine-specific by design. A correct write-up reports: (a) touching one byte per 4 KiB page across the region yields about one minor fault per page (on the book's machine, 25,672 minor faults and 0 major faults to touch 100 MiB), shown via `/usr/bin/time -v`; (b) `VmRSS` is tiny right after a large `mmap` and grows to about the mapping size after touching it all (on the book's machine, 1,364 kB → 525,976 kB for 512 MiB) — reservation vs residency; (c) if you dare to exceed RAM, major-fault and swap-I/O rates climb while CPU utilization drops, the signature of thrashing — stop before the box wedges or the OOM killer fires. Label kernel/page size/RAM. *Tempting wrong answer:* "RSS should equal the mapping size right after `mmap`" — it should not; that is exactly what demand paging prevents. **Feed-forward:** §17.2, §17.4.

Chapter 18 — Solutions

18.1 A **signal** is an asynchronous notification the kernel delivers to a process to indicate an event (a software interrupt). It differs from a normal function call in that the program never *calls* the handler: the kernel *injects* it into the control flow at an arbitrary instant, suspending whatever was running, then resumes that code when the handler returns. *Tempting wrong answer:* "a signal is just a callback you invoke" — you register it, but delivery is forced by the kernel, not called by your code. **Feed-forward:** §18.1, §18.2.

18.2 **Terminate** (`SIGTERM`, `SIGINT`); **Terminate and dump core** (`SIGSEGV`, `SIGABRT`); **Ignore** (`SIGCHLD`'s default); and **Stop/Continue** (`SIGSTOP`/`SIGCONT`). *Tempting wrong answer:* "the default is always to terminate" — several signals default to ignore or to stop/continue, not termination. **Feed- forward:** §18.1.

18.3 **`SIGKILL`** and **`SIGSTOP`** cannot be caught or ignored. If a process could catch them, it could refuse to die or to be stopped — a runaway or malicious program would be unkillable, so the OS would lose its last-resort control. Keeping these two unblockable guarantees the kernel can always terminate or halt any process. *Tempting wrong answer:* "you can catch `SIGKILL` to clean up first" — you cannot; that is exactly why `SIGTERM` (catchable) exists for graceful shutdown and `SIGKILL` for the forced kill. **Feed-forward:** §18.1.

18.4 When `Ctrl-C` sends `SIGINT`, the kernel suspends the blocked `read`, runs the handler, and then the `read` returns `-1` with `errno == EINTR` ("interrupted system call"). It did not resume waiting because the kernel had to unblock the process to deliver the signal, and the default is to report the interruption rather than silently re-block. The two standard fixes: (1) **retry** the call when `errno == EINTR` (a loop), or (2) install the handler with **`SA_RESTART`** so the kernel auto-restarts many interrupted calls — though not all, so robust code still handles `EINTR`. *Tempting wrong answer:* "treat `EINTR` as a real read error and abort" — it is not an error about the data; the correct response is to retry. **Feed-forward:** §18.3.

18.5 It is unsafe because a handler can run in the middle of arbitrary code, including inside `printf`/ `malloc` themselves; those functions hold internal locks / mutable state and are not reentrant, so the handler's re-entry can deadlock or corrupt the heap. A function callable from a handler must be **async-signal-safe** (essentially reentrant — the `signal-safety(7)` list, e.g. `write`, `_exit`). The safe pattern: the handler sets a `volatile sig_atomic_t` flag (or does one `write` to a self-pipe) and returns; the main loop notices and does the real work synchronously. *Tempting wrong answer:* "it's fine if you only `printf` occasionally" — it only has to interrupt `printf` once, under load, to hang; rarity is what makes the bug dangerous, not safe. **Feed-forward:** §18.4.

18.6 Standard signals (1–31) are represented by a single pending bit each, so they do **not queue**: if the same signal is generated multiple times while pending/blocked, the process sees it once when it is delivered; the extra instances coalesce. So if several children exit close together, the parent may receive a single `SIGCHLD` for all of them — reaping exactly one child per signal would leave the others as zombies (Chapter 14). A correct handler therefore loops: `while (waitpid(-1, &status, WNOHANG) > 0) ;` reaps *all* currently-exited children per delivery. *Tempting wrong answer:* "one `SIGCHLD` per child, so reap one" — signals coalesce, so the count of deliveries does not match the count of events. **Feed-forward:** §18.4, and Ch 14.

18.7 (a) Setting a `volatile sig_atomic_t` flag is a single, atomic, async-signal-safe store — it cannot deadlock or corrupt anything — whereas doing the work directly risks re-entering unsafe libc; and it is cheap (one write). (b) The self-pipe / `signalfd` approach adds a little latency (a `write` then the main loop's next `read`) but buys correctness: the real work runs in the synchronous main flow where all functions are safe, and it integrates signals into an ordinary event loop. (c) It can never be a reliable counter because standard signals do not queue — multiple occurrences while pending collapse into one delivery, so the number of handler runs under-counts the number of events. *Tempting wrong answer:* "increment a counter in the handler to count events" — coalescing makes that count wrong under load. **Feed-forward:** §18.4.

18.8 **Step 4:** The bug is an **async-signal-safety violation**: the `SIGHUP` handler calls non-reentrant functions (`malloc`, stdio `fprintf`), so when the signal interrupts a request thread that is already inside one of them, the handler re-enters it and **deadlocks** (or corrupts the heap). It appears only under load because it requires the signal to land *during* an unsafe call; light testing rarely hits that window, but heavy request traffic keeps threads inside `malloc`/stdio a large fraction of the time, so the odds rise — the same schedule-dependent, "passes tests / breaks in production" pathology as a data race. The fix: make the handler trivial (set `volatile sig_atomic_t reload_requested = 1;` or `write` a self-pipe byte and return), and perform the config re-read, `malloc`, and logging in the **main loop** when it observes the flag, where libc is safe to call. *Tempting wrong answer:* "add a lock around the handler's `malloc`" — the handler may already hold or be blocked on that very lock via the interrupted call; locking cannot make a handler reentrant. **Feed-forward:** §18.4.

18.9 (a) **Signal** (`SIGHUP` by convention) — a content-free "reload" notification is exactly what a signal is for. (b) **Real IPC — a pipe** — a signal carries no data; a byte stream needs a pipe (or socket). (c) **Signal** — this *is* a signal (`SIGCHLD`); the parent then `wait`s. (d) **Real IPC — a socket** (or a pipe pair) — structured request/reply is a conversation, and a signal is only a doorbell. *Method-selection cue:* a content-free notification → signal; moving data or a request/reply → pipe/socket/shared memory. **Feed-forward:** §18.1, §18.5.

18.10 (a) **False** — the kernel injects the handler asynchronously; the program does not call it. (b) **False** — `SA_RESTART` restarts *many* but not all interrupted calls, so robust code still handles `EINTR`. (c) **False** — `SIGKILL` cannot be caught, so no cleanup runs; use `SIGTERM` for graceful shutdown. (d) **True** — an invalid memory access causes the kernel to deliver `SIGSEGV`, whose default disposition terminates (and cores) the process. *Tempting wrong answer:* calling (d) false because "a segfault is a hardware thing" — the hardware fault is *translated* by the kernel into the `SIGSEGV` signal. **Feed-forward:** §18.1, §18.2, §18.3, and Ch 11.

18.11 (a) `volatile` stops the compiler from caching the flag in a register or optimizing the read away, so the main loop actually re-reads memory the handler wrote; `sig_atomic_t` guarantees the read/write is a single uninterruptible operation, so the handler cannot catch a half-written value. Without both, the compiler might never observe the change, or the main loop might see a torn value. (b) It is *similar* to a data race: two flows of control touch the same memory with no ordering, so the shared access needs care. It is *different* in that there is one thread — the handler cannot run truly simultaneously with the main flow, only interrupt it — so the concern is atomicity of a single access and compiler visibility, not simultaneous multi-core writes (which is why `sig_atomic_t` suffices here where full synchronization is needed between threads). (c) Because the handler is a second, asynchronously-scheduled flow of control sharing memory with the main flow, a single-threaded program with signals already has concurrency — the interruption can happen between any two instructions. *Tempting wrong answer:* "a single-threaded program can't have concurrency bugs" — signals introduce asynchronous control flow, which is concurrency. **Feed-forward:** §18.4, and Ch 15.

18.12 Machine-specific by design. A correct write-up reports: (a) with a plain handler (no `SA_RESTART`), the `read` interrupted by `SIGALRM` returns `-1` with `errno == EINTR` (on the book's machine, "errno=4 (Interrupted system call)"); reinstalling with `SA_RESTART` makes the kernel restart the `read` so it keeps blocking (no `EINTR` surfaced). (b) The counting handler runs **once**, not five times, because the five blocked `SIGUSR1`s coalesce into one pending bit ("handler count so far = 0" while blocked, then "ran 1 time(s)" after unblock). (c) The first shows a caught signal interrupts a blocking call; the second shows standard signals do not queue. Label compiler/kernel. *Tempting wrong answer:* "the handler should run five times" — non-queuing standard signals collapse the repeats. **Feed- forward:** §18.3, §18.4.